

The logo for CANalyzer. It consists of the word "CAN" in white, bold, sans-serif capital letters, enclosed within a dark gray rectangular box. To the left of this box is a small red square. To the right of the box, the word "alyzer" is written in a red, italicized, sans-serif font.

User Guide

Version 3.1

International Quality Standard Certificate

Vector Informatik GmbH, 70499 Stuttgart, Germany, was awarded ISO 9001 certification on 19.08.1998. The ISO 9001 standard is a quality standard of the British Standards Institute that is recognized worldwide.

Certificate number: 70 100 F 1498 TMS

Sales associates list:

Subsidiaries

USA

Vector CANtech, Inc.
Suite 190
39500 Orchard Hill Place
Novi, MI 48375
Tel.: +1 248 449-9290
Fax: +1 248 449-9704
<http://www.vector-cantech.com>

Japan

Vector Japan Co., Ltd.
Nishikawa Bldg. 2F
3-3-9 Nihonbashi, Chuo-ku
Tokyo 103-0027
Tel.: +81-3-3316-7850
Fax: +81-3-3316-7855
<http://www.vector-japan.co.jp>

Distributors:

France

NSI S.A.
6, Avenue du Pré de Challes
"Parc des Glaisins"
74943 Annecy le Vieux
Tel.: +33 4 5009-4630
Fax: +33 4 5009-4631
<http://www.nsi.fr>

Germany

Softing GmbH
Richard-Reitzner-Allee 6
85540 Haar
Tel.: +49 89 45656-0
Fax: +49 89 45656-399
<http://www.softing.com>

Great Britain

Accutest Ltd.
Suite 2
9 George Street
Glossop
Derbyshire SK13 8AY
Tel.: +44 14 5786-6613
Fax: +44 14 5785-6789
<http://www.accutest.co.uk>

Israel

Dan-EL Technologies Ltd.
58 Amal Street
Petach Tikva 49513 Israel
Tel.: +9 72 3 927-1888
Fax: +9 72 3 927-1666
<http://www.danel.co.il>

Italy

MICROTASK Embedded S.r.L.
Via Paolo Bassi, 1
20159 Milano
Tel.: +39 02 6680-2557
Fax: +39 02 6900-8174
<http://www.microtask-embedded.it>

Korea

Hanil Mech. Eng. Consultants.
Automatic Control Division
140-012, 173, 2-KA, Hangang-RO, Yongsan-Ku
Seoul
Tel.: +82 2 709-4810
Fax: +82 2 709-4813
<http://www.hanilacd.co.kr>

Netherlands

Micro-key
Industrieweg 28
NL-9804 Noordhorn
Tel.: +31 594 50-3020
Fax: +31 594 50-5825
<http://www.microkey.nl>

Sweden

Kvaser AB
P.O. Box 4076
51104 Kinnahult
Tel.: +46 32 0-15287
Fax: +46 32 0-15284
<http://www.kvaser.se>

Copy Protection

The software is not copy protected. The licensee is entitled to make a (single) backup copy. The licensee is obligated to note the copyright of Vector Informatik GmbH on this copy. It is expressly prohibited to make additional copies and/or transfer the software to third parties. The conditions of the Vector licensing agreement must be observed.

The supplied software is provided with a serial number and a text entry identifying the licensee. Both entries are displayed briefly when the program is started. They can also be called up by the menu command *Help-Info*. The licensee must take appropriate precautions to ensure that no unlicensed copies can be made from his version. If it becomes known to the licensee that a copy of his version is being used by a third party, he is obligated to immediately stop this use or inform Vector of such use. Otherwise, the licensee is liable to Vector for consequential damages.

Typographic Conventions

Text formatting	Meaning	Example
Italics	Menu, command and dialog box names	<i>File Associate database</i>
Bold	Examples of CAPL syntax	on message 0x100 {...}
In angle brackets	Key names	<TAB>
Upper-case	File names	CAN.INI
Fixed-width font	CAPL syntax description	"DLC" "=" <integer>

Overview

Chapter 1, *Introduction* introduces CANalyzer and takes beginners on a short tour that provides an overview of the most important functions.

Chapter 2, *Working with CANalyzer* explains the concept of the measurement setup and explains basic CANalyzer functions for transmitting and analyzing data.

Chapter 3, *CAPL Programming* discusses the CAPL programming language, with which you can supplement and expand basic CANalyzer functions in a nearly unlimited manner. The application possibilities of this programming language are clarified by means of numerous examples.

Chapter 4, *Special Topics* gives you more in-depth information for a better understanding of CANalyzer functionality and offers you additional tips and tricks for using CANalyzer in special situations.

Chapter 5, *Appendix* you will find instructions on how to troubleshoot errors.

CANalyzer Feature Matrix

	CANalyzer/JUN	CANalyzer/FUN	CANalyzer/PRO
CAN Channels	1	n	n
Databases	1	n	n
Trace window / Trace Watch	1	n	n
Bus Statistics window	✓	✓	✓
Statistics window / Statistics report	—	n	n
Logging (without triggering)	1	n	n
Filter block	✓	✓	✓
Message Explorer & Signal Explorer	✓	✓	✓
Data window	1	n	n
Generator block	✓	✓	✓
Trigger block	—	n	n
Graphic window	—	n	n
Offline mode	✓	✓	✓
Replay block	—	✓	✓
CANdb Editor	✓	✓	✓
CAPL	—	—	✓

Legend:

- ✓ Available
- Not available
- 1,2 One / two of these available
- n Several of these available

Contents

1 Introduction.....	1-1
1.1 Overview	1-1
1.2 CANalyzer Tour.....	1-3
1.2.1 Preparations.....	1-3
1.2.2 Setting Up the CAN Bus.....	1-4
1.2.3 Transmitting Data.....	1-6
1.2.4 Evaluation Windows.....	1-9
1.2.5 Working with Symbolic Data.....	1-11
1.2.6 Analysis of Signal Values in the Data Window	1-13
1.2.7 Analysis of Signal Responses in the Graphics Window.....	1-15
1.2.8 Use of the Database in Transmitting Messages	1-15
1.2.9 Analysis of an Engine Area Simulation.....	1-16
1.2.10 Logging a Measurement.....	1-17
1.2.11 Evaluating a Log File.....	1-18
1.2.12 Tips for Solving Your Own Tasks	1-19
1.3 Basic Operation.....	1-19
1.3.1 Menu Operation.....	1-20
1.3.2 Dialog Boxes	1-20
1.3.3 Control of the Measurement Setup.....	1-21
1.3.4 The Help System.....	1-23
2 Working with CANalyzer	2-1
2.1 Preparations for the Measurement.....	2-1
2.1.1 Program Start.....	2-1
2.1.2 The CANalyzer Screen.....	2-2
2.1.3 Data Flow in the Measurement Setup.....	2-4
2.1.4 Configuration of the Measurement Setup	2-5
2.1.5 Working with Evaluation Blocks in the Measurement Setup	2-5
2.1.6 Measurement Start.....	2-6
2.1.7 Working with Configurations.....	2-7
2.1.8 Representation Formats.....	2-8
2.2 Interface to the Hardware	2-9
2.2.1 Configuring the Hardware	2-9
2.2.2 Programming the Bus Parameters	2-10
2.2.3 Acceptance Filtering.....	2-13
2.2.4 Card and Driver Options.....	2-14
2.3 Transmit and Receive Data.....	2-15
2.3.1 The Transmit Branch.....	2-15
2.3.2 The Evaluation Branches	2-15
2.3.3 Message Attributes.....	2-16
2.4 Use of Databases.....	2-18
2.4.1 Creating and Modifying Databases.....	2-19
2.4.2 Access to Database Information.....	2-19

2.5 The Measurement Window	2-20
2.5.1 Trace Window	2-20
2.5.1.1 Standard Configuration of the Trace Window.....	2-21
2.5.1.2 Configuration of the Columns in the Trace Window	2-23
2.5.1.3 Trace Window Options from the Toolbar.....	2-24
2.5.1.4 Trace Watch Functionality and Trace Watch Window	2-24
2.5.1.5 Optimizing the Trace Window	2-24
2.5.2 The Data Window.....	2-25
2.5.2.1 Configuration of Signals	2-25
2.5.2.2 Display Types.....	2-26
2.5.2.3 Activity Indicator	2-28
2.5.2.4 Peak Indicator	2-28
2.5.2.5 Optimization of Data Display	2-28
2.5.3 Graphic Window.....	2-29
2.5.3.1 Selecting Signals.....	2-29
2.5.3.2 Arrangement of Signals.....	2-30
2.5.3.3 Signal Layout	2-31
2.5.3.4 Configuration of the Measurement	2-32
2.5.3.5 Measurement and Display Functions	2-33
2.5.3.6 Signal Modes	2-33
2.5.3.7 Measurement Modes.....	2-34
2.5.3.8 Display Modes.....	2-35
2.5.3.9 Layout Functions.....	2-35
2.5.3.10 Export of Signals.....	2-37
2.5.3.11 Toolbar of the Graphics Window	2-37
2.5.3.12 Optimization of the Graphics Window	2-37
2.5.4 Statistics.....	2-39
2.5.4.1 Direct Display in the Statistics Window	2-40
2.5.4.2 Statistics Report.....	2-41
2.5.4.3 Choosing a Histogram.....	2-41
2.5.5 Bus Statistics.....	2-42
2.5.6 Write Window.....	2-43
2.5.7 Layout and Fonts of the Measurement Windows.....	2-43
2.6 Hotspots and Insertable Function Blocks	2-44
2.6.1 CAPL Nodes	2-45
2.6.2 Filter Block	2-46
2.6.3 Channel Filter.....	2-47
2.6.4 Replay Block	2-48
2.6.5 Generator Block	2-49
2.6.5.1 Configuration of Triggering.....	2-50
2.6.5.2 Configuration of Transmit List	2-50
2.6.5.3 Entry of Signal Values.....	2-51
2.6.5.4 Entry of Mode-Dependent Signals.....	2-52
2.6.5.5 Function Generator for the Transmit List.....	2-52
2.6.6 Interactive Generator Block	2-54
2.6.6.1 Configuring the Interactive Generator Block.....	2-55
2.6.6.2 Transmit List	2-56
2.6.6.3 Trigger Condition	2-56
2.6.6.4 Generating a High-Load Situation	2-57
2.6.6.5 Entering Signal Values.....	2-57
2.6.6.6 Entering Mode-Dependent Signals.....	2-57
2.6.6.7 Keyboard Control	2-58

2.6.7 Break.....	2-58
2.7 Logging and Evaluation of Measurement Files.....	2-59
2.7.1 Logging Trigger.....	2-59
2.7.1.1 Trigger Modes.....	2-60
2.7.1.2 Trigger Events.....	2-62
2.7.1.3 Time Window	2-63
2.7.1.4 Configuration of the Logging Buffer	2-63
2.7.2 Log Files	2-64
2.7.3 Event Types in Log Files	2-66
2.7.4 Data Analysis in Offline Mode	2-67
2.7.4.1 Flow Control in Offline Mode	2-68
2.7.4.2 Configuration of Online and Offline Modes.....	2-69
2.7.5 Trigger and Search Conditions.....	2-70
2.7.5.1 Condition Primitives	2-70
2.7.5.2 Entry or Change of Primitives (without Database).....	2-70
2.7.5.3 Entry of Change of Primitives (with Database)	2-71
2.7.6 Exporting and Converting Log Files.....	2-72
2.7.6.1 Export	2-72
2.7.6.2 Conversion.....	2-72
3 CAPL Programming.....	3-1
3.1 Overview	3-1
3.1.1 Potential Applications of CAPL Programs.....	3-1
3.1.2 Integration of CAPL Programs	3-2
3.1.3 Use of the Symbolic Database in CAPL	3-3
3.2 Introduction to CAPL.....	3-3
3.2.1 CAPL Variables	3-4
3.2.2 Declaration of Messages.....	3-6
3.2.3 Event Procedures.....	3-7
3.2.4 Expressions in CAPL.....	3-12
3.2.5 The Key Word <code>this</code>	3-13
3.2.6 Interpretation of Signal Values	3-13
3.2.7 Access to Attributes in CAPL.....	3-14
3.2.8 Working with Symbolic Signal Values.....	3-15
3.2.9 Working with Multiple Databases	3-16
3.2.10 Functions in CAPL.....	3-16
3.3 Creating a CAPL Program.....	3-17
3.4 Browser for Creating and Compiling CAPL Programs	3-19
3.4.1 Opening Browser.....	3-20
3.4.2 Browser Window	3-21
3.4.3 Compiling CAPL Programs.....	3-21
3.4.4 Searching for Run-Time Errors.....	3-21
3.4.5 Access to the Database	3-22
3.4.6 Importing and Exporting ASCII Files.....	3-22
3.4.7 Browser Options.....	3-23
3.5 CAPL Language Reference	3-23
3.5.1 Expanded Backus-Naur Form (BNF).....	3-23
3.5.2 Program Structure.....	3-24

3.5.3 Declaration of Variables	3-24
3.5.4 Event-Driven Procedures (Event Procedures)	3-27
3.5.5 CAPL Subroutines	3-35
3.6 Special Topics in CAPL	3-36
3.6.1 Integrating a Windows DLL in CAPL	3-36
3.6.2 Handling of Run-Time Errors	3-38
3.7 List of Intrinsic Functions	3-38
3.7.1 Access to CAN and Other Hardware	3-38
3.7.2 User Interaction	3-43
3.7.3 Logging Functions	3-46
3.7.4 Standard / Extended Identifiers	3-47
3.7.5 Flow Control	3-48
3.7.6 Time Management	3-49
3.7.7 Accessing CANoe Environment Variables	3-54
3.7.8 Intel/Motorola Formats	3-57
3.7.9 Trigonometry and Mathematical Functions	3-57
3.7.10 File Functions	3-59
3.7.11 Sequential File Access	3-61
3.7.12 String Functions	3-64
3.7.13 Speech Support and Debugging	3-65
3.8 Examples	3-66
3.8.1 Transmission of Messages	3-66
3.8.2 Gateway Application	3-70
3.8.3 CAPL-Driven Triggering	3-71
3.8.4 Use of the Symbolic Database	3-72
4 Special Topics	4-1
4.1 System Description	4-1
4.1.1 Overview of the Programs	4-1
4.1.2 CANalyzer Architecture	4-1
4.2 CANalyzer in Load and Overload Operation	4-3
4.2.1 Behavior in Load Situations	4-3
4.2.2 Behavior with Data Loss	4-3
4.2.3 Fill Level Indicator	4-3
4.2.4 Optimizing Performance	4-4
4.2.5 Configuration Options at High Bus Load	4-5
4.3 Working with Databases	4-6
4.3.1 Associating the Database	4-6
4.3.2 Use of Multiple Databases	4-7
4.3.3 Resolving Ambiguities	4-8
4.3.4 Checking for Consistency of Symbolic Data	4-8
4.4 Working with Multiple Channels	4-9
4.4.1 Channel Definition	4-9
4.4.2 Channels in Online Mode	4-9
4.4.3 Channels in Offline Mode	4-10
4.5 The DDE Interface	4-10

4.5.1 DDE Services	4-11
4.5.2 The DDE <i>System</i> Topic	4-12
4.5.3 The DDE <i>Execute</i> Transaction	4-12
4.5.4 Loading Configurations with the Explorer	4-14
4.5.5 COM-Server	4-15
5 Appendix	5-1
5.1 Particularities of the Demo Version	5-1
5.2 File Name Extensions Used	5-2
5.3 Troubleshooting	5-3
5.4 System Messages in the Write Window	5-4
5.5 List of Error Messages to the CAN Interface	5-8
6 Index	6-1

1 Introduction

1.1 Overview

CANalyzer is a universal development tool for CAN bus systems, which can assist in observing, analyzing and supplementing data traffic on the bus line. Based on its powerful functions and user-programmability, all needs are covered - from initial trial runs with CAN to selective error localization in complex problems.

You can choose to work with CANalyzer on the byte level with bus-like raw data format, or on the application level with logical/physical data representation. A CAN database is used to convert raw data. This database has become a de facto standard in the motor vehicle industry. The user-friendly database management program CANdb++ is included with CANalyzer.

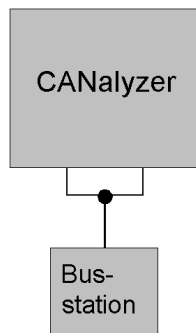
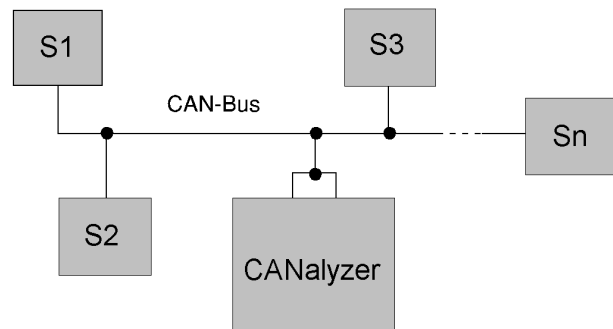
Even the basic built-in functions - which can be used without any programming knowledge - provide for an abundance of possible applications. These include listing bus data traffic (Tracing), displaying data segments of specific messages, transmitting predefined messages and replaying recorded messages, statistically evaluating messages, and acquiring statistics on bus loading and bus disturbances, as well as recording messages for replay or offline evaluation.

Furthermore, the user can expand CANalyzer's functionality as desired by means of user-programming. Program blocks can be inserted at any point in the data flow diagram. The application-oriented, C-like language CAPL (CAN Access Programming Language) serves as the programming language. A special event procedure concept and CAN-adapted language tools enable the user to develop quick solutions to specific problems. CANalyzer contains an interactive development environment that makes it easy to create, modify and compile CAPL programs.

Programmability results in numerous potential applications:

Emulation of a bus station:

CANalyzer can emulate those functions of a bus station which are relevant for data traffic, e.g. transmitting messages in response to certain events.

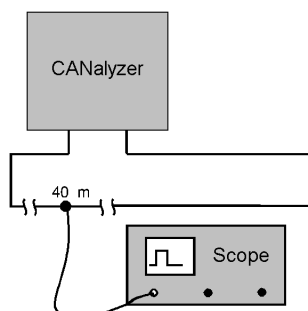
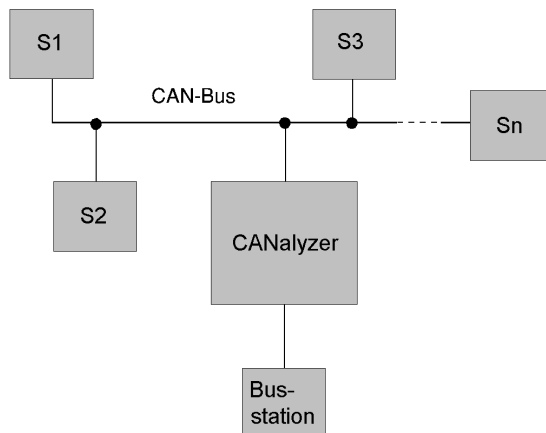


Emulation of system environment for testing a bus station:

In development equipment with bus connections the problem arises that the remaining bus participants not available yet for testing purposes. The data traffic of all remaining stations can be emulated with the help of CANalyzer to emulate the system environment.

Link between two buses:

This provides for exchange of data between CAN buses which may have different speeds. Another target application is to interpose CANalyzer between a bus station to be tested and the actual bus, in order to observe and manipulate the data transfer in a test environment.



Test generator for studying the physical layer:

It is possible to distribute messages intentionally between CANalyzer's two bus connections and to connect a real (i.e. long) bus line. This makes it possible to conduct very simple experiments involving arbitration or line reflections.

CANalyzer is controlled and configured from the data flow diagram in the measurement setup window. For further information on this please refer to section 2.1.4.

1.2 CANalyzer Tour

If you are starting up CANalyzer for the first time, and its functionality and controls are still completely new to you, the following tour will help you to become familiar with its operating concept and its most important features.

For this tour you will first set up a very simple CAN bus where CANalyzer assumes the roles of both sender and receiver. In the first step CANalyzer is configured as a data source, i.e. as a transmitting station. You will then learn about CANalyzer's analysis options by studying the generated data in the measurement windows afterwards.

In complex real systems CANalyzer typically also assumes both roles. You can utilize the program as a data source to transmit data to other controllers, but you can simultaneously use it to observe, log and evaluate the data traffic on the CAN bus.

In the last part of the tour you will become familiar with the CAPL programming language and expand CANalyzer functionality by adding a simple CAPL program.

1.2.1 Preparations

To start CANalyzer, call CANW32.EXE by double clicking the appropriate icon in the CANalyzer program group.

CANalyzer has various evaluation windows (Trace, Data, Graphics, Statistics and Bus Statistics windows) as well as a measurement setup window that shows you the data flow in CANalyzer and simultaneously allows you to configure CANalyzer.

You can access all program windows from the *View* menu on the main menu bar.



Figure 1: *View* Menu on Main Menu Bar

The data flow diagram of the CANalyzer measurement setup contains the data source on the left - symbolized by the symbol of a PC-card - and various evaluation blocks on the right serving as data sinks. That is, the data flow is from right to left. Connection lines and branches are drawn between the individual elements to clarify the data flow.

The information arriving at each evaluation block are displayed in the block's evaluation window. For example, the Trace window displays all information arriving at the trace block, while the Graphics window shows you information arriving at the graphics block.

The only exception is the logging block, which is not assigned a window but rather a file in which the data arriving at the block are logged.

On the left side of the measurement setup, CANalyzer's transmit branch branches off after the card icon. Data can be sent onto the bus from here. The data flow in the transmit branch always runs from top to bottom.

In the data flow diagram you will also recognize small black rectangles: . At these insertion points (Hotspots) you can insert additional function blocks for manipulating the data flow (Filter, replay and generator blocks, or CAPL program blocks with user-definable functions).

Make sure that you begin this tour with a new configuration by selecting the menu item *File|New configuration*.

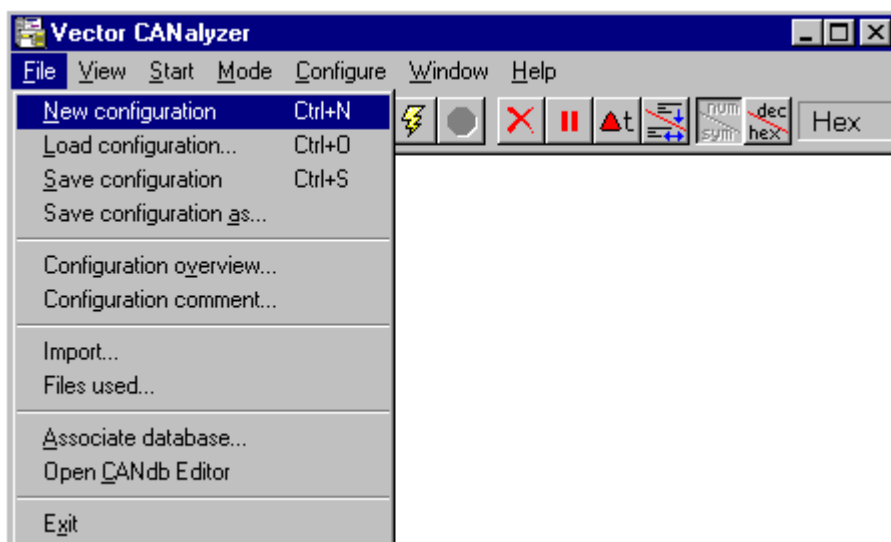


Figure 2: Menu Item *File|New configuration*

1.2.2 Setting Up the CAN Bus

To start up CANalyzers it is advisable to use a test setup with only two network nodes that is independent of existing CAN bus systems. The two CAN controllers on the supplied PC-card can serve as the network nodes.

First, connect the two D-Sub-9 connectors of your CAN card to one another. A connection cable with two bus termination resistors of 120Ω each for the high-speed bus interface is provided with the CANalyzer product. For a low-speed interface you will simply need a 3-conductor cable to interconnect the pins of the two controllers that are assigned to the bus lines CAN-High, CAN-Low and Ground.

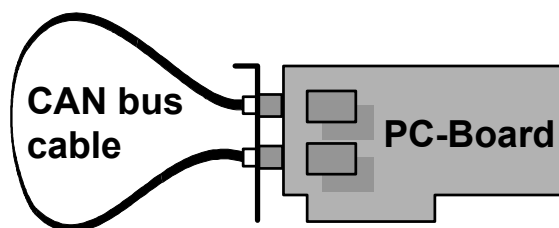


Figure 3: PC-Card with Connection Cable

Consequently, the CAN bus that you use during this tour will consist of a short 2-conductor or 3-conductor cable that connects the two CAN controllers of the CAN card to one another. This is necessary as a minimal configuration, since the CAN protocol requires - in addition to a sender - at least one receiver that confirms the correct receipt of messages with an acknowledge.

Up to this point we have not considered definitions of bus parameters (Transmission speed, sampling point, etc.) which must be set for each of the two participating controllers. To do this, from the *View* menu bring the measurement setup to the foreground and click the right mouse button on the PC-card icon at the left of this window.

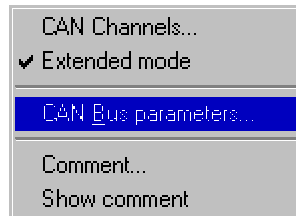


Figure 4: Popup Menu of the PC-Card Icon

In the popup menu you initially select bus parameters for the first controller CAN 1 and first set the baud rate in the configuration dialog. The *Baud rate selection* button takes you to the baud rate direct selection dialog, where you enter the value 100 kBaud.

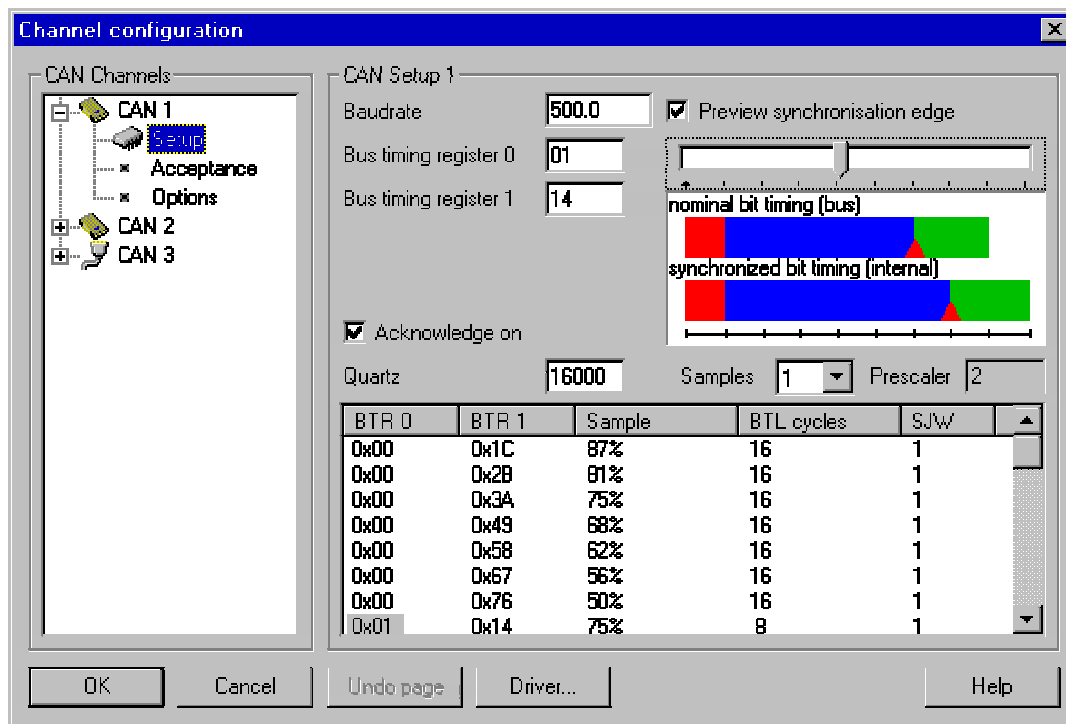


Figure 5: Configuration of Bus Parameters and Direct Baud Rate Selection

This makes sense for both high-speed and low-speed buses. The CANalyzer recommends default values for the controller registers. When you do this - besides the transmission speed of 100 kBaud - you also implicitly define other controller parameters (Sampling point, BTL cycles, and synchronization jump width). For the overall system to function properly, the same exact values must be assumed for the second controller CAN2. Accept the parameters with *OK*.

1.2.3 Transmitting Data

Since your current test setup still does not have a data source, your first task is to set up a data source in CANalyzer which places information on the bus cyclically.

Exercise 1: Configure CANalyzer so that - after the measurement start - a CAN message with identifier 64 (hex) is placed on the bus every 100 milliseconds. In this case the message should contain exactly four data bytes with the values D8 (hex), D6 (hex), 37 (hex) and 0.

You can solve this task by inserting a generator block in the CANalyzer transmit branch which generates the message to be transmitted. This is done by clicking with the right mouse button on the hotspot in the transmit branch directly above the block labeled *Transmit* and - from the hotspot's popup menu - inserting a generator block in the transmit branch.

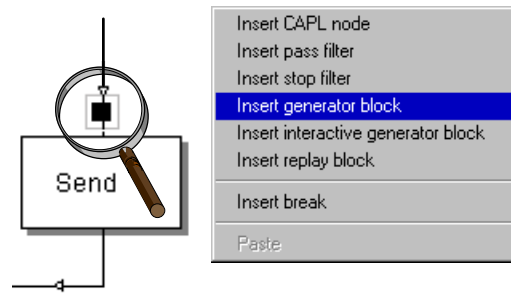


Figure 6: Hotspot in the Transmit Branch with Popup Menu

Afterwards, this appears as a rectangular block with the label G. You can then configure this block from its popup menu, which you access by pressing the right mouse button.

Afterwards, this appears as a rectangular block that is connected to the simulated bus (red line). You can then configure this block from its popup menu, which you access by pressing the right mouse button.

First, fill out the transmit list. You enter 64 as the identifier. (Check to see whether the numbering format is set to *Hex* using the *Options* button .) Then enter the value 4 in the DLC box as the data length entry. Finally, set the values of the data bytes in the four data boxes that follow by entering the values D8, D6, 37 and 0 there.

	ID	DLC	DATA				CAN			
▶ 1	64	4	d8	d6	37	0				1 ▼

Figure 7: Transmit List of Generator Block

Exit the transmit list with *OK* to accept the values in the configuration. In the generator block's popup menu, you must now still configure triggering for the transmit action. On the second line check the box *With Period* and then enter the value 100 in the input box to the right of this.



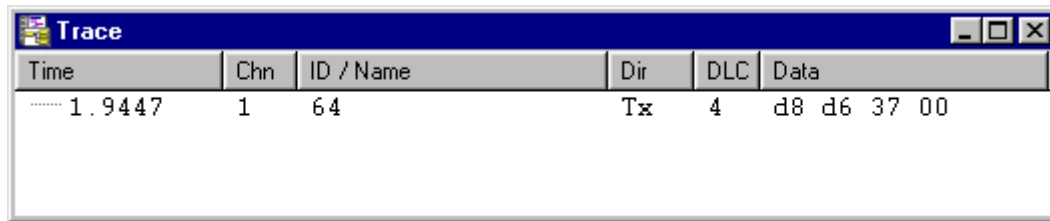
Figure 8: Triggering of Generator Block

These values are assumed into the configuration with *OK*.

Before you start the measurement you should save the configuration that you have prepared to this point with the menu command *File|Save configuration*. You can then reload this configuration at any time and resume your work precisely at this point.

Start the measurement by pressing the start button  on the toolbar. CANalyzer immediately begins to cyclically transmit the message you have configured in the generator block. You can recognize this in the Trace window, which automatically jumps to the foreground after the start of measurement and can now be seen at the lower right of the main program window: In the first line you see the message that is

sent by the generator block, whereby the first column shows the transmit time relative to the measurement start.



Time	Chn	ID / Name	Dir	DLC	Data
1.9447	1	64	Tx	4	d8 d6 37 00

Figure 9: Trace Window

The next column shows you which of the two CAN controllers was used to transmit. This value (CAN 1) agrees with the default value assigned in the generator block's transmit list of messages to be transmitted.

Afterwards, this message is also received by the second CAN controller over the bus. The question arises: Why is this not also displayed in the Trace window? You will find the answer in the configuration dialog for the acceptance filter for the second controller. In turn, you can open this dialog from the PC-card icon's popup menu under the entry *CAN Bus parameters/CAN2/Acceptance*.

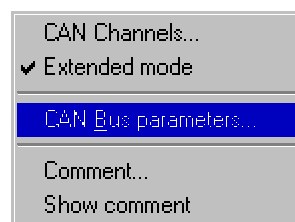


Figure 10: Popup Menu of the PC-Card Icon

The acceptance filter options support hardware-side filtering of messages. The default options block most message receiving. You can open the filter by entering the value X in the upper line. After a new measurement start you can now also see that the message transmitted via channel 1 (Transmit attribute Tx [= Transmit] in the Trace window) was received by the second controller (Receive attribute Rx [= Receive] in the Trace window).

We will now expand the task and additionally transmit a message with modified data:

Exercise 2: Expand the configuration of the last task such that, additionally, a message with identifier 3FC (hex) is transmitted every 200 milliseconds. The value of the first data byte of this message should cyclically assume values from 1 to 5.

You can solve this task by inserting another generator block in the transmit branch. For this task it does not matter whether you insert this generator block before or after the first one. Select *200 ms* as the value for cyclic triggering. The transmit list should appear as shown below:

	ID	DLC	DATA									CAN
▶ 1	3fc	1	1									1 ▼
2	3fc	1	2									1 ▼
3	3fc	1	3									1 ▼
4	3fc	1	4									1 ▼
5	3fc	1	5									1 ▼

Figure 11: Transmit List for Generator Block

Do not forget to stop the measurement before you reconfigure the measurement setup. During a running measurement it is not possible to make changes to the configuration of the data flow. The menu items of the relevant popup menus appear in gray shading.

Besides the generator block, CANalyzer also offers two additional block types as data sources. With a replay block you can play back data on the bus that were logged with CANalyzer's logging function. A program block allows you to integrate your own transmit functionalities - which may be quite complex - into CANalyzer with the CAPL programming language (cf. chapter 3).

1.2.4 Evaluation Windows

Evaluation windows are used to analyze data generated by the generator blocks in the transmit branch.

You have already learned about the Trace window. Data that reach the trace block of the measurement setup are displayed here as CAN messages in bus-oriented format. Besides the time stamp, this includes the number of the CAN controller, the identifier, an attribute for differentiating transmitted and received messages, and the data bytes of the CAN message. You can configure the Trace window - like all other analysis windows - from the popup menu that is accessed by clicking the right mouse button on the window or on the appropriate block.

Furthermore, the four buttons on the right of the toolbar can be used to configure the Trace window. For example, with  you can toggle from "stationary" mode to the scroll mode, in which each message arriving at the trace block is written to a new line.

With  you can toggle between absolute and relative time representation. In relative time representation, the time difference between two successive messages ("transmit interval") is shown in the first column. Of course, in this display format it is also easy to find the transmit interval that you entered previously in the generator block: 100 milliseconds.

The Statistics window also offers you bus-related information. Here you can observe the transmit frequencies for messages, coded by identifiers. If you have configured the transmit branch as in the two last tasks, then you should see two vertical lines in the Statistics window after the measurement start, which show the transmit frequencies of the two generated messages 64 (hex) and 3FC (hex).

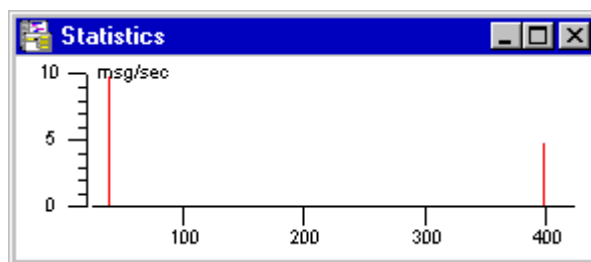


Figure 12: Statistics Window

10 messages per second were recorded for identifier 64, and half as many were recorded for identifier 3FC. This result corresponds to the cyclic periods of 100 and 200 milliseconds set in the generator blocks.

If the Graphics window display is too imprecise, the statistics block offers you a statistical report that gives you more precise information on the transmit interval for each message. Stop the measurement and activate the statistical report in the popup menu of the Statistics block (*Statistic report... Activate*).

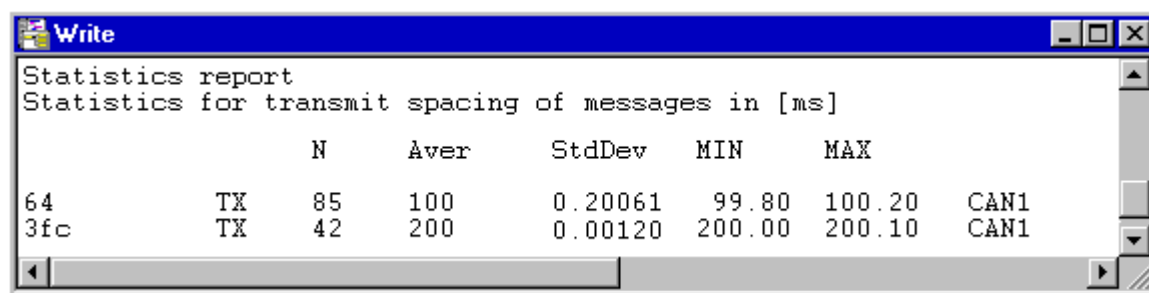
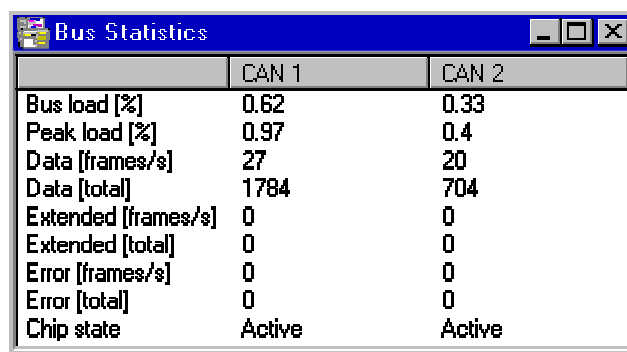


Figure 13: Statistics Report

If you now restart the measurement, statistical information is gathered in background, which you can output to the Write window after the next measurement stop using the popup menu command *Display statistics report*.

Besides showing the total number of messages for each identifier, the statistics report also shows the mean value, standard deviation, and minimum and maximum for the recorded transmit interval.

Another bus-related window, the Bus Statistics window, provides an overview of bus data traffic. Displayed here are the total frequencies of data, remote, error and overload frames, bus loading and CAN controller status. Since in our case one message is sent every 100 ms and the second message every 200ms, the total frequency of all messages is 15 frames per second. With an average data length of about 70 bits per frame, approx. $15 * 70 \approx 1000$ bits are placed on the bus in one second. At a baud rate of 100 kBit/sec the bus load in our example would be on the order of magnitude of one percent.



	CAN 1	CAN 2
Bus load [%]	0.62	0.33
Peak load [%]	0.97	0.4
Data [frames/s]	27	20
Data [total]	1784	704
Extended [frames/s]	0	0
Extended [total]	0	0
Error [frames/s]	0	0
Error [total]	0	0
Chip state	Active	Active

Figure 14: Bus Statistics Window

1.2.5 Working with Symbolic Data

Before we discuss the remaining windows in detail, let us have a look at the capabilities offered by CANalyzer for the symbolic description of data. Of primary interest in the analysis of CAN systems - besides bus-related information such as messages, error frames and message frequencies - is information on useful data, i.e. signals such as RPM, temperature and engine load, which are provided by individual controllers, and are sent on the bus with the help of CAN messages.

To describe this information symbolically, CANalyzer provides you with the database format DBC and a database editor with which you can read, create and modify CAN databases. Please refer to the CANdb++ manual and the CANdb++ online help included with the CANalyzer product for more information on the CANdb++ editor.

At this point we would like to associate a prescribed database to the active CANalyzer configuration. This database will be used to interpret the data bytes of the messages generated by the generator blocks in the transmit branch. The database MOTBUS.DBC is located in CANalyzer's demo directory DEMO_CL which was created parallel to the system directory EXEC32, provided that you installed the demo configurations. You associate this database to the active CANalyzer configuration by choosing the menu command *File|Associate database*. You can now open the database using the  button on the toolbar. The CANdb++ Editor is opened, and the contents of the database MOTBUS.DBC are shown in the Overall View window of the CANdb++ Editor.

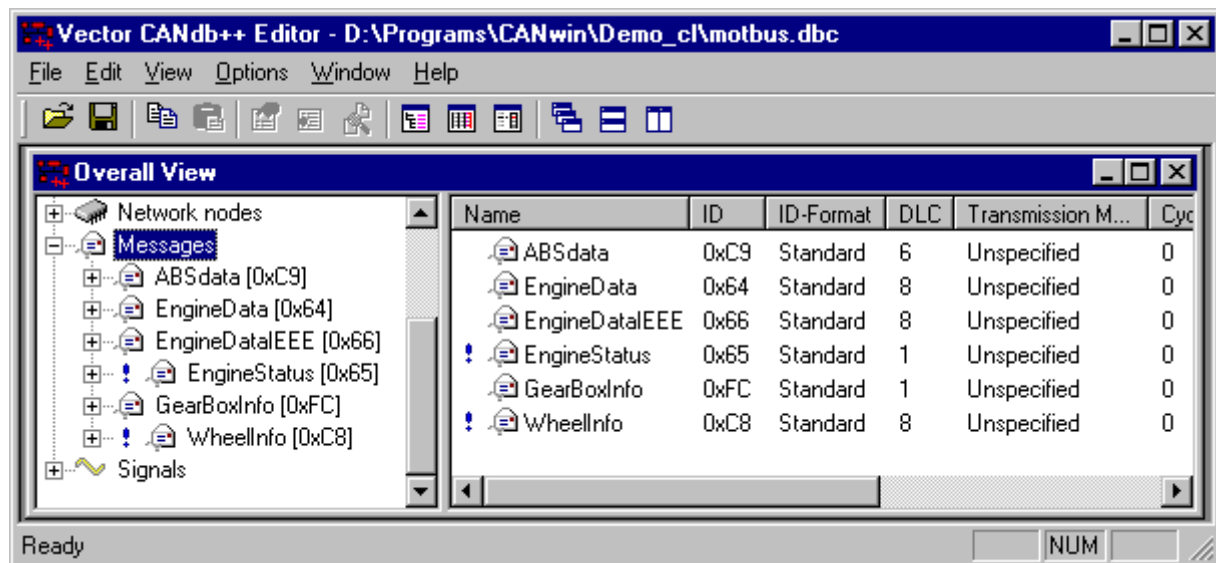


Figure 15: Overall View Window of the CANdb++ Editor

Double click the Messages object type in the area on the left side of the Overall View window. The subordinate structural level is then shown in this area. In the area on the right the available messages are shown with their system parameters (e.g. symbolic name, identifier, etc.). First, toggle the numbering format from decimal to hexadecimal in the *Options|Settings* menu item. We can deduce from the symbolic names of the messages that the system under consideration involves a description of communications in a rudimentary engine area system.

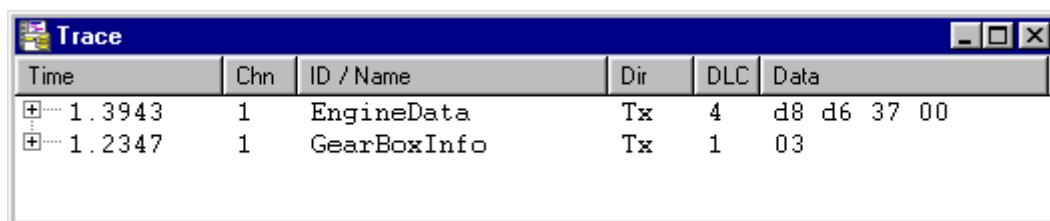
Click the message *EngineData* in the left area of the overall view window. The system parameters of signals transmitted in this message are shown in the area on the right side of the Overall View window.

The temperature *EngTemp* for example is a 7 bit signal. To obtain the physical value in degrees Celsius, the bit value must be multiplied by the factor 2, and the offset 50 must be subtracted from the result. The idle switch signal *Idle Running* in the last bit of the third data byte is a binary signal (one bit), which can assume the value 0 or 1.

With the help of this symbolic information the data contents of messages can now be interpreted in CANalyzer. Please note that this only makes sense if the database information describes the system that you are currently observing. Of course, you can also associate a different database to CANalyzer. The observed CAN data traffic is then interpreted according to the information in that database, even if it does not make any sense. You yourself are responsible for ensuring that the database associated to the configuration matches the real CAN network.

Messages that you generate in the two generator blocks can be interpreted with the database MOTBUS.DBC. Please note that the message you generated in the first task has the identifier 64 (hex). This agrees with the identifier of the message *EngineData* that we just examined in the database editor. If you now start the measurement, you can toggle the program to symbolic mode by activating the  button.

In the Trace window you will now see the symbolic message name in addition to the identifier.



Time	Chn	ID / Name	Dir	DLC	Data
1.3943	1	EngineData	Tx	4	d8 d6 37 00
1.2347	1	GearBoxInfo	Tx	1	03

Figure 16: Trace Window

1.2.6 Analysis of Signal Values in the Data Window

Besides the use of symbolic message names, the associated database can also be used to analyze signal values. The purpose of the Data window is to assist in the study of momentary signal values.

This explains why the Data window is initially empty in a new configuration. The signal values to be displayed are exclusively dependent upon information from the database. You as the user must decide which signal values should be displayed.

Exercise 3: Configure the Data window to display the signal values of the message *EngineData* (ID 64 hex) that is generated in the transmit branch.

To solve this task, first open the Data window's popup menu and then start the configuration dialog. Initially, the signal list in this dialog is still empty. With the *New Signal* button you start the Signal Explorer, which makes it possible for you to select a signal from the database. The object hierarchy on the left side of the dialog allows you to search for a specific signal. On the right side are the signals of the selected object.

To configure the Data window, first select *EngineData* from the list of all messages.

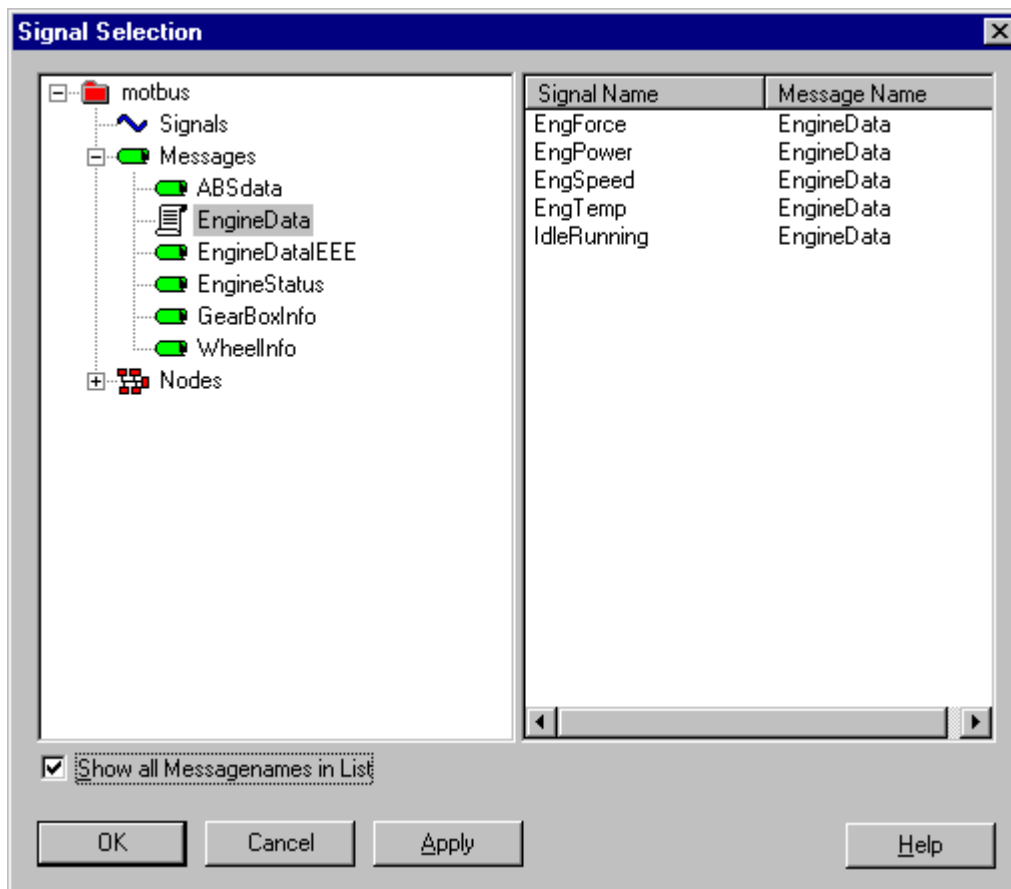


Figure 17: Selecting Signals with the Signal Explorer

Afterwards, select and accept all signals of this message from the dialog list on the right.

When you close the Data window's configuration dialog you will see that the signal names are now entered in the window. After the measurement start the generator block begins to cyclically send the message *EngineData* with data bytes D8, D6, 37 and 0 onto the bus. According to the message description in the database, the data block in the measurement setup now interprets these byte values as engine speed, temperature and idle switch and displays the appropriate signal values in the Data window in physical units.

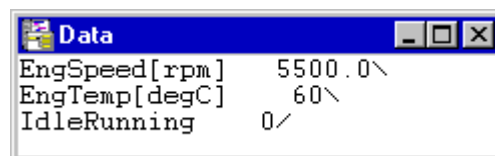


Figure 18: Data Window

With the help of the conversion formula in the database, engine speed is shown in RPM, while temperature is shown in degrees Celsius. The values of all three signals remain constant over time, since the message is constantly transmitted with the same data bytes D8, D6, 37 and 0.

1.2.7 Analysis of Signal Responses in the Graphics Window

While the Data window displays momentary signal values, you can have the time responses of signal values displayed in the Graphics window. After the end of measurement the signal responses are available for study by user-friendly analysis functions.

Exercise 4: Configure the Graphics window so that signal values are displayed for message 3FC (hex) that is generated in the transmit branch.

The second message generated in the transmit branch is also described in the associated database. In the database it will be apparent to you that the identifier 3FC is associated with the symbolic message name *GearBoxInfo* containing the signals *Gear*, *ShiftRequest* and *EcoMode*.

You can now observe the time responses of these signals in the Graphics window. The Graphics window can be configured exactly like the Data window. Here too you open the configuration dialog for signals from the window's popup menu. In the signal selection dialog you select the 3 signals of the message *GearBoxInfo*. In the Graphics window you see that the signals are now entered in the legend on the left side of the window. After the measurement start you observe that the signal *Gear* cyclically assumes values from 1 to 5, while the other two signals remain constant over time.

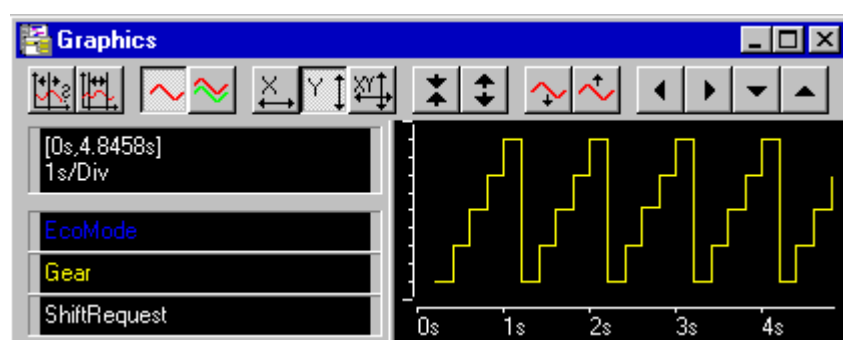


Figure 19: Graphics Window

This corresponds to the five values that you entered in the generator block as part of task 2. The values remain in the Graphics window after the end of the measurement. The measurement functions that the window provides for post-analysis are described in section 2.5.3.5.

1.2.8 Use of the Database in Transmitting Messages

Until now you have only used the symbolic database to observe signal values. However, the application capabilities reach well beyond this. For example, double click the generator block of task 1 to open the transmit list. Instead of the identifier that you previously entered in the transmit list, you will now recognize the associated symbolic name in the first column. In fact, you can now enter a message directly from the database using the *Symbol...* button, without having to work with the identifier.

Signal values can also be edited directly in the transmit list now. Select the first line of the transmit list and then activate the *Signal....* button. In the values dialog you can

now enter the signal values directly. It will also be apparent to you, once again, that the byte values D8, D6, 37 and 0 from the first line correspond to the signal values *EngSpeed* = 5500 rpm, *EngTemp* = 60 degrees Celsius and *IdleRunnning* = 0.

	name	enum	value
1	IdleRunning	Running	0
2	EngTemp		60
3	EngSpeed		5500

Figure 20: Values Dialog in the Generator Block

If you now set - for example - the value of *EngSpeed* to 1000 rpm, the generator block automatically uses the database information to compute the corresponding data bytes (10, 27, 37 and 0).

1.2.9 Analysis of an Engine Area Simulation

Included with the CANalyzer product are several sample configurations, which are provided to assist you in startup. In the directory DEMO_CL you will find a CAPL program block which simulates a portion of the data communication on the engine area bus of a motor vehicle. You have already added the database underlying the model, MOTBUS.DBC, to your configuration in Task 3.

Exercise 5: The CAPL program MOTBUS.CAN simulates the RPM, vehicle speed, and engine temperature for a motor vehicle. Study these signals in the Data and Graphics windows while you shift the vehicle's gears during the measurement run with the '+'- and '-' keys.

To solve this task, first delete both generator blocks from CANalyzer's transmit branch and insert - in the transmit branch in their place - the CAPL program MOTBUS.CAN from your demo directory DEMO_CL: To do this, choose *Insert CAPL program* from the popup menu of the hotspot in the transmit branch. Afterwards, in the configuration dialog for the inserted program block, you would press the *File...* button to assign the program file DEMO_CL\MOTBUS.CAN. Finally, you must still compile the program (Menu command: *Configuration|Compile all nodes*). The transmit branch is now prepared.

Now, in the Data and Graphics windows you configure the signals for engine speed (*EngSpeed*) and temperature (*EngTemp*) to the message *EngineData*, the vehicle speed signal (*CarSpeed*) to the message *ABSData* and the gear signal (*Gear*) to the message *GearBoxInfo*.

Once you have started the measurement, you can view the bus traffic directly in the Trace window. The messages *EngSpeed* and *ABSData* are transmitted cyclically, while the message *GearBoxInfo* is only transmitted spontaneously and is transmitted only once at the measurement start and with each gear shifting action, i.e. when the '+'- or '-' key is activated.

You can observe the signal values in the Data and Graphics windows. After the measurement start the temperature rises slowly to a maximum value, while the vehi-

cle speed and engine speed vary between two values. The ratio of the two signal values is determined by the selected gear.

You will find an introduction to CAPL programming and a detailed presentation of the programming language in chapter 3.

1.2.10 Logging a Measurement

CANalyzer has extensive logging functions for data logging. In the standard measurement setup the logging branch is shown at the very bottom of the screen. You can easily recognize it by the file icon  that symbolizes the log file. The log file is filled with CAN data during the measurement.

Exercise 6: Log - in ASCII format - all CAN data traffic that is generated in a short measurement (approx. 20 sec.) by the generator blocks in the transmit branch.

To log the data that arrive in CANalyzer's measurement setup to a file, first activate the logging branch. Also remove the break that separates the logging block of a new configuration from the data source. From the popup menu of the file icon located at the far right of the logging branch, open the configuration dialog. Here you can enter the file name for the measurement log as well as its format. Select ASCII format here.

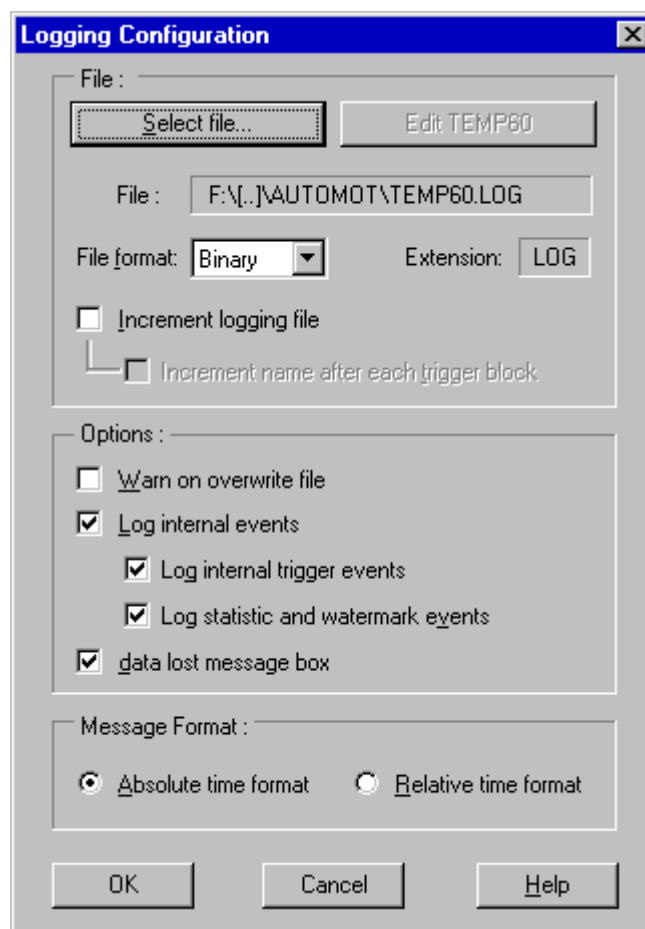


Figure 21: Configuration Dialog in the Logging Branch

Logs in binary format take up less space on your hard drive, but they cannot be read by normal text editors. The program's Offline mode offers you the same evaluation options for logs in both formats.

Besides the file icon, you can also specify trigger conditions for file logging in the logging block. This is often advisable, since frequently it is not the data traffic on the can bus over the entire measurement period that is of interest, but rather only certain time intervals, e.g. when there are implausible signal values or when error frames occur.

A description of how to define trigger conditions and time windows around these conditions is presented in section 2.7. To log the entire measurement it is sufficient to change the mode from *Single Trigger* to *Entire Measurement* in the trigger configuration dialog.

Exit the dialog with *OK* and then start the measurement, which you stop again after 20 seconds. Now with a double click on the log file icon you can open the logged ASCII file. Besides the logged messages you can see that statistical information was also logged. These lines correspond exactly to the information that is displayed in the Bus Statistics window during a measurement.

1.2.11 Evaluating a Log File

Log files in ASCII format can indeed be viewed with text editors, but often it is more sensible to utilize the capabilities that CANalyzer provides for offline analysis of log files.

Exercise 7: Play back the log file recorded for the last task in Offline mode, and observe the signal response in the Graphics window.

To solve this task, first switch CANalyzer to Offline mode. In the main *Mode* menu you will find two entries for this: *To Offline* and *To Offline (Copy)*. Since you can use the Graphics window configuration you prepared in Online mode here too, it is advisable to copy all configuration options of the analysis branch to Offline mode with *To Offline (Copy)*.

Now shown as the data source in the measurement setup - instead of the PC-card icon - is a file icon. Of course, the transmit branch is omitted here. Otherwise, all of measurement setup options of Online mode have been assumed. You can configure the data source by double clicking the file icon at the left of the measurement setup and entering the name of the log file selected in the last task.

You can now play back the measurement with the F9 key. In contrast to Online mode, here CANalyzer also offers you the option of replaying the measurement in slow motion (*Start|Animate* menu item or F8 key) or in Single-Step mode (*Start|Step* menu item or F7 key).

The same analysis functions are available to you in Offline mode as in Online mode. That is, the logged data are displayed in bus-related format in the Trace window, while you can observe the log's signal responses in the Graphics window.

Of course, you can also insert filters or CAPL programs in the measurement setup to further reduce the data or introduce additional user-defined analysis functions.

1.2.12 Tips for Solving Your Own Tasks

This small tour should make you aware of the fundamental control concepts and most important features of CANalyzer.

Remember that CANalyzer's measurement setup window represents the data flow plan for your actual measurement task. Besides associating a database, you can configure all other options directly in this window: From the data source on the left side, to the transmit branch, to the evaluation blocks on the right side of the window. You can always access the popup menus of all measurement setup objects by pressing the right mouse button.

All data arriving at an evaluation block are - with the exception of the logging block - displayed in the corresponding window. The evaluation windows can also be configured by pressing the right mouse button. You can save all configuration settings in a configuration file. Simply load such a prepared configuration file to prepare CANalyzer for another measurement task.

If you are using CAPL for the first time, perhaps to write your own analysis functions or to study the bus behavior of a controller, you will find a brief introduction to CAPL in section 3.2.

In the CAPL manual you will find detailed explanations of the program's transmit and analysis functions, which are only discussed briefly here, and explanations of CAPL programming. The context-sensitive Help function (F1 key) describes all menu items and explains the displays and controls of all dialogs.

1.3 Basic Operation

The basic operating procedures for using CANalyzer are explained in this section. If you are working with Windows for the first time, you should first become familiar with the basics of operating Windows applications. To do this, select the Windows tutorial program under the Help menu in the Windows Program Manager.

Essentially, CANalyzer can be operated by both mouse and keyboard. For example, you can select a main menu by clicking it with the left mouse button. Then you can click again on an item in the submenu which appears, and the associated action is executed.

As an alternative, the main menu can be activated by pressing the <Alt> key. You can now select an item with the cursor keys (<Arrow right>, <Arrow left>, <Arrow up> and <Arrow down> and execute the associated action by pressing the Enter key.

You can deactivate a selected menu entry again by pressing <ESC> or by clicking outside of the menu area with the mouse button.

All of the windows described above can be moved, enlarged, reduced, opened and closed again at any time, i.e. also during the measurement. To move the window simply drag (= press the left mouse key and hold it down while the mouse is moved) the title bar of the particular window to the new position. To change the window size, drag on the sides or corners of the window.

As an alternative you can also perform these actions with the keyboard after calling the system menu (pressing <Alt>-<SPACE> or <Alt>-<->). See the Windows manuals or Windows online Help for further details.

1.3.1 Menu Operation

CANalyzer is operated using the main menu. The individual menu commands are described in detail in online Help

Additionally, there are other context-sensitive menus in the evaluation windows described above and in the data flow plans in the simulation and measurement setup windows. These menus allow the user to specifically configure certain objects. These menus can be opened by clicking the active block in the active window or in the measurement setup window with the right mouse button. Using the keyboard this is done by pressing <F10>.

Most blocks in the measurement and simulation setups can be parameterized by selecting the first item in the context menu ("Configuration"). The block's configuration dialog is opened for this purpose. You can also start this dialog directly, without going through the context menu, by double clicking on the active block or pressing the Enter key.

1.3.2 Dialog Boxes

In addition to command inputs, which are usually made using menus, there are also parameter inputs. As a rule, parameters are entered in dialog boxes. A dialog box generally consists of six types of fields, each of which can occur more than once:

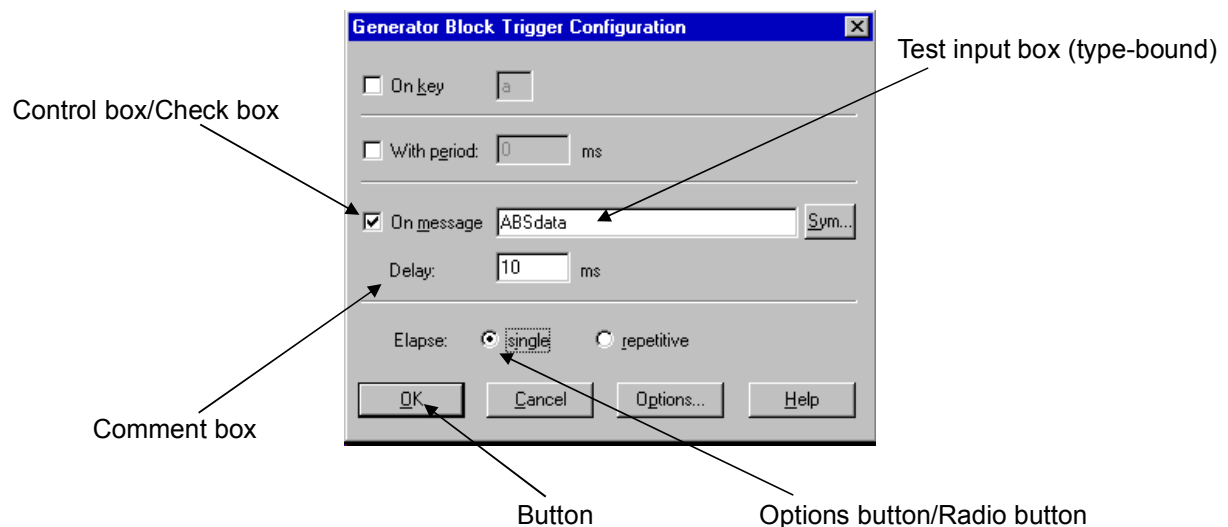


Figure 22: Box Types in Dialogs

Comment box	This tells the user what is to be input. The boxes behave passively when clicking on them with the mouse. They cannot be accessed by keyboard either.
Text input box (type-bound)	Alphanumeric boxfield, e.g. for entering file names. Numeric boxfield, e.g. for entering integer or floating point numbers.
Drop-down list	After clicking on the arrow along the right border of the box, a list drops down, from which you can select a value from a prescribed set of values.
Options button/Radio button	These buttons represent mutually exclusive options. You can only select one option at a time. If you select another option, the previous selection becomes inactive. The currently selected option button is identified by a black dot.
Control box/Check box	A check box next to an option indicates that this option can be activated or deactivated. In this case you can activate as many check boxes as desired. Activated check boxes are identified by an "x" or  .
Button	Buttons serve to execute certain actions, e.g. to exit the dialog box or to open a subordinate dialog box.

All dialogs have action buttons labeled *OK*, *Cancel* and *Help*. If you press *OK*, the settings you have made in the dialog are accepted into the configuration of the particular block. If you press *Cancel*, all settings made since the dialog box was last opened will be lost. With the *Help* button you can obtain a help text about the dialog box you are currently working with. After the Help window has been closed you can continue with the dialog. All settings are preserved.

Most CANalyzer dialogs also have an *Options* button. When this button is activated another dialog appears with which you can modify the CANalyzer global options (decimal/hexadecimal number representation, symbolic/numeric mode).

Note: Modification of the global options from a configuration dialog affects data representation in all system windows and dialogs.

Where there are multiple input and action boxes in a dialog box, first the desired box must be selected. Using the mouse this is done by clicking on the appropriate box. For input boxes this causes the text cursor to be placed at the mouse pointer position in the box. Check boxes change their state, and for action boxes the action is executed. With keyboard operation the particular box is selected with <Tab> or <Shift-Tab>. Check boxes can be then be toggled using the spacebar. The <Enter> key closes the dialog box and executes any actions selected in action boxes.

1.3.3 Control of the Measurement Setup

Measurements and evaluations are primarily configured in the measurement setup window which shows CANalyzer's data flow plan for the particular operating mode.

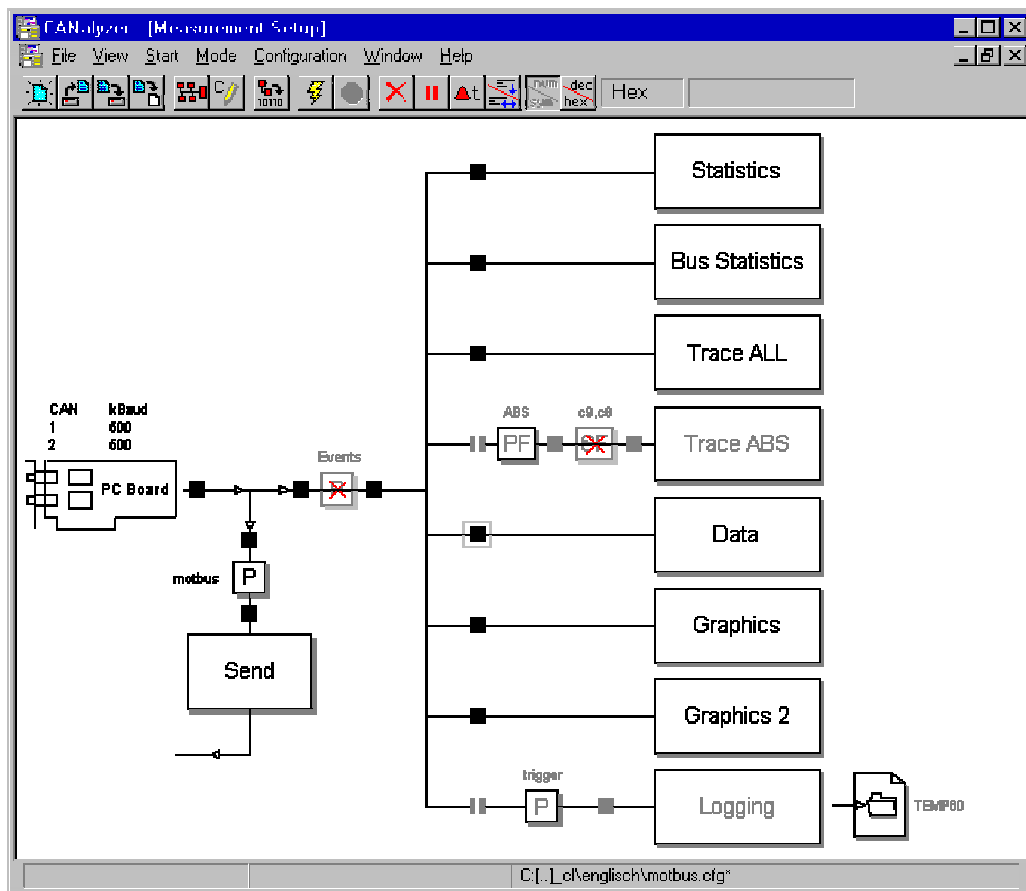


Figure 23: CANalyzer Measurement Setup

Mouse Operation

All blocks and some images in the active measurement setup window are mouse sensitive. When selected by clicking the left mouse button, the element preselected in this manner is identified by a frame as the *Active Element*. When the right mouse button is then clicked, a popup menu appears in which the object is configured by the methods described above. As an alternative, the configuration dialog for the active block can be called directly by double clicking with the left mouse button.

Keyboard Operation

When the measurement setup window is active, if <Tab>, <Shift-Tab> or one of the cursor keys is activated, the preselect frame around the currently active element is indexed forward. <Tab> results in forward indexing (<Shift-Tab>: Reverse indexing) of the internal processing sequence. The cursor keys index forward to the next closest element geometrically in the direction of the arrow. When <F10> is activated the popup menu of the active element appears. As an alternative, the Enter key can be used to call the configuration dialog of the active block directly.

The spacebar can be used to deactivate the preselected function block in the measurement setup; it can be reactivated by pressing the spacebar again.

With <Ctrl-F6> and <Ctrl-Shift-F6> you can bring any opened CANalyzer window to the foreground and activate it.

1.3.4 The Help System

Selecting the main menu item *Help* opens a Help contents window, which contains basic information and references to other Help pages. You can select references by clicking with the mouse or indexing through them with <TAB> and then pressing the Enter key.

The CAPL Browser and CANdb++ Editor each have their own Help system with another main *Help* menu. Activate this from the particular program.

Activating the <F1> key causes a Help topic to appear for the element that is active or preselected at the time the key is pressed. This context-sensitive Help function is provided for all dialogs, all program window panes and for all menu items, both in the main menu and in popup menus.

2 Working with CANalyzer

2.1 Preparations for the Measurement

2.1.1 Program Start

At the program start of CANalyzer the program CANW32.EXE is called by double clicking the appropriate icon in the CANalyzer program group. CANalyzer can only operate troublefree if the system directory contains all necessary files and the hardware has been installed properly (compare enclosed instructions on hardware installation).

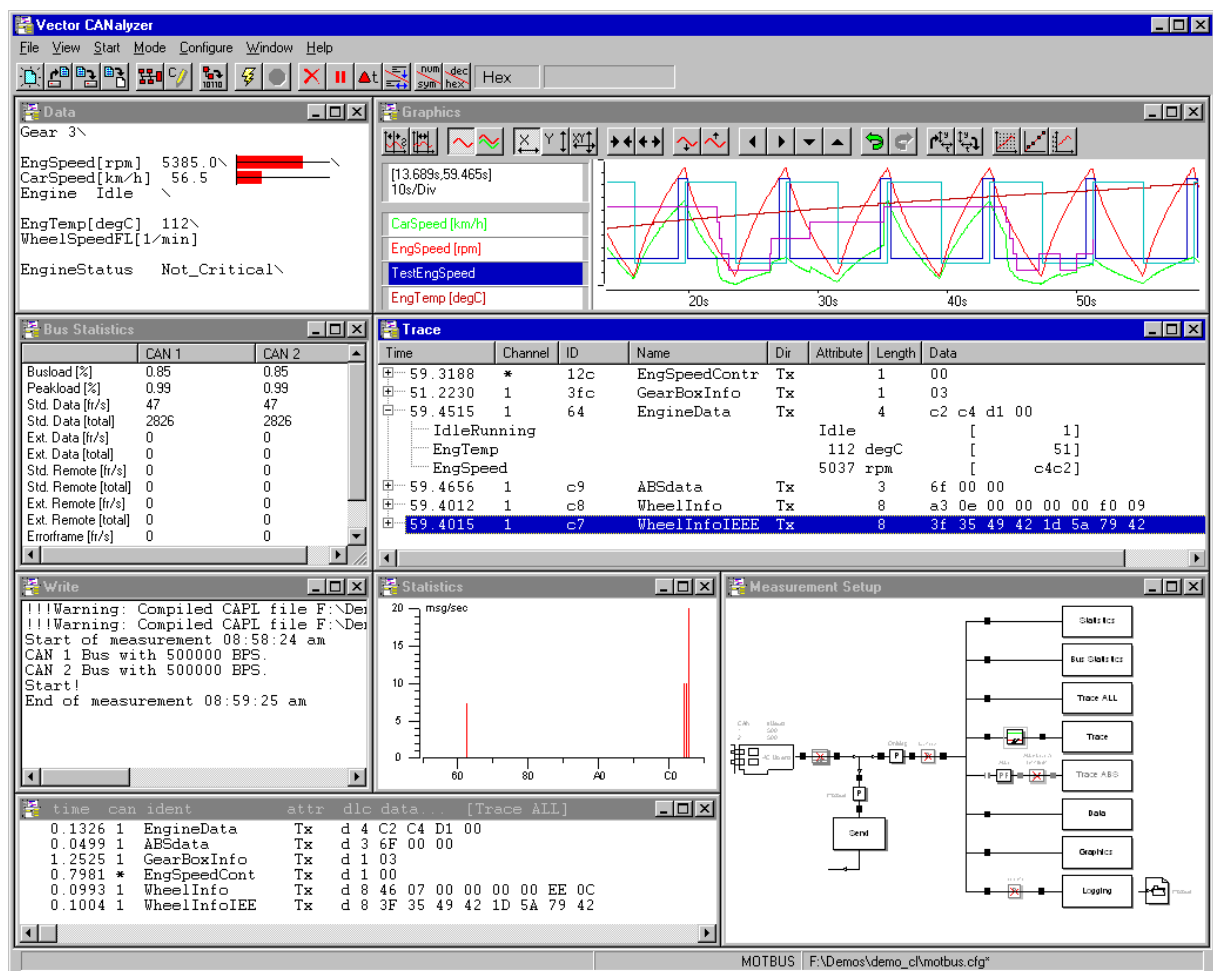


Figure 24: CANalyzer during a measurement run

At the program start CANalyzer first reads information on hardware settings, start paths, editors used, etc. from the project file CAN.INI. in the system directory. Afterwards, a configuration file *.CFG is read in. This file, which contains all information on the currently active configuration of CANalyzer, is updated automatically after a prompt at each program stop.

You can specify a working directory in the program icon. To have this file loaded automatically at the start you can also enter the name of a configuration file in the command line for program names. This method can be used to configure CANalyzer differently at the start by using multiple icons. If no entries were made in the command line, the last opened configuration is automatically loaded.

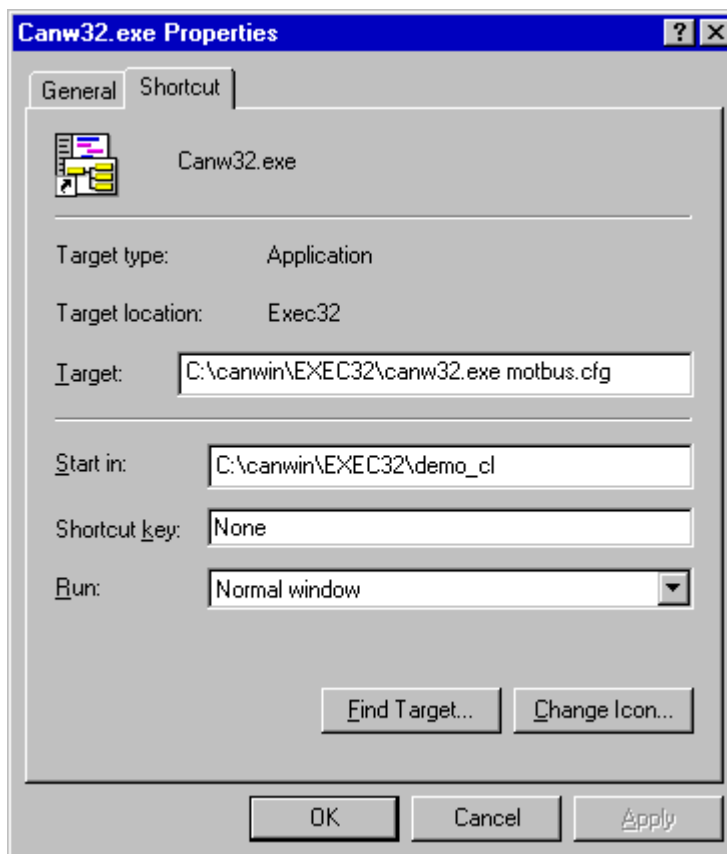


Figure 25: Automatic loading of the configuration MOTBUS .CFG at program start

2.1.2 The CANalyzer Screen

The CANalyzer screen consists of the main menu bar and the toolbar in the upper portion of the screen, the status bar at the bottom of the screen, and the data flow window and various measurement windows. You can gain access to all CANalyzer windows by double clicking the specific evaluation block in the measurement setup or by selecting the window from the *View* menu.

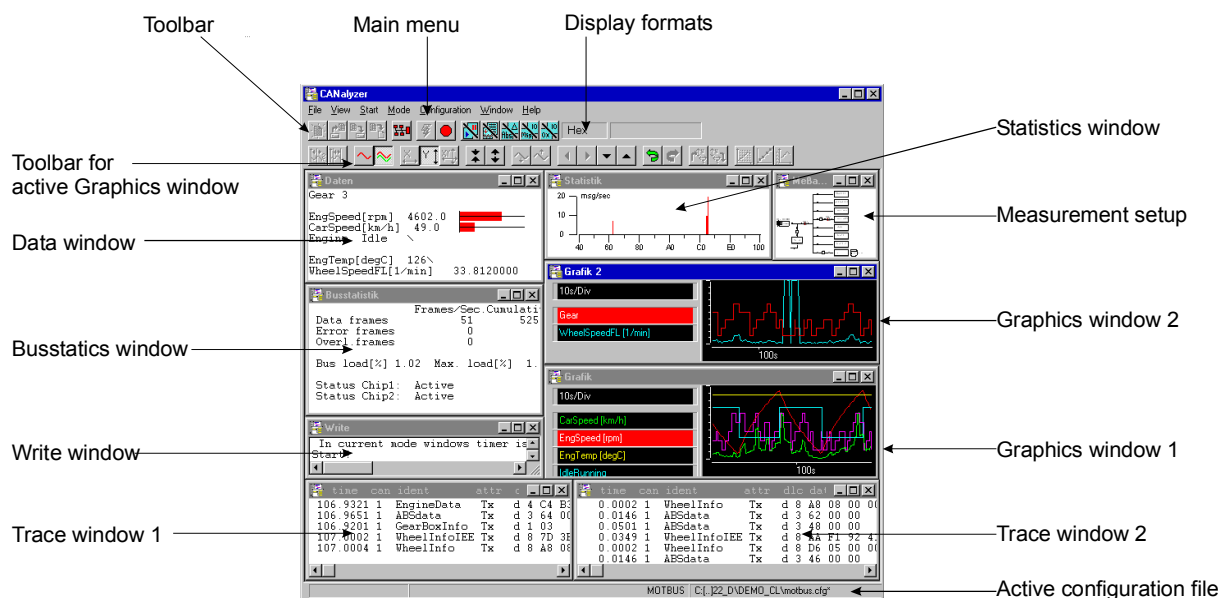


Figure 26: The CANalyzer screen

After selecting an entry the relevant window is activated and is displayed in the foreground.

Menu line	Used to select basic functions
Toolbar	Used for quick selection of important commands and also contains status indicators for the number system being used (decimal or hexadecimal) and to display of keyboard entries made during the ongoing measurement.
Measurement setup	The measurement setup displays the program's data flow. All options are set in this window for parameterizing a measurement or evaluation.
Trace window	Bus activities are recorded here. The user can scroll in this window after a measurement has been completed.
Statistics window	The mean transmit frequencies of messages are displayed as line spectra above the identifier axis in this window. As an option, the user can toggle over to mean transmit spacing. The window can be zoomed for detailed evaluations.
Data window	Preselected data segments of messages can be displayed here.
Graphics window	Graphic representation of signal time responses, which are displayed in a X-Y diagram above the time axis. After the end of measurement a measurement cursor and a difference cursor are provided, with which you can examine the coordinates of all measurement points or differences between two measurement points precisely.

Write window	Important information on the progress of the measurement can be output here (e.g. triggering of logging function). Furthermore, all outputs that the user places with the Write command in CAPL programs are written to this window.
Bus statistics window	Hardware-related information such as number of data and remote frames, error frames and bus load are displayed here. Availability of this information depends the CAN PC-card being used.
Status bar	The names of the active configuration file and the database being used are displayed here.

2.1.3 Data Flow in the Measurement Setup

The data flow diagram of the measurement setup contains data sources, basic function blocks, hotspots, inserted function blocks and data sinks. Connection lines and branches are drawn in between the individual elements to clarify the data flow.

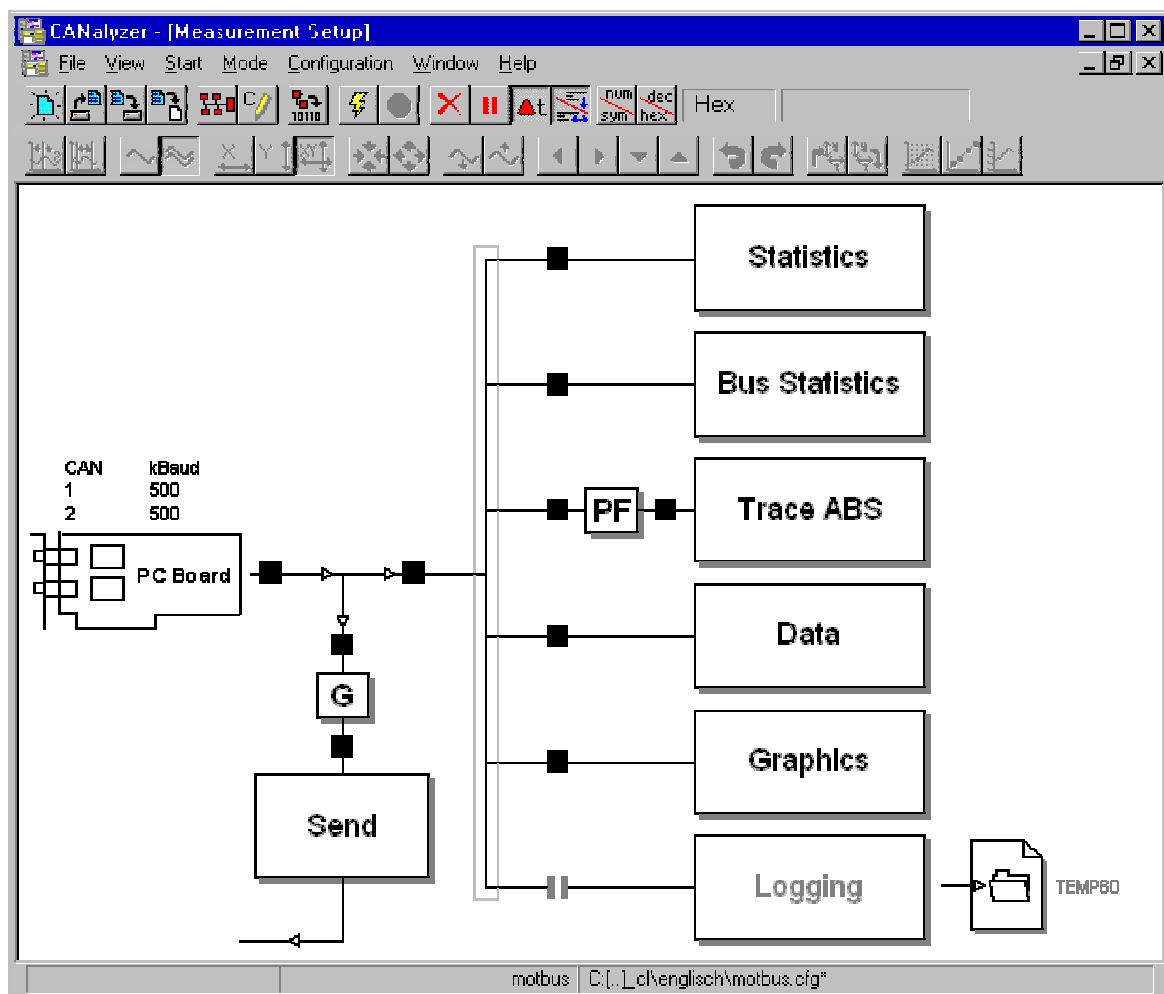


Figure 27: CANalyzer Measurement Setup

In Online mode the PC-card serves as the data source. It registers CAN messages on the bus and passes them on to CANalyzer.

Moreover, some of the supported PC-cards also provide additional information such as the detection of Error and Overload flags, the values of error counters, the bus load and external trigger signals.

2.1.4 Configuration of the Measurement Setup

Besides such functions as loading and saving configurations or associating CAN databases, which you call directly from items in the main menu, the data flow diagram and the function blocks in the measurement setup window are used primarily in the configuration of CANalyzers

You configure the measurement setup by activating a block in the data flow diagram (clicking it with the left mouse button or moving the selection frame with the <TAB> or arrow keys). Clicking on the selected hotspot with the right mouse button or pressing <F10> opens a popup menu with all configuration options.

For example, new function blocks such as filters or generator blocks can be inserted at the black rectangular insertion points (hotspots) in the data flow, and the PC-card's CAN controller can be configured from the PC-card icon at the far left in the measurement setup.

To copy and move blocks there are the functions *Copy* and *Cut* in each block's popup menu, as well as *Paste* in the popup menu for a hotspot.

If you wish to exclude a function block from the measurement, you can deactivate it before the measurement with the spacebar or with the *Node active* line in the popup menu. This is especially helpful if you have already configured a block and only want to disable it for certain measurements without deleting it. Deactivated blocks are shown as a different shape to differentiate them from active blocks. A node can be reactivated by pressing the spacebar again or by selecting the same popup menu line again.

A brief look at the measurement setup window gives an overview of the configuration options provided by CANalyzer and shows you how your actual measurement configuration appears ("Graphic menu").

The measurement setup can be displayed in two different modes:

- Automatically fitted to the window size.
- Fixed magnification with scroll bars if necessary.

You can select these modes from the measurement setup's popup menu. In fixed magnification mode the dimensions of the measurement setup are preserved. If the window is too small to show the entire measurement setup, you can scroll to the hidden area with the help of the scroll bars.

2.1.5 Working with Evaluation Blocks in the Measurement Setup

All evaluation blocks on the right side of the measurement setup are displayed above one another. The standard evaluation blocks, Statistics and Bus Statistics, always appear exactly once each. Other evaluation blocks (Trace, Data, Graphics and Logging) appear at least once each.

To insert new evaluation blocks in the measurement setup, click the branch with the right mouse button and select the new window from the popup menu. This places the new block after the last block of the same type. It gets the standard name with a serial number. The first Trace window is called Trace, the second gets the name Trace 2, etc.

As an alternative, you can insert evaluation blocks by opening the popup menu for one of the evaluation blocks and selecting a new block there.

You can also delete the block from the measurement setup via its popup menu, provided that there is more than one evaluation block of that basic type in the measurement setup. When the block is deleted, the entire branch is always deleted, including all of the insertable evaluation blocks there.

To open the window assigned to the evaluation block, double click the block with the left mouse button or choose *Show Window* in the block's popup menu. Multiple windows of the same type are shown cascaded in the standard layout.

2.1.6 Measurement Start

The measurement is started by pressing the F9 key, by choosing *Start|Start* in the main menu or by activating the start button  on the toolbar. In Online mode data can now be received and evaluated from the bus or can be sent out onto the bus. The data flow diagram shows you which analysis and transmit blocks are active during the particular measurement.

At the start of an Online measurement, first the CAN board is initialized. If this cannot be done, an error message is output and the measurement is terminated.

During the measurement the user can configure the Trace block, Data block and the scaling of the Statistics window and Data window. However, the menu items in the popup menus of the remaining blocks are masked out for the duration of a measurement. You cannot parameterize these blocks until after the measurement run has ended. All keyboard inputs during the measurement are passed through directly to the function blocks (CAPL programs, generator block, etc.). They are shown in the relevant status window on the toolbar. The only available program controls are the <Esc> key (terminate measurement) and all of the key combinations with the <Alt> key (Window control under Windows).

You can stop the measurement by pressing the <ESC> key, selecting the main menu item *Start|Stop* or activating the  button on the toolbar.

Internally, the measurement can be terminated by the elapse of the post-trigger time after triggering has occurred, or by the call of the **stop()** function from a CAPL program.

Note: During high system loads the stopping process may take a certain amount of time, since the system's message buffer must be emptied. A repeated stop command (double click) causes the buffered data to be ignored, and the measurement is terminated immediately, even under high system loading.

2.1.7 Working with Configurations

All options that you configure (configuration of the measurement windows, transmit branch, PC card, etc.) while working with CANalyzer can be saved to a configuration file with the extension CFG. Thus, you can work with different configurations to perform specific measurements and tests with CANalyzer.

To save changes of a specific configuration to a new configuration file choose the menu bar item *File|Save configuration as*. With the menu item *File|Load configuration* you can reload configuration files which you previously saved in CANalyzer. In the demo directory DEMO_CL you will find some prepared demo configurations that can serve as models when you start up CANalyzer and during the learning phase.

To obtain an overview of the files belonging to your project (configuration files, log files, CAPL programs, databases, , etc.) and to allow you to run them on another computer if necessary, it is advisable to create a separate project directory for each project (also called a working directory in Windows). Be sure to save all files resulting from your work in this directory. If you are working on several different CAN projects, multiple project directories are also advisable. With large projects it might even be easier to distribute the databases and configuration files of a project to different sub-directories.

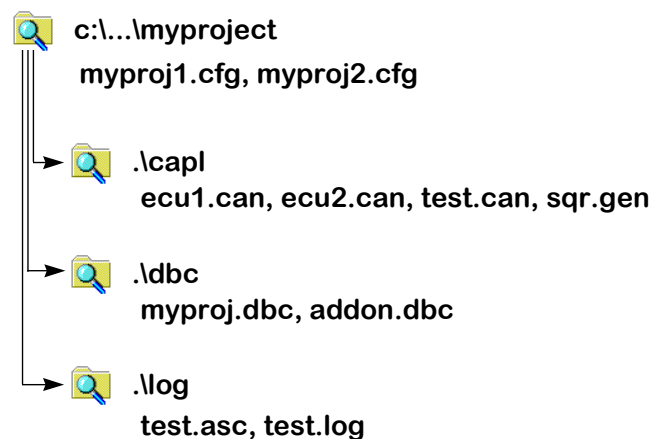


Figure 28: Example of a Directory Tree for a CANalyzer Project

References to other project files (e.g. to database files *.DBC or to CAPL programs *.CAN) are also saved in the configuration files. CANalyzer works internally with *relative paths*. That is, referenced files are searched and read-in relative to the configuration file during loading. Consequently, if you move your project directory with *all* associated files to another location in the directory tree or to another drive, the databases and CAPL programs referenced by the configuration file are found and correctly read-in at the next loading.

Note: To document or archive configurations, from the menu item *File|Files used* you can have a list generated of all files belonging to the active configuration (databases, CAPL programs, etc.) or have a ZIP archive generated.

The last configurations you worked with are saved in the [LastCANalyzerConfigurations] section of the CAN.INI file. The list of last opened configurations

in the *File* menu allows you to access these configurations. If you do not specify a start configuration on the command line, the last edited configuration of this list is used at the program start.

Saving Configuration in Older Formats

Older CANalyzer versions cannot read the current configuration file format. However, if you still want to work with older CANalyzer versions you can save the configurations in formats compatible with those versions.

Select the desired version in the *File Type* list. However, please note that the older the selected format, the more configuration information that will be lost. Of course, the active loaded configuration remains unaffected by this.

Please refer to online Help to learn the most important differences between the versions.

Importing Configuration Descriptions

As an alternative to loading configuration files (*.CFG), CANalyzer offers you the option, via the menu item *File|Import*, of importing configuration descriptions (*.CIF). For the most part, the format of configuration descriptions corresponds to the format of INI files.

Configuration descriptions allow you to specify configurations directly in a configuration description file, and to make adaptations there, e.g. to add more Trace windows or change the paths of CAPL program files.

In Help you will find an example of the structure of a configuration description.

2.1.8 Representation Formats

The main menu item *Configuration|Global Options* opens a dialog for entering the representation formats: Here you can select the numbering system (hex/dec) and decide whether you wish to have CAN messages displayed as identifiers or - provided that you have associated a database (cf. section 2.4) - whether you wish to have them displayed symbolically.

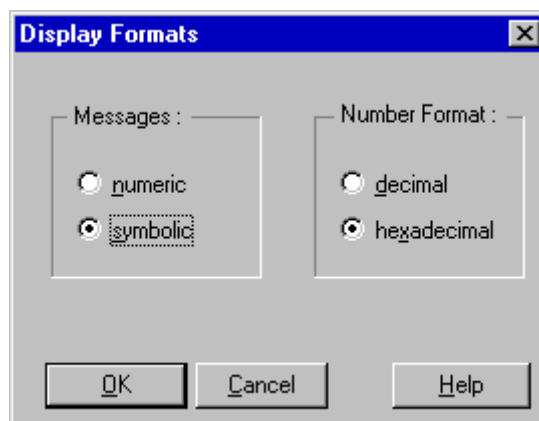


Figure 29: Dialog for configuring representation formats

These options affect the representation formats throughout the entire program.

Note: Please note that the numbering system in CAPL programs remains unaffected by these options. Identifiers with the prefix **0x** are interpreted as hexadecimal values, analogous to the C programming language. Otherwise the CAPL compiler always assumes that they are decimal numbers.

2.2 Interface to the Hardware

In Online mode¹ the data source of CANalyzer is the CAN PC-card. It provides messages received from the CAN bus with the time of reception. Furthermore, if desired it can also acknowledge transmit requests with the exact time of the request and provide transmitted messages with the exact time of transmission.

Depending on the hardware used, other events might be provided by the board. These may include, e.g. detection of Error or Overload frames, measurements of bus load or the arrival of external trigger signals.

CANalyzer supports a maximum of 32 (virtual and real) channels. Consequently, you can also use multiple CAN cards to transmit and receive messages. The number of CAN channels to be used is set in the channel definition (cf. section 4.4.1).

The hardware is initialized at the start of a measurement. You must communicate the parameters required for this to CANalyzer before the measurement start. Parameters are configured from the menu bar with *Configure|Hardware* or from the popup menu of the card icon on the right side of the data flow plan, which you can obtain with the right mouse button or by <F10> on the keyboard (cf. section 1.3.1). Parameterization is hardware-dependent. It is described below for CANcardX, which contains two SJA 1000 CAN controllers. For other cards (CAN-AC2, CANcard, etc.) the description is supplied with the card. Operation of the various dialogs and the meanings of input boxes are explained in online Help.

Each channel can be parameterized independently. Consequently, applications can be supported which have independent bus systems with differing speeds.

When creating a new measurement setup, default values are configured automatically for the particular controller type.

2.2.1 Configuring the Hardware

You can obtain the dialog for hardware configuration from *Configure|CAN bus parameters...* on the menu bar.

On the left side of the dialog is an overview of the channels defined in the active configuration. The number of channels can be set in the Channel Definition Dialog (cf. section 4.4.1).

When you click on a channel symbol with the left mouse button, you will see the chip allocation for this channel on the right side of the dialog. The chip allocation is the allocation of the application channels to the CAN chips registered in the system. This can be configured in the CAN hardware configuration section of your computer's Control Panel.

¹ The Offline mode for the analysis of log files is described in section 2.7.4.

After the sub-icons drop down in response to double clicking on the channel icon or clicking once on the [+] symbol, the chip icon appears beneath the channel symbol with the label *Setup* and two items with the labels *Acceptance filter* and *Options*.

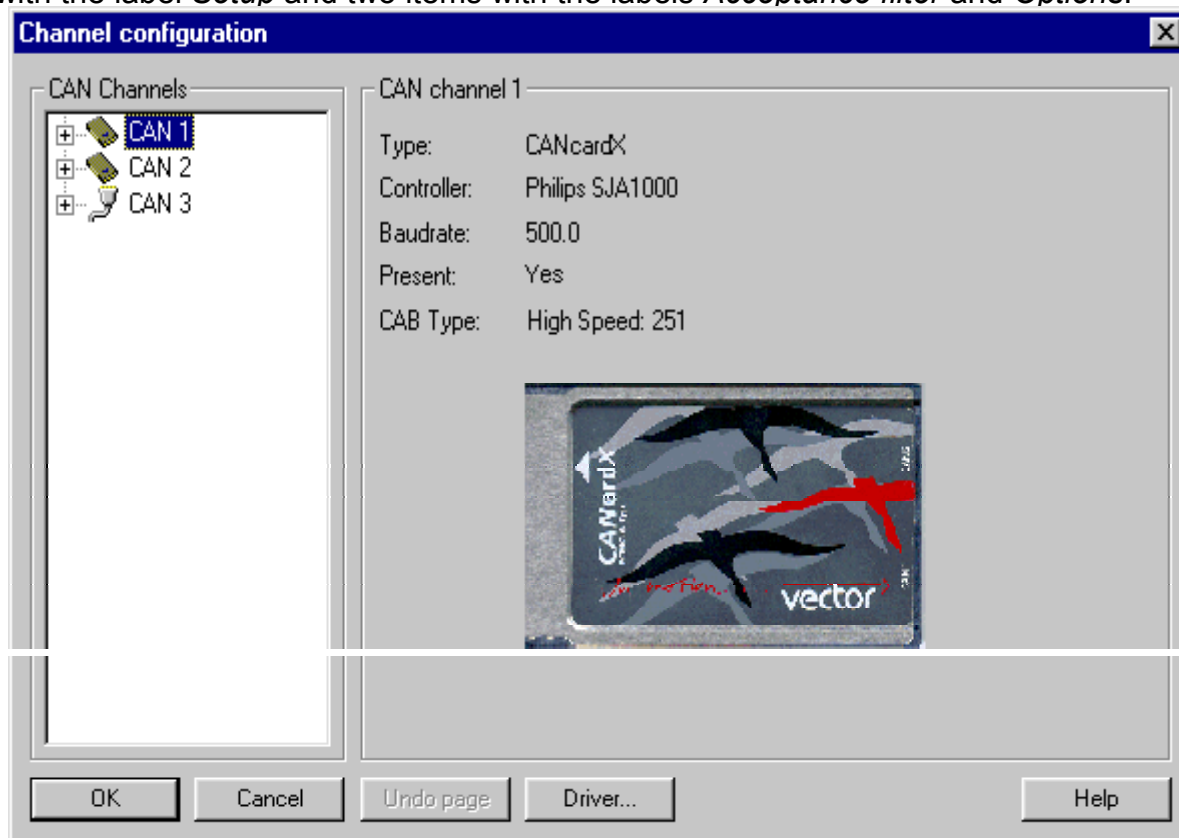


Figure 30: Configuration of Hardware

When you click the *Setup* chip icon the sub-dialog for bus parameterization appears on the right side, with which you can configure the chip's baudrate and bus registers (cf. section 2.2.2). When you click the *Acceptance Filter* item, on the right side you see the sub-dialog for configuring the chip's message acceptance filtering (cf. section 2.2.3). Finally, with the *Options* item you obtain a sub-dialog for configuring driver and card options (cf. section 2.2.4).

You can confirm all entries with the *OK* button or reject them with *Cancel*. If you only wish to undo the entries of one of the sub-dialogs mentioned above, press the *Undo Page* button in the particular sub-dialog. If a channel icon is selected *Undo* will cancel all changes (*Setup* / *Mask* / *Options*) made to this channel.

2.2.2 Programming the Bus Parameters

In the *Setup* sub-dialog you can directly configure the desired *Baudrate* for the CAN chip in the relevant input box.

As soon as you leave the input box (by <TAB> or mouse click in another option box) the associated register values are calculated and all relevant option boxes are updated automatically.

There may be additional input boxes - depending on the particular type of chip - for directly programming the CAN chip registers:

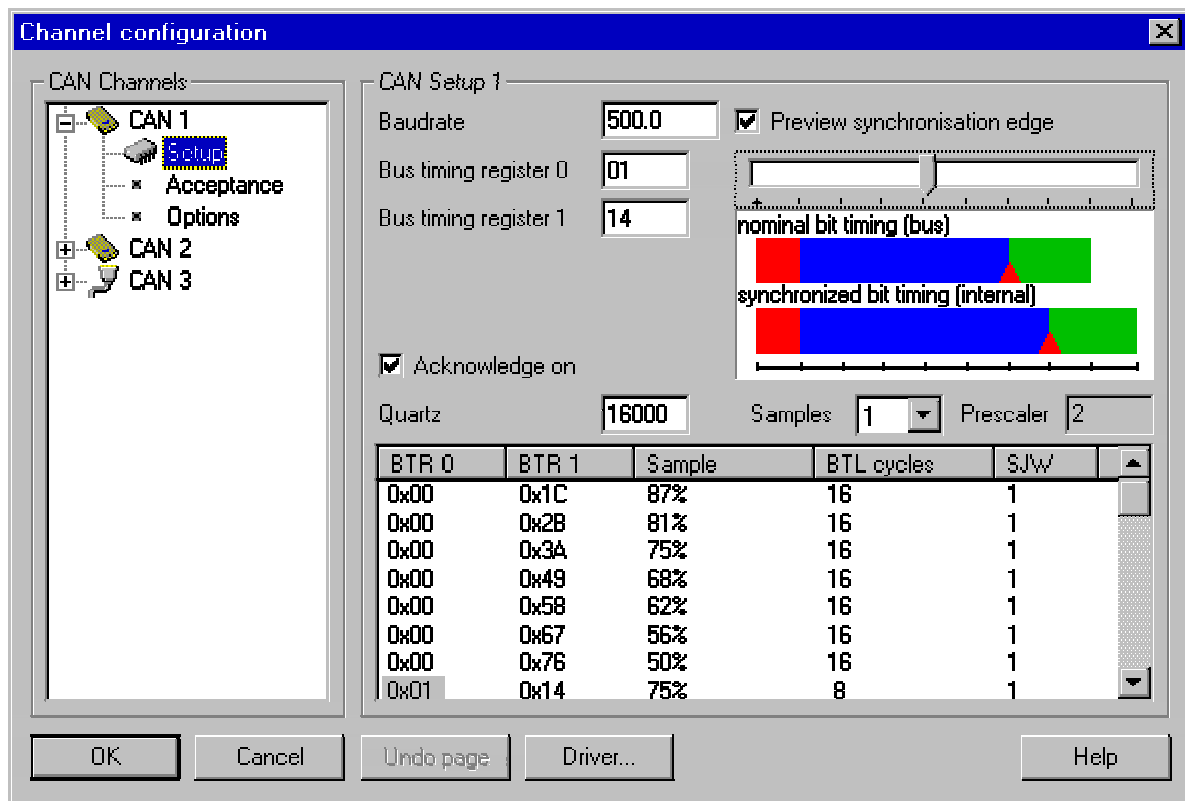


Figure 31: Bus Parameters Dialog

Above the list you see the input boxes for clock frequency and number of samples. Shown after the list is the prescaler resulting from the register values.

The CAN controllers are provided with 16 MHz clocks when delivered, and as a rule 16000 (kHz) should be entered in this box. However, it may be necessary to change out the clocks for special applications. Then the appropriate frequency must be entered here.

In the *Samples* input box you set the number of bus samples per bit time. Possible values are:

- *1 Sample*. This setting is recommended for High Speed Buses (SAE Class C).
- *3 Samples*. This setting is recommended for Low/Medium Speed Buses (Classes A and B) to filter out level peaks on the bus.

The two Bus Timing Registers define how an individual bit of the serial bit stream is assembled on the bus. Please refer to the data sheet for the CAN controller (SJA 1000/82C200/82527/72005) for the values that should be entered here. Input the values as hexadecimal numbers. On the right side of the dialog box you can determine whether the entered values are plausible (e.g. whether the desired baudrate has been achieved).

There are multiple Bit Timing Register pairs for a given baudrate which determine the timing of the CAN controller with regard to the sampling time point, number of BTL cycles and synchronization jump width (SJW). You can view a selection of allowable register pairs in the list of sampling options.

Displayed in this list are all Bus Timing Register values for the configured baudrate and sample count. Shown next to the register values are associated values such as the sampling time point (*Sample*) in percent of bit time after the beginning of the bit, number of BTL cycles (*BTL cycles*) and the synchronization jumpwidth (*SJW*).

Once you have changed the baudrate or sample count, click on the list box to update the list. Afterwards you can select the desired sampling option.

Representation of Bit Timing

The figure in the upper right area of the sub-dialog sketches the timing of the CAN controller resulting from the configured register values. It depicts the bit time schematically, subdivided into three regions *Sync* (orange), *Tseg1* (blue) and *Tseg2* (green).

The number of subdivisions corresponds to the value that is set for BTL cycles. The sampling time point (border between *Tseg1* and *Tseg2*) is identified by one or three small red triangle(s) according to the setting for sample count. The ratio between the bit length up to the sampling point and the overall length of the represented bit time corresponds to the percent value of the actively selected list entry.

Above the figure you see the *Preview Synchronization Edge* selection box which is deactivated by default, whereby the slider beneath it is also deactivated. Please note that this slider does not have any functionality as a control; it is only used for previewing purposes.

Click on the *Preview Synchronization Edge* selection box to activate previewing. With the help of the slider above the figure you can examine the effects of the synchronization edge and synchronization jump width on CAN controller timing. The slider is intended to represent the time point of a synchronization edge on the bus, i.e. the beginning of a bit with recessive-dominant edge. The upper area of the figure shows the nominal bit timing, i.e. a bit on the bus is depicted as it is expected by the controller. The lower area of the figure shows the internal controller timing, i.e. the bit time interval from the controller's perspective. The length of this bit interval depends on the time point of the arriving synchronization edge:

If the edge falls within the *Sync* region of nominal timing, then the chip is running synchronously. If the edge falls within the *Tseg1* region of nominal timing, then resynchronization must be performed. In this case the *Tseg1* region is lengthened by up to *SJW* (Synchronization Jump Width) BTL cycles. If the edge falls within the *Tseg2* region of nominal timing, then resynchronization must be performed. In this case the *Tseg2* region is shortened by up to *SJW* (Synchronization Jump Width) BTL cycles. If no edge falls within nominal timing, this bit time is not utilized for resynchronization.

Disconnecting the Transmit Branch

Since the CAN controller on the PC card represents the interface between the analysis software and the CAN bus, the bus is influenced by the measurement process. In particular, the CAN controller gives its acknowledge for correctly recognized messages by placing a dominant level on the bus at the relevant slot in the CAN message. To reduce the influence on the system, this functionality can be explicitly deac-

tivated on the SJA 1000 controller. However, please note that when the acknowledge is deactivated communication can only occur over the bus if at least one other bus node sends an acknowledge.

Note: On the CAN-AC2 card (with 82C200 or i82527 controller) the acknowledge can only be deactivated by moving the appropriate jumper.

2.2.3 Acceptance Filtering

With all Basic-CAN controllers on the PC card (SJA 1000/82C200/82527/72005) a mask controls which messages can be transmitted and which can be received.

For example, the SJA 1000 has one acceptance filter for standard identifiers and one for extended identifiers, and it expects separate acceptance mask and acceptance code. The acceptance mask indicates which bit of the ID should be compared with the acceptance code. If the bit is 1 in the mask, then that particular bit is irrelevant for the comparison. If it is 0, that bit of the ID is compared with the corresponding bit of the acceptance code. If these two bits are identical, the message is received; otherwise it is filtered out. Both the mask and code are entered as hexadecimal numbers.

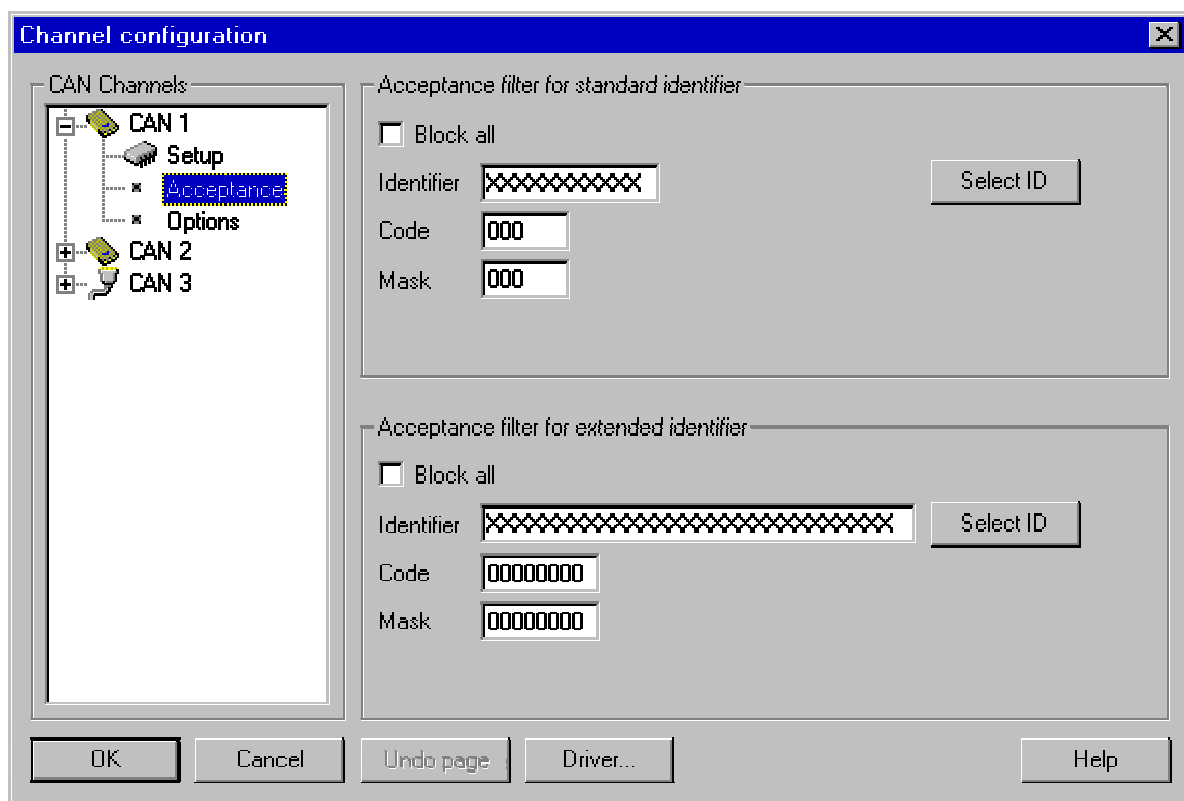


Figure 32: Acceptance Filtering with the SJA1000 Controller

Instead of entering the mask and code directly, you may program acceptance filtering in CANalyzer using a logical mask (*Acceptance filter for standard indentifiers* or *Acceptance filter for extended identifiers*). You can enter one of the values 0, 1 or X for each bit in this mask. A message occuring on the bus is only be received if all mask bits given as 0 or 1 agree with the corresponding message bits. Bits shown as X are not utilized in the comparison ("don't care"). The values of the acceptance

mask and acceptance code are automatically calculated and displayed after inputting the logical mask.

You will find a detailed explanation of acceptance filtering for all supported CAN controllers, with examples, in online Help.

2.2.4 Card and Driver Options

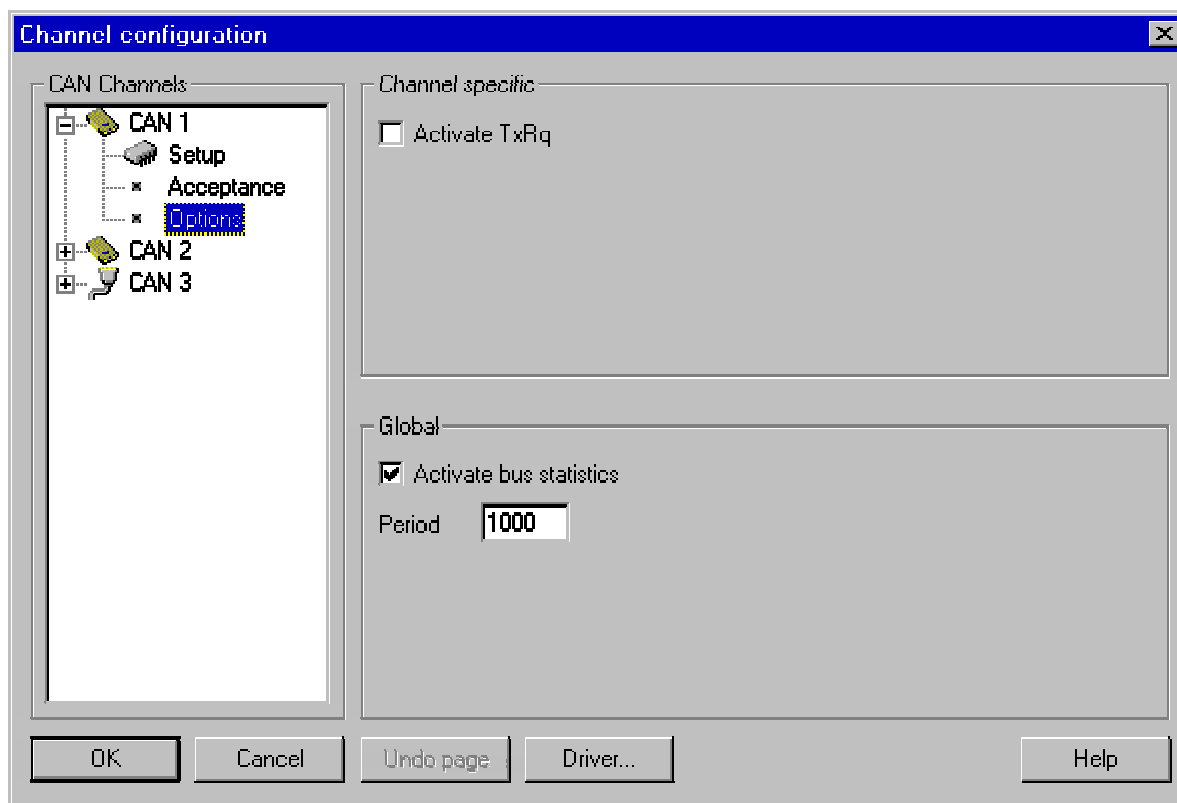


Figure 33: Card and Driver Options Dialog

Depending on the CAN PC card used, various options may be configured in this dialog. These include:

- Reporting the time point of a transmit request (TxRq) to measure delay times (cf. section 2.3.1).
- Choosing whether bus statistics should be displayed, as well as their
- Display refresh rate (cf. section 2.5.4.3).

The context-sensitive Help function offers more detailed descriptions for your particular hardware platform.

Note: The actual display of statistical data can be deactivated separately by disconnecting the Bus Statistics window from the data flow. This will enhance performance. In contrast to a complete disabling of bus statistics support, data can still be logged and evaluated later in Offline mode.

2.3 Transmit and Receive Data

2.3.1 The Transmit Branch

On the left side of the measurement setup, CANalyzer's transmit branch branches off after the card icon. Here you can insert function blocks to feed data onto the bus. The data flow in the transmit branch always runs from top to bottom. The small arrows in the measurement setup indicate the direction of data flow.

The transmit branch is only accessible in Online mode, and there is has the task of forwarding messages that arrive at its input to the card driver as transmit tasks. The transmit block itself cannot be parameterized. When an attempt is made to select the block by mouse click or by keyboard, neither a menu nor a dialog box appears.

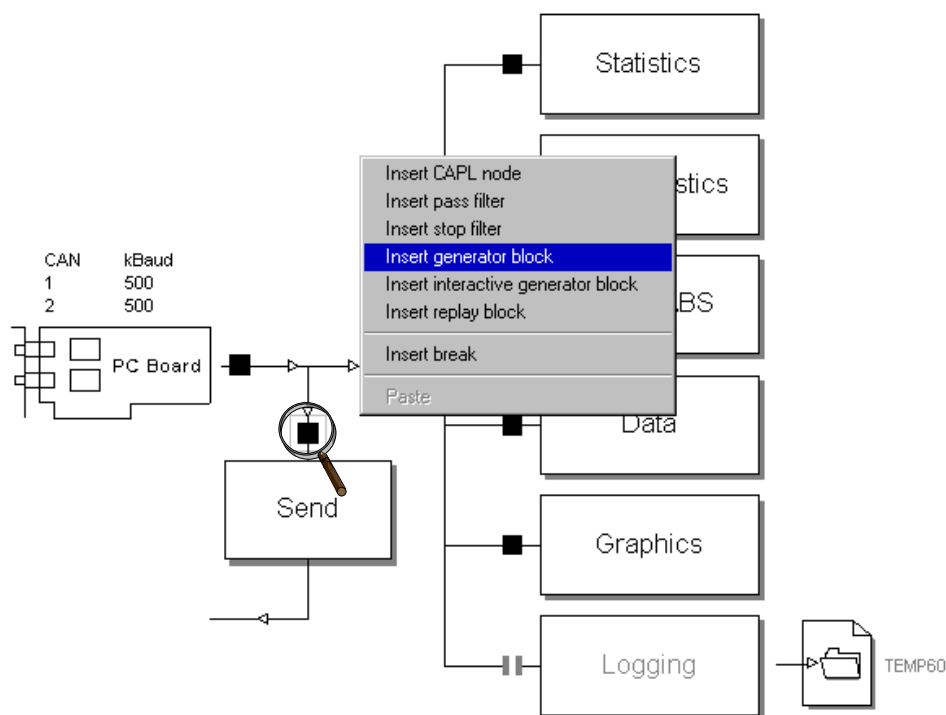


Figure 34: Inserting a generator block in the transmit branch

For typical applications, a program block, generator block or replay block is inserted at the hotspot before the transmit block. You can use this to specify what should be transmitted.

Note: When transmitting from CAPL programs, generator blocks and replay blocks, you can explicitly specify which of the two CAN controllers (bus connections) should be used to transmit the message. This is observed accordingly by the card driver.

2.3.2 The Evaluation Branches

In the evaluation branches of the measurement setup, data are passed from left to right to the measurement setup's evaluation blocks, where they can be visualized and analyzed with various functions.

Filters or user-defined analysis programs can be inserted in the data flow diagram before the evaluation blocks. As a result, the data flow can be configured in many ways to suit the particular task.

Each evaluation block has a measurement window in which the data arriving in the block are displayed. The functions of all measurement windows are described in detail in the sections below. Only the logging block is not assigned its own window. Instead, a log file is assigned to it for the purpose of logging bus data traffic and then studying it "offline" (cf. section 2.7.1).

Located between the function blocks are insertion points (hotspots), at which blocks can be inserted for to manipulate the data flow (Filter, replay, generator block, CAPL program block with user-definable functions). Before and after the block inserted in this manner, new hotspots appear, so that additional blocks can be inserted. The data flow can also be broken at the hotspots. You will find a description of all insertable function blocks in section 2.6.

Figure 27 shows a possible CANalyzer configuration in Online mode², whereby a generator block is provided in the transmit branch (e.g. for cyclic transmission of one or more messages). A filter is inserted in the trace branch, so that only certain messages will be displayed in the Trace window. The signal branch with data and graphics windows, the statistics branch and the bus statistics branch each receive all data, while the logging branch is broken.

Note: The data flow in the measurement setup is always directional. It runs from the left, starting with the card icon, to the evaluation windows on the right, and in the transmit branch from top to bottom to the transmit block, and from there to the card. Messages generated in the transmit branch (e.g. by an inserted generator block) arrive first at the CAN controller on the PC-card; from there - once transmission has been completed successfully - they are fed back in on the left side of the measurement setup at the card icon.

2.3.3 Message Attributes

Messages that were not transmitted by CANalyzer's CAN PC-card (Receive messages), get the attribute Rx and a time stamp from the card's clock when they are received. Afterwards they are passed to CANalyzer via the card driver and, finally, they are shown in the evaluation windows. The time stamp and Rx message attribute can be seen in the Trace window.

² Data flow and functions in Online and Offline modes only differ in the data source and in the transmit block. Refer to section 2.7.4 for a description of Offline mode.

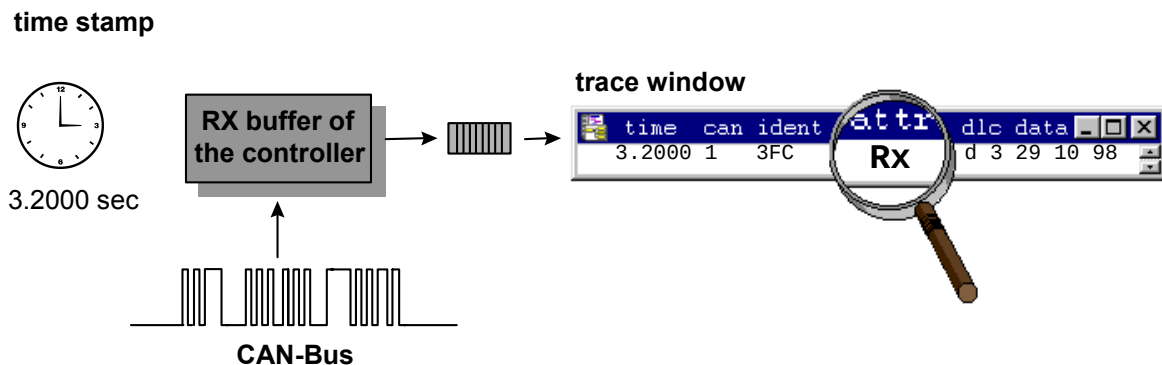


Figure 35: Receiving messages

The messages to be transmitted are passed from the transmit block via the card driver to the *CAN PC-card*. If your hardware supports the card and driver option *Activate TxRq* in the *Options* item of the PC-card icon's popup menu, and you have activated this, the driver returns the time of the transmit request assigned to the CAN microcontroller to you. In the Trace window, for example, you would see the message to be transmitted with the attribute *TxRq*.

After successful transmission the message is returned with the actual time of transmission and the attribute *Tx*, so that the transmit messages can be displayed and/or logged in the Trace window. If these messages reach the transmit block directly again, they are not retransmitted. This prevents unintentionally forming infinite loops which might severely load the bus under certain conditions. If an infinite loop is to be programmed intentionally (e.g. as a base load) this can be done by recopying the message in a program block.

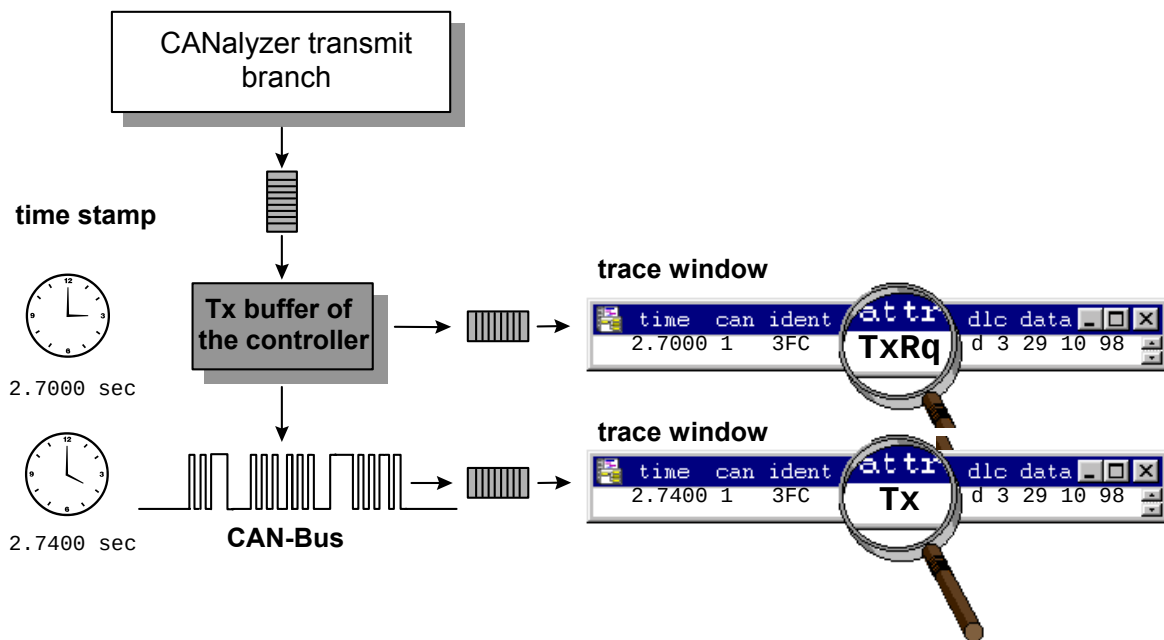


Figure 36: Transmission of messages

The *TxRq* display permits measurements of the difference between the time of transmit request and time of transmission. The time between the message with Tx attribute and TxRq attribute is essentially the transmission time of a message, i.e. the time that the CAN controller needs to place a message completely on the bus. It is a function of baud rate and message length. The transmission time also grows as a result of lost arbitration events, which can be observed more for low-priority messages at high bus loads.

Since the (very small) latency time of the card driver interrupt must be added to the transmission time, the following general formula applies:

$$t_{Tx} - t_{TxRq} = \text{Transmission time} + \text{Latency time}$$

Note: Under high load conditions the display of messages might be delayed in the evaluation windows under some circumstances. However, the time stamps for the messages remain unaffected by this, since they are already assigned to the messages when they are received on the card.

2.4 Use of Databases

When performing large-scale studies on the CAN bus, it is a great help to the user if - in addition to the bus-oriented raw data format with numerical identifiers and data contents - a symbolic interpretation of the message event is also provided.

CANalyzer supports the use of symbolic databases. You can make this information available by associating one or more databases to the active configuration (Menu item *File|Associate database*). Afterwards you can access the information in measurement windows, insertable function blocks and CAPL programs.

2.4.1 Creating and Modifying Databases

The database management program CANdb++ is available to you for inputting and modifying databases. It is included with the standard CANalyzer product.

In a database, names are assigned to CAN messages. In CANalyzer you can then address the messages using these names. For example, the clear text *EngineData* is shown in the Trace window instead of the identifier *100*.

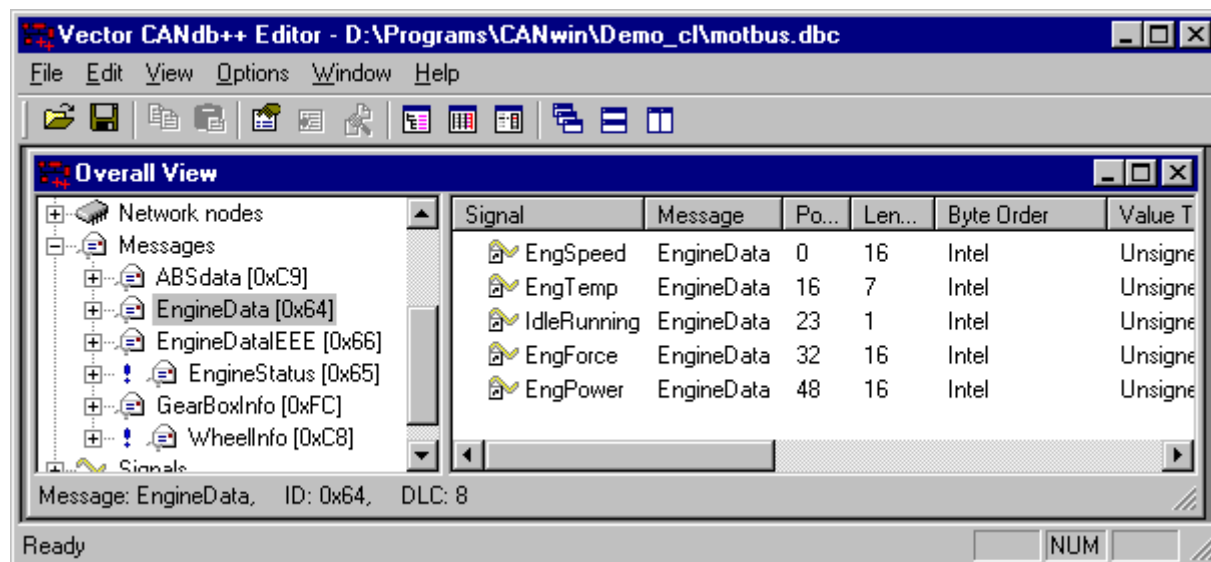


Figure 37: Symbolic description of the message *EngineData* in CANdb++

Moreover, so-called signals are defined in the database. A signal is a symbolic description of a data segment within a message. Besides identifying the data segment, the signal definition also incorporates characteristics such as machine format (Motorola/Intel), sign handling, a conversion formula and physical unit of measurement. This permits direct display of physical dimensions in the data window, such as:

"Speed = 810.4 rpm".

Please refer to CANdb++ Editor documentation or CANdb++ Editor online Help for further details on creating and modifying a database. Important aspects of working with databases are explained in section 4.3.

2.4.2 Access to Database Information

Besides the individual text input boxes, in general there are also small buttons for the purpose of entering symbolic message or signal names in function blocks. When you press one of these buttons you start the Message Explorer with all symbols defined in the database. You can choose one or (in the case of the filter configuration dialog) multiple names from the Message Explorer.

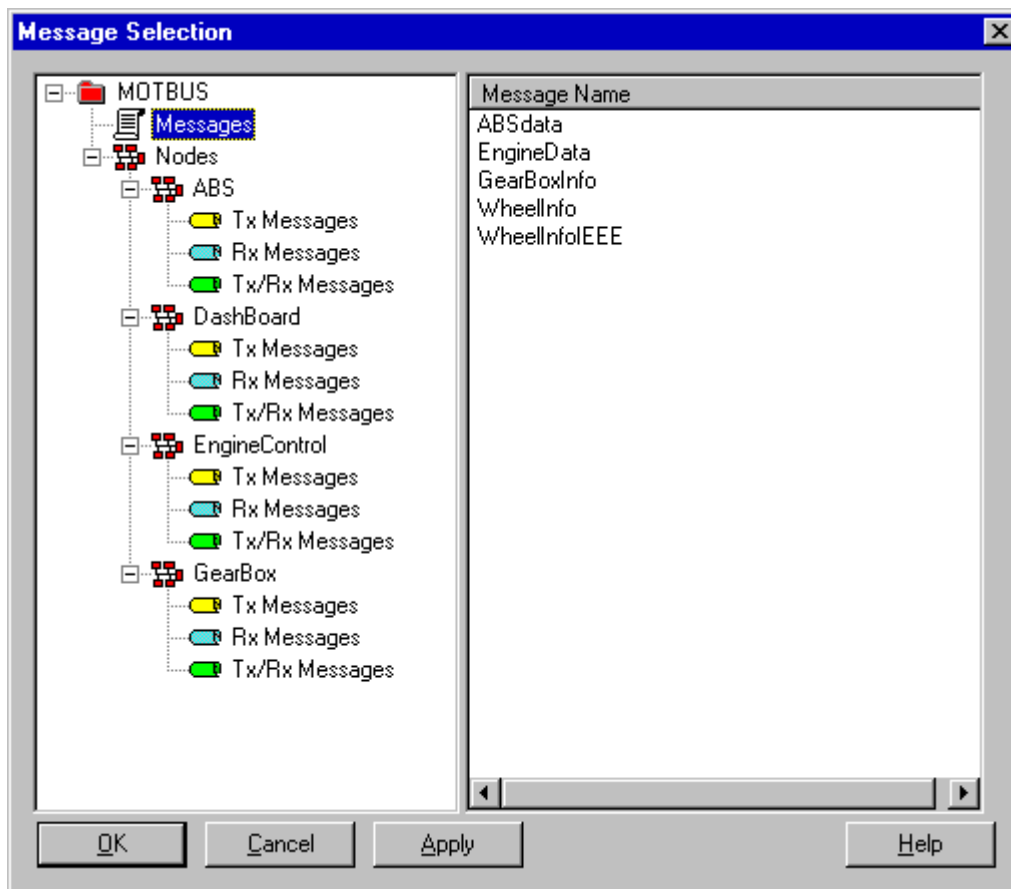


Figure 38: Message and Signal Explorer

If the database has been filled out completely, you can search in this Explorer for e.g. a node, a message or even a list of all signals in the database.

Please refer to online Help for further instructions on using the Message Explorer and Signal Explorer.

2.5 The Measurement Window

2.5.1 Trace Window

All messages arriving at the input of the Trace block are displayed as text lines in the Trace window.

Trace ALL						
Time	Chn	ID	Name	Dir	DLC	Data
11.3061	*	12c	EngSpeedContr	Tx	1	01
0.0082	1	3fc	GearBoxInfo	Tx	1	05
11.4392	1	64	EngineData	Tx	4	f2 df 2b 00
11.4158	1	c9	ABSdata	Tx	3	7e 01 00
11.4014	1	c8	WheelInfo	Tx	8	8e 22 00 00 00 00 73 21
11.4017	1	c7	WheelInfoIEEE	Tx	8	c5 e0 32 43 60 e5 3e 43

Figure 39: Trace Window

Figure 39 shows an example of a Trace window. Depending on the specific CANalyzer.

Time	Point in time when the information arrived at the CAN card (Receive, transmit or transmit request). If the message was generated by a CAPL program, the time set in the program is displayed. Output is in seconds from the start of measurement.
Chn	Number of the CAN controller chip which provided the message. Generally the number 1 or 2 is output in this column. If the message was generated by a CAPL program block, whereby the CAN number was not declared explicitly, the character * is output.
ID	Identifier for the message. Representation is decimal or hexadecimal according to the preselection. An X is appended to an extended identifier.
Name	If a symbolic database is used the message names appear. The columns "ID" and "Name" can also be shown as a combined column.
Dir	Possible values here are: RX = Receive, TX = Transmit or TXRQ = Transmit request
Attr	Special messages are described by add-on attributes: WU = Wake up or TE = Transceiver error
DLC	The DLC (Data length code) specifies the length of the CAN-data field in bytes.
Data	Listing of the message data bytes. Representation is decimal or hexadecimal according to the preselection. If a Remote message is involved, the text "Remote Frame" appears at this point.

Besides these main columns there may also be other configurable columns, depending on the specific CANalyzer option.

A number of other events are output in the Trace window:

Error frames:

When error frames occur a message will appear in the Trace window.

2.5.1.1 Standard Configuration of the Trace Window

To configure the Trace window, select the Trace block in the data flow diagram by mouse or keyboard. Then choose the menu item *Window configuration* that appears, which opens a dialog box for configuring the window. Different display modes are available.

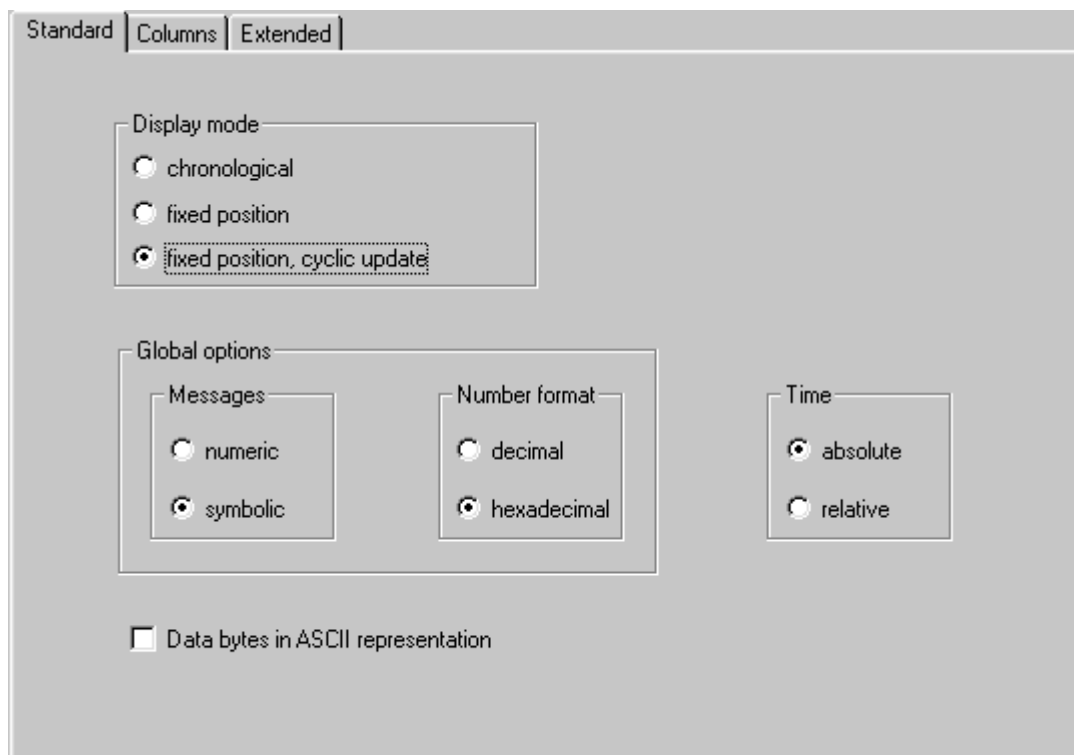


Figure 40: Configuring the Trace window

chronological output mode

The arriving messages are shown sorted by time. When the list reaches the window border it is scrolled, which can make it difficult to observe during fast bus traffic.

fixed position output mode

Here messages are assigned to fixed lines. As they arrive, new messages overwrite older messages of the same type on these lines. This mode has the advantage that even fast bus traffic can be tracked in a user-friendly manner. Furthermore, less computing time is required.

fixed position, cyclic update output mode

Here fixed lines are assigned to the messages, as in the *Fixed Position* mode. However, the Trace window is not updated with each message any longer, but rather only cyclically with a time constant. This saves on computing time during high bus loading, making bus observations possible on computers with weaker performance in this mode.

At any time you can switch back and forth between the two representations, even during a measurement. Thus, the user can switch over to chronological output mode after the measurement to page forward and backward through the received messages in the Trace window.

Note: The Trace window can be stopped by the toolbar icon  during a measurement run, so that you can analyze its contents in a user-friendly manner without terminating the measurement.

Furthermore, the Trace window offers you two different time formats. You can select whether the time stamp for messages should be displayed as absolute, i.e. in seconds since the start of measurement, or relative to the message preceding it. In the latter case, in fixed mode the time differences always refer to the same line. As a result, in this mode you can read off the transmission intervals between messages with the same identifiers. In *Chronological* mode the time differences are always shown referenced to the last message displayed in the window.

2.5.1.2 Configuration of the Columns in the Trace Window

Depending on the CANalyzer option or the purpose of the Trace window it may be advisable to show more or fewer columns. Therefore the Trace window offers a set of possible columns that can be layed out in any desired way.

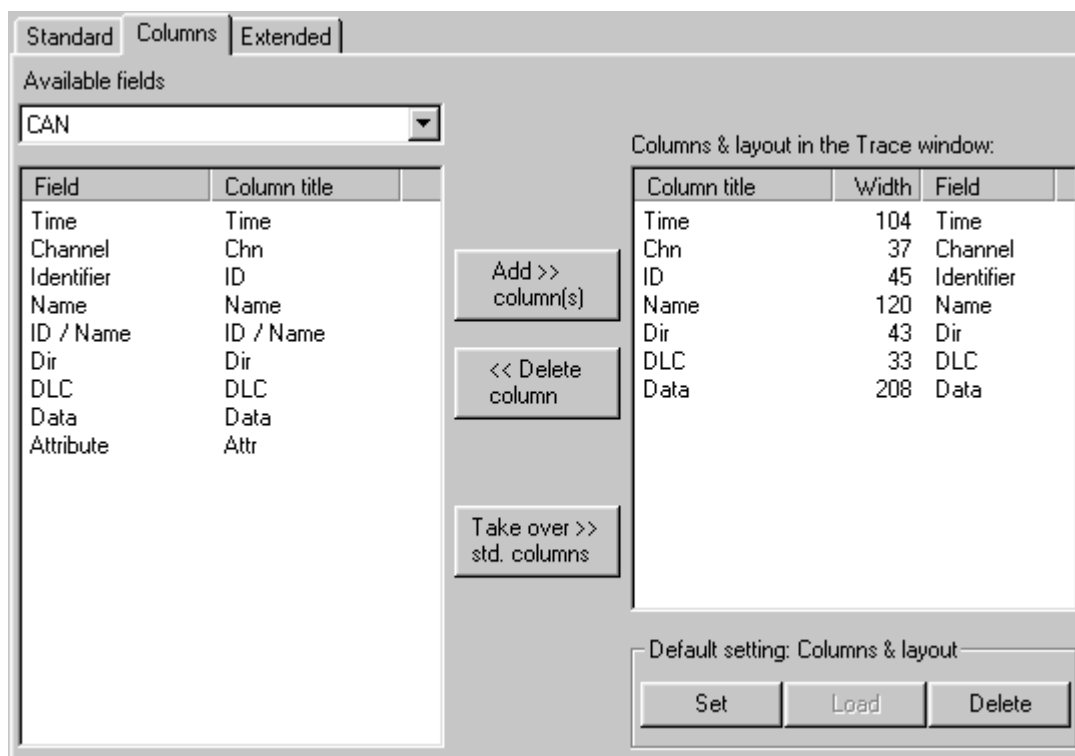


Figure 41: Columns Configuration in the Trace Window

A user-defined columns configuration is also possible as the standard. This means that all new Trace windows you create with this CANalyzer version are created with these same settings. Please refer to online Help for the exact procedure for configuring columns and selecting the individual configuration sets.

Note: Not all messages or events (e.g. Error frames) occurring in your system will fill the same columns or all columns. For example, the "Destination" column is not filled out for a CAN message. However, a J1939 message will show its "Destination address" in this column.

2.5.1.3 Trace Window Options from the Toolbar

The toolbar contains five buttons with which you can directly configure the Trace window³:

-  Delete Trace window
-  Update/Stop Trace window
-  Toggle Trace window mode: *Chronological/Fixed position*
-  Toggle Trace window mode for time representation: *Absolute/relative*
-  Toggle Trace window mode for messages: *Symbolic/Numeric*
-  Trace window mode for numbering format: *Decimal/Hexadecimal*

If you are working with multiple windows, the actions always act only on the active Trace window. If another measurement window is active, when one of these buttons is activated the window of the first Trace block is activated, and the function is executed in this window. If you are only working with one Trace block, the action is executed there directly without the window being activated.

2.5.1.4 Trace Watch Functionality and Trace Watch Window

Trace Watch functionality allows you to study individual messages in more detail. If additional information exists for a message in the Trace window (e.g. signals, if a database is associated), small [+] symbols appear in front of the Trace lines in Fixed mode or Pause mode. All of this information is shown to you when you expand the tree.

The Trace Watch window offers user-friendly methods to further examine the contents of the Trace window after the end of a measurement or in Pause mode. You can open and close the Trace Watch window from the Trace window's popup menu.

The trace watch window displays an exploded representation for all messages from the database: Besides displaying the time, message identifier, symbolic name and raw data for the message, it also displays physical signal values for the message in the active line of the Trace window.

When the cursor position is moved the contents of the trace watch are automatically updated.

As an additional CANalyzer main window, the trace watch window is always located above all other CANalyzer windows, including above the currently active pane.

2.5.1.5 Optimizing the Trace Window

Since the Trace display requires a lot of computing time, at high bus loading the data might be displayed in a delayed manner. However, the messages will still have a cor-

³ The buttons for numbering format (hex/dec) and message format (numeric/symbolic) change the associated options globally for the entire program.

rect time stamp that is already assigned to it by the PC-card. At high base load the message buffer might overflow and messages could be lost. A warning is output in the Write window. To prevent a buffer overflow when "Tracing along", it is recommended that all unnecessary branches (Logging, Statistics window, etc.) be disconnected. At high bus load, switch the Trace window to the fixed mode to economize on computing time (see also 4.2 regarding this).

In offline mode computer power plays a subordinate role. Here the trace branch can always be active. An animate function is also available, with which the entire measurement can be repeated in slow motion, and the entries can be tracked in the Trace window.

2.5.2 The Data Window

Data windows serve to display signal values. Signals are understood to be data segments of a message which carry specific information (e.g. engine RPM for CAN buses in motor vehicles). In the Data window you can view signals in a user-friendly manner. When a symbolic database is used the values of the signals specified there can even be displayed as physical variables. For example, engine RPM can be viewed in revolutions per minute, or temperature in degrees Celsius.

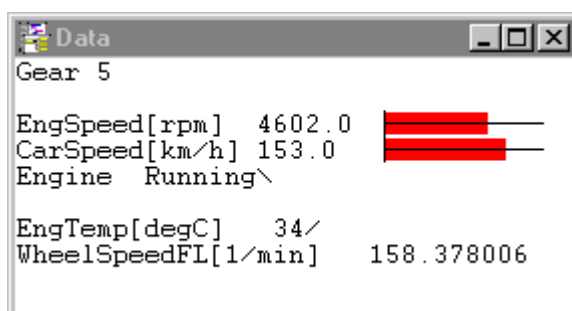


Figure 42: Data Window

Always displayed are the signal name - which can be set in the configuration dialog - and its associated value. You can also decide whether the value should be displayed as a raw datum (hexadecimal or decimal), as a physical value with accompanying unit of measurement, or as a bar chart.

The signal names and values displayed in the window are context-sensitive. When the mouse pointer is moved over them, the element below the pointer is identified by a frame. This element can be dragged to any window location with the mouse, allowing you to group signals and values according to your needs.

2.5.2.1 Configuration of Signals

From the popup menu of any data block or window, a configuration dialog can be opened in which you can enter the signals that should be displayed in the particular window, add or delete signals, or set their representation parameters.

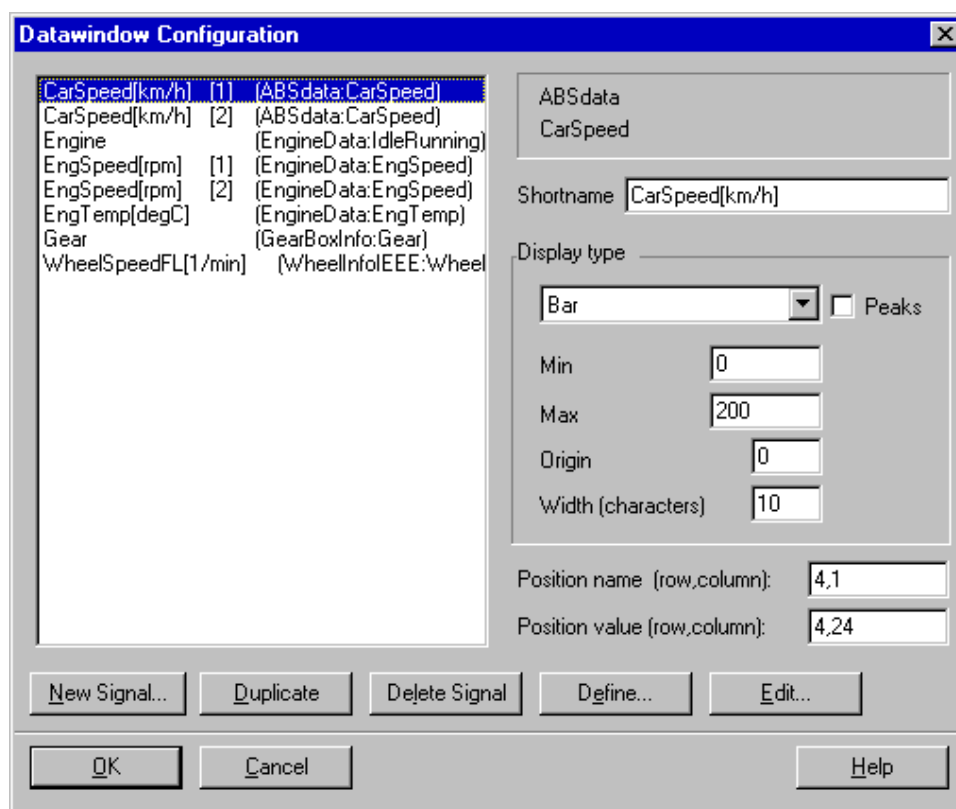


Figure 43: Data Window Configuration

All signals of the signal list are shown in the list box at the upper left of the dialog. Here you can accept new signals from the associated database. When copying, the abbreviated signal name is modified such that no ambiguities result. The abbreviated name appears during the measurement as the signal identifier in addition to the actual signal value.

With the *Define* and *Edit* buttons you can enter a new signal description - independent of the database - in the Data window or modify an existing signal description.

2.5.2.2 Display Types

Signal values can be displayed in the Data window in the modes shown below. The output width of the signal value is automatically calculated from the bit length and conversion formula. CANalyzer takes the necessary signal data from the database:

Physical

The raw data are extracted from the CAN message, scaled by the (linear) conversion formula, and displayed as decimal fixed point numbers.

Example: Signal T consists of 5 bits, is unsigned, has a factor of 10 and an offset of 0. Then the possible raw value range is 0-31, and the physical value range is 0-310. Therefore, the necessary output width is 3 characters.

Display type: Decimal

The raw data are extracted from the CAN message and displayed as decimal numbers.

Example: Signal T consists of 5 bits and is unsigned. Consequently, the possible raw value range is 0-31, and the necessary output width is 2 characters.

Display type: Hexadecimal

The raw data are extracted from the CAN message and are displayed as hexadecimal numbers.

Example: Signal T consists of 8 bits. The possible raw value range is then 0-255, or in hexadecimal representation 0-FF. The necessary output width is therefore 2 characters.

Display type: Bit

The raw data are extracted from the CAN message and displayed as binary numbers.

Example: Signal T consists of 8 bits. The necessary output width is also 8, since all bits are shown individually.

Display type: Bars

The physical signal value is displayed in the form of an analog bar. The following parameters must be configured for the display:

Min, Max: Minimum and maximum signal values to be displayed by the bars.

Width: Size of the bar that is displayed. The unit corresponds to the width of one character in the Data window.

Center: This defines the zero position of the bar.

With the help of parameters Min, Max and Center, it is possible to display just a section of the signal's value range. For example, this could be the working point of a signal to be monitored.

Display type: C-Style

You should only select this option if you are familiar with the programming language C or C++. In the dialog's format input box you would enter a format string that is used to display signal values in the Data window.

The signal's raw data (without conversion formula) are extracted from the CAN message, converted to a 32 bit value, and output with the help of the `sprintf()` C function:

```
sprintf(buffer, FORMATSTRING, (unsigned long)SIGNALVALUE)
```

The format string is compatible with the `printf()` format string of C. Due to the transformation to unsigned long an L-modifier must always be entered. Legal interpretations are "lx", "lX", "ld", "f" and "F". You can also use width and precision modifiers.

Examples of format strings: `%5.2f`, `%8lx`, `%-3ld`

The Data block attempts to recognize width specifiers in the format string to determine the output width for the screen. If this is not possible, the maximum width is assumed. This does not influence the output in the Data window.

Please note that invalid format strings can lead to unpredictable results as serious as system crashes. As a user you are fully responsible for the validity of the format string!

2.5.2.3 Activity Indicator

The signal values of the last message received in the Data block remain visible in the Data window until they are overwritten by new values. With signal values that change slowly, the user can determine whether the signal was updated with the same value or whether the particular message was not transmitted any longer and consequently an older signal value is being displayed in the Data window. This information is provided by the Data window's activity indicator: If a message is received and recorded with an unchanged signal value an alternating slash ('/' or '\') appears after the value. If the slash is not displayed, this means that the displayed constant signal value is not current, since the particular message was not received.

2.5.2.4 Peak Indicator

With signal values that change very quickly over time, transient minima and maxima can easily be overlooked. To make these transient peaks visible, the Data window therefore has a peak display. This allows you to recognize very short signal fluctuations as "afterglows" in the bar-type display. You configure the "afterglow duration" in the *Configure Timers* dialog, which is called from the Data window's popup menu.

With the *Peaks* option box you can configure - in the Data window - whether minimum and maximum values should be displayed in addition to the active value. If the active signal value is greater than the previous maximum, the maximum is updated immediately to the new value. It is not reset to the active value until after a specific time interval. The same principle applies to the minimum.

The minimum and maximum values are only shown in bar-type displays. With all other display types only the screen color changes when the minimum or maximum value deviates from the active value.

2.5.2.5 Optimization of Data Display

From the Data window's popup menu you can open a dialog for configuring the display's time behavior during the measurement.

In the lower area of the dialog are option buttons used to enhance the performance of the Data window for large quantities of data. In the normal case you should have the default option *Refresh Immediately* activated. In this case the Data window output is updated after a message is received. Additionally, the window is redrawn cyclically every 2 seconds.

To permit the processing of large quantities of data you should activate the option *Only Refresh Cyclically*. This results in the window only being updated cyclically. If there are high message rates, this can lead to a significant reduction of system loading. However, to obtain reasonable outputs in the Data window you should not select a cycle time greater than 500 ms.

2.5.3 Graphic Window

The Graphics windows serve to display signal responses over time. As was the case for the Data block, if a symbolic database is used you can have the values of signals specified there displayed directly as physical variables. For example, the engine speed response could be viewed in units of RPM or the temperature pattern over time could be viewed in degrees Celsius.

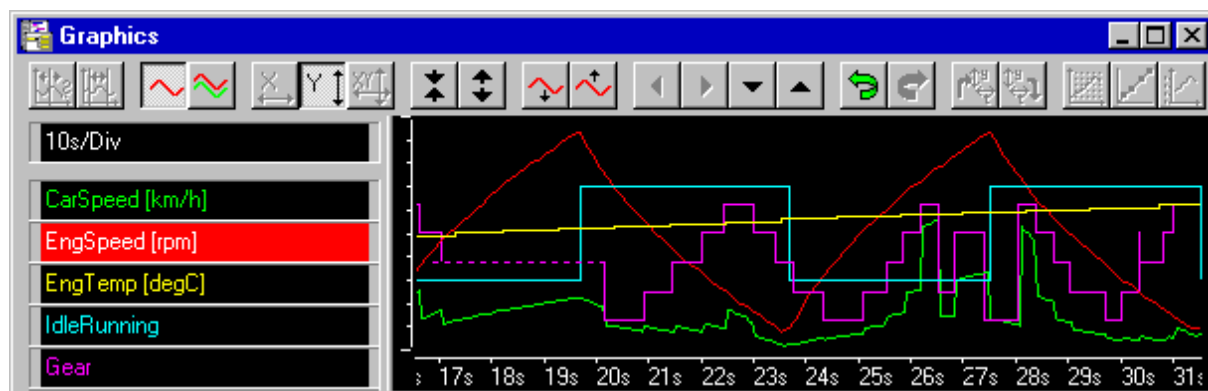


Figure 44: Graphics window

Signal-time responses are displayed graphically in the Graphics window. They are displayed in a X-Y diagram above the time axis. The measurement data remain in the Graphics window after a measurement stop and can be examined using special measurement cursors.

The Graphics window has a legend in which the selected signals are shown with their value ranges and colors. It also has a toolbar from which you can easily call the most important measurement functions. Both the legend and toolbar can be configured in the window's popup menu and can be enabled or disabled from there.

In the Graphics window there is exactly one active signal identified by inverted font in the legend. You can make a signal the active signal using the Tab key, by Page Up/Down or by clicking the signal name with the mouse. If *Single-signal mode* is enabled, all commands - such as Measure, Zoom and Scroll - refer to the active signal. In *Multisignal mode* the commands refer to all signals of the Graphics window.

2.5.3.1 Selecting Signals

In the Graphics window's signal selection dialog you can - just like in the dialog for the Data window - add signals, delete signals and enter display parameters for them. Open the dialog from the popup menu of the Data/Graphics block in the measurement setup, from the popup menu of the Graphics window, or by double clicking the legend at the right of the window with the mouse.

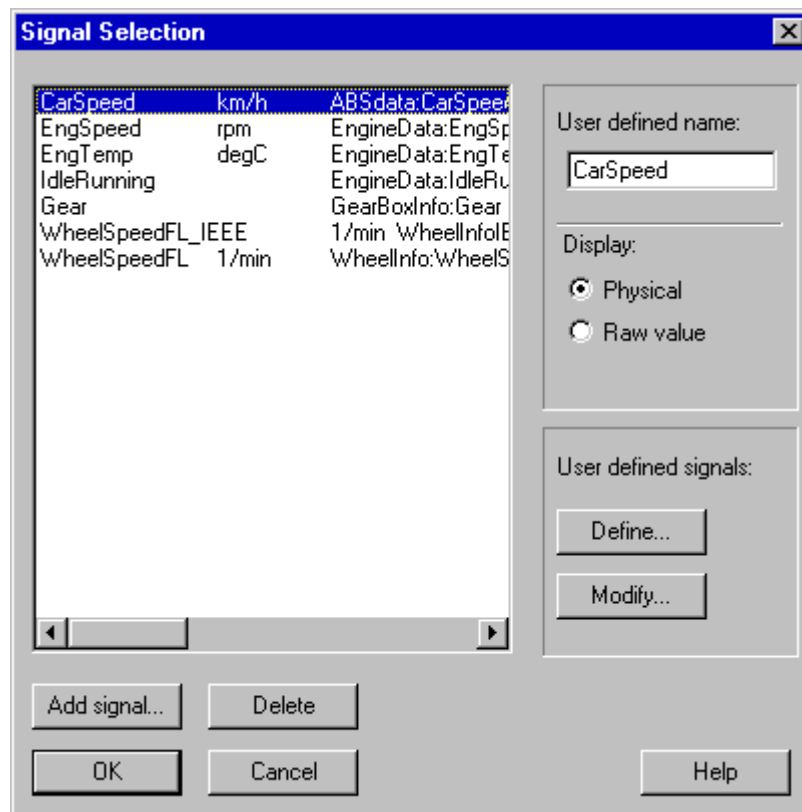


Figure 45: Signal selection dialog in the Graphics window

All Graphics window signals are displayed in the list box at the upper left of the dialog. With the *New Signal ...* button you first open a list for selecting CAN messages. In a second list you can then import the signals from the message you initially selected into the Graphics window.

Pressing the *Delete* button removes the highlighted signals from the list. With *Define...* you can define a signal, if you wish to display it without using the database. The *Edit...* button allows you to modify an existing signal description.

In the text input box *Short name* you enter a short, descriptive name for the signal, which is then output as the signal name in the Graphics window legend.

The display modes physical and decimal are available to you. In the physical display mode the raw data are extracted from the CAN message, scaled with the (linear) conversion formula, and displayed as physical values in the Graphics window. The necessary signal data are obtained from the database. In the decimal display mode, on the other hand, the raw data are only extracted from the CAN message and displayed as decimal numbers. There is no scaling by a conversion formula.

2.5.3.2 Arrangement of Signals

To exclude certain configured signals from the next measurement, you can deactivate them before the measurement. To do this, select the desired signal from the legend. Then in its popup menu choose the function *Deactivate*. You can also reactivate any deactivated signal from the same place.

Moreover, you can also change the sequence of signal entries in the legend of the Graphics window. This is useful if you have configured a rather large number of signals, but the window is too small to show them all in the legend. Additionally, you can move the deactivated signals to the lower border so that you can still see as many of the active signals in the legend as possible, even with relatively small window dimensions.

To move a signal entry you would select it with the mouse and choose the desired direction from the popup menu or by the key combination <Alt+Arrow up> or <Alt+Arrow down>.

2.5.3.3 Signal Layout

The signal layout for signals shown in the Graphics window can be controlled by a number of functions. You can set basic options in the *Graphics Window Options* dialog, which is opened from the window's popup menu or by double clicking the window with the right mouse button. All functions used to arrange the signals in the window or to study them after the end of the measurement are opened from the Graphics window's popup menu or the toolbar and are described in the next section.

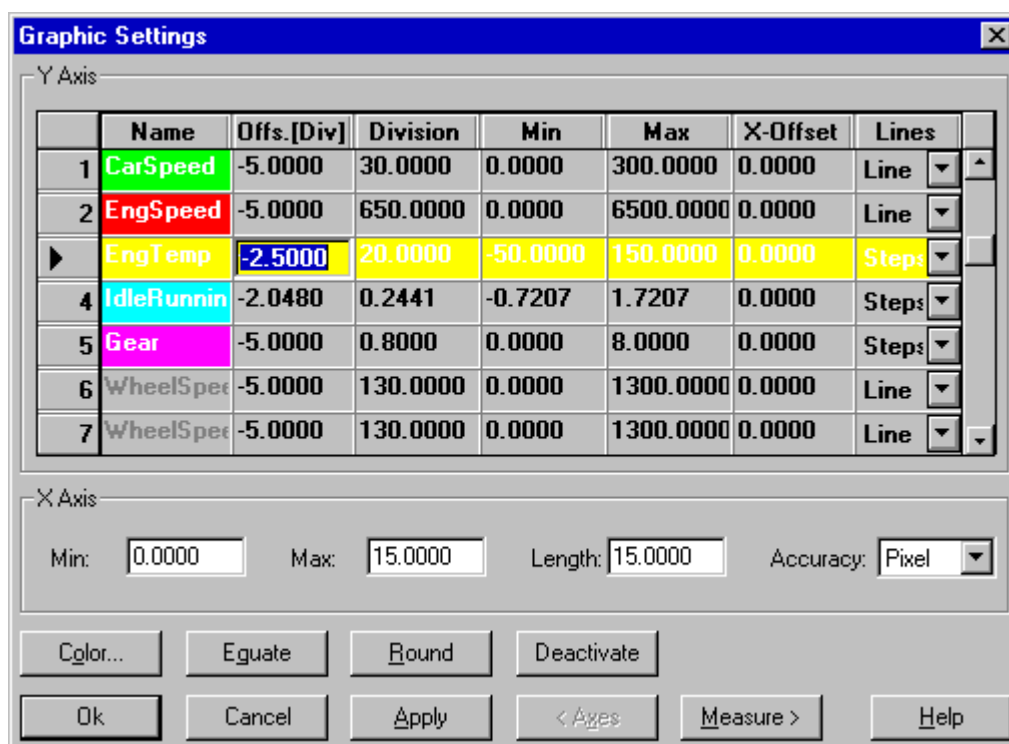


Figure 46: Configuring the axes in the Graphics window

For all signals you have entered in the Graphics window's signal configuration dialog, values are automatically assigned for Y-scaling, signal color and time axis, as well as the lines type. The values for Y-scaling and lines type are assumed from the database. In the Graphics window's *Options* dialog you can configure all of the options to satisfy your work requirements. The dialog consists of two parts, the axes options and the measurement options. You can toggle between the two parts with the action buttons *Measurement >* and *< Axes*.

Listed in the dialog's signal list are all those signals which were entered in the signal selection dialog.

Line Types

The line type in the last column identifies the display type for the display of a signal curve.

With *Line* the measurement points are connected by a line. This representation results in a continuous curve. It is the default option for displaying physical signals that are at least one byte in length.

With the *Steps* option, after a measurement point a horizontal line is output to the time of the next measurement point and from there a vertical line to the measurement point. This display type is especially suitable for digital signals. *Steps* is the default option for signals that are less than 8 bits in length.

With the *Horizontal* option, after a measurement point a horizontal line is output up to the time of the next measurement point. With *Dots* only the measurement points are marked.

Display Modes

Beneath the signal list you will find four input boxes for configuring the time axis. With *Output* you can define the display mode.

If multiple measurement points fall together within the time range of a single screen pixel, then in *Pixel* mode only measurement points at the borders of this range are displayed. This leads to faster output if there are many measurement points in a small space. Under some circumstances, however, individual peaks might not be shown in the signal response, if the measurement point with the extreme value lies within this range. In *Full* mode, all measurement points are output even if they lie within the time range of the same screen pixel at the active scaling. When there are many measurement points in a small space, this will lead to a lower output speed, but all extreme values of the signal response will be displayed.

The output mode setting has no influence whatsoever on measurement value acquisition.

2.5.3.4 Configuration of the Measurement

You can access the *Graphics Window Options* dialog by activating the *Measurement* button from the window layout dialog. Here you can configure the behavior of the Graphics window during the measurement.

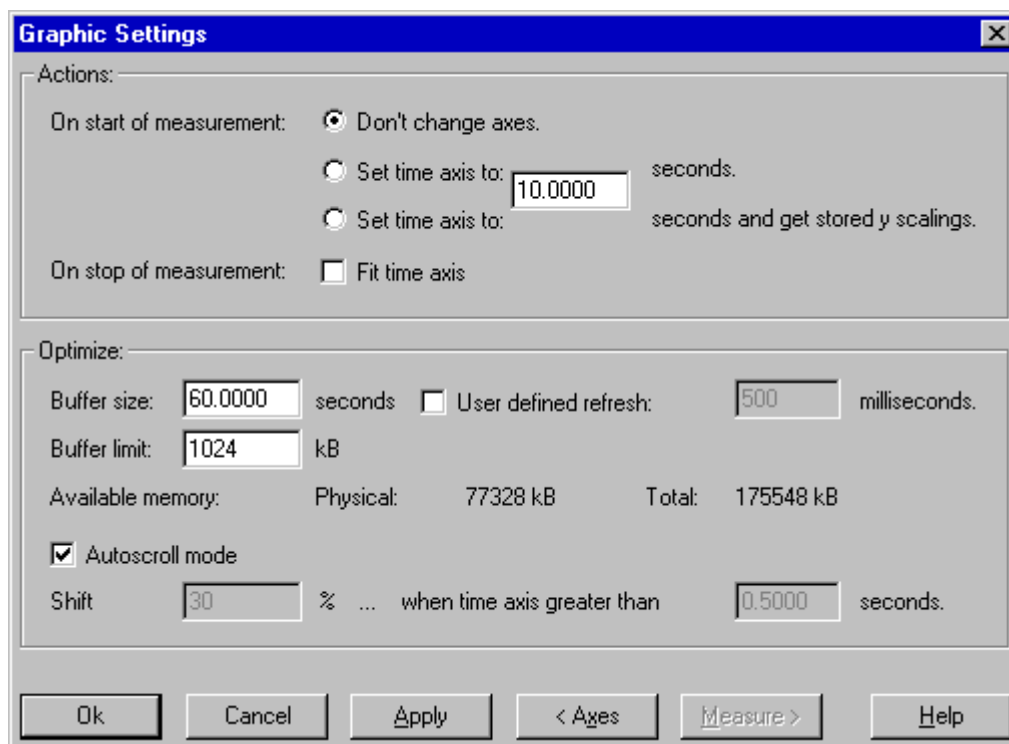


Figure 47: Measurement Options in the Graphics Window

If you select the option *Do not change axis options* under *Actions at measurement start*, then the last configured Graphics window setup is assumed at the start of the measurement. *Set with time axis* sets the Graphics window - at the measurement start - to a time range selected by you. Also, with *Set time axis and home configuration* at the measurement start you can set the Y-axis range you had previously defined in the popup menu with *Save home configuration*.

If you check the option *Fit time axis* under *Actions at end of measurement*, the signal responses over the entire measurement duration are displayed at the end of the measurement. Otherwise, the displayed time interval remains on the screen at the end of the measurement.

2.5.3.5 Measurement and Display Functions

You can call measurement and display functions of the Graphics window, either from the popup menu or by activating the appropriate toolbar button. From the popup menu you can decide whether you wish to have the toolbar shown in the Graphics window or in the main window.

2.5.3.6 Signal Modes

You can activate single-signal mode from the popup menu with the *Single* function. In single-signal mode, functions such as *Zoom-in*, *Zoom-out*, *Home configuration* and *Fit* only act on the active signal of the Graphics window, which you select by clicking in the legend with the mouse. In multisignal mode (*All* function) the same functions act on all signals of the active Graphics window. Inverted output of the name of the active signal in the legend is disabled.

Note: It is always the case that either single-signal mode or multisignal mode is active. The operations that change time axis scaling are always executed for all signals - regardless of the setting single-signal / multisignal mode - since there is only one time axis for all signals in the Graphics window.

2.5.3.7 Measurement Modes

Point Measurement Mode

You can activate point measurement mode in the Graphics window with the function *Measurement marker* from the popup menu or by the appropriate toolbar button. If this mode is already active, it is deactivated by selecting the function again.

A measurement cursor (vertical line in the window) is displayed, which you can position by clicking and holding down the left mouse button. If the mouse pointer is located above the measurement cursor, it changes its form to a horizontal double arrow. If the mouse button is pressed at a point not located above the measurement cursor, a rectangle is dragged open when the mouse is dragged. The contents of this rectangle is then displayed magnified when the mouse button is released (Zoom function). You can also move the measurement cursor by keyboard with the key combinations <Shift-Arrow left> and <Shift-Arrow right>.

While the mouse button is held down a small square is visible which highlights the next closest measurement value. The measurement time, signal name and value are shown in the upper legend for this measurement point. In the legend with signal names, the signal values of all signals are displayed for this particular time point. The measurement cursor considers Single-Signal or Multisignal mode. In Single-Signal mode the small box only jumps to measurement points of the active signal; in Multisignal mode the box jumps to the next closest measurement point of all signals.

Difference Measurement Mode

To evaluate the in measurement value differences between two points in time, you use the difference measurement mode, which you activate by the *Difference markers* item from the popup menu or by the appropriate toolbar button.

The measurement cursor and a difference cursor (vertical lines in the window) are displayed. If Difference mode is enabled, the cursors are shown at their active positions if they lie within the visible screen area. Otherwise they are moved to the viewing area. By clicking and holding the left mouse button you can position the cursors. If the mouse pointer is located above a cursor, it changes its form to a horizontal double arrow. If the mouse button is pressed at a point not located above the cursors, a rectangle is dragged open when the mouse is dragged. The contents of the rectangle are then displayed magnified when the mouse button is released (Zoom function). The cursors can only be positioned within the viewing area. However, the viewing area can be shifted by the arrow keys.

You can also move the measurement cursors by keyboard with the key combinations <Ctrl-Arrow left> and <Ctrl-Arrow right>.

While the key is pressed a small square is visible, which highlights the next closest measurement value. The measurement time, signal name and absolute value (not

the difference) of this measurement point are shown in the upper legend. In the legend with signal names, the differences in signal values for all signals are shown for the time points that have been set. The two time points and the time difference are also displayed. The measurement cursor considers the option Single-Signal or Multisignal mode. In Single-Signal mode the small box only jumps to measurement points of the active signal; in Multisignal mode the box jumps to the next closest measurement point of all signals.

2.5.3.8 Display Modes

Three different display modes are available to you for layout functions such as *Zoom-in*, *Zoom-out*, *Home configuration* or *Fit*:

In X-mode the functions *Zoom-in*, *Zoom-out*, *Home configuration* and *Fit* only act on the time axis (X-axis) of the signals. In Y-mode the same functions act only on the values axis (Y-axis), while in XY-mode they act simultaneously on both axes.

2.5.3.9 Layout Functions

The Graphics window provides you with a number of functions for changing the window layout. Some of the functions available to you via the popup menu include:

Fit all

Independent of the preset mode, the signals are scaled such that they are completely visible. To do this, the program determines the actual minimum and maximum values for each signal and the time range of all signals, and scaling is adjusted accordingly.

Zoom-in/Zoom-out

This command magnifies or reduces, by a factor of 2, either the active signal (in Single-Signal mode) or all signals (in Multisignal mode). The size is changed according to the preselected axis mode, either for only one axis (in X-mode or Y-mode) or for both axes simultaneously (in X/Y-mode).

Operations that change the scaling of the time axis are always executed for all signals (independent of the option Single-Signal/Multisignal mode), since there is only one time axis for all signals in the Graphics window. Axes can also be scaled individually for each signal in the Graphics window's options dialog.

Fit

The signals are scaled such that they are completely visible. This involves determining the actual minimum and maximum values of each signal as well as the time range for all signals, and the scaling is set accordingly. The active modes (X-mode, Y-mode or X/Y-mode, and Single-Signal or Multisignal mode) are taken into consideration. This fits the entire graphic optimally in the window.

Round

Scaling of the displayed value range for all signals is rounded-off. This involves rounding-off the active *Division* value to a valid whole number ($\text{Division} = n * 10x$

, $n = [1 \text{ to } 9]$, $x = \text{whole number}$). The lower and upper range limits are rounded-off to the precision of the *Division* value.

Rounding always affects all signals of the Graphics window. The active mode (X-mode, Y-mode or XY-mode) is considered. Scaling of the signals is rounded-off to a whole number value.

Get scalings/Store scalings

You can save the scaling configuration of the Graphics window (i.e. the time range displayed for all signals and the value ranges of each individual signal) as a home configuration. The default home configuration has a time range from 0 to 5 seconds and the Min/Max value range of each signal taken from the database.

With the function *Store scalings* the current scaling of the active Graphics window is saved. In doing so, the active modes are considered (X-mode, Y-mode or XY-mode, and Single-Signal or Multisignal mode). Afterwards, you can reset all scalings in the Graphics window to this stored configuration at any time with the function *Get Scalings*.

Marker

This command activates or deactivates measurement point marking. The color of the measurement points matches the preset signal color.

Grid

This command enables/disables grid lines in the Graphics window. The grid lines match the subdivisions of the Y-axis. You can set the color of the grid lines in the *Color Options* dialog.

Y-Axis

This command enables/disables labeling of the Y-axis in the active Graphics window.

When labeling is disabled, a Y-axis with 10 subdivisions is displayed for all signals. In the legend the following are shown for each signal: Lower and upper values of the viewing area, and the value amount between any two subdivision tick marks.

If labeling is enabled, the subdivision of the Y-axis is calculated automatically for the active signal and is displayed in the color of the signal.

The tick marks are set to whole number values. If a signal is shown as decimal (i.e. raw signal values without conversion), then only whole numbers are displayed. In this case, if an area between two adjacent numbers is shown magnified, no subdivision tick marks will be seen any longer along the Y-axis. For the physical display type subdivisions less than 1 are also shown.

When labeling is enabled the legend also shows the following for each signal: Lower and upper values of the viewing area (i.e. not necessarily the values of the lowermost and uppermost Y-axis tick marks) and the value amount between any two tick marks.

The grid lines in the graphics display always match the tick marks on the Y-axis. Grid lines are enabled or disabled by the *Grid* command in the window's popup menu.

Colors

Here you select the background color for the window (white or black). Furthermore, you can open the *Options* dialog for configuring signal colors.

Signal Legend

With this function you choose whether the legend for signals should be displayed on the left side of the Graphics window or not.

2.5.3.10 Export of Signals

With the help of this function you can save the data of one or all signals of the Graphics window to a file. Depending on the activated signal mode (i.e. single-signal or multisignal mode) the Export either applies to the currently active signal or to all signals. This function is only available if data exist for the active signal.

CSV format (Comma Separated Values Format) is supported for export. Possible delimiter symbols are comma, semicolon, tab [TAB] and space [SPACE].

2.5.3.11 Toolbar of the Graphics Window

The graphics popup menu offers you the option of showing the graphics toolbar either in the main window or in the graphics windows. If you have the graphics toolbar shown in the main window, the functions act on the active graphics window. If no Graphics window is active, the graphics toolbar is deactivated. Click a graphics window with the mouse to activate it.

2.5.3.12 Optimization of the Graphics Window

The Graphics window is a very powerful but equally complex window in the measurement setup. Its performance capabilities are closely interrelated to the graphics card of your computer and the graphics driver installed. Under certain conditions effects may occur that disturb the graphics display when working with the window. Generally these unpleasant effects in the display can be eliminated if you have Windows redraw the window, for example by modifying the size of the window.

Note: Please make sure that the Graphics window is not covered up by other measurement windows, menus or dialogs during the measurement. Under some circumstances the performance capabilities of the output may be impaired severely by such overlapping.

Depending on the configuration and bus load the powerful symbol algorithms used in the Graphics window may place a relatively high demand on computer resources. Since individual requirements vary widely, the Graphics window provides special configuration options for optimizing the display.

To optimize the Graphics window, open the *Options* configuration dialog from the popup menu or the window's toolbar. After pressing the *Measurement* button you will be presented with the measurement options, where you can optimize the following *Parameters* of the Graphics window:

Buffer size

Here you set the time interval that is to be saved (in seconds) for the signal responses of all signals configured in the Graphics window. For example, if you enter the value 10 here, the last 10 seconds of your measurement in the Graphics window are always saved and will be available to you after the end of measurement for further evaluation. Consequently, high values for this parameter will - particularly when displaying many signals - result in a large memory requirement, but will allow you to track and evaluate the signal response over a correspondingly large time period after the end of measurement.

After the measurement stop, with large signal buffers some functions may become lethargic, such as moving or fitting signals, since in this case large quantities of data must be redrawn. Therefore, you should select a value for the buffer size that is as small as possible.

Buffer limit

In addition to buffer size, you can also specify a buffer limit in kB. This defines the maximum memory usage by the Graphics window during the measurement. Above all, this is advisable if you have specified a relatively large time span as the buffer size. Without this maximum limit, more and more memory is demanded by the operating system over the course of the measurement. This can lead to severe loading of the overall system due to swapping out of storage. Please refer to online Help for further details.

User-defined refresh

Here you define how often the Graphics window display should be updated. Small values result in continuous display of the signal response, but on the other hand they place a high demand on computing resources and may lead to performance problems with slower computers. High refresh values lessen the demand on computing resources, particular when many signals are being displayed, but they lead to more erratic display of signal responses. You should only input a refresh value if your measurement setup places special demands (high bus load, simultaneous display of many signals, etc.) on the Graphics window. If the Refresh check box is not checked, the Graphics window automatically determines a favorable default value. As long as no load problems are occurring during the measurement (See section 4.2) you should not modify these options.

Scrolling

If the signal curves run into the right border of the Graphics window after the start of measurement, this command will result in automatic tracking of the curves. This involves shifting the time axis to the left, to make room for the measurement signal on the right side. You can configure this behavior, referred to as scrolling, in the Graphics window. Continuous scrolling is activated as a default, whereby the time axis is only shifted minimally to the left to give the impression of continuously flowing signal curves.

To shift the time axis in jumps, thus maintaining the graphic diagram at a fixed location between these jumps, you would deactivate the autoscroll mode in the Graphics Window Settings dialog. In the line below this you can set the percent-

age of the displayed time interval by which the time axis should be shifted. The smaller the value, the more evenly the view is scrolled, but it is shifted more frequently. If the time interval displayed in the Graphics window is smaller than the value you specified, the entire Graphics window is reconstructed, i.e. the image is shifted by 100%. This prevents the Graphics window from demanding too much computing power due to frequent scrolling for very short time intervals.

The scrolling procedure always demands more computer resources for smaller time intervals, since scrolling must be done more quickly. Therefore, there is a minimum refresh rate for updating the window contents. If the displayed time interval is of the same order of magnitude as this rate, the signal curves are displayed again in an increasingly jumpy manner, since the window contents are not always updated. You can set the minimum refresh rate in the project file CAN.INI. This involves setting the following value in the [System] section:

GraphicWindowMinAutoRefreshrate= <Cycle time in ms>

However, please note that the smaller the value you select, the more system loading will increase. Before modifying the automatic refresh rate, it is therefore essential that you determine an optimal refresh rate with the help of the User-Defined Refresh function.

Note: Some driver-specific problems in displaying the measurement cursors and dotted lines, such as those used to draw extrapolated signals and window grid lines, can be resolved by special options in the file CAN.INI. Refer to documentation in section [Debug] of the project file regarding this.

2.5.4 Statistics

The Statistics block fulfills three different functions. One of these is to display the average time between the sending of messages (secs/msg) during a measurement. It can also display the messages per second. These are done by constructing a continuously updated line histogram above a message identifier's axis. A sliding scale averaging method with an adjustable averaging time is used. The other function keeps statistics on all bus activities in the background; these results can be reported either as a statistical report in the Write window or stored via a histogram function and then processed further.

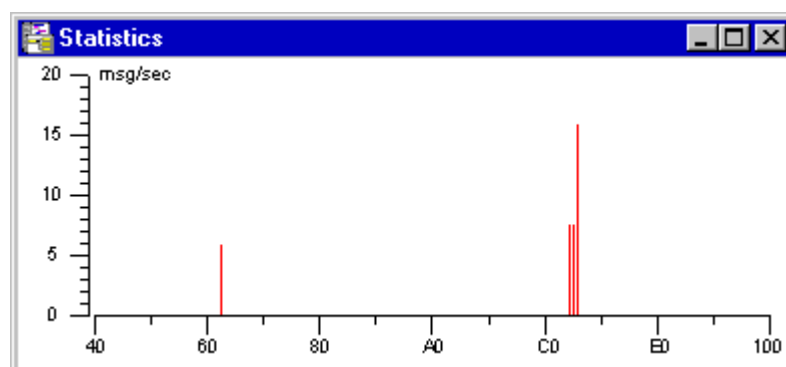


Figure 48: Statistics Window

The Statistics window displays (see Figure 48) the mean message rates existing at the end of the measurement. The Write window contains the statistics report (see Figure 50).

2.5.4.1 Direct Display in the Statistics Window

During the measurement either the mean transmit interval or the mean message rate is displayed in the Statistics window. Smoothed averaging is used with adjustable averaging time. The message identifiers are output on the horizontal axis, and the corresponding rates on the vertical axis. The IDs are distributed according to whether they originated from controller CAN1 or CAN2, and by message attributes Rx and Tx:

	Rx	Tx
CAN1	Green	Red
CAN2	Gray	Blue

In the statistics configuration dialog you can define whether the window diagram should show the mean transmit interval (sec/msg) or its inverse, the mean message rate (msg/sec). Also configurable is the averaging time which defines the time interval at which the display is refreshed. Averaging is most precise with a low value, but this demands a lot of computing time. With high values the statistics display lags behind. An averaging time of approx. 1000 ms gives satisfactory results for standard applications.

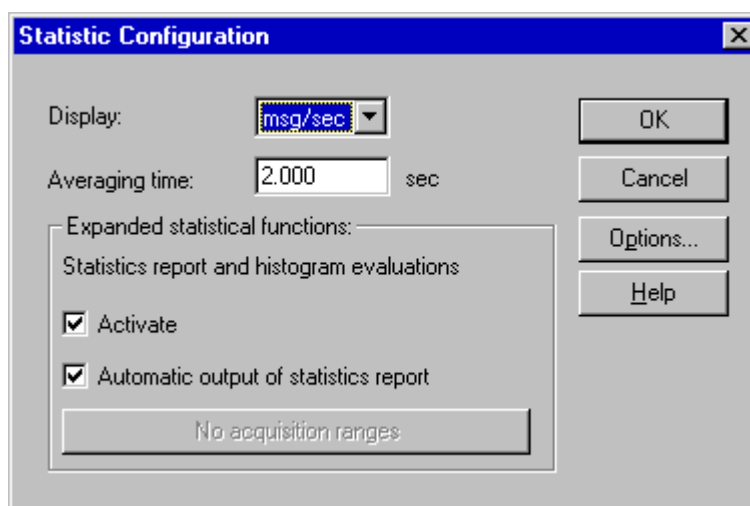


Figure 49: Configuration of the Statistics Block

You can scale the Statistics window from the popup menu. The functions available for this, such as *Zoom*, *Fit*, *Basic Image* and *Manual Scaling* are described in detail in online Help.

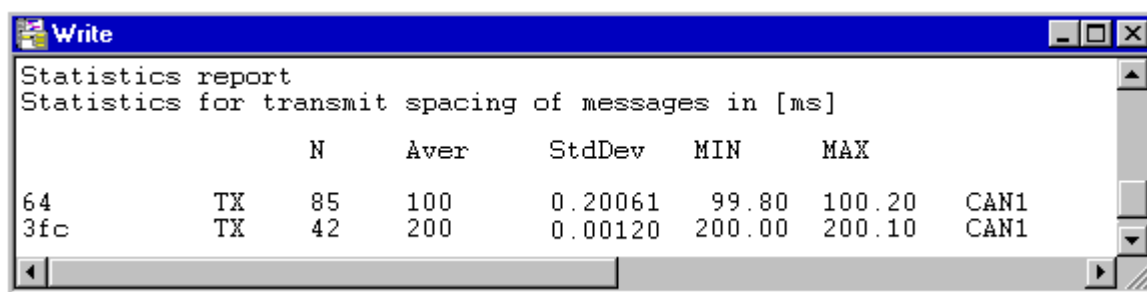
Note: If the CAN card used supports extended identifiers, the function *Basic Image* is split. The user can choose whether scaling will be over the range of standard identifiers or over the entire range.

2.5.4.2 Statistics Report

In background, statistics are kept on all bus actions, and the results can be reported to the Write window after conclusion of the measurement. A list is constructed (sorted by message identifiers) which contains information organized separately separately for receive messages, transmit messages, transmit requests and transmit delays: Number of messages, mean time spacing, standard deviation, minimum spacing, maximum spacing.

When you select the option *Activate* under *Expanded statistical functions* in the statistics configuration dialog all data will be accumulated for the report. However, this of course requires computer resources and can slow down very fast bus traffic.

You can output the statistics report to the Write window either automatically or via the menu entry *Display statistics report* after the end of a measurement.



Statistics report						
Statistics for transmit spacing of messages in [ms]						
		N	Aver	StdDev	MIN	MAX
64	TX	85	100	0.20061	99.80	100.20 CAN1
3fc	TX	42	200	0.00120	200.00	200.10 CAN1

Figure 50: Statistical evaluation of a measurement in the statistics report

Afterwards, the Write window shows - for each occurring ID coded by attributes - the total number of messages, the mean transmit interval, its standard deviation, minimum and maximum.

Note: The function *Display statistics report* can only be selected if the *Statistic report... Activate* option was selected during the measurement.

2.5.4.3 Choosing a Histogram

When the upgraded statistics functions are activated, data is collected for interpretation during measurement runtime. Several time periods for collecting data can be recorded via the histogram function. The beginning and end of these time periods can be determined from *New evaluation* and *Stop evaluation* in the popup menu of the Statistics block or with appropriate CAPL functions.

Note: Stopping the measurement also ends data gathering for the the time range being measured at the time. Evaluations can be recorded from several measurements started one after the other in the same configuration.

The data obtained can be further evaluated with the histogram dialog box (menu item *Evaluation output*) by choosing regions of interest from the data included in the time range.

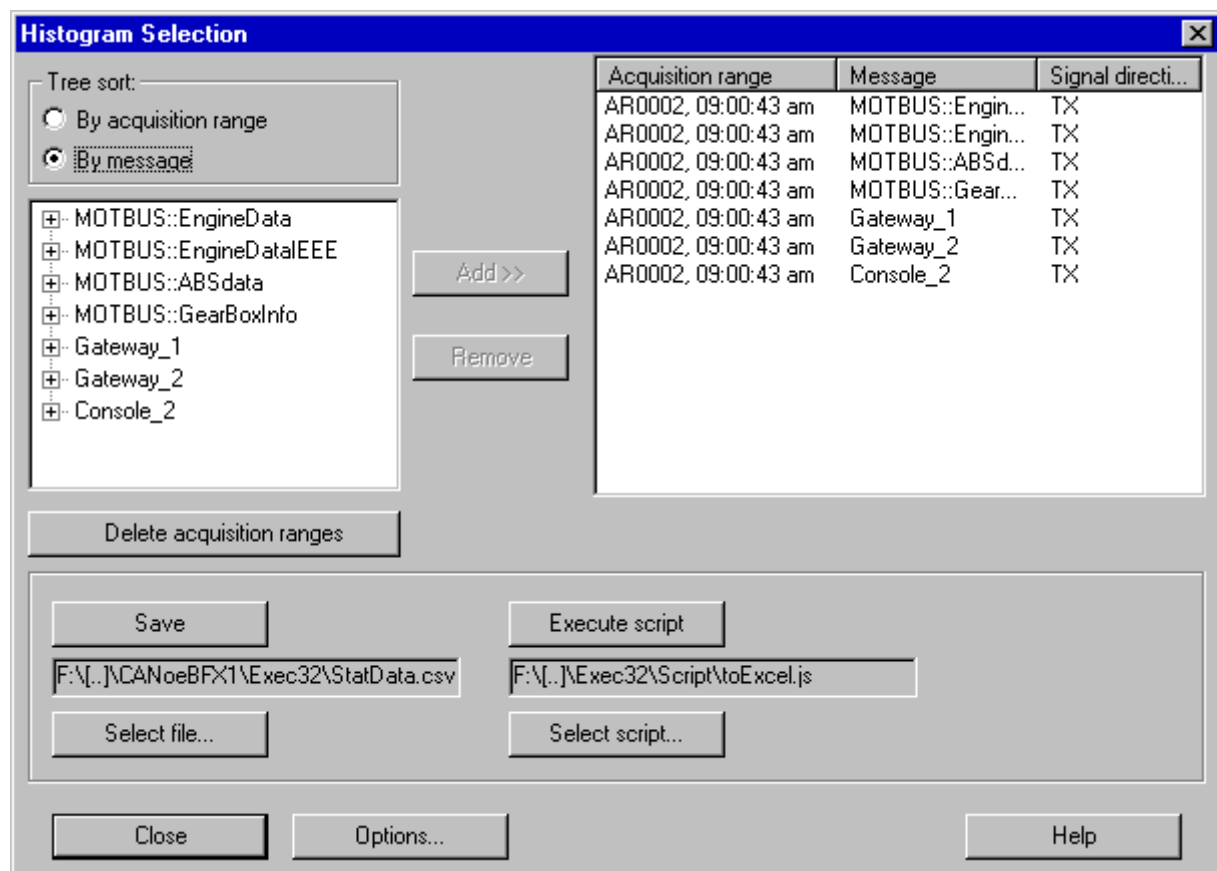


Figure 51: Histogram Selection

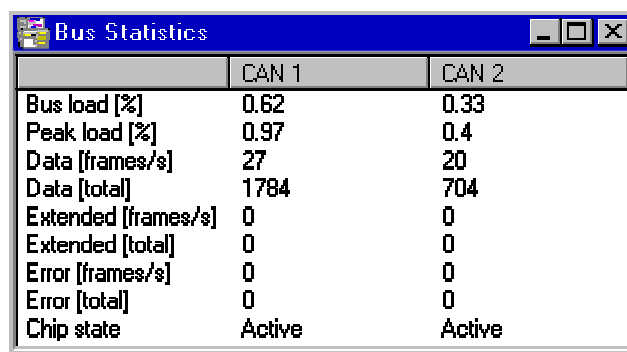
The appropriate data selected can be stored as a CSV file (values separated by commas) or transferred to Excel with the script toExcel.js. The data can also be manipulated with the user's own Java or Visual Basic script.

The collected data can be reseted in the configuration dialog box of the statistics block.

Note: Changing the configuration deletes all recorded ranges of coverage.

2.5.5 Bus Statistics

CANalyzer acquires statistical data over all CAN channels used. The data acquired depend on the hardware used. The results are displayed in the Bus Statistics window. The statistical functions include the rates and number of Data and Remote frames (11 and 29 bit), Error frames and Overload frames. Also displayed are the values for momentary and maximum bus load. The state of the CAN controller is shown as ACTIVE, ERROR PASSIVE or BUS OFF. Also displayed for certain hardware platforms are the value of the CAN controller's Tx error counter and Rx error counter.



	CAN 1	CAN 2
Bus load [%]	0.62	0.33
Peak load [%]	0.97	0.4
Data [frames/s]	27	20
Data [total]	1784	704
Extended [frames/s]	0	0
Extended [total]	0	0
Error [frames/s]	0	0
Error [total]	0	0
Chip state	Active	Active

Figure 52: Bus Statistics Window

Under *Options* in the card icon's popup menu in the measurement setup window, the user can configure the time interval at which the card passes bus statistics information to CANalyzer. This interval defines the frequency of the bus load measurement and thereby also the averaging time. The default value is one second. For measurements with extreme data throughput bus statistics may be deactivated to increase performance. Above all, this affects the FIFO buffer between CANalyzer and the CAN card. The error message "Rx buffer overrun" could possibly be prevented by doing this.

Bus statistics information is also recorded in logging (cf. section 2.7). To include this information in logging activate the *Log internal events* check box in the configuration dialog for the log file. When the file is played back in Offline mode this information is then displayed again in the Bus Statistics window. The Bus Statistics window remains empty in Offline mode if the data source does not contain any bus statistics information.

2.5.6 Write Window

The Write window has two functions in CANalyzer: First, important system messages on the progress of the measurement are output here (e.g. start and stop times of the measurement, preset baud rate, triggering of the logging function, statistics report after the conclusion of measurement). Secondly, all messages which you, as the user, place in CAPL programs with the function `write()` are output here.

The popup menu command *Copy contents to Clipboard* accepts the contents of the Write window in the clipboard. Write window messages serve as both a supplemental report for your measurements and – should problems occur – as a basis for error analysis by our customer service.

For a description of the most important CANalyzer system messages that are output to the Write window, please refer to the Appendix under section 5.4

2.5.7 Layout and Fonts of the Measurement Windows

You can modify the arrangement of windows (Window layout) from the *Window* main menu:

With *Standard Layout* the standard window layout is loaded. It makes sense to call this function if you have changed the window sizes and positions and wish to quickly return to the standard configuration. In contrast to the function *File|New configuration*

this function only acts on the window layout. All other configuration settings are preserved.

Select the function *Set user-defined layout* to define a window arrangement (Layout) according to your working requirements. this window layout is saved in the file CAN.INI. With the menu item *User-defined layout* you can finally arrange the windows of each opened configuration according to the settings made with this function.

In the lower menu area all recently opened windows of CANalyzers are shown. Additionally, the active window is marked with a .

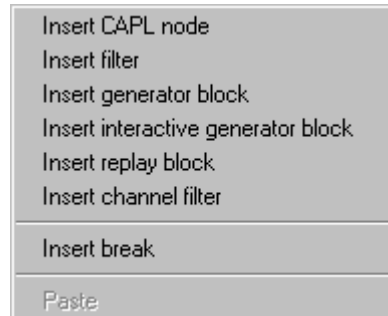
For Trace windows and the Write window, the window's font can be set from the window's popup menu with the function *Font|Select*. The fonts of function blocks and function block comments can be configured in the measurement setup.

The predefined font can be set for a window with the function *Font|Reset*.

2.6 Hotspots and Insertable Function Blocks

In the measurement setup there are square points (hotspots) between the basic function blocks, at which additional function blocks can be inserted or the data flow can be blocked. The hotspots themselves allow all data to pass unhindered.

When a hotspot is clicked on with the right mouse button (or selected with the cursor keys and then activated with <F10>), a popup menu appears with the following menu commands:



If one of the first five menu commands is clicked on, a block is inserted in the data flow plan which satisfies the particular function. New hotspots appear before and after this block, so that additional blocks can be inserted. The last menu item is a special case. If it is activated a broken hotspot appears, which is intended to show that the flow of information is blocked at this point.

Function blocks can be recognized by their appearance or by their labels in the data flow chart. A "P" stands for a CAPL node (*Program block*), "PF" and "SF" designate the *Pass* and *Stop Filters*, "PE" and "SE" are the corresponding filters for environment variables, a "G" refers to a *Generator block*, an "IG" to the *Interactive Generator block* and "R" stands for a *Replay block*. The channel filter is represented with a special icon.

All blocks have popup menus which the user can open by either clicking with the right mouse button or by selecting the block in the data flow plan and then pressing <F10> . The first menu item opens a configuration dialog for the particular block, which only serves to parameterize the block (there are two configuration dialogs for

the generator block, which can be opened by the first two menu items). By selecting the last item in the popup menu "Delete this node", you can remove the block from the data flow plan. All configuration information is lost in the process. However, the CAPL source files of the CAPL node and the log file of the replay block are not deleted.

From the popup menu you can assign a comment to each function block and analysis block. The comment is displayed by default. With the function *Show comment* in the popup menu you can enable and disable the display separately for each block.

Please note that the size of the displayed comment depends on the actual position of the function block and the space available for the display. Independent of this, it is always the case that only the first two lines of the comments are displayed. It is advisable to use only short key words in the first two lines and to enter more elaborate information further below in the text.

If you have not entered any comment, but the display is nevertheless activated, the predefined comment is displayed.

2.6.1 CAPL Nodes

A CAPL node is a universal function block whose characteristics the user defines by writing a CAPL program. See section 3 for a detailed description of CAPL program blocks and instructions on creating CAPL programs.

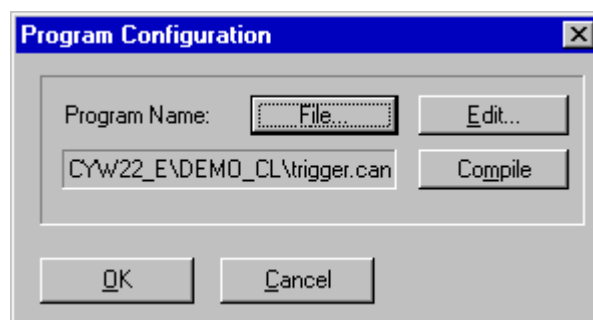


Figure 53: Configuration dialog for CAPL programs

An important use of program blocks is e.g. to preset transmit messages in the transmit branch. However, even very simple program blocks can be written for data reduction (Example: Only pass every tenth message) or for monitoring.

Note: A CAPL node blocks all messages in a data flow branch which are not explicitly output with **output()**. A program that is transparent for all messages must therefore contain the following message procedure:

```
on message * {
    output(this); /* Pass all messages */
}
```

Consequently, filters can also be programmed in the evaluation branch with CAPL programs by the intentional use of **output(this)**. The functionality of these programs can be much more complex than that of normal pass or blocking filters.

Program blocks appear in the data flow plan as small blocks with the label *P*.

In the configuration dialog, first assign a CAPL file name (Extension *.CAN) to the program block.

Press *Edit* to open the CAPL Browser. Browser is an easy-to-use tool for creating, modifying and compiling CAPL programs and is described in detail in section 3.4.

Before you start the measurement you must compile the CAPL file. To do this, press the *Compile* button or choose *Configuration|Compile all nodes* in the main menu to compile all CAPL programs at once.

Note: It is permissible to reference the same CAPL programs in different program blocks. For example, this may be of interest if the identical data manipulations are to be performed in two different data flow branches (e.g. data reduction operations).

You can deactivate the node by pressing the spacebar or by selecting the line *Node active* in the CAPL node's popup menu. The node can be reactivated by repeating the same action. The menu command *Delete this node* removes the CAPL node from the measurement setup.

Note: When a CAPL node is removed from the measurement setup, the CAPL source file is not deleted.

2.6.2 Filter Block

The volume of data can be selectively reduced by using the filter block. Toggling between a pass filter and a stop filter will pass or block those identifiers and/or identifier ranges that are specified. This is done with the button "Filter type". All messages of a network node can be filtered as well.

In addition, the message type affected by the filtering function can be set for the identifier, as well as whether filtering should also apply to error frames.

Filter blocks are entered in the data flow plan by right clicking a hotspot, and appear as small blocks with the label PF (Pass Filter) or SF (Stop Filter). When these blocks are double clicked a configuration dialog appears in which the filter can be parameterized or selectively deleted by specifying messages (ranges), error frames, network nodes and attributes.

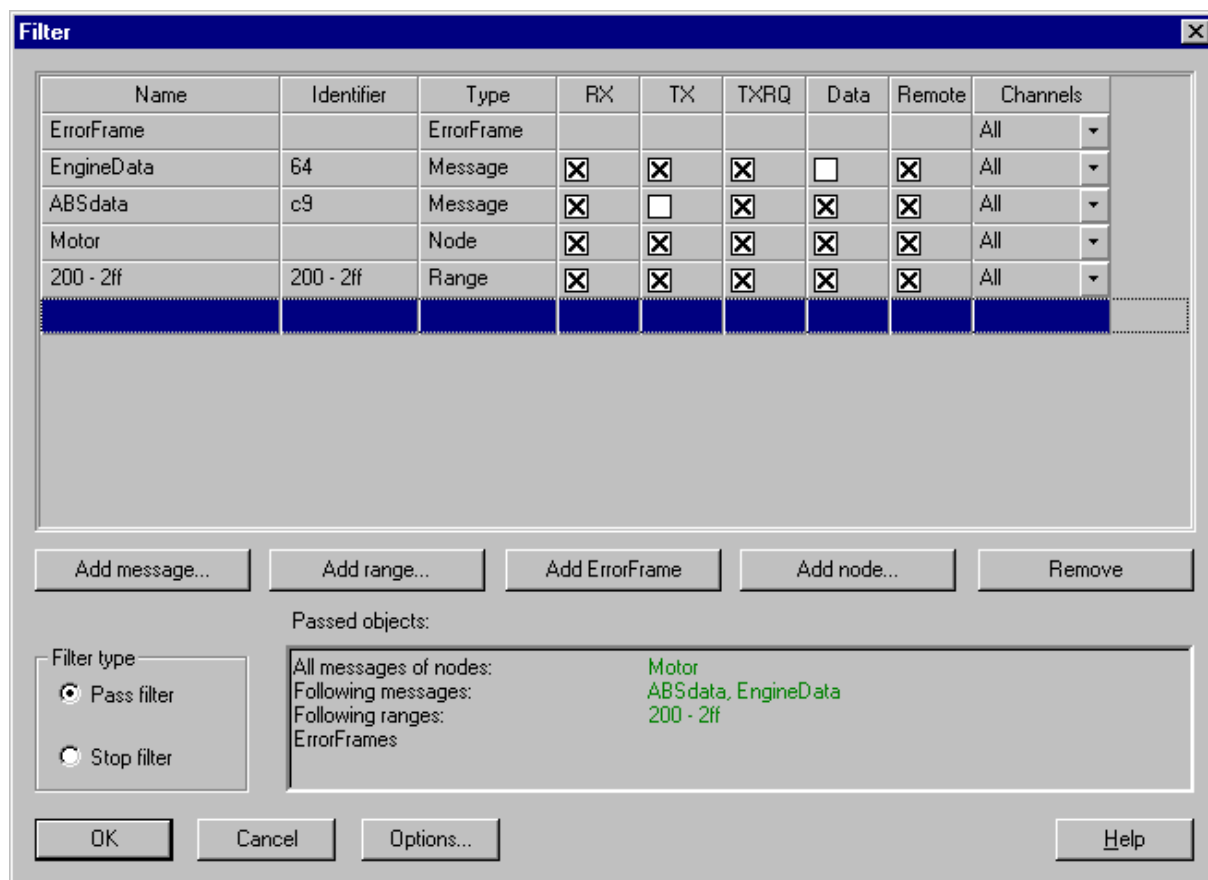


Figure 54: Filter Configuration Dialog

The filter configuration is lost when a filter block is removed from the measurement setup (by choosing the command “Delete this node” in the popup menu or “Del”).

Note: In keeping with its function, a pass filter which is not configured (empty) does not pass any messages and so blocks all message traffic. If older configurations are opened with Version 3.1, their old pass and stop filters will reappear.

2.6.3 Channel Filter

It is possible to completely block all messages on a channel or to let them pass with a channel filter.

The channel filter can be seen in the data flow plan as a small block in which the actual setting is graphically shown. Blocked channels are represented by broken red lines; those which are not blocked appear as green lines.



Figure 55: A Channel Filter in the Data Flow Plan

Double clicking on the filter symbol opens a configuration dialog in which the available channels are seen and can be set.

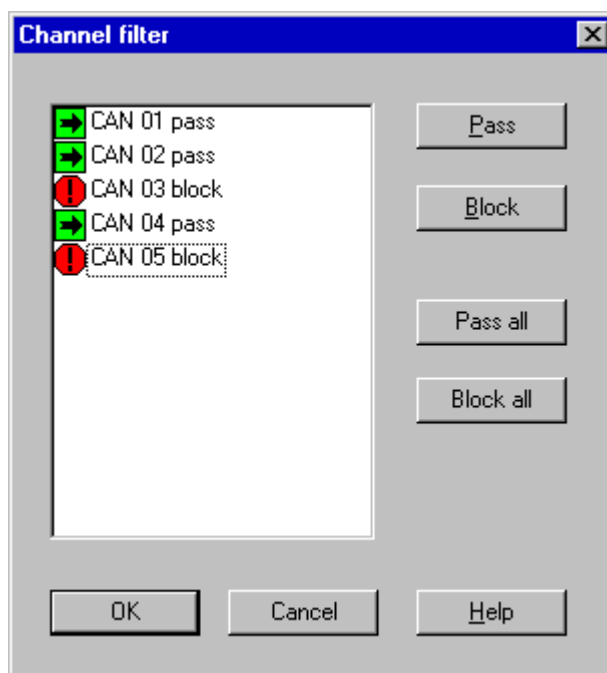


Figure 56: Configuration Dialog For Channel Filter

The filter configuration is lost when a channel filter is removed from the measurement setup (by choosing the command “Delete this node” in the popup menu “Del”)

Note: A channel filter which has not been configured can be used in the data flow plan to simply show the number of channels being used.

2.6.4 Replay Block

The replay block makes it possible to replay measurement sequences which have already been recorded. To do this, the user specifies a log file. Messages contained in it are introduced into the data flow. The most important application is replaying a recorded data stream onto the CAN bus. To do this, the replay block must be inserted in the transmit branch.

Replay blocks appear in the data flow plan as small blocks with the label *R*. Double clicking these blocks causes a dialog box to appear, in which the replay block can be configured.

The user can specify whether RX messages or TX messages are to be transmitted or not. The user can also choose whether messages originating from CAN controller 1 should be transmitted on CAN 1 or CAN 2, or should not be transmitted at all, and similarly for messages originating from CAN 2.

The file can be transmitted once or cyclically. For cyclic transmission, transmission resumes with the first message after the end of the file is reached.

There are three possibilities to define the start of transmission of the first message of the file.

<i>Immediate</i>	The first message is transmitted at the start of measurement.
<i>Original</i>	The time of transmission is defined by the time saved with the message in the file.
<i>Specified</i>	The user sets the time explicitly in milliseconds since the start of measurement.

In all three cases the time spacing between messages within the file is preserved. If it is less than one millisecond, transmission is delayed accordingly.

The configuration is lost when a replay block is removed from the measurement setup (by selecting the command "Delete configuration" in the popup menu). This only affects the options set in the dialog box. The replay file itself is not deleted.

Note: To make a replay block capable of sending data onto the bus, you must insert it in CANalyzer's transmit branch. If you insert it in the evaluation branch, the data are indeed sent to the evaluation blocks to the right of the replay block and are displayed in the associated windows. However, the data do not reach the bus, due to the left-to-right directional flow of data.

2.6.5 Generator Block

The purpose of the generator block is to create messages for transmission. Trigger conditions and a transmit list are defined for this purpose. A received message, a key press or a time period can be entered as a trigger condition. The trigger conditions can also be used in combination. In the transmit list, messages (and error frames) are entered in the sequence in which they are to be transmitted.

Whenever a trigger condition occurs the next message is transmitted. When the end of the list is reached, transmission may resume at the start of the list, depending on the configuration.

Since generator blocks can be configured to be very elaborate, the popup menu offers you the option of saving a generator block as a file and later reloading it, possibly from a different configuration. As a result, you can easily exchange generator blocks between different configurations.

Generator blocks appear in the data flow plan as small blocks with the label *GB*.

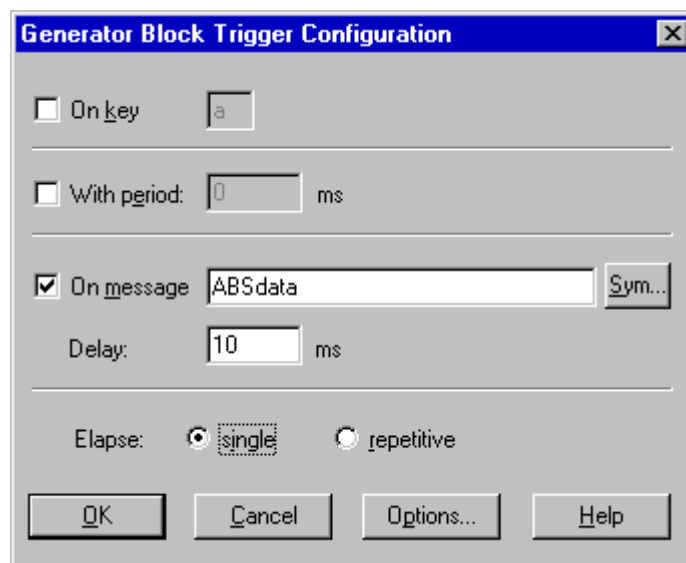


Figure 57: Generator Block Configuration - Triggering

Note: To have a generator block send data onto the bus, you must insert it in CANalyzer's transmit branch. If you insert it in the evaluation branch the data will indeed be sent to the evaluation blocks to the right and be displayed in the appropriate windows. However, they will not reach the bus due to the left-to-right directional data flow.

2.6.5.1 Configuration of Triggering

In the popup menu select the item *Triggering* to define a trigger condition for transmission of messages.

You can select one or more of the three possible conditions by marking the appropriate check box(es). For *On Message* you indicate which identifier should trigger transmission, and the delay that should occur. For *On Key* you indicate the key that should trigger; and for *On Period* the period in milliseconds.

In the line *Flow* you define whether the transmit list should be run through exactly once or cyclically.

2.6.5.2 Configuration of Transmit List

In the popup menu select the item *Transmit list* or double click the generator block to create the list of messages to be transmitted:

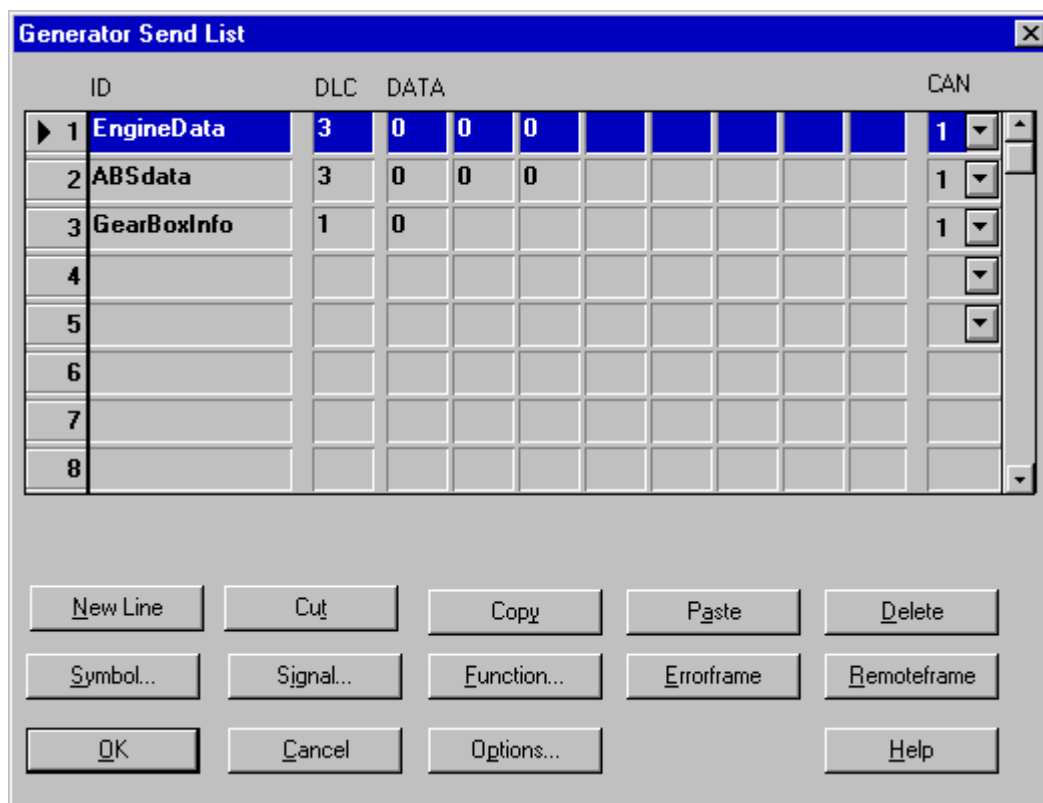


Figure 58: Generator block Configuration of Transmit list

When one of the trigger conditions occurs, the next element of this list is transmitted. When the end of the list is reached the program resumes with the first element if the run mode has been set to *cyclic*. The list may also consist of only one element.

Nine lines are displayed in the dialog box. The *active line* - to which the dialog buttons refer - is identified by the "▶" symbol at the beginning of the line. The active line is shifted automatically by activating the <TAB> key or by clicking the dialog entry boxes with the mouse.

Each line of the list consists of 11 columns. In the first column you enter the desired identifier. The DLC field defines the number of data bytes in the message. After that come the fields for data bytes. Only the number of bytes specified by the DLC are accepted. The rest are ignored. The last column is a combination box for selecting the controller by which the message is to be transmitted.

2.6.5.3 Entry of Signal Values

You can configure the generator block symbolically, provided that you are working with a database. Use the option button *Symbol...* to select a message from the database which is to be transmitted from the generator block.

Then the physical signal values of the message can be defined with the *Signal...* button.

Displayed in the signal value input dialog are the individual signals of the message that was on the active line of the transmit list. If the message only contains standard signals, the signal list consists of three columns. The individual signals are listed in the lines. In the last column you can prescribe a signal value. If you have assigned a

value table to the signal in the database, you can use the relevant symbolic descriptor instead of a numeric value. You can select this from the signal value table in the middle column.

Physical dimensions are stored in discrete form in the CAN messages. However, it may not always be possible to represent the numeric value entered in the *Value* box as a discrete value. In such cases, when exiting the line or activating the *OK* button, the two next possible physical values are displayed in a dialog. Then the entered value is rounded to the next possible closest value.

Example:

The signal EngineData.RPM is defined as a 16 bit unsigned with an offset of 0 and a factor of 10. If the value to be compared is entered as 1015, the raw value would have to be $1015/10 = 101.5$. Since only discrete values may occur, either 101 or 102 must be used, which correspond to the physical values 1010 or 1020. It is these two numbers which appear in the dialog.

In spite of discrete memory storage, quantities which have digits after the decimal point can be valid. In the example above, if a factor of 10.5 is used for calculation 1008 and 1018.5 are recommended as possible values.

2.6.5.4 Entry of Mode-Dependent Signals

Generally there is a direct, fixed assignment of a message's data segment to a signal. However, for multiplex messages different signals are transmitted in the data segments, depending on a mode signal. Only a subset of all possible signals of the message is defined for a mode value.

In the dialog for inputting signal values, the standard signals and mode-dependent signals are shown in two separate list boxes for ease of input. Additionally, all those signals not defined in the active mode are filtered out in the mode-dependent list box.

The *Mode signal* and the *Mode* are displayed in the associated Edit boxes. If the mode value is changed, the list with mode-dependent signals is reconstructed. The mode signal itself cannot be changed here.

2.6.5.5 Function Generator for the Transmit List

It is often the case that the user wishes not only to place certain signal values on the bus, but rather entire signal responses. The generator block has a signal response generator for this purpose, which you configure with the *Response...* button.

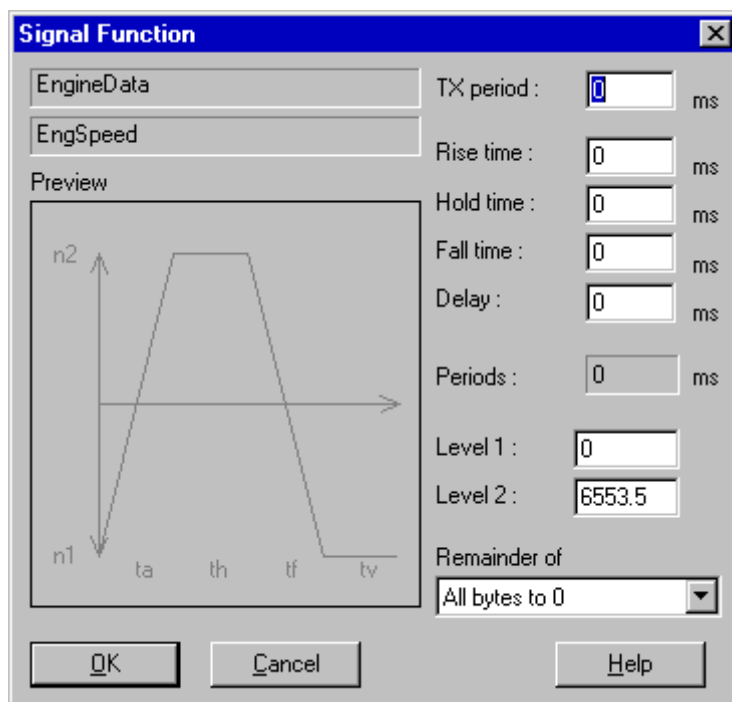


Figure 59: Signal Response Generator

The configuration dialog allows you to parameterize a trapezoidal function. When the dialog is exited with *OK* the corresponding lines are automatically generated in the generator block transmit list. The following signal responses can be generated with certain parameters:

Signal response	re-Parameter
Square	$ta = tf = 0$
Triangular	$th = tv = 0$
Saw-tooth	$th = tf = tv = 0$ or $ta = th = tv = 0$
Constant	$n1 = n2$

Displayed in the upper part of the dialog box are the message and the signal. Shown below this, in the preview box, are a trapezoid and the meanings of its parameters. By setting individual parameters to zero, the following responses can be generated:

The levels $n1$ and $n2$ must be entered physically. The relevant limitations apply here (see for example generator block signal values).

The entry for transmit interval identifies the interval between any two messages, and this corresponds to the entry in the generator block dialog *Trigger initiation period*. Since CAN is message-based and not signal-based, all signals of a message get the same transmit interval!

Since this signal characteristic generator is only a tool for the generator block and is not a block itself, the following must be observed when using different period lengths for several signals within one message:

The generator creates a list of messages with:

$$\text{No. of messages} = \text{Period} / \text{Transmit spacing}$$

This is exactly one period. If the transmit list contains more they are rejected. Using the combination box *Remainder of* the user can define how remaining signals are to be handled for supplementally generated messages:

All bytes to 0:

All signals are set to the raw value 0.

Continue cyclically:

During generation the previous messages are copied as often as necessary until the new period length is reached.

Example:

The original list contains 3 messages, and the new period length requires 9 messages. The new signal (byte 1) is a saw tooth.

Original Transmit List:	New Transmit List:
0 0 0 4 0	1 0 0 4 0
0 1 0 2 0	2 1 0 2 0
0 2 0 4 0	3 2 0 4 0
	4 0 0 4 0
	5 1 0 2 0
	6 2 0 4 0
	7 0 0 4 0
	8 1 0 2 0
	9 2 0 4 0

The recommended procedure for generating multiple signals is therefore as follows:

The signals are defined beginning with the shortest period length. Then only integer multiples of this period length are used.

2.6.6 Interactive Generator Block

The purpose of the interactive generator block is to generate and transmit messages.

Messages can also be configured and interactively transmitted during a measurement (Online). This makes the interactive generator block especially well suited for influencing a measurement in a quick and improvised way. In many cases, with the interactive generator block you can achieve your goal without the use of traditional generator blocks and without CAPL blocks.

Interactive generator blocks appear in the data flow plan of the measurement setup as small blocks with the label *IG*. Just like traditional generator blocks they are permeable to all data in the data flow diagram. That is, they do not filter the data flow like filter blocks or CAPL blocks do, rather they act in a purely additive manner.

Tip for Working with CANalyzer:

In each of your CANalyzer configurations, insert an interactive generator block in the transmit branch of the measurement setup. This provides you with a flexible tool, even during a measurement, with which you can spontaneously influence the message traffic on the bus.

For examples of interactive generator block implementation please refer to the CANalyzer Help function.

2.6.6.1 Configuring the Interactive Generator Block

To configure the interactive generator block you use a non-modal configuration dialog, in which you input a transmit list of messages with trigger conditions. Each message is assigned a signal list, in which signal values can be configured. All lists are taken from an associated database.

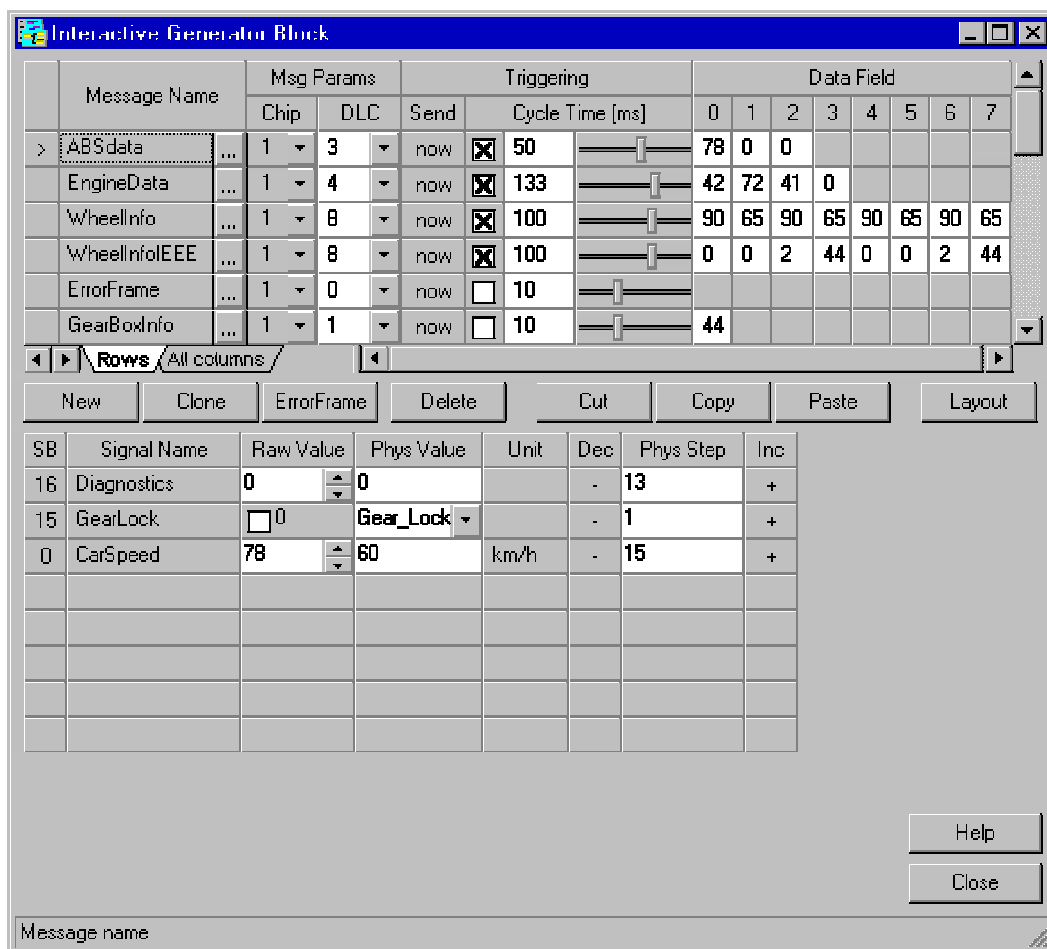


Figure 60: Configuration of the Interactive Generator Block

Appearing at the top of the configuration dialog is a list of messages to be sent - the transmit list - with the message name, message parameters, trigger conditions and data field of the particular message. Below this, a list of signals appears for the active message (marked on the left side of the transmit list with ">"). In this signal list it is easy for you to configure the values of individual message signals.

All input boxes are explained in the status bar at the lower border of the dialog. The explanation is always shown for the currently selected box entry. There are also keyboard shortcuts for use without the mouse; these are described in section 2.6.6.7.

2.6.6.2 Transmit List

The list of messages to be sent (Transmit list) is available in two views: *Rows* and *All Columns*. In the *Rows* view the most important columns of the transmit list are summarized in an easy-to-read format, while in the *All Columns* view even seldomly used message parameters and trigger conditions appear. The widths of the columns can be adjusted from their borders. The *Layout* button is used to have the columns automatically configured to the most easy-to-read layout.

The *New* button is used to input messages in the transmit list. The procedure differs depending on whether or not a database is associated to the CANalyzer configuration:

If there is a database, messages from the database can be added to the transmit list. Clicking the *New* button opens the Message Explorer with which messages can be selected symbolically. They are then assumed in the transmit list. Afterwards the signals of the messages can be configured individually in their signal lists. An alternative to activating the *New* button is to double click with the left mouse button in the first empty line of the message name column of the Message Explorer.

Without a database you can transmit any desired messages by manually entering the CAN identifier of the desired message in the transmit list (*Identifier* column). To generate a new message click in the first empty cell of *Identifier* column with the left mouse button or press the *New* button. After selecting the desired number of data bytes for the message (*DLC* column) you can then input the data bytes (*Data field* columns). The signal list remains empty.

Note: The *ErrorFrame* button can be used to enter an Error frame in the transmit list. This allows Error frames to be transmitted on the bus. Remote frames can be created from normal messages in the transmit list, except that in the *Frame* message parameter column the description *Remote* is selected instead of *Data*.

2.6.6.3 Trigger Condition

The trigger condition (*Triggering* columns) is, in contrast to the traditional generator block, entered separately for each message.

You can choose between manual interactive triggering (*Send* button), key press (*Key* column) and interval-driven repetition (Timer; *Cycle time* column [*ms*]). Additionally, the user can configure the number of messages to be sent at the time of triggering (*Burst* column). The default value for Burst is 1, i.e. exactly one message is sent per triggering.

2.6.6.4 Generating a High-Load Situation

A high load situation can be initiated with the *High Load* trigger condition. With this type of triggering, as soon as a message has been transmitted successfully, the next transmit request for the same message is already shifted in again. This continuously refills the transmit queue on the chip, so that it is filled with the number of messages set in the Burst column. This results in the high bus load.

Note: When high load is configured no other simultaneous triggering of the same message should be performed manually by keyboard or by timer, since the transmit queue might overflow. Furthermore, the associated receive acknowledgments (Tx or TxRq events) in front of the interactive generator block in the data flow plan must not be filtered out; otherwise the transmit queue will not be refilled.

2.6.6.5 Entering Signal Values

The list of message signals is taken from the database from which the particular message originates. If no database is associated, the list remains empty.

The list contains - for each signal - the signal name, raw value, physical value and units. The signal name and units are taken from the database. The raw value and physical value are adjustable. Possible value ranges are shown in the status bar at the bottom of the dialog.

Whenever an entry is made the raw value and physical value are adjusted to one another using the signal's conversion formula. As a result, you only need to set the value you prefer, and the other value will be calculated automatically. If a physical signal value is entered whose raw value equivalent lies between two raw values and therefore cannot be represented, it is automatically rounded up or down to the next closest value.

If the signal is of the type *Enumeration* (Enum), the defined value descriptions are presented to you in a selection list in the input box for selection. This permits symbolic selection of the signal value. For bit signals a check box is provided, which makes it easy to toggle the bit value.

The three columns on the right represent a decrementer and incrementer, with which the physical signal value can be easily changed by an amount that is set in the *Phys. Step* column. Shown at the far left is the start bit *SB* of the signal within the message.

2.6.6.6 Entering Mode-Dependent Signals

As a rule, a message's data segment has a direct, fixed assignment to a signal. However, with multiplex messages different signals, so-called mode-dependent signals, are transmitted in the data segments as a function of a mode signal. The mode signal carries the active mode value that forms the basis for the currently valid mode-dependent signals. That is, only a portion of all possible signals of the message are defined for a given mode value.

In the configuration dialog for the interactive generator block mode signals and mode-dependent signals of a message appear highlighted in color in the signal list. Mode signals are light green, and mode-dependent signals are shown in dark green.

When the mode is changed in the mode signal, its associated signals are shown. All signals not defined in the currently set mode are filtered out.

2.6.6.7 Keyboard Control

The following keyboard shortcuts apply to work in the interactive generator block:

<i>Spacebar</i>	Toggles the states of check boxes and activates the Send buttons. In message name boxes the key opens the Message Explorer, with which another message is entered.
Arrow keys (←,→,↑,↓)	Navigates through the input boxes of the dialog. The box entries are explained in the status bar at the dialog's lower border.
<Ctrl><↑> <Ctrl><↓>	Increments or decrements the value of the active box (by 1 for timers and data bytes of a message, and by the lowest possible raw value difference for signal values on a signal line)
<Ctrl><←> <Ctrl><→>	Logarithmic calibration of the slider in the timer columns of the transmit list
<Page ↑> <Page ↓>	Increments or decrements a signal value on a signal line by the configured step width (Phys. Step).
<Esc>	Exits the editing box, whereby any change is canceled and the previously set value is restored. Attention: <Esc> during a running measurement will terminate the measurement!
<F9>	Starts the measurement

While the configuration dialog is open and selected by the insertion point, all keyboard entries except <F9> and <Esc> are used for editing within the dialog. A transmit triggering of the generator blocks or activation of CAPL program nodes cannot be performed by keyboard press until the insertion point is moved to one of the main windows of CANalyzer or the test mode is explicitly enabled in the configuration dialog.

2.6.7 Break

If certain branches of the data flow in the measurement setup should not be run through, then a hotspot can be converted to a breakpoint. This is advisable, e.g. in online mode, if all functions (above all in the Trace window) cannot be serviced any longer without loss of data due to a high data rate.

When a break is created the configuration after the break is fully preserved, so that the old state can be reinstated after the break is deleted. Therefore, the break provides a very quick means for temporarily disconnecting certain data paths and thereby saving computer time.

At the start of a measurement the currently valid data flow plan is converted to an internal tree structure. In this conversion process breaks which are encountered in a path are propagated forward to the next branching point. Therefore, it is irrelevant to processor loading whether a break is placed at the front or back of a path. The same path is always masked out.

2.7 Logging and Evaluation of Measurement Files

CANalyzer offers you the option of saving the CAN data traffic in a log file, so that you can evaluate it later in Offline mode.

Logging blocks are provided to you for this purpose. The task of a logging block is to store data arriving at its input to a file. You can configure the log file in the measurement setup by the file symbol icon at the far right in the logging branch.

Each logging block is equipped with user-friendly triggering to reduce the amount of data as much as possible even before acquisition. This permits the formulation of a trigger condition, and data are only saved near the time of the trigger. During each measurement multiple triggers can be initiated for various events, whereby the user can prescribe pre-trigger and post-trigger times. The trigger condition is user-programmable. You can configure triggering in the measurement setup via the *Logging* function block.

CANalyzer has an Offline mode for analyzing log files. In contrast to Online mode the data source here is a file, e.g. a file created by logging in Online mode. All measurement and evaluation functions of Online mode are also available to you in Offline mode.

2.7.1 Logging Trigger

You can open the configuration dialog for the logging trigger by clicking the logging block in the data flow plan or by selecting the logging block with the cursor keys and then pressing the Enter key.

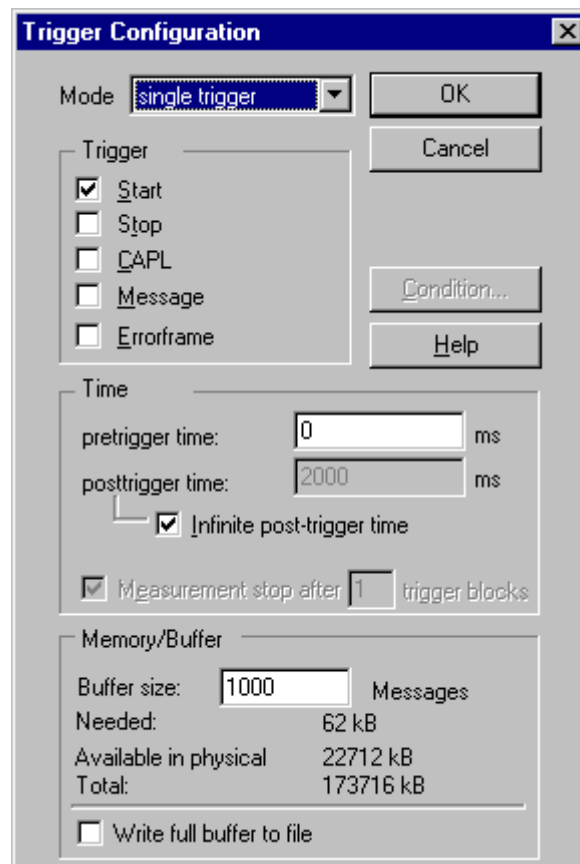


Figure 61: Trigger Configuration Dialog

2.7.1.1 Trigger Modes

You can operate the triggering mechanism for logging in three modes:

- Entire Measurement
- Single Trigger
- Toggle Trigger

In the *Entire Measurement* mode the entire measurement is logged. Pre-trigger and post-trigger times cannot be selected. Activating the button *Write full buffer to file* causes a full data buffer to be saved intermediately for long measurements. You can avoid data loss in this manner.

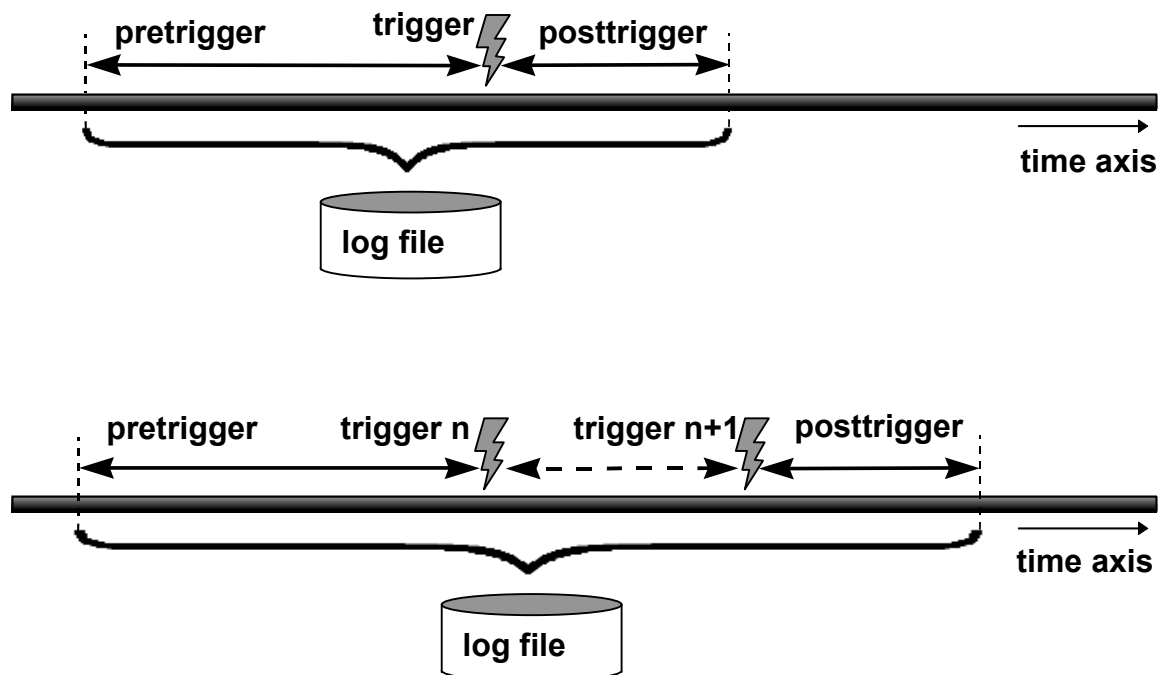


Figure 62: Time Window for the Trigger in Single Trigger Mode (top) and in Toggle Trigger Mode (Bottom)

In *Single Trigger* mode the user-definable time window between the pre-trigger and post-trigger times is set around each trigger. Pre-trigger and post-trigger times then relate to the same trigger. All data falling within this time window are recorded in the log file.

In *Toggle trigger* mode the time window is described by two successive triggers (start-of-block and end-of-block triggers). The first trigger activated during measurement is the start-of-block trigger, and the second is the end-of-block trigger. Afterwards, another start-of-block trigger follows, etc. The pre-trigger time in toggle trigger mode is referenced to the start-of-block trigger, and the post-trigger time is referenced to the end-of-block trigger.

Figure 63: Log file with 2 trigger blocks. Pre-trigger: 50ms, post-trigger: 100ms, Trigger types: Message *GearBoxInfo* and Error Frames

You can determine whether the trigger action should be executed once or multiple times. If the option *Measurement stop after n triggers* is selected in the trigger's configuration dialog, the measurement process is stopped after the post-trigger time of the nth trigger has elapsed.

You can define trigger events using the five check boxes in the configuration dialog. The following types are available to you:

- Trigger on start. Triggering occurs unconditionally and immediately at the start of measurement. The logging time period is defined exclusively by the post-trigger time in this case.
- Trigger on stop. Triggering is executed at the end of the measurement. Triggering can be activated by the CAPL function *stop()* or by the <Esc> key. The logging time period is defined exclusively by the pre-trigger time in this case.
- Trigger by a CAPL program: Using *CAPL programming* methods (see section 3.8.3) arbitrarily complex conditions can be formulated, which can depend on the occurrence of various events. Triggering is activated when the intrinsic function *trigger()* is called by a procedure.

- Trigger when certain messages occur. Additionally, attributes can be selected in dialog boxes, and masks can be entered for ID's and data bytes, etc. A separate section (see section 0) is devoted to these trigger conditions, which can also be used for searches in offline mode. Triggering is activated when one of the conditions specified there occurs.
- Trigger on error frames. Triggering is executed whenever an error frame occurs.

2.7.1.3 Time Window

In the configuration dialog you also define the time window (the pre-trigger and post-trigger times in milliseconds) around each trigger.

The pre-trigger time defines the time interval before activation of the trigger condition which is to be saved. If the trigger condition occurs so early that the pre-trigger time is greater than the time since the start of the measurement, only those data occurring prior to the trigger can be saved. Valid pre-trigger times must lie in the range of 0 to 180000 ms.

The post-trigger time indicates how long data continue to be acquired and saved after the trigger condition occurs. The measurement is terminated automatically after the post-trigger time has elapsed. Valid post-trigger times must lie in the range of 0 to 180000 ms.

2.7.1.4 Configuration of the Logging Buffer

CANalyzer initially saves the data arriving in the logging branch to a ring buffer in the PC's main memory.

In the last line of the dialog you can define the size of the buffer in which events are saved intermediately during a measurement. Approximately 50 bytes of memory is taken per message; however, this memory is only allocated as needed during the measurement. Consequently, for very large buffer sizes Windows may swap-out portions of the main memory to the hard drive during the measurement, and this may lead to significantly delayed program execution.

After the trigger condition occurs the ring buffer is filled with raw data until the post-trigger time has elapsed. The user can define whether the triggering process should be run once or multiple times. If the option "Measurement stop after n trigger blocks" is chosen, then the measurement process is stopped by the logging block after the post-trigger time of the nth trigger has elapsed. At that time it transfers data from the ring buffer to the actual log file. Depending on the quantity of data, this memory transfer process may involve substantial waiting times.

Since the ring buffer is limited in size, data can be lost if the pre-trigger and/or post-trigger times are too long. Then only the last data are recorded up to the point where the post-trigger time elapsed, and an error message is output. If necessary the measurement can be repeated with a data reduction filter inserted upstream.

Note: If you wish to write all data to file for a long-duration measurement you should select the option *Write full buffer to file*. This option causes a full buffer to be written to file during the measurement. The buffer size is of decisive importance with this option. To prevent excessive system loading during saves to the buffer, and to prevent data losses, the buffer should not be configured to be very large. Buffers of approx. 1000 messages have proven effective in practice. This button is only accessible in *Entire Measurement* mode or in *Single Trigger* mode with the trigger type *Start*.

To prevent data losses, especially at high system loading, you should observe the following points:

- Close all applications that run in background and demand system time;
- Switch all blocks in the measurement setup over to *Cyclic Update* or disable them entirely.
- Utilize filters for data reduction.
- Do not execute any unnecessary user actions during the measurement, such as moving windows.

If a data loss occurs in spite of this, you can make this apparent to the user with the option *Data Lost Message Box* in the Logging dialog. This option results in a dialog box being displayed after the end of measurement if data loss occurs. If no message appears you will know that all data were logged.

With the help of this option the corrupt locations in the log file are marked with a ‘*’ as a special symbol. The symbol for data loss only disappears when the overload condition has ended.

2.7.2 Log Files

The icon for the log file (file symbol) indicates that the data flow ends in a file.

Choosing the menu item *Log file* in the popup menu of the File icon to the right of the Logging block opens the configuration dialog for the log file.

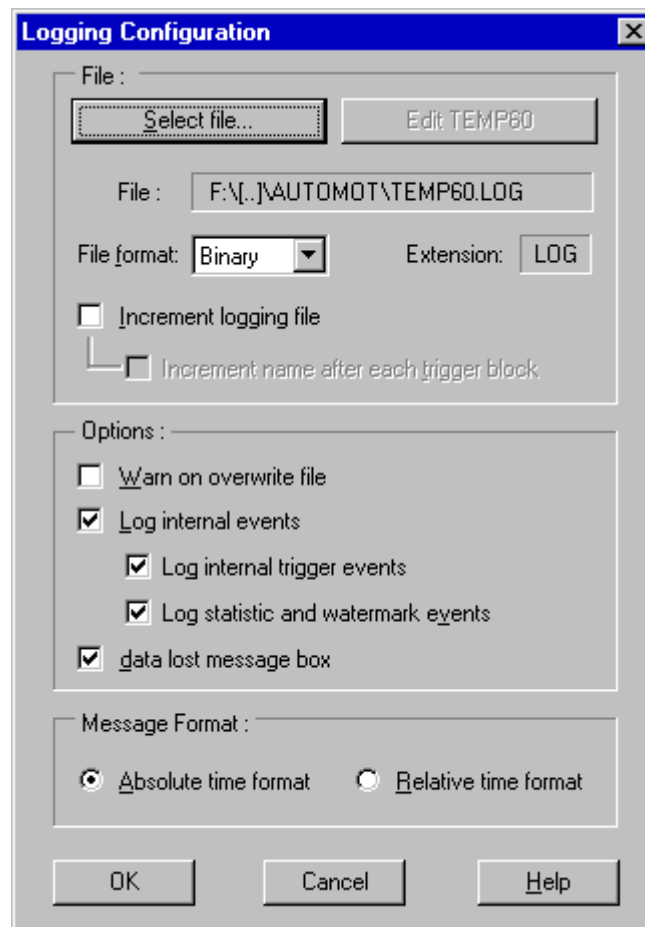


Figure 64: Configuration of the Log File

The format of the log file is defined by the *File format* combination box. The user can choose between *binary* and *ASCII* format. The *binary* format is recommended for online measurements, since this format is faster in offline evaluation and also generates smaller log files. When ASCII format is chosen, the data are saved as readable text. The setting of the global switch determines whether decimal or hexadecimal representation is used. Among other things, ASCII format can be used for data exchange with external programs or for incorporating trace data into documents.

The Offline mode data source can be configured to be either binary or ASCII files.

The automatically preset extension of the file name is .LOG for a binary log file or .ASC for an ASCII file. The recommended default name is CANWIN.LOG or CANWIN.ASC.

To view or edit an ASCII file, double click the file icon or press the *Edit file* button in the configuration dialog for the log file. You can use your own text editor for this. To do this, enter the following line in section [Environment] of the file CAN.INI:

```
LogEditor=MYEDITOR.EXE
```

whereby you must enter the name of your own editor for MYEDITOR.EXE.

You can configure logging, such that the selected log file name is automatically incremented after the end of measurement or at the end of a trigger block. This prevents overwriting files that already exist:

Example:

Log file before the measurement: C:\CANWIN\MESS_01.ASC

Log file after the measurement: C:\CANWIN\MESS_02.ASC

In the configuration dialog you can indicate whether data losses in overload situations should be reported to you (cf. section 4.2). In the log file, the faulty lines are marked with a '*' as a special symbol.

Note: Analogous to output in the Write window with the CAPL function `write()`, you can output text lines from CAPL programs to ASCII log files using the functions `writeToLog()` and `writeToLogEx()`.

2.7.3 Event Types in Log Files

In the following table you will find an overview of all events that are recorded in the log file. When the function *Log internal events* is active in the data icon's configuration dialog the internal events generated by the program (e.g. bus statistics information, triggers, etc.) are also logged. To have bus statistics information generated the relevant option must be activated in the *Card and Driver Options* dialog. There you also set how frequently these events should be generated.

In the first column you will find all event types that are recorded by logging. In the second column is the format of the particular event in ASCII logging. The third column provides information on whether the event can also be recorded in binary MDF log files. You can determine from the last column whether the function *Log internal event* must be activated to log the event.

Event type	Format in ASCII Logging	Binary	Comments
CAN message	<Time> <CAN> <Name or ID> <Attributes> <DLC> <Data>	Yes	
Error Frame	<Time> <CAN> ErrorFrame	Yes	
Overload-Frame	<Time> <CAN> Overloadframe	Yes	
CAN error	<Time> CAN <CAN> Error: <Error message>	Yes	Internal event
Bus statistics	<Time> <CAN> Statistic: <Data>	Yes	Internal event
Measurement start	<Time> Start of Measurement	No	
Logging Trigger	Statistic: <Time> log trigger event	Yes	Internal event
Trigger block Start	Begin trigger block <Time and date>	No	

Event type	Format in ASCII Logging	Binary	Comments
Trigger block End	End trigger block <Time and date>	No	
Overload symbol	'*' at beginning of line after the data loss occurred.	No	Internal event, cf. section 4.2

2.7.4 Data Analysis in Offline Mode

To study recorded log files switch CANalyzer to Offline mode with the main menu item *Mode|To Offline*. Figure 65 shows the corresponding data flow diagram.

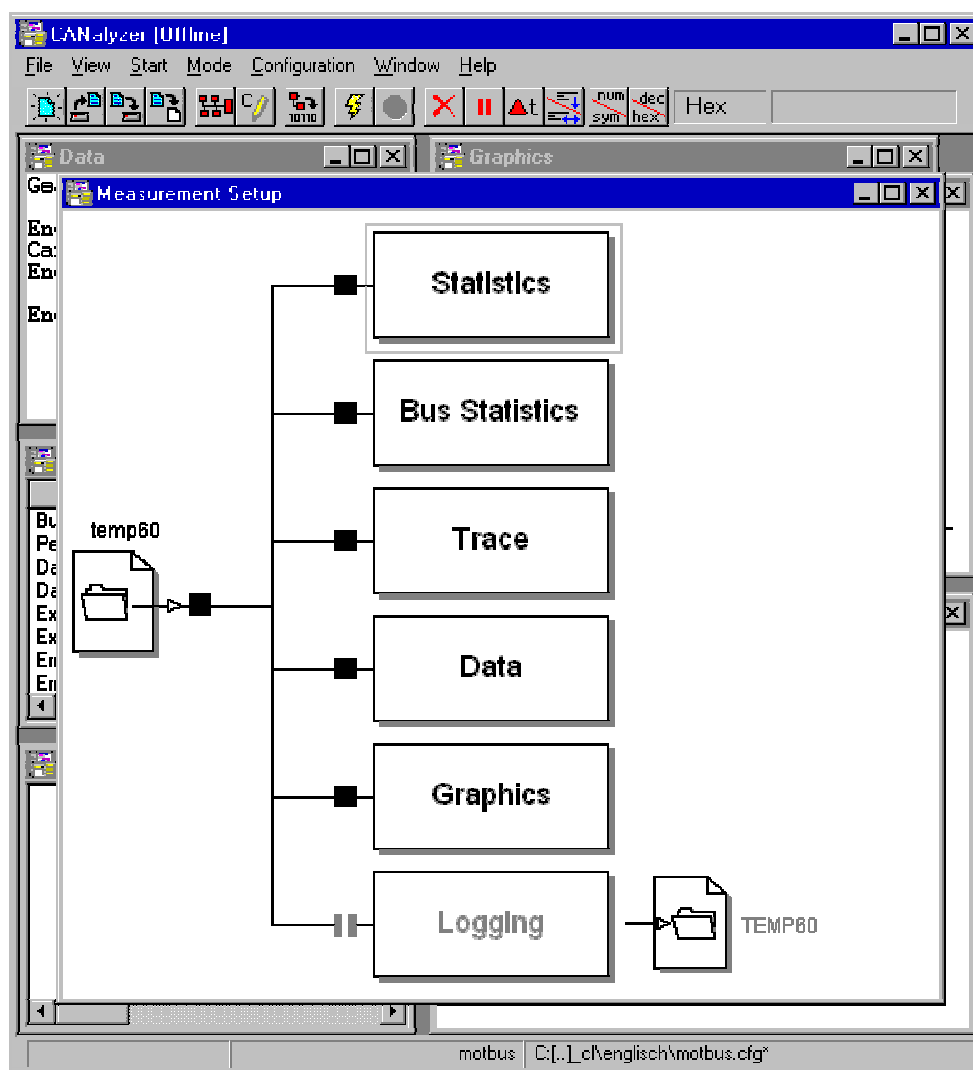


Figure 65: Data flow diagram for the Offline Mode

The data source in Offline mode is a file, e.g. generated by logging in Online mode. Analogous to Online mode, all measurement and evaluation windows are also available to you in Offline mode. The only option omitted is the possibility of sending data over the bus. Furthermore, Offline mode provides a powerful search and break function, with which you can intentionally stop the replay of the log file. This is described

in section 2.7. In the logging block, which is also available in Offline mode, data can be resaved to a new file, whereby targeted data reduction can be achieved by means of insertable data manipulation blocks.

You can enter the name for this file under the *Configuration...* item in the data icon's popup menu on the left side of the offline measurement setup. CANalyzer supports both binary and ASCII logging formats for this.

The menu item below this, *Break conditions...*, opens a dialog in which you can set an interruption point – a *breakpoint*. When the condition you have specified occurs, the offline replay is interrupted until you resume replay of the file with the function *Run* (F9), *Animate* (F8) or *Step* (F7).

You can configure the break condition with the *Conditions...* button, which is described in more detail in section 2.7.5. If the tools provided in the configuration dialog are inadequate, the CAPL language – with the function **stop()** – allows you to program a breakpoint yourself.

2.7.4.1 Flow Control in Offline Mode

The following functions of the main *Start* menu are available to you in Offline mode to track the recorded bus proceedings on the screen in slow motion:

Start

The individual messages of the data source are read-out and are sent as quickly as possible through the components of the measurement setup. In Offline mode the measurement can be resumed after a break. *Reset* must be called for a new start.

Reset

After a measurement has been run through either partially or completely in Offline mode, it can be reset to the beginning again with *Reset* and can thereby be studied from the beginning again.

Animate

Instead of reading data from the source file as quickly as possible, in Animate mode only about 3 entries per second are read from the source file. This results in a slow-motion representation of the processes. All of the display windows may be active during this process, e.g. so that the user can easily observe the sequence of messages in the Trace window. The Animate run can be interrupted by the <Esc> key or the menu command *Start|Break*. The speed of the Animate mode can be set with the following line in section [Offline] of the file CAN.INI:

```
AnimationDelay=nnn
```

where *nnn* describes the time between the replay of two successive events in milliseconds. (The default value is 300 [ms])

Break

In Offline mode this menu item interrupts the replay of data from the source file (Animate run). The same can be achieved by the <Esc> key. A restart resumes the replay at the point where it was interrupted by Break.

Step

This menu item (or the <F7> key) causes a single step of the measurement to be run. Each time this is activated only one additional message is read from the log file and processed in the data flow plan.

Within a measurement the user can switch back and forth whenever desired between *Start*, *Animate* and *Step*.

2.7.4.2 Configuration of Online and Offline Modes

In principle, there are different measurement setups and completely separate parameter sets for Online and Offline modes, so that the two modes - including their window layouts - can be configured differently.

Nevertheless, it is often the case when switching between Online and Offline modes that the user would like to assume the configuration settings made in one mode in the other mode. Therefore CANalyzer provides the functions *Online(Copy)* and *Offline(Copy)* in the main *Mode* menu.

Online

Toggles to Online mode. In Online mode a data flow plan is shown in the measurement setup window, whereby the CAN PC-card serves as the data source. Moreover, there is also a transmit block. The time basis in Online mode is real time, and clocks in CANalyzer and on the CAN board are synchronized at the start. All time data are referenced to the start of measurement. Data acquisition cannot be resumed after a break in Online mode, rather it may only be restarted. The Animate mode is not possible.

Online (Copy)

Toggles from Offline mode to Online mode, whereby the data flow plan of Offline mode is assumed with all of its function blocks. The corresponding portion of the last Online configuration is lost in the process. Since the Offline mode does not have any transmit branch, that section remains unchanged.

Offline

Toggles to Offline mode. In Offline mode a data flow plan is shown in the measurement setup window, in which a data icon serves as the data source. Evaluation of this file is begun by *Start*, and the evaluation can be resumed after a break with <ESC>. Animate mode and Single-Step mode are also possible. The time basis in Offline mode is based on the times recorded for the data in the file.

Offline (Copy)

Toggles from Online mode to Offline mode, whereby the portion of the data flow plan of Online mode located after the branch to the transmit block is assumed. This affects all window configurations and function blocks. The last Offline configuration is lost in the process.

The mode switchover functions with copy represent a user-friendly method for assuming elaborate options such as trigger conditions, data display configurations or CAPL programs, and it allows the user to begin immediately with analysis or a new recording.

2.7.5 Trigger and Search Conditions

Conditions can be formulated for triggering logging and for interrupting an Offline measurement. These conditions are always input in the same dialog.

2.7.5.1 Condition Primitives

A condition is composed of condition primitives that can be combined with logical *or* or *and*.

Example:

1.Primitive: ID > 100

2.Primitive: D0 == 0

A logical *OR* combination results in all those messages being found whose IDs are greater than 100 or whose first data byte is 0. For a logical *AND* combination all those messages are found whose IDs are greater than 100 and whose first data byte is 0.

The *Define* and *New Prim.* buttons are provided (only when a symbolic database is used) for entering a new primitive.

The dialogs for entering and modifying primitives differ, depending on whether you are working with a symbolic database (*New Prim.* button) or not (*Define* button). The desired attributes can be input in the upper line. The identifier and data bytes are input below this. Furthermore, a time window can be specified within which the search is to take place.

2.7.5.2 Entry or Change of Primitives (without Database)

The same rules apply to both identifiers and data bytes. There is a toggle switch (*Relation*) in the column after the object name, which defines the desired relation. If it is empty, the object is not considered. If a relation is to be made, the user selects one of the following symbols:

Symbol	Meaning
==	equal to
!=	not equal to
<	less than

Symbol	Meaning
<=	less than or equal to
>	greater than
>=	greater than or equal to

After this is a mask to be executed, and the user must choose the numbering format to be used:

Number format: Decimal

The desired value is entered here as a decimal number. To find the identifier 100 the following is entered:

ID == 100

The entry is illegal if there are still X (undefined) or \$ (invalid) characters in the input box.

Number format: Binary

A mask is entered here which specifies bits to be set, bits not to be set, and bits that are masked out, i.e. should not be considered. The characters have the following meaning:

X	Masked out
1	Bit is set
0	Bit is not set

To find all messages whose first data byte is odd, the following must be entered:

D0 == XXXXXX1

Note: It is possible to use >, < etc. in binary mode, but this is difficult to interpret in mask conditions, and is only of interest to users who have a knowledge of precise machine-internal data representation.

Number format: Hexadecimal

Here also masks can be specified. Integers can be entered in hexadecimal format, or nibbles (half bytes) can be masked out by X. The character \$ represents invalid. For example, it can result in the following case:

XXXXXXX1 is entered in binary mode, then a switch is made to hex mode. The four X's on the left are converted to an X as a nibble. The lower values XXX1 cannot be interpreted as a completely defined nibble. Therefore, the result is X\$.

2.7.5.3 Entry of Change of Primitives (with Database)

When a symbolic database is used it is possible to trigger to messages and signals.

The *SYM* buttons *Message name* and *Signal name* after the text input boxes are used to call the symbolic selection dialog, in which you can select the message - and if applicable also the signal from the database - to which triggering should occur.

If you only enter a message name in the symbolic selection dialog, logging is triggered whenever the message occurs on the bus.

If you select a signal name in the input box below this, a decision is made based on the signal value regarding when logging should be triggered. The comparison of signal values is made based on their physical interpretations. This involves choosing a comparison operator and entering a numeric value.

Physical dimensions are stored in discrete form in the CAN messages. Consequently, the specified numeric value may not always be capable of being mapped to a discrete value. In such cases, after pressing the Enter key or the *OK* button the two next possible physical values are displayed in two boxes below your entry.

Example:

The signal *Enginedata.rpm* is defined as a 16-bit unsigned value with an offset of 0 and a factor of 10. If 1015 is input as the comparison value, the raw value would have to be $1015/10 = 101.5$. Since only discrete values may occur, either 101 or 102 is used, which corresponds to physical values of 1010 or 1020, respectively. It is these two values that appear in the two information boxes.

In spite of discrete saving, valid values may still have digits after the decimal point. If a factor of 10.5 were used in the above example, then 1008 and 1018.5 would be recommended as possible values.

You can also enter a time range for each primitive, within which it should be active. Outside of this time range the condition would never be satisfied.

2.7.6 Exporting and Converting Log Files

The contents of log files can be exported or converted to other file formats with the help of signal-based Logging Export. The configuration dialog for Logging Export can be opened by the *Export* menu item in the popup menu of the File icon to the right of the Logging block.

2.7.6.1 Export

The user can limit the export to specific signals. This involves selecting the desired signals in the *Signals* selection list.

In the *Expanded options* dialog that is opened by pressing the *Expanded* button in the Logging Export Configuration dialog, the user can define in the *Actions* box those programs that can be started after an export.

2.7.6.2 Conversion

Conversion of log files is supported in both directions, i.e. ASCII->Binary and Binary->ASCII.

3 CAPL Programming

3.1 Overview

The universal applicability of CANalyzer results in large measure from its user programmability.

The CAN Access Programming Language CAPL is a C-like programming language, which allows you to program CANalyzer for individual applications. In the development of network nodes, for example, the problem arises that the remaining bus nodes are not yet available for tests. To emulate the system environment, the data traffic of all remaining stations can be simulated with the help of CAPL.

You can also write programs for problem-specific analysis of data traffic with CAPL, or you can program a gateway - a connecting element between two buses - to exchange data between different CAN buses.

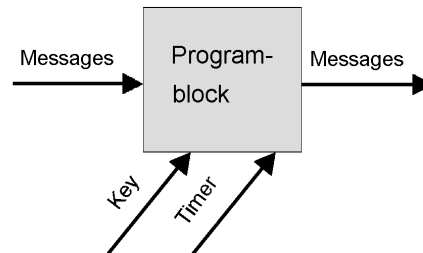
CAPL nodes are inserted in the data flow plan as function blocks. Event procedures serve as inputs in CAPL. These procedures can react to external events (e.g. the occurrence of specific messages). You send messages by calling the function **output()**. These language tools and symbolic access to the various variables in the database make it possible to create simple prototypical models of nodes. The event procedures can be edited in the user-friendly Browser.

We start with a discussion of potential applications of CAPL programs in section 3.1.1. If you are working with CAPL for the first time, section 3.2 that follows offers you an introduction to the programming language. You will find a small but complete example of creating a CAPL program in section 3.3. Section 3.3 then gives an overview of CAPL Browser, the tool for creating and modifying CAPL programs that is included with the standard product.

Finally, section 3.5 serves as a reference, providing a language description. All references in this manual to environment variables or CAPL functions for environment variables do *not* apply to CANalyzer. Environment variables are supported exclusively by CANoe.

3.1.1 Potential Applications of CAPL Programs

CAPL programs have an input through which messages pass as events into the block. Appearing at the output are all messages that either pass through the program or are generated by it. Furthermore, the program block can react to keyboard inputs (Key), time events (Timer) and - with CANoe - to changes in environment variables such as switches or slider positions.



Therefore, you can utilize a CAPL program to develop monitoring and testing for your special problem task. The CAPL program reacts to messages that CANalyzer registers on the CAN bus, and afterwards you can call your own analysis and test functions.

You can also use a CAPL program to emulate the system environment for a controller. The CAPL program reacts to both messages on the CAN bus and to your keyboard inputs, responding with certain CAN messages according to the event registered. It is entirely up to you to determine which actions are performed in response to which events.

Another possible application of CAPL is to program a gateway - that is a connecting element between two buses - to exchange data between different CAN buses and moreover to correct erroneous data occurring in this exchange.

Last but not least, the logging block can also be triggered by a CAPL program. Conditions of any desired complexity can be formulated for triggering. Triggering is initiated by a call of the intrinsic function **trigger()**.

3.1.2 Integration of CAPL Programs

A CAPL program can be inserted in the measurement setup at all hotspots. To do this, select the menu command *Insert CAPL node* from the hotspot's popup menu, and enter the name of the CAPL program file you wish to assign to this node in the configuration dialog. If you want to create a new CAPL program you can enter the name of a file that does not exist here yet. This file is then automatically created when editing.

You open the CAPL Browser by pressing the *Edit...* button in the configuration dialog or by double clicking the CAPL node. You can create and modify CAPL programs with this Browser.

Note: If you prefer to use your own editor to edit CAPL programs, enter it in the [Environment] section of the CAN.INI file.

Before you start the measurement you must compile all CAPL programs of the configuration. You can start the CAPL compiler from the CAPL Browser or from the configuration dialog. To compile all nodes at once, simply choose the main menu item *Configuration|Compile all nodes*.

Please note that a CAPL program may react completely differently, depending on the point at which you place it in the measurement setup. For example, a CAPL program

located to the right of CANalyzer's transmit branch can indeed generate messages, but it cannot send them on the bus. Since the data flow is directed from left to right, these messages are only passed to the function blocks to the right of the CAPL program. Only messages generated by CAPL programs located in CANalyzer's transmit branch⁴ can be sent out on the bus. This completely logical behavior - which may at first seem surprising - applies equally to the generator block, which - when it is located on the right side of the measurement setup - similarly generates messages without affecting the bus.

Therefore, in general those CAPL program blocks that exclusively serve analysis purposes should be inserted on the right side of the measurement setup, while program blocks for transmitting CAN messages should be inserted in CANalyzer's transmit branch.

3.1.3 Use of the Symbolic Database in CAPL

Just like other function blocks in the measurement setup, from CAPL you also have access to the symbolic information in the database. For example, instead of using the identifier 100 in your CAPL program you could use the symbolic name *EngineData* at all locations, provided that you have assigned this name to the identifier 100 in your database.

Use of the symbolic database makes your programs essentially independent of information that only relates to the CAN protocol, but has no meaning for the applications. Let us assume, for example, that during the development phase you determine that certain CAN identifiers in your system should be reassigned to change message priorities, and that in your system the message *EngineData* should now get the higher priority identifier 10 instead of the identifier 100.

In this case, let us assume that you have already developed test configurations and CAPL programs for your system which are exclusively based on the symbolic information (which do not use the identifier 100 anywhere, but rather always refer to the name *EngineData*). After modifying the identifiers in the database, you can incorporate the new information in the configuration by recompiling the CAPL programs. It is not necessary to adapt the CAPL programs to the new identifiers, since you only used symbolic names (e.g. *EngineData*), and not CAN identifiers (previously ID 100, now ID 10).

Therefore, it is advisable to manage all information relating only to the CAN bus in the database, and to use application-relevant symbolic information in CANalyzer exclusively.

3.2 Introduction to CAPL

CAPL is a procedural language whereby the execution of program blocks is controlled by events. These program blocks are known as event procedures.

The program code that you define in event procedures is executed when the event occurs. For example, you can send a message on the bus in response to a key press

⁴ CAPL programs which are inserted at hotspots to the left of CANalyzer's transmit branch can also send messages on the bus. The part of the data flow between the card icon and the branch to the transmit block must, in strict terms, therefore also be included as part of the transmit branch.

(**on key**), track the occurrence of messages on the bus (**on message**), or execute certain actions cyclically (**on timer**).

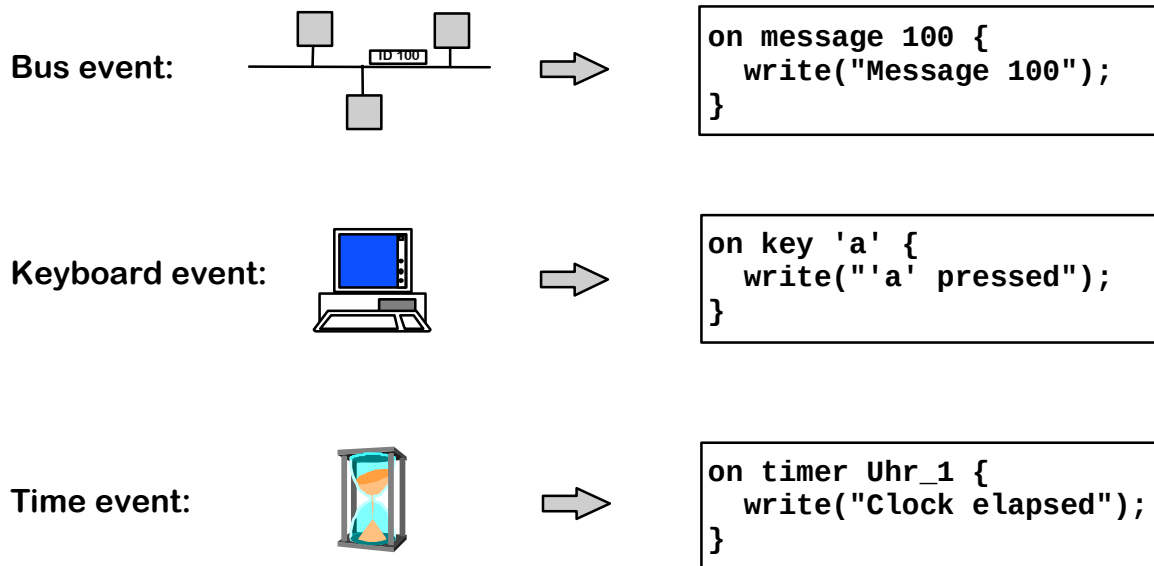


Figure 66: Examples of CAPL Event Procedures⁵

A CAPL program consists of two parts:

1. Declare and define global variables
2. Declare and define user-defined functions and event procedures

Variables in CAPL and the event procedure concept are described in further detail in the sections below.

3.2.1 CAPL Variables

Data types available for variables include integers (**dword**, **long**, **word**, **int**, **byte**, **char**), floating point numbers (**float** and **double**), CAN messages (**message**) and timers (**timer** or **msTimer**). Except for the timers, all other variables can be initialized in their declarations.

```
variables {
    int msgCount;           // Is set to 0 at measurement start
    message 34 sendMsg = { // Declare message with Id 34
        dlc = 1,           // Set Data Length Code = 1
        byte(0) = 1       // Set 1st data byte = 1
    };
}
```

Integer and floating point numbers are used as in other programming languages. They are compatible with one another, and arithmetic is performed with 32-bit resolution for integers and 80-bit for floating point numbers.

⁵ Besides keyboard events, in CANoe you can - with event procedures of the type **on envvar** also react to actions that you perform yourself on user-defined control panels.

In the Browser, global variables are defined in the global variables pane on the upper right of the screen. The data types **dword**, **long**, **word**, **int**, **byte** and **char** can be used analogous to their use in the C programming language. The data types **float** and **double** are synonyms and refer to 64-bit floating point numbers conforming to the IEEE standard.

Variables can be initialized when they are declared, whereby you can use either simple notation or brackets {}. With the exception of timers, the compiler initializes all variables with default values (unless otherwise defined: 0).

Local variables are - in contrast to C - always created as static. This means that an initialization is only performed at the program start. The next time they enter the procedure, variables have the value they had when they last exited the procedure.

CAPL permits the declaration of arrays (arrays, vectors, matrices), analogous to their declaration in the C programming language.

```
variables {  
    int vector[5] = {1,2,3,4,5};  
    int matrix[2][2] = {{11,12},{21,22}};  
    char progname[10] = „CANalyzer“;  
}
```

Arrays are passed formally, as in C. But as an expansion, it is also possible to pass multidimensional arrays. The number of elements in one dimension can be queried with the function **elCount()**. The compiler executes a value range check. If an index is less than zero or greater than the number of elements, a run-time error is generated by the function **runError()**.

Variables of the type **timer** (based on seconds) or **msTimer** (based on milliseconds) serve to generate time events. Timer variables are not automatically initialized at the program start, rather they must be "set" explicitly with the function **setTimer()**.

```
variables {  
    timer delayTimer; // Declaration of a second timer ...  
    msTimer cycTimer; // ... and a millisecond timer  
}  
  
...  
setTimer(delayTimer,3); // Set timer to 3 sec  
setTimer(cycTimer,100); // Set timer to 100 msec  
...
```

In contrast to C, CAPL does not permit the use of pointers. This language feature was avoided intentionally to make CAPL programs more robust. For example, memory protection violations can be brought under significantly better control this way: Instead of a crash in the program's high-priority real-time library, the measurement is terminated in a controlled manner when programming errors occur, and a message is output to the Write window.

Since the emphasis in modeling with CAPL lies in the description of the bus behavior of nodes, the scope of the CAPL language is normally sufficient. Nevertheless, expanded functions in C, C++ or PASCAL can also be implemented in a custom Windows DLL and integrated in CAPL. See section 3.6 for detailed instructions on this.

3.2.2 Declaration of Messages

Messages to be output from the CAPL program are declared with the key word **message**. The complete declaration includes the message identifier or - when working with symbolic databases - the message name. For example, you might write the following to output messages on the bus that have identifier A (hex) or 100 (dec) or the message *EngineData* defined in the database.

```
message 0xA m1;           // Message declaration (hex)
message 100 m2;          // Message declaration (dec)
message EngineData m3;   // Symbolic declaration
message * wcrd;          // Declaration without Id
...
output(m1);              // Transmit message m1
output(m2);              // Transmit message m2
output(m3);              // Transmit message m3
wcrd.id = 0x1A0;         // Define Id...
output(wcrd);            // ... and transmit message
```

The designations for messages are entered as message identifiers, whereby an identifier is an integer number in either decimal or hexadecimal representation. For extended identifiers a x is appended to the identifier. The entry of * means that initially this variable does not contain any message identifier. It must then be defined in another manner before the object is sent out. Such objects might be used for filtering tasks, e.g. to save all objects that are to be passed unchanged. (The received message is then copied, including the message identifier.)

The data field of a message can be accessed by entering the data type as a data selector and the offset as a byte value (starting with 0). Possible data selectors are: **long**, **dword**, **int**, **word**, **char** and **byte**.

For example, the following would transmit a message 100 with DLC = 1 and the first data byte 0xFF on the bus:

```
message 100 msg;
msg.DLC = 1;
msg.BYTE(0) = 0xff
output(msg);
```

Note: CAPL programs you wish to use to send messages on the CAN bus *must* always be inserted in CANalyzer's transmit branch in the measurement setup. Messages sent by CAPL programs further to the right in the measurement setup will *not* be output on the bus.

The components of objects can be accessed by means of component selectors. If you wish to define an object for a specific chip (for CAN cards with more than one

CAN chip), you would enter the appropriate selector (CAN1 or CAN2) followed by a dot (.) in front of the message name.

You can access control information for the CAN message objects using the following component selectors:

ID	Message identifier
CAN	Chip number
DLC	Data Length Code
DIR	Direction of transmission, possible values: RX , TX , TXREQUEST .
RTR	Remote Transmission Request; possible values: 0 (No RTR), 1 (RTR)
TYPE	Combination of DIR and RTR for efficient evaluation. $(TYPE = (RTR \ll 8) \mid DIR)$
TIME	Time point, units: 10 microseconds

For examples of the use of component selectors for messages, please refer to the CAPL Browser's online Help.

3.2.3 Event Procedures

You can react to the following events in CAPL using event procedures:

Event	Event procedure
Receipt of a CAN message	on message{}
Press of a key	on key{}
Initialization of measurement (before measurement start)	on preStart{}
Measurement start	on start{}
End of measurement	on stopMeasurement{}
CAN controller goes to ErrorActive	on errorActive{}
CAN controller goes to ErrorPassive	on errorPassive{}
CAN controller reaches the warning limit	on warningLimit{}
CAN controller goes to Bus Off	on busOff{}
Elapse of a timer	on timer{}
Occurrence of an error frame	on errorFrame{}

When the specified event occurs, the relevant procedure body is executed. This can contain the declaration of local variables, C-like arithmetic instructions and control instructions, and calls of intrinsic default procedures.

React to Messages

The event procedure type **on message** is provided to react to the receipt of CAN messages in the CAPL nodes.

on message 123	React to message 123 (dec), Receiving chip is not considered
on message 0x123	React to message 123 (hex); receive chip is not considered
on message EngineData	React to message EngineData
on message CAN1.123	React to message 123, if it is received by chip CAN1
on message *	React to all messages
on message CAN2.*	React to all messages that are received by chip CAN2
on message 100-200	React to all messages with identifiers between 100 and 200

Within an **on message** procedure the key word **this** is available to you for accessing the data of the message just received.

In the following example, a simple gateway is programmed with CAPL. The gateway should transmit all messages between Bus 1 and Bus 2 in both directions, but it should modify the message with ID 34 by setting byte 4 to 0.

```
on message CAN1.34 {
    message CAN2.34 sendMsg; // Local message variable with
                             // name sendMsg, identifier 34,
                             // target controller CAN 2
    sendMsg = this;          // Copy all data and attributes
                             // of received message (this)
                             // to message to be transmitted
    sendMsg.byte(4) = 0;     // Change byte 4,
                             // always enter 0
    output(sendMsg);         // Transmit message
}
```

Please note that at this point it would be insufficient to change the fourth byte on the received message **this** directly. As described in section 3.2.5, all changes to **this** are local and therefore are not considered in the **output()** function. Consequently, **this** is copied to a local message variable.

A bridge function is implemented for all other messages: All messages from Bus 1 are transmitted to Bus 2 and vice versa.

```
on message CAN1.* { // All messages that are not pro-
                    // cessed in any other procedure
                    // are marked by '*'.
    message CAN2.* sendMsg;
    sendMsg = this;
    output(sendMsg);
}

on message CAN2.* {
    message CAN1.* sendMsg;
    sendMsg = this;
    output(sendMsg);
}
```

CAPL programs are, by default, impermeable to bus events. This means that with a CAPL node in the evaluation branch of the measurement setup, in effect you block the data flow to the right side of the node. You must explicitly program the passing of messages for CAPL nodes in the evaluation branch. You would write the following to make the CAPL node permeable to messages:

```
on message * {
    output(this);
};
```

In CAPL a maximum of one procedure is called for each received message. If multiple procedures exist in a CAPL program which match the message, that procedure is tasked which has the most specific name. The following rules apply here:

- Entry of a controller number has precedence over no entry
- Entry of a message number has precedence over the entry of a wildcard.

An example of the priority sequence (the search for the first applicable procedure is made from top to bottom):

```
on message CAN1.123 // React only to message 123
                    // from CAN1
on message CAN2.* // React to all messages from CAN2
on message 123    // Is never called
on message*       // React to all messages from CAN1
                    // except message 123
```

The sequence in the example corresponds to the representation of the event procedures in the Browser. Two event procedures with the same name are not allowed in CAPL.

React to Keyboard Events

With **on key** procedures you can execute certain actions by key press.

on key 'a'	React to press of 'a' key
on key ' '	React to press of spacebar
on key 0x20	React to press of spacebar
on key F1	React to press of F1 key
on key ctrlF12	React to press of Ctrl-F12
on key PageUp	React to press of Page Up
on key Home	React to press of Home
on key *	React to any key press

The code for a key press can either be input as a character, number or a predefined name for a function key.

In the following example, all messages with the (hexadecimal) identifier 1A1 are counted, and this count is output to the Write window when the 'a' key is pressed. Please note that this program is written for the evaluation branch of the measurement setup and was made permeable to all messages there by the call of **output(this)**. Without **output(this)** the messages would not be passed to the function blocks further to the right, and of course they also would not be displayed in the corresponding windows.

```
variables {
    int counter = 0;
}

on key 'a' {
    write("A total of %d messages 0x1A1 counted",counter);
}

on message 0x1A1 {
    counter++;
    output(this); // Only in the evaluation branch
}

on message * {
    output(this); // Only in the evaluation branch !!!
}
```

In **on key** procedures, the key word **this** is used to determine the active key code. For example, you could write the following code to consolidate all keyboard handling in a single event procedure.

```
on key * {
    switch(this) {
        case 'a' : ... break;
        case F10: ... break;
        ...
    }
}
```

React to Time Events

You can define time events in CAPL. When this event occurs, i.e. after a certain time duration elapses, then the corresponding **on timer** procedure is called. You can program cyclic program sequences by resetting the same time event within these **on timer** procedures.

With the function **setTimer()** set a timer which you have previously defined. In CAPL there are second-based timers (**timer**) and millisecond-based timers (**msTimer**) available to you. When the timer elapses the corresponding **on timer** procedure is called. The maximum time is 1799 sec. for variables of the **timer** type and 65.535 msec for variables of the **msTimer** type. With the function **cancelTimer()** you can stop a timer that has already been set and thereby prevent the assigned **on timer** procedure from being called.

In the following example, the message 100 is always sent on the bus 20 milliseconds after pressing the 'a' key.

```
msTimer myTimer;    // Define millisecond timer
message 100 msg;     // Define message to be transmitted
...
on key 'a' {         // React to key press of 'a'...
    setTimer(myTimer,20); // ... Set timer to 20 ms
}
...
on timer myTimer {   // Send message after timer...
    output(msg);      // ... has elapsed
}
```

React to System Events

The event procedures **on preStart**, **on start** and **on stopMeasurement** are called before the start, during the start or when the measurement ends, respectively. In these procedures you can e.g. initialize variables, read-in initialization data from files with the functions **fileReadXXX()** and **seqFileXXX()**, output a start message to the Write window with the **write()** function, set timers, or output statistics after the end of measurement.

Please refer to CAPL Browser's online Help for further instructions and examples on the use of system event procedures.

React to CAN Controller Events

The event procedures **on errorActive**, **on errorPassive**, **on warningLimit** and **on busOff** are called during a state transition or in response to a change in the CAN controller's error counter. Use these procedures to monitor the error counter (e.g. output a warning), to terminate the measurement if necessary, or to execute a reset after a transition to the Bus-Off state.

The CAN controller's error counters can be accessed within the CAN controller's event procedures using the key word **this**:

```
on errorPassive {  
    ...  
    write("CAN Controller ist in errorPassive")  
    write("    errorCountTX = %d", this.errorCountTX);  
    write("    errorCountRX = %d", this.errorCountRX);  
};
```

Note: At this time, access to the values of the error counter is only supported by systems with CANcardX.

3.2.4 Expressions in CAPL

CAPL syntax is based on the C programming language. The following expressions are permitted as they are in C:

- Instruction blocks: { ... }
- **if** { ... } and **if** {...} **else** { ... }
- **switch, case, default**
- **for...while...do...while** loops
- **continue** and **break**
- **return**
- **?:** expression

You will find examples of these expressions in CAPL Browser's online Help.

The same arithmetic, logical and bitwise operators (**+**, *****, **+=**, ***=**, **|**, **++**, etc.) are provided in CAPL as they are in C.

The Cast instruction can be used (as in C) to make an explicit type conversion. A Cast with entry of a **message** denotation can be used to assign messages with different identifiers.

An implicit type conversion is performed when evaluating expressions containing **long**, **dword**, **int**, **word**, **char** and **byte**. The following rules apply:

8-bit variables are first converted to 32 bits (**char** ⇒ **long**, **byte** ⇒ **dword**)

16-bit variables are first converted to 32 bits (**int** ⇒ **long**, **word** ⇒ **dword**)

3.2.5 The Key Word **this**

The key word **this** is used to refer to the the data structure of an object within an event procedure for receiving a CAN object or environment variable.

For example, the following accesses the first data byte of message 100 which is just being received:

```
on message 100 {  
    byte byte_0;  
    byte_0 = this.byte(0);  
    ...  
}
```

Analogously, in CANoe you can read out the new value of the just changed integer environment variable *Switch* with the following:

```
on envVar Switch {  
    int val;  
    val = getValue(this);  
    ...  
}
```

You should not change the value of **this** within an event procedure. Nevertheless, so that you can use **this** as a parameter, value changes to **this** are not prohibited by the CAPL compiler. However, please note that such write accesses to **this** only have local validity (i.e. within the event procedure). You will receive an appropriate warning when compiling. As a result, if - after a change to **this** - you call the function **output(this)** in an **on message** event procedure, the unchanged original of **this** is output and not its changed value.

3.2.6 Interpretation of Signal Values

When using a project database an individual signal can be accessed by entering its name. The compiler determines the bit offset and bit length as well as the machine format conversion (Intel or Motorola). The values are interpreted in raw format. If the physical interpretation is to be used, the operator **phys** must be appended. The compiler then scales the value according to the conversion formula given in the database (Factor, Offset).

The following example assumes that you have defined the message *EngineData* with a signal *EngineSpeed* in the associated database, and furthermore that you have entered a conversion formula.

```
message EngineData m;  
char    errSymbol[32];  
...  
m.rpm=100;           // Assignment of a raw value  
m.rpm.phys=123.5;    // Use of conversion formula
```

In the first line, the raw value 100 is assigned to the data bits which you have allocated to the signal *RPM* in the database. In the second line, the conversion formula assigned in the database is used to calculate the bit value.

When working with signals please note that signal values are always saved in discrete form. If a physical value is assigned, the next closest discrete raw value is entered after scaling. When the signal is read-out afterwards, the result may not necessarily be the original value.

For example, let us assume that the signal *RPM* of the message *EngineData* is defined as a 16-bit unsigned with an offset of 0 and a factor of 10. If the physical value 1015 is assigned, the raw value would have to be $1015/10 = 101.5$. However, since only discrete values may occur, 101 is saved. When this value is read out again it is scaled to $101 \cdot 10 = 1010$.

3.2.7 Access to Attributes in CAPL

Besides the data bytes that are transmitted over the bus with a CAN message, in certain systems supplemental attributes for messages, signals or network nodes are of interest. For example, with attributes it is possible to describe the cycle times of all cyclically transmitted messages of a CAN system. This involves defining an integer attribute *Cycle time [in ms]* for all messages and then setting a value for this attribute for all messages to be transmitted cyclically.

The CAN database supports attributes for the following object classes:

- Messages
- Signals
- Network nodes

Attributes are described by an attribute name, a data type, a value range and a default value. Possible data types are INTEGER, HEX, FLOAT, ENUM and STRING. The value range for the type ENUM is entered by an enumeration of valid names, else by minimum and maximum. You can assign values to each attribute of each database object. The attributes of all objects to which you have not assigned any attribute values are given the default values.

In CAPL you can select the attribute value of a database object by entering a dot (.) between the database object and attribute names. Let us consider the following example:

```
float rpm;
int time, n;
char buff[100];
...
time = EngineData.CycleTime;
n = ABS.NodeNumber;
rpm = EngineData.Speed.Max_rpm;
strncpy(buff, motordat.db_name, elcount(buff)-1);
```

This portion of a CAPL program reads out the values of the message attribute **CycleTime** (Object: message **EngineData**, type: INT), the signal attribute

Max_rpm (Object: Signal **Speed** of the message **EngineData**, type: FLOAT), the network node attribute **NodeNumber** (Object: Network node **ABS**, type: INT) and the database attribute **db_name** (Object: Database **motordat**, type: STRING).

Attribute values of the ENUM type are handled like strings in CAPL. Therefore, the last line of the example above is used to copy the value of the ENUM message attribute to the buffer **buff**. Please note that all attributes share the same name area. Therefore, it is not possible for example to use a database attribute and a message attribute which have the same name.

Note: If you read out the attribute value of an object in CAPL without having explicitly set this value for the object in the database, you get the default value that you entered for each attribute when you defined it in the database editor.

3.2.8 Working with Symbolic Signal Values

Besides using symbolic names for messages, signals and environment variables, you can also use symbolic names for integer values of signals and environment variables. This involves assigning symbolic names to the specific integer values of signals and environment variables in the database editor.

In CAPL you can use these names instead of the values by entering the signal or environment variable name followed by two colons (::) in front of these names. Let us assume that you have defined the values of the signal *AmpelStatus* of the message *LightData* as follows in the database AMPEL . CFG

Value	1	2	3
Name	Red	Yellow	Green

You would write the following code in CAPL to output a message *LightData* with the symbolic value *Red* (=1) of the signal *LightStatus* on the bus.

```
message LightData msg;
msg.AmpelStatus = LightData.LightStatus::Red;
output(msg);
```

If you would like to read the symbolic name of a signal value, use the **sys** operator:

```
on message LightData
{
    char buf[64];
    strncpy(buf, this.AmpelStatus.sym, elCount(buf)-1);
    write("this.AmpelStatus = %s", buf);
}
```

Analogously, you can also work with symbolic values for environment variables of the Integer type, provided that you defined these in the database.

3.2.9 Working with Multiple Databases

With large systems it may be advisable to distribute the description of messages, signals and environment variables to multiple partial databases. When analyzing two CAN systems simultaneously, it is also advisable to describe each system by its own database.

CAPL supports the simultaneous use of multiple databases. In Browser you configure a database and the CAN chip assigned to this database with the menu command *File|Associate database*. Afterwards you can use symbolic names from all databases for messages, signals and environment variables.

Note: If you start Browser from CANalyzer, perhaps by double clicking a CAPL node, the databases configured in CANalyzer are automatically associated properly.

If the symbolic message name *EngineData* is in a database that is explicitly assigned to controller CAN2, then the following code transmits the message **msg** via Chip 2:

```
message EngineData msg; // EngineData ⇔ Chip2
...
output(msg); // msg is transmitted via Chip 2
```

If you change the controller assignment of the database to CAN1, the message is transmitted via Chip 1 when the same CAPL code is run again after recompiling.

When using more than one database, you can resolve ambiguous symbols by qualifying them. This involves simply entering the database name followed by two colons (: :) before the symbolic name. For example, let us assume that the message 0x10F is named *DiagData* in the database *Infodat*. In the database *Motordat*, on the other hand, the name *DiagData* is used as a name for the message 0x4E1. With **Infodat::DiagData** for the message 0x10F and with **Motordat::DiagData** for the message 0x4E1 you get unique designations for the two messages.

Please refer to online Help for further instructions on resolving ambiguities when working with more than one database.

3.2.10 Functions in CAPL

Some of the purposes of CAPL functions include encapsulation of code for its re-use and the formation of interfaces. The syntax is C-like, but in contrast to C it offers a number of supplemental features:

- A missing result type is interpreted as void
- An empty parameter list is permitted as in C++
- Overloading of functions (i.e. more than one function with the same name but different parameter lists) is possible as in C++
- A parameter check is performed as in C++
- Arrays of any dimension and size may be passed

For example, the following function outputs a matrix in the Write window:

```
void write(int m[][])
{
    int i,j;
    for(i=0;i<elCount(m);i++) {
        for(j=0;j<elCount(m[0]){
            write("%ld", m[i][j]);
        }
    }
}
```

Please note that there is already an intrinsic CAPL function **write()** with which you can output formatted text to the Write window. The function defined in the example overloads the intrinsic function and instead of always calling it, a two-dimensional array is passed instead.

You will find a complete library of intrinsic CAPL functions in CAPL Browser's online Help. The following table offers a brief overview of the most important functions.

Transmission of frames	output(), output(errorFrame)
Output to Write window	write()
Output to log file	writeToLog(), writeToLogEx()
Measurement control	trigger(), stop()
Time management	setTimer(), cancelTimer(), timeNow()
Array size determination	elCount()
Environment variable access	getValue(), putValue()
PC access	outport(), inport()
Calculations	sin(), cos(), abs()

3.3 Creating a CAPL Program

Let's suppose you want to create a CAPL program with which you can count the number of messages of the type *EngineData* (ID 64 hex) and output the counted number of messages to the Write window in response to a key press.

First you must decide where you wish to insert your CAPL program in the data flow plan. In principle, any hotspot in the measurement setup is available to you. However, since this program is solely for analysis purposes and does not generate any messages itself, but only counts them, it is advisable to insert the program on the right side of the measurement setup, perhaps before the Statistics block. In the hotspot's popup menu choose the function *Insert CAPL node*. A function block with the program symbol *P* now appears at the selected point in the measurement setup. You can also access the node's configuration dialog via the popup menu. First, select a program name, e.g. COUNTER.CAN, and then start the CAPL Browser either from the

configuration dialog's *Edit...* button or directly by double clicking the program block in the measurement setup.

If you choose a new file the CAPL Browser, which is described in detail in section 3.4, starts with its sub-windows ("Panels"). It allows you to create and edit CAPL programs quickly and easily.

For your program you will first need an integer variable which counts the messages. For example, you could name it **counter**. Go to the upper right Browser pane and enter this name in the variables block. The following should now appear in this pane:

```
variables {  
    int counter;  
}
```

Like all global variables, this variable is automatically initialized to zero at the measurement start.

In the next step, this variable should be incremented whenever an *EngineData* message is registered. Therefore, you must expand the CAPL program to include an event procedure of the type **on message** ("React to message event"). To do this, click the event type **Messages** in the Browser tree using the right mouse button and insert a new event procedure of this type using the *New* command in the popup menu

Now a procedure template appears in the Procedures Text Editor. First replace the text **<newMessage>** by the symbolic name **EngineData**, which you could also assume directly from the database via the popup menu item *CANdb Message*. During compilation the CAPL compiler replaces the symbolic name by the corresponding identifier 0x64.

Now you only need to define which actions should be performed when the event occurs. Since the program is to count messages, the variable **counter** must be incremented whenever a message is registered. The complete procedure appears as follows:

```
on message 0x100  
{  
    counter++;  
}
```

As a last step, the output to the Write window must still be implemented. Finally, the program should not just count messages, but also keep track of how many messages have been counted.

The output to the Write window should occur when the 'a' key is pressed. Therefore, you must define another event procedure for the event "Press key 'a'". In the Browser tree you select the type **Keyboard**. This causes the previously defined **on message** procedure to disappear, since it belongs to a different event type. Of course it still remains a component of the CAPL program and will appear again as soon as you select the **Messages** event type again.

Now insert a **Keyboard** event in the CAPL program from the popup menu. A new procedure template will appear in the Procedures Text Editor, which you fill out as follows:

```
on key 'a'
{
    write("%d 0x100 messages counted",counter);
}
```

The format entry %d refers to the integer variable **counter**, which is entered after the comma. For the most part, this format string conforms to the C function `printf()`.

That completes the program. Save it and then start the compiler either with the F9 key, or the main menu command *Compiler|Compile* or by the lightning icon button on the toolbar. If you have made an error in creating the program, a message window will open showing you the error. Double click this error message to go to the location where the error occurred. After you have corrected it and saved the program file again, recompile the program. Once the program has compiled without errors, the message *Compiled* appears in the status bar at the bottom of Browser's main window.

Now start the measurement. The generator block in the transmit branch begins to cyclically transmit messages of the type *EngineData*, which are now counted by your program. Whenever you press the 'a' key the text "n EngineData messages counted" can be seen in the Write window, whereby *n* represents the number of messages counted.

3.4 Browser for Creating and Compiling CAPL Programs

A special Browser is integrated in CANalyzer as a user-friendly means to create, modify, and compile CAPL programs. Browser in this context is meant to signify a program for fast and targeted editing of CAPL programs.

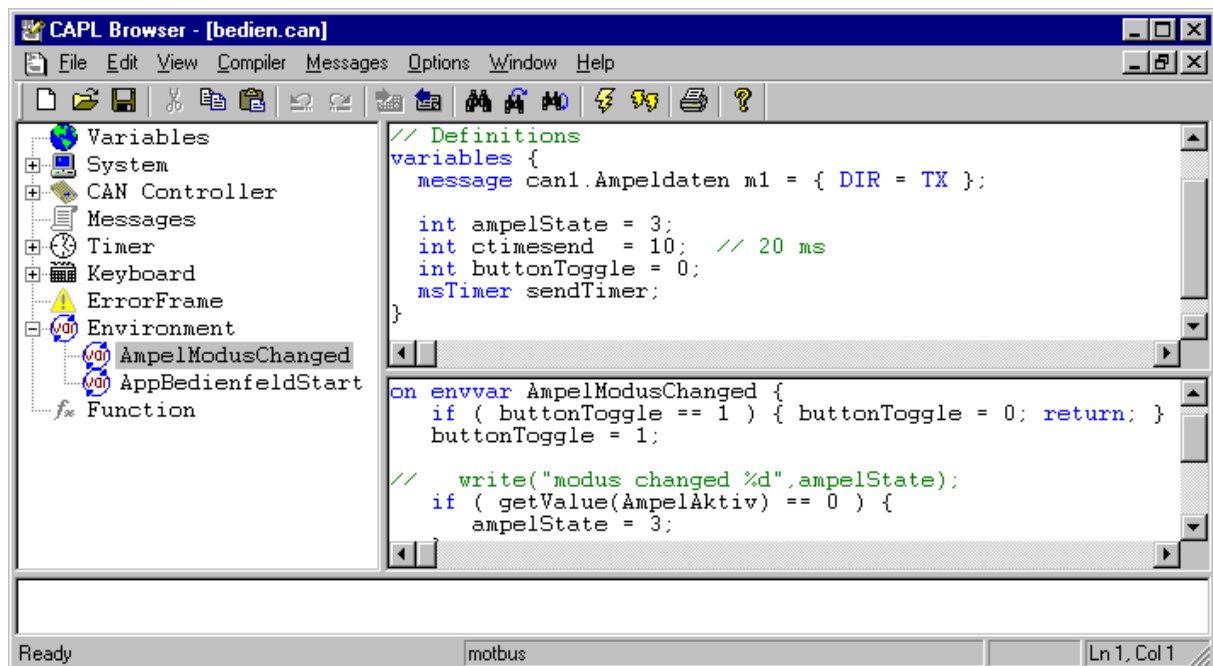


Figure 67: The Browser screen

A special Browser is integrated in CANalyzer for the user-friendly creation and modification of CAPL programs. This Browser shows you the variables, event procedures and functions of a CAPL program in structured form.

Multiple Browser windows with different CAPL programs can be opened simultaneously.

The CAPL compiler is started from Browser's main menu or toolbar. Compilation time is very short, even for larger programs. When an error is detected, the faulty program section is shown, and the cursor is positioned at the location of the error. This makes it very easy to make corrections.

3.4.1 Opening Browser

Browser is located in the system directory (canbr32.exe) and can be started as a stand-alone application like any other Windows program. However, it is recommended that you always start Browser from CANalyzer, since a number of important parameters for the program start (database name, compiler options, hardware parameters, CAPL-DLLs, etc.) must be passed.

When you start Browser by double clicking a CAPL node in the measurement setup, the file PARBROW.INI with all start parameters is automatically generated and made available to Browser. Therefore, always start Browser from CANalyzer, and furthermore make sure that the file PARBROW.INI is never given write protection, or else it might be impossible to pass important start parameters properly the next time Browser is started.

Note: A CAPL program file can be dragged from the File Manager on a CAPL Browser that has already been started ("Drop and drag"). The file is then displayed in a new window.

3.4.2 Browser Window

A Browser window is subdivided into up to four sub-windows, so-called "Panels".

At the upper left is the Browser Tree, which contains the CAN event types as drop-down nodes. In turn, each of these nodes contains the procedures that can be assigned to the CAN event types. The upper part of the text editor to the right of the Browser tree shows the global variables for the CAPL program; the lower part shows the procedure text for the procedure selected in the Procedures List. The text editor can also be configured so that global variables and procedures can be edited in a common editing window. Along the lower border is the Messages window that is used to display compiler messages for the CAPL program.

You can access the most important functions for each of the panes from the popup menu by pressing the right mouse button. In the editor panes you have access, via the popup menu, to the intrinsic CAPL functions and to the objects defined in the database. Furthermore, you can copy text to the Clipboard from the popup menu, and from there you can paste it in your program.

CAPL programs which are not available in Browser-specific file format are displayed in unstructured format in a normal text window and can be edited there. As in the editors of the Browser window, the program text can be edited via the menu command *Edit* or from the popup menu. In section 3.4.6 you will learn how to import such CAPL programs in Browser format.

3.4.3 Compiling CAPL Programs

To generate an executable program file in CBF format (CAPL Binary Format, file extension *.CBF) from a CAPL program, you must compile the program with the CAPL compiler.

Compilation of the program is started by the command *Compile* from the compiler menu, or by pressing the Compile icon () on the toolbar or by pressing the F9 key. If the CAPL program still does not have a file name, you are prompted to enter a name for the CBF file. If the program already has a name, the name of the CBF file is composed of the program name and the extension CBF.

All messages during the compilation process are output in the Message window.

If errors or warnings occur during the compilation process, the Message window automatically jumps to the foreground with the relevant error message. Double click the message or select the line and execute the command *Go to* from the *Messages* window to position the cursor at the location where the error occurred. After you have corrected it and saved the program file again, you recompile the program. If the program compiles without errors, the status *Compiled* appears in the status bar at the bottom of Browser's main window.

3.4.4 Searching for Run-Time Errors

Usually more difficult than the search for syntax errors that can already be found at compile time is the search for run-time errors. Nevertheless, CAPL offers you support here in two ways.

On the one hand, a number of possible run-time errors are automatically monitored in CAPL programs. These include:

- Division by zero
- Exceeding upper or lower array limits
- Exceeding upper or lower offsets in the data fields of messages
- Stack overflow in the call of CAPL subroutines

If such a run-time error is discovered, the measurement is terminated in a controlled manner. You get a message in the Write window containing the name of the CAPL program, the error type and an error index. With the help of this error index you can easily find the location in the CAPL source text that caused the error: Enter the index under the menu item *Compiler|Find run-time errors*.

On the other hand, as a user you have the option of generating run-time errors yourself with the CAPL function **runError()**, thereby making specific critical program sections fault-tolerant. In doing so, you can also give the function an error index which is then output to the Write window upon completion of the measurement.

3.4.5 Access to the Database

Provided that you open Browser directly from the measurement setup, Browser gets information regarding the databases associated to the configuration via the temporary file PARBROW.INI. But you can also associate one or more databases to the CAPL program directly in Browser from the *File* menu.

In Browser you have access to the database's messages and signals via the Message Explorer and Signal Explorer. You can open the selection dialog from the popup menu item *CANdb Message* or *CANdb Signal* in the editor panes on the right side. The selected object is assumed directly into the CAPL program text after exiting the dialog with *OK*. Of course you may also enter the signal and message names directly as clear text in the editor panes instead of using the symbolic selection dialog.

If you are using more than one database, the objects in the databases following the first database are qualified by including the database name in the symbolic selection dialogs. But you only need these qualified names to resolve ambiguities. As long as the symbolic names are unique in all databases, you can refrain from qualifying symbolic names when editing CAPL programs.

You can read about special considerations when using multiple databases in section 4.3.2.

3.4.6 Importing and Exporting ASCII Files

Browser offers an import function and an export function in the *File* menu to permit editing of your CAPL programs with text editors other than Browser.

The structural information that Browser uses to display CAPL programs to you in Browser format is contained in special comment lines. Browser automatically generates this information during compilation.

With *File|Import* a CAPL program that only exists in ASCII format can be loaded in the CAPL Browser. The procedures defined in the program are assigned to the CAPL procedure groups (e.g. Message, Timer, CAPL function, etc.). The command opens a file selection dialog in which you select the file to be imported. When import-

ing programs make sure that the programs can be compiled error-free, since otherwise it might not be possible to recognize function names or procedure names unambiguously.

With *File|Export* you can save CAPL programs in ASCII format. The generated file does not contain any structural information of the CAPL Browser and can only be displayed in a Browser window by importing it later.

3.4.7 Browser Options

In the menu *Options|Editor...* you will find commands for individual configuration of the editors in the Browser panes.

The menu item *Options|Compiler...* opens a dialog in which compiler options for the active CAPL program can be changed. These options are automatically configured for you, provided that you have opened Browser from the measurement setup. That is, normally you should not change the option settings here.

3.5 CAPL Language Reference

In this section the syntax of language elements in expanded BNF representation is described, and the semantics are described in the form of explanatory text.

Syntactic constructs are based on the C programming language wherever this was reasonable.

3.5.1 Expanded Backus-Naur Form (BNF)

An expanded BNF is used to describe the grammar of the language:

- A nonterminal is represented as a simple identifier:

`varDeclList msgDenotation`

- `::=` is used as a logical inference symbol in production rules.

- A terminal (Key word) is enclosed in quotation marks:

`"VARIABLES" "TIMER" "{"`

- When the program is input no differentiation is made between upper and lower case characters in key words.

- Items that are not formally defined are enclosed in angle brackets:

`<integer> <varId>`

- An optional item is enclosed in square brackets:

`varDecl [ varDeclList ]`

- Alternate items are delimited by `|` :

`"WORD" | "INT" | "BYTE" | "CHAR"`

- Explanations are enclosed in comment symbols

`/* unsigned 16 Bit */`

- The C++ comment symbol `//` can also be used. The rest of the line then serves as a comment:

```
// Comment line
```

3.5.2 Program Structure

A CAPL program consists of the declaration of (possibly zero) global variables and a set of event procedures (at least one to be meaningful).

```
CAPLProgram ::= varDeclarations procedures
```

3.5.3 Declaration of Variables

```
varDeclarations ::= "VARIABLES" "{"
                  [ varDeclList ]

                  "}"
```

In Browser global variables are declared in the upper right browser window.

```
varDeclList ::= varDecl [ varDeclList ] |
                constDeclaration [ varDeclList ] |

varDecl ::= intTypeSpecifier intIdList ";" |
            FLOAT intIdList |
            DOUBLE intIdList |
            "TIMER" idList ";" |
            "MSTIMER" idList ";" |
            "MESSAGE" msgDenotation msgIdList ";"

intTypeSpecifier ::= "DWORD" | /* unsigned 32 Bit */
                    "LONG" | /* signed 32 Bit */
                    "WORD" | /* unsigned 16 Bit */
                    "INT" | /* signed 16 Bit */
                    "BYTE" | /* unsigned 8 Bit */
                    "CHAR" | /* signed 8 Bit */
```

The data types **dword**, **long**, **word**, **int**, **byte** and **char** can be used analogous to their use in the C programming language.

The data types **float** and **double** are synonymous and designate 64 bit floating point numbers conforming to the IEEE standard. Internal floating point arithmetic is performed with 80 bits.

Example:

```
int counter;
double speed, temperature;
```

With **timer** a clock is created which does not run until it is set in an event procedure. When time "0" is reached the associated event procedure is called. A variable of the timer type can only be accessed by predefined functions. For **timer** the time is decremented in one-second cycles, and for **msTimer** in one-millisecond cycles. For reasons of speed, a 16-bit value (32 bits internally to count the ticks) is used for memory storage. The maximum time for **timer** is 1799 s (limited by internal processor accuracy), and for **msTimer** 65.535 s (limited by 16 bits).

Example:

```
timer timer1s;
msTimer timer100ms;
```

CAN objects (messages, J1939 parameter groups) to be output from the CAPL program are declared with **message** or **pg**. Object components are accessed by selectors (see below).

```
msgDenotation      :=
[ chipSelector "." ] <integer> |           // Standard Id
[ chipSelector "." ] <integer>"x" |        // Extended Id
[ chipSelector "." ] "*" |                 // Wildcard
[ chipSelector "." ] <msgSymbol>           // Symbolic name
[ chipSelector "." ] <dbName>":"<msgSymbol>
// Symbolic name with database qualification

chipSelector       ::= "CAN1" | "CAN2"
```

The name you enter for a CAN object is the message identifier or parameter group number (<integer>) in decimal or hexadecimal representation. For extended identifiers a x is appended. Entry of * means that this variable initially does not contain any message identifier. It must then be defined in another way before the object is sent out. Such objects may serve, for example, in filtering tasks to save all objects to be passed unchanged. (The received message is then copied, including the message identifier.)

When a symbolic database is used, the name from the particular database is used instead of the number.

Example:

```
message 100 msg100;
message Enginedata msgEnginedata;
message 6563x msgFirst, msgSecond;
message CAN1.ABSData msgABS;
message Motbus::EngineData msgEngineData;
```

If the object is defined for a specific chip (for CAN cards with more than one CAN chip), this can be entered as **CAN1** or **CAN2**.

```
idList              ::= <varId> [ "," idList ]
intIdList           ::= intIdDecl [ "," intIdList ]
intIdDecl           ::= declarator [ "=" declInit ]
declarator          ::= <varId> | declarator "[" <integer> "]"
declInit            ::= "{" declInitList "}" | simpleType | <string>
declInitList        ::= declInit [ "," declInitList ]
simpleType           ::= signedInteger | signedFloat
signedInteger       ::= <integer> | "+" signedInteger | "-" signedInteger
signedFloat         ::= <float> | "+" signedFloat | "-" signedFloat
```

Variables can be initialized when they are declared. Either the simplified writing style or bracketing with "{ }" is possible here. Variables which are not explicitly initialized are set to 0 in CAPL. It is possible to declare arrays (arrays, vectors, matrices). They are used and initialized analogous to the C language. In particular, arrays of the type CHAR are initialized by assigning a string.

```

msgIdList      ::=msgIdDecl [ "," msgIdList ]
msgIdDecl      ::=msgDeclarator [ "=" msgDeclInit ]
msgDeclarator  ::=<varId> | msgDeclarator "[" <integer> "]"
msgDeclInit    ::= "{" msgInitList "}" | "{" msgDeclInitList "}"
msgDeclInitList ::=msgDeclInit [ "," msgDeclInitList ]
msgInitList    ::=msgInit [ "," msgInitList ]
msgInit        ::=intTypeSpecifier
                "(" <integer> ")" "=" signedInteger |
                "DLC" "=" <integer> |
                "RTR" "=" <integer> |
                "DIR" "=" specialConstant |
                "TYPE" "=" specialConstant |
                "SIMULATED" "=" <integer> |
                "CHARACTERISTIC" "=" <integer> |
                "TIME" "=" <integer> |
                <signalSymbol> "=" simpleType |
                <signalSymbol> ".PHYS" "=" simpleType |
                "SA" "=" <integer> |
                "DA" "=" <integer> |
                "PRIO" "=" <integer> |
                "R1" "=" <integer> |
                "DP" "=" <integer>

```

Message variables can be initialized when declared. The data range can be accessed by entering the type and byte offset. Symbolic identifiers are provided for initializing the control ranges.

Example:

```

message 100 msg100 = {DLC=2, BYTE(0)=8, BYTE(1)=16 };
message Enginedata = { Speed.phys = 53.8 };

```

Arrays or matrices of message variables can also be created.

Symbolic names for numeric constants are designated as "specialConstant". Their use is comparable to define constants in C.

The meaning of control information and constants is explained in more detail below.

<integer>

Integer constants which can be written as decimal numbers, hexadecimal numbers (with 0x, etc.) or as character constants ('A'). An X is appended to the number when assigning a message identifier that is an extended ID (29 bits).

<float>

Floating point constant as in C.

<varId>

Identifier for a variable. Must begin with an alpha character or '_' and can contain "any quantity" of alphanumeric characters or '_'. Upper/lower case formatting is relevant. Umlauts and 'ß' are not accepted as alpha characters.

<constant expression>

Constant expression as in C.

```
constDeclaration ::=
  "CONST" [ intTypeSpecifier | "DOUBLE" ] <varId> "=" <constant expression>
```

This is how constants are declared. The method corresponds to the #define directives of C with supplemental type information and type checking. If no type is given 'long' is assumed. Constants can be used any place where the use of <integer> or <float> is allowed.

3.5.4 Event-Driven Procedures (Event Procedures)

Reactions to events (arrival of a message, elapse of a timer, etc.) can be implemented in procedures.

```
procedures ::= procedure [ procedures ]
procedure ::= eventProcedure | caplProcedure
eventProcedure ::=
  "ON" "MESSAGE" msgDenotation procBody |
  "ON" "TIMER" varId procBody |
  "ON" "KEY" keyDenotation procBody |
  "ON" "PRESTART" procBody |
  "ON" "START" procBody |
  "ON" "STOPMEASUREMENT" procBody |
  "ON" "ERRORFRAME" procBody |
  "ON" "ERRORACTIVE" procBody |
  "ON" "ERRORPASSIVE" procBody |
  "ON" "WARNINGLIMIT" procBody |
  "ON" "BUSOFF" procBody
keyDenotation ::= keyName | <integer> | "*"
keyName ::=
  "F1" | "F2" | "F3" | "F4" | "F5" | "F6" |
  "F7" | "F8" | "F9" | "F10" | "F11" | "F12" |
  "shiftF1" | "shiftF2" | "shiftF3" | "shiftF4" |
  "shiftF5" | "shiftF6" | "shiftF7" | "shiftF8" |
  "shiftF9" | "shiftF10" | "shiftF11" | "shiftF12" |
  "ctrlF1" | "ctrlF2" | "ctrlF3" | "ctrlF4" |
  "ctrlF5" | "ctrlF6" | "ctrlF7" | "ctrlF8" |
  "ctrlF9" | "ctrlF10" | "ctrlF11" | "ctrlF12" |
  "ctrlPageUp" | "ctrlPageDown" |
  "PageUp" | "PageDown" | "End" | "Home"
```

msgDenotation permits the same specifications as in a variable declaration, but only one event procedure may be declared for each object over the entire program. In this context, specifying '*' means that a message was received for which no event procedure was declared (default event procedure). It is also possible to enter a list of messages delimited by commas. Ranges are also permitted.

Example:

```
on message 100,200,250-300
on message EngineData
on message *
on message Motbus::EngineData
```

The code for a key press ("keyDenotation") can either be entered as a number or with the help of predefined names for function keys. The option *on key ** serves as a wild card. All key presses are passed to this function.

Note: Numbers may also be entered as character constants, e.g. 'A'. This permits problem-based input for the "normal" keys.

Example:

```
on key 65
on key 'a'
on key pageDown
on key *
```

The defined timer variable must be entered as a selection criterium for the reaction to a timer.

Example:

```
on timer Timer1sec
on timer Timer100ms
```

The procedure body is based on the capabilities of conventional programming languages.

```
procBody      ::= "{"
                [ localVarDecls ]
                [ statements ]
                "}"
localVarDecls ::= varDecl [ localVarDecls ]
```

Local variables are always created as static in CAPL (in contrast to C). This means that initialization is only performed at the program start. When the variables enter the procedure they have the same value they had when they last left the procedure.

It is not advisable to declare local variables of the **Timer** type. The purpose of these event procedures is to trigger, but they can only react to global **Timers**.

```

statements      ::= statement [ statements ]

statement       ::= exprStatement      |
                  compoundStatement    |
                  selectionStatement   |
                  iterationStatement   |
                  jumpStatement        |
                  caseStatement

compoundStatement ::= "{" statements "}"

selectionStatement ::= "IF" "(" expr ")"      statement [ "ELSE" statement ] |
                      "SWITCH" "(" expr ")" statement

caseStatement      ::= "CASE" signedInteger ":" statement | "DEFAULT" ":"

iterationStatement  ::= "WHILE" "(" expr ")" statement      |
                      "DO" statement "WHILE" "(" expr ")" ";" |
                      "FOR" "(" [ expr ] ";" [ expr ] ";" [ expr ] ")"
                          statement

jumpStatement      ::= "CONTINUE"      |
                      "BREAK"          |
                      "RETURN" [ expr ]

```

The types of statements provided have been adopted from C and have the same syntax and semantics. To simplify CAPL, **goto** and the option of declaring variables after { were not implemented. As in C the programmer must be careful not to program infinite loops.

```

exprStatement    ::= [ expr ] ";"

expr              ::= leftValue "=" expr      |
                    leftValue "|=" expr       |
                    leftValue "&=" expr        |
                    leftValue "+=" expr        |
                    leftValue "%=" expr        |
                    leftValue "*=" expr        |
                    leftValue "-=" expr        |
                    leftValue "/=" expr        |
                    leftValue "<=" expr         |
                    leftValue ">=" expr         |
                    leftValue "^=" expr        |
                    dyadicOperation           |
                    dyadicOperation "?" expr ":" expr

leftValue         ::= unaryExpr

dyadicOperation   ::= dyadicOperation [ dyadicOperator castExpr ]

```

```

dyadicOperator      ::= "|" |
                        "&&" |
                        "|" |
                        "^" |
                        "&" |
                        "==" | "!=" |
                        ">=" | "<=" | ">" | "<" |
                        ">>" | "<<" |
                        "+" | "-" |
                        "*" | "/" | "%" |
                        /* conditional or */
                        /* conditional and */
                        /* bitwise inclusive or */
                        /* bitwise exclusive or */
                        /* bitwise and */
                        /* equality operators */
                        /* relational operators */
                        /* shift operators */
                        /* add, sub */
                        /* mul, div, modulo */

```

The dyadic operators conform to C syntax. In the rule above, the operators with the same level of precedence are shown on the same line. Precedence increases from top to bottom.

The range of values must be considered for shift operators:

0 <= Shift value <= 31.

For reasons of run time this is not checked by the compiled code.

```

castExpr            ::= "(" typeName ")" castExpr |
                        unaryExpr

typeName            ::= "intTypeSpecifier" |
                        "FLOAT" | "DOUBLE" |
                        "message" msgDenotation |
                        <msgSymbol> "." <signalSymbol> "." "phys" |
                        <msgSymbol> "." <signalSymbol> "." "hex"

```

A type conversion can be made explicitly (as in C) using the cast statement. A cast specifying msgDenotation may be used to assign messages with different identifiers. The use of symbolic notation is explained below.

When evaluating expressions that contain long, **long**, **dword**, **int**, **word**, **char** or **byte**, an implicit type conversion is performed. The following rules apply:

8-bit values are first converted to 32-bit values (**char**⇒**long**, **byte**⇒**dword**)

16-bit values are first converted to 32-bit values (**int**⇒**long**, **word**⇒**dword**)

If an expression contains mixed signed and unsigned values, the operation is executed as unsigned after the expansion.


```

unaryExpr ::=      monadicOperator castExpr | postfixExpr

monadicOperator  ::=
    "+" | "-" |          /* plus, minus */
    "~" | "!" |         /* bit not, logic not */
    "++" | "--"         /* Increment, Decrement */

postFixExpr      ::=
    primaryExpr
    postfixExpr "[" expr "]"
    postfixExpr "++"
    postfixExpr "--"
    postfixExpr "." selection

primaryExpr      ::=
    <varId>
    "THIS"
    <integer>
    <float>
    specialConst |
    "(" expr ")" |
    functionCall

selection ::=
    intTypeSpecifier "(" <integer> ")" |
    "CAN"
    "ID"
    "PGN"
    "DLC"
    "TYPE"
    "DIR"
    "RTR"
    "TIME"
    "SIMULATED"
    "PF"
    "SA"
    "DA"
    "PRIO"
    "R1"
    "DP"
    "CHARACTERISTIC"
    "ERRORCOUNTTX"
    "ERRORCOUNTRX"
    <signalSymbol> [ "." "PHYS" ]
    <attributeSymbol>
    <valueDescription>

specialConstant ::=
    "RX" | "TX" | "TXREQUEST" | "RXREMOTE" | TXREMOTE |
    "TXREQUESTREMOTE" | "RXDATA" | "TXDATA" | "TXREQUESTDATA" |
    "NOW" | "LPT1" | "LPT2" | "LPT3" | "ERRORFRAME" | "PI"

```

The data structure of an object is designated by the key word **this** within an event procedure for receiving the CAN object.

Note: **this** should not be modified (read only)! But the use of **this** as a parameter for CAPL procedures (see below) is generally not prohibited any longer. For write accesses to **this**, a warning appears that this change will only be made locally. If, for example, **output(this)** is called after a change, it will pass the unchanged original of **this**.

The data range of a message can be accessed by inputting the data type as the selector and the byte value offset (which starts with "0"). The following can be used as selectors **long**, **dword**, **int**, **word**, **char** and **byte**.

When a project database is being used, an individual signal can be accessed by inputting the name of the signal. The compiler assumes the task of determining the bit offset and bit length as well as the task of converting the machine format (Intel $\leftrightarrow$ Motorola). The values are interpreted in raw format. If physical interpretation is to be used, the **.phys** operator must be appended. The compiler then scales the value according to the conversion formula (factor, offset) specified in the database.

Example:

```
message EngineData m;
double           x;
long            i;

m.rpm = 100;          /* Assignment of a raw value */
m.rpm.phys = 123.5; /* With use of the phys. formula */
```

Note: It must be noted that signal values are always saved in memory as discrete values! If a physical value is assigned, then the next closest discrete raw value is entered after scaling. If the signal is read out afterwards, the result will not necessarily be equal to the original value.

Example:

The signal EngineData.rpm is defined as 16-bit unsigned with an offset of 0 and a factor of 10. If 1015 is assigned as the physical value, the raw value would have to be $1015/10 = 101.5$. But since only discrete values may occur, the value 101 is saved. When the value is read out it is scaled to $101 \cdot 10 = 1010$.

This problem is of a general nature. Memory storage of floating point numbers is also executed in discrete format (in this case conforming to IEEE specification). This can result in inaccuracies:

```
double x, y;
x = 1.4;
y = 0.7;
if( x != y*2) write("Error x = %g, y*2 = %g",x,y*2);
```

In this example an error message could result. The correct comparison would be:

```
if (abs(x-2*y) < 0.001) write ...
```

The following selectors can be used to access control information:

Selector	Meaning	Possible Values (Comment)
ID	Message identifier	
PG	Parameter Group Number	(Only J1939)
CAN	Chip number	1, 2
DLC	Data Length Code	0, 1, ... ,8
DIR	Direction of transmission, event classification	RX, TX, TXREQUEST.
RTR	Remote Transmission Request	0: No RTR, 1: RTR
TYPE	Combination of DIR and RTR for an efficient evaluation	(TYPE = (RTR << 8) DIR)
TIME	Time point, units: 10 microseconds	
SIMULATED	Indication of whether message was sent by a simulated node	0: Unsimulated node, 1: Simulated nodes
PF		(Only J1939)
SA	Sender entry in CAN identifier of a parameter group	(Only J1939)
DA	Receiver entry in CAN identifier of a parameter group	(Only J1939)
PRIO	Priority entry in CAN identifier of a parameter group	(Only J1939 at initialization of parameter group)
R1	R1 field in CAN identifier of a parameter group	(Only J1939 at initialization of parameter group)
DP	Data page field in CAN identifier of a parameter group	(Only J1939 at initialization of parameter group)
CHARACTERISTIC	Combination of DP, R1 and PRIO in the CAN identifier of a Parameter Group	(Only J1939: for accesses outside of the parameter group initialization)
ERRORCOUNTTX	Transmit error counter of the CAN chip	
ERRORCOUNTRX	Receive error counter of the CAN chip	

The following symbolic constants are available to remain independent of the actual coding of DIR and RTR:

For DIR:

RX	Message was received	DIR == RX
TX	Message was sent	DIR == TX
TXREQUEST	Transmit request was set for message	(DIR == TXREQUEST)

For TYPE:

RXREMOTE	Remote message was received	(DIR == RX) && RTR
----------	-----------------------------	--------------------

TXREMOTE	Remote message was transmitted	(DIR == TX)&&RTR
TXREQUESTREMOTE	Transmit request was set for remote message	(DIR == TXREQUEST) && RTR
RXDATA:	Data message was received	(DIR == RX)&&!RTR
TXDATA:	Data message was transmitted	(DIR == TX) &&!RTR
TXREQUESTDATA:	Transmit request was set for data message	(DIR == TXREQUEST) && !RTR

In the project database the CAN objects (messages, signals, etc.) can be assigned attributes and symbolic value descriptions. The attribute values of objects can be accessed with the attribute name as object selector.

Example:

```
on message EngineSpeed {
    // 'msCycleTime' defined as message attribute
    if (this.msCycleTime == 100) { ... }

    // 'notAvailable' defined as signal attribute
    if (this.EngineSpeed.notAvailable) { ... }
}
```

Value descriptions can be used analogous to signal values. It is possible to use symbolic descriptions as substitutes for integer values (analogous to a *enum* definition in C).

Example:

```
// Use value description 'active' for signal 'State'
message ABSData msg = { State = ABSData.State::active };
```

In **on key** event procedures, **this** serves to determine the current key code.

Example:

```
on key * {
    switch(this) {
        case 'a': .... break;
        case F10: ... break;
        ...
    }
}
```

this is not meaningful for event procedures which react to timers, start or error frames.

3.5.5 CAPL Subroutines

In CAPL there are a number of built-in functions, the so-called intrinsic functions (see below). `caplFunction` refers to subroutines defined by the user.

```
functionCall      ::= intrinsicFunctionCall |
                    caplFunctionCall         |
                    externalFunctionCall

caplFunctionCall  ::= <varId> "(" argExprList ")"

argExprList       ::= argExpr [ "," argExprList ]

argExpr           ::= expr
                    |
                    <string>
```

User-defined subroutines are called in the same way as intrinsic functions.

```
caplProcedure     ::= [resultType]<varId>("[funcParDeclList]") procBody

resultType        ::= intTypeSpecifier          |
                    "float" | "double"          |
                    "void"

funcParDeclList   ::= funcParDecl [ "," funcParDeclList ] | "void"

funcParDecl       ::= intTypeSpecifier declarator |
                    "float" | "double"           |
                    "TIMER" <varId>              |
                    "MSTIMER" <varId>            |
                    "MESSAGE" msgDenotation msgDeclarator
```

`procBody` is described for `eventProcedures`. Simple types (**int**, **float**) are passed as 'call by value', and structured types (**message**, **timer**, **arrays**) are passed as 'call by reference'. Simple types can be returned as functional results (or by global variable or by an auxiliary construct array of length 1). All variables are declared as static. Function parameters are passed to the stack. This must be taken into consideration in recursive calling of functions!

If the result type is not specified for a function, `void` is assumed, contrary to C. The parameter list may be empty; this corresponds to a void parameter.

Examples of function headers:

```
int func1(int i)           // Parameter and result integer
int func2(int i, double x) // Parameter integer and float,
                           // Result integer
void func3(void)           // No parameter, no result
func4()                   // No parameter, no result
func5(char str[])         // Parameter char string of any
                           // length, no result
float func6(int matrix[][] // Parameter 2-dim. matrix,
                           // result float
```

Functions can be defined with overloading (as for intrinsics). This means that several functions can be declared with the same name but different parameters: The type of the result must always be the same.

```
void print(int i)
void print(int j[])
void print(int j[][]);
```

This method may be used, for example, to define a uniform output function for all data bytes, which is always called under the same name. The compiler then always calls the function that is appropriate for the particular data type. Existing intrinsic functions can be expanded in this manner (e.g. **write(int)**, **write(float)**, ..., **output(float)**, ...).

Use of arrays as parameters:

Arrays are passed formally, as in C. But the program has been expanded to permit passing of multidimensional arrays. The number of elements of a parameter can be queried using the *elCount()* function. The compiler executes a range check. If an index is less than zero or greater than the number of elements used, a runtime error is generated by *runError()*.

Example:

```
void printMatrix(int m[][]) {
    int i,j;
    for(i = 0; i < elCount(m); i++) {
        for(j = 0; j < elCount(m[0]); j++) {
            write("%ld", m[i][j]);
        }
    }
}
```

3.6 Special Topics in CAPL

3.6.1 Integrating a Windows DLL in CAPL

In CAPL programs you can call functions you have implemented in your own Windows DLL. The functions of the DLL are thereby exported via a function table.

For the compiler and Browser the name of the DLL must be entered in section [SIMULATION] of the file CAN.INI. For example, if the name of your DLL is CAPLDLL.DLL, you would enter the following section in the CAN.INI file:

```
[SIMULATION]
CAPLDLL1=capldll.dll
```

Note: When you integrate custom DLLs in CAPL programs all paths for accessing system resources are open to you (memory management, hardware, etc.). However, due to the many problems associated with this method only experienced programmers should make use of this capability. Please understand that Vector Informatik cannot provide any support for the creation of your own DLLs.

Description of the CAPL Export Table

Name:

CAPL_DLL_INFO_LIST

Function:

Export of functions to CAPL

Description:

Column 1 contains the name of the function to be exported

Column 2 contains the address of the function in the DLL. All functions must be defined and exported with the PASCAL calling convention.

Column 3 contains a description of the function parameters and return values. The following character symbols are possible:

Symbol	Meaning	Comment
V	void	
L	long	
D	unsigned long	
I	int	Only together with field size > 0
W	unsigned int	Only together with field size > 0
B	unsigned char	Only together with field size > 0
C	char	Only together with field size > 0
F	float	Means "double" in 8 byte 80387 format)
0xabcd	Symbol for valid table	

A pointer to the address of this table must be returned in the function `capDllGetTable()`. This function must be compiled with extern C linkage.

Note: Return values (`long`, `int`) for the functions must be copied to the `_EAX` register. Please note that - with the exception of arrays - all parameters are of the type `long`.

Example:

```
unsigned long _export pascal caplDllGet(void)
{
    unsigned long temp;
    calcRc(&temp);
    _EAX = temp;
    return _EAX;
}
```

The complete example is included in the CAPLDLL subdirectory of your system directory.

Note: A DLL function cannot be directly accessed from CAPL. In order to access the functionality implemented in CAPL-DLL, the mapping between the name of the new CAPL function and the DLL must be entered in the CAPL export table.

Example:

The function

```
appPut ({"dllPut", (CAPL_FARCALL)appPut, 'V', 1, "D"})
```

must be accessed in CAPL via `dllPut` and not via `appPut`.

3.6.2 Handling of Run-Time Errors

A number of run-time errors are monitored in CAPL:

- Division by zero
- Exceeding upper or lower array limits
- Exceeding upper or lower offsets in the data fields of messages
- Stack overflow when CAPL subroutines are called

If a run-time error is detected, the intrinsic function **runError()** is called. This outputs a message to the Write window containing the name of the CAPL program, the error type and an error index. The location of the particular CAPL source text which caused the error is found with the help of the error index. The measurement is terminated after output of the message.

The function **runError()** can also be called directly by the user to generate assertions.

3.7 List of Intrinsic Functions

Below is a list of predefined functions ("Intrinsic Functions") organized by subject area.

3.7.1 Access to CAN and Other Hardware

output(message msg)
output(errorFrame)

Function:

Outputs a message or an error frame from the program block.

Parameters:

Variable of the type "message" or "errorFrame"

Example:

```
message 0x100 m100;  
...  
output (m100);  
output (errorFrame);
```

resetCan()

Function:

Resets the CAN controller. Can be used to reset the CAN controller after a BUSOFF or to activate configuration changes. Since execution of this function takes some time, and the CAN controller is disconnected briefly from the bus, messages might be lost here.

Example:

```
on key 'r' {  
    // After BUSOFF the controller is reset  
    resetCan();  
}
```

setBtr(long channel, byte btr0, byte btr1)

Function:

Sets a different baud rate. The values do not become active until the next call of the "resetCan()" function. This function can also be used to automatically set the baud rate.

It should be noted that the values will depend on the CAN Controller used.

Parameters:

CAN channel:

0: Both controllers

1: Channel 1

2: Channel 2

BTR0: Value of Bit Timing Register 0

BTR1: Value of Bit Timing Register 1

Example:

```
setBtr(0, 0x00, 0x3a); // 500 kBaud for 82C200  
resetCan();           // Activate
```

setOcr(long channel, byte ocr)

Function:

Sets the Output Control Register. The values do not become active until the next call of the function "resetCan()".

Please note that the values depend on the CAN platform used.

Parameters:

CAN channel:

0: Both controllers

1: Channel 1

2: Channel 2

OCR: Value of the Output Control Register

Example:

```
setOcr(0, 0x02);           // Set  
resetCan();                // Activate
```

Sets the CAN controllers to "passive monitoring" when the CAN AC2/200/ANA is used.

outport(word addr, byte value)

Function:

Outputs a byte to a port.

Parameters:

Port address or "LPT1"/"LPT2"/"LPT3" and value to be output.

Example:

```
outport (0x3f8, 12); /* Outputs 12 to port 0x3f8 */  
outport (LPT2, 'x'); /* Outputs 'x' to LPT2 */
```

outportLPT(word addr, byte value)

Function:

Outputs a byte to a parallel port. This function automatically switches the parallel port to the correct transmission direction.

Parameters:

Port address or "LPT1"/"LPT2"/"LPT3" and value to be output.

Example:

```
outportLPT (LPT1, 1);
```

byte inport(word addr)

Function:

Reads in a byte from a port.

Parameters:

Port address

Return:

Byte that is read-in

Example:

```
val = inport (0x3f8); /* Reads out port 0x3f8 */
```

byte inportLPT(word addr)

Function:

Reads-in a byte from a parallel port. This function automatically switches the parallel port to the correct transmission direction.

Parameters:

Port address

Returns:

Byte that was read in

Example:

```
val = inportLPT (LPT1); /* Reads a character from the  
                        first LPT port */
```

Note: The CAPL functions **inport()**, **inportLPT()**, **outport()** and **outportLPT()** make it possible for the 32-bit version to access the parallel port under Windows 95 and Windows NT too. Under Windows NT, first a driver must also be installed for the parallel port. You will find this driver together with installation instructions in the directory Exec32\GpioDrv.

long getCardType()

Function:

Determines the type of CAN platform being used. This is required, for example, in programming the BTR / OCR values.

Return:

Value	Board Type
0	DBB196 - Daimler-Benz Board FullCAN
1	DBB196B - " with BasicCAN
2	CANIB - Bosch CANIB
3	DEMO - Demo driver
6	CANAC2 - Softing AC2/200/ANA
7	CANAC2X - Softing AC 2/527/ANA
8	CPC/PP = EMS Wish Module
9	INDIGO - Silicon Graphics Indigo2
10	CANCARD - PCMCIA 11 Bit
11	CANCARDX - PCMCIA 29 Bit
12	CANAC2B - Softing AC2/527 11 Bit
13	VAN462 - NSI VAN Card
14	VANDEMO - VAN demo driver
16	Vector CAN-Dongle
17	Vector CANCardx
18	Virtual CAN driver
19	Caesar
20	Softing CANcard-SJA

Additional types may be added!

Example:

```
switch( getCardType( ) ) {
    case 6:  setOcr(0,0x02);break;
    case ....
    default:
        write("Unknown driver %d",getCardType());
        break;
}
```

long getChipType(long channel)

Function:

Determines the type of the CAN controller being used.

Parameters:

Value:	CAN Channel:
0:	Both controllers
1:	Channel 1
2:	Channel 2

Return:

Value	Type of Controller
5	NEC 72005
200	Philipps PCA82C200
462	MHS29C462 VAN Controller
526	Intel 82526
527	Intel 82527
1000	SJA 1000 (CANcardX)
1001	SJA 1000 (CANcard)

Other types may occur. Demo versions return the result 0 or simulate one of the existing types. The function result is 0 if an attempt is made to access a nonexistent channel (e.g. Channel 2 with CPC/PP) or if the driver used does not support this function.

3.7.2 User Interaction

write (char format[], ...)

Function:

Outputs a text message to the Write window.

Parameters:

Format string, variables or expressions

Write is based on the C function "printf".

Valid format entries:

"%ld", "%d":	Decimal representation
--------------	------------------------

"%lx", "%x":	Hexadecimal representation
"%lX", "%X":	Hexadecimal representation in upper-case characters
"%lu", "%u":	Unsigned decimal representation
"%lo", "%o":	Octal representation
"%s":	String output
"%g", "%lf":	Floating point
"%c":	Output of a character
"%%":	Output of %

The compiler cannot check the format string. Invalid format entries lead to undefined results.

Example:

```
i = 10; j = 12;
write ("d = %ld, h = 0%lxh", i, j);
/* Yields: "d = 10, h = 0ch" */
```

beep(int freq, int duration);

Function

Sound output. In the Windows version of CANalyzer and in CANoe the parameter *duration* has no effect.

Parameters

freq = Integer for sound pitch.

In the Windows version the parameter *freq* establishes which sound is to be output. These sounds are defined in section [SOUND] of the file WIN.INI. This entry is made in brackets.

Value of freq	Meaning
0x0000	SystemDefault
0x0010	SystemHand
0x0020	SystemQuestion
0x0030	SystemExclamation
0x0040	SystemAsterisk
0xFFFF	Standard Beep

Return: None

Note: If no sound card is installed in your computer, Windows always generates the same signal sound whenever the function is called. In this case the *freq* parameter has no effect.

sysExit()

Function:

Exits CANalyzer from a CAPL program.

sysMinimize()

Function:

Minimizes or restores the application window of CANalyzer. On the first call of the function after system start, the application window is minimized; afterwards it is alternately restored to normal size and minimized.

dword keypressed()

Function:

This function returns the key code of a currently pressed key. If no key is being pressed it returns 0. For example, this can be used in a timer function to query pressing of a key. A reaction can be made to the release of a key too.

```
variables {
    msTimer Zeitgeber;           // Timer
    message 100 botschaft;       // CAN message
}

on key F1 {
    setTimer(Zeitgeber,100);     // Start 100ms timer
    write("F1 pressed");        // Output Write window
}

on timer Zeitgeber {
    if(keypressed()) {
        setTimer(Zeitgeber,100); // Restart timer
        // Transmit as long as key is pressed
        output(botschaft);
    }
    else {
        // Reaction to release of key
        write("F1 released");
    }
}
```

3.7.3 Logging Functions

trigger()

Function:

Initiates triggering of logging.

Example:

```
on key 't' // Triggers to key press
{
    trigger();
}
```

Note: Please note that the call of the **trigger()** function must be made from a CAPL program in the logging branch or from CANalyzer's transmit branch.

void writeToLog(char format[], ...);
void writeToLogEx(char format[], ...);

Function:

Outputs a text message to an ASCII Log file. The functions are based on the C function `printf()`. The compiler cannot check the format string. Invalid format entries lead to undefined results.

The function **writeToLog()** outputs a comment symbol (`//`) and time entry at the beginning of the line. The function **writeToLogEx()**, on the other hand, leaves the output format entirely up to the user.

Parameters:

Format string, variables or expressions

Valid format entry: (cf. **write()** function)

setLogFileName(char filename[])

Function:

Sets the name of the log file

Parameters:

New name of the log file. This name can be entered as an absolute path or as a pure file name. If a path is entered, all path levels that do not already exist are created. Entry of a file name causes the log file to be created in the directory of the active configuration. The directories within the path must be delimited by a double backslash (`\\`). The file name may not contain any file extension; this is determined automatically by the system.

Example:

```
// Sets the name of the log file to "newlog"
// in the directory of the active configuration
setLogFileName("newlog");
...
// Sets the absolute path of the log file
setLogFileName("c:\\canwin\\demo_cl\\newlog");
...
```

3.7.4 Standard / Extended Identifiers

long isStdId(dword id)
long isStdId(message m)
long isExtId(dword id)
long isExtId(message m)

Function:

Checks the parameter for extended identifier (29 bit) or standard identifier (11 bit)

Parameters:

Variable of the type "message" or ID portion of a message

Return:

1 if check is successful (true), else 0

Example:

```
if(isExtId(this)) { ... }
std = isStdId(m100.id);
```

long valOfId(dword id)
long valOfId(message* m)

Function:

Returns the value of a message identifier regardless of its type.

Parameters:

Variable of type **message*** or ID portion of a message

Return:

Identifier as Long variable

Example:

```
id = valOfId(this); // Also functions with ext.IDs
```

dword mkExtId(dword id)

Function:

Returns an extended identifier

Parameters:

ID portion of a message

Return:

Extended identifier

Example:

```
msg.id = mkExtId(this.id);
```

3.7.5 Flow Control**stop()**

Function:

In Online mode the running measurement is stopped. In Offline mode the call of **stop()** causes a programmed break in the measurement run. Afterwards, the measurement can be resumed by the main menu functions *Start|Start*, *Start|Animate*, or *Start|Step*.

Example:

```
stop();
```

canOnline()**canOffline()**

Function:

The function **canOffline()** disconnects the node from the bus. Then it is no longer possible to transmit and receive messages. After this function has been called, the node can be reconnected to the bus with the function **canOnline()**. Messages transmitted by the node are then passed to the bus again.

Note: These functions are only available in CANoe's simulation setup. For example, they may be used there with network management - in simulations of the remainder of the bus - to assure that a node simulated in CAPL does not send messages any longer after the transition to bus quiet.

setStartdelay(int delay)

Function:

Sets the value of Startdelay for this network node. This function can only be called in the event procedure preStart. After that, the value of Startdelay can not be changed any more.

Parameter:

delay = an integer for the start delay in ms. This value can be between 0 und 999999. If this value is larger or smaller than this range a warning will appear in the Write window.

int getStartdelay()

Function:

Determines the value set for Startdelay for this network node in the simulation setup.

Return:

Start delay in ms. If no Startdelay was set, the function returns null.

Example:

```
int val;  
on preStart  
{  
    setStartdelay(10000);    // Set Startdelay to 10 Seconds  
}  
val = getStartdelay();    // val is allocated the value of  
                           Startdelay
```

3.7.6 Time Management

setTimer(msTimer t, long duration) **setTimer(timer t, long duration)**

Function:

Sets a timer

Parameters:

Timer or msTimer variable and an expression which specifies the duration of the timer.

Example:

```
/* msTimer t; Timer t1; */  
setTimer(t, 200); /* Set the timer t to 200 ms */  
setTimer (t1, 2); /* Set the timer t1 to 2 s */
```

cancelTimer(msTimer t)

cancelTimer(timer t)

Function:

Stops a running timer

Parameters:

Timer or msTimer variable

Example:

```
/* msTimer t; Timer t1; */  
cancelTimer(t); cancelTimer(t1);
```

Note: A call of **cancelTimer()** is relatively time intensive. Therefore, this function should be used sparingly.

setDrift(int drift)

Function:

A constant variance (deviation) can be set for all timers of a network node with this function. Values between –100.00 % and 100.00 % can be entered. If the value does not lie within this range, an announcement will appear in the Write window.

Parameter:

drift = an integer for constant deviation.

int getDrift()

Function:

Determines the constant deviation when the drift is set.

Return:

Drift in +/- 1000,00%.

Example:

```
int val;  
setDrift(3550);    // sets the drift at 35.5 percent  
val = getDrift();  // assigns the drift to val
```

Note: Setting a drift causes any jitter that may be present to be reset.

setJitter(int min, int max)

Function:

The jitter interval for all timers of a network node can be set with this function. Both values can lie between -100.00 and 100.00 percent. If one of the values is outside this range, an announcement will appear in the Write window.

Parameter:

min = an integer for the lower limit.

max = an integer for the upper limit.

Example:

```
setJitter(-400, 400);    // Sets a jitter of +-4  
                          percent  
setJitter(1300, 2100);   // When the drift is 17  
                          percent (17.00), sets a  
                          jitter of +-4 percent
```

Note: Setting a drift causes any jitter that may be present to be reset. To use jitter and drift at the same time, follow the example.

int getJitterMin() int getJitterMax()

Function:

Determines the upper or lower limit for the permissible deviation when the jitter is set.

Return:

Upper or lower deviation in percent.

Example:

```
int val1, val2;
val1 = getJitterMax();           // assigns the upper jitter
                                // value to val1
val2 = getJitterMin();           // assigns the upper jitter
                                // value to val2
```

setMsgTime(message m, NOW)
setMsgTime(message m1, message m2)

Function:

Sets the acquisition time for a message as the acquisition time for another message or as the actual time. This function is obsolete and its only purpose is to provide compatibility with older versions.

Parameters:

Variable of the type "message" or "now".

Example:

```
setMsgTime(m100, this); /* CANalyzer 1.xx & 2.xx */
m100.time = this.time; /* CANalyzer 2.xx */
setMsgTime (m101, now);
```

dword timeNow()

Function:

Returns the current system time

Return:

System time in units of 10 µsec. This time is determined with the help of the PC timer with a resolution of 1 msec.

Example:

```
float x;
x = timeNow()/100000.0; //Actual time in seconds
```

long timeDiff(message m1, NOW)
long timeDiff(message m1, message m2)

Function:

Time difference between messages or between a message and the current time in ms. The difference can also be calculated directly in units of 10 microseconds (**msg2.time-msg1.time** bzw. **now-msg1.time**).

Parameters:

1. Variable of the type **message**
2. Variable of the type **message** or **now**.

Return:

Time difference in ms;

Example:

```
diff = timeDiff(m100,now); // Time diff. with
                           timeDiff()
diff = this.time-m100.time; // or directly
```

getLocalTime(long time[])

Function:

Provides information on the current time and date in an array of the type long.
The components of the array are filled with the following information:

No.	Meaning	Value Range
1.	Seconds	0 - 60
2.	Minutes	0 - 60
3.	Hours	0 - 24
4.	Day of month	1 - 31
5.	Month	0 - 11
6.	Year	Beginning at 1900
7.	Day of week	0 - 7
8.	Day of year	0 - 365
9.	Flag for Daylight Savings Time entry	0 - 1, 1 = Daylight Savings Time

Parameters:

An array of the long type with at least 9 entries.

Example:

```
long tm[9];
getLocalTime(tm);
// tm now contains the following entries:
// tm[0] = 3; (Seconds)
// tm[1] = 51; (Minutes)
// tm[2] = 16; (Hours)
// tm[3] = 21; (Day of month)
// tm[4] = 7; (Month beginning with 0)
// tm[5] = 98; (Year)
// tm[6] = 5; (Day of week)
// tm[7] = 232; (Day of year)
// tm[8] = 1; (Daylight Savings Time)
```

getLocalTimeString(char timeBuffer[]);()

Function:

Returns the current date and current clock time in the form *ddd mmm dd hh:mm:ss yyyy* (e.g. "Fri Aug 21 15:22:24 1998,") in the buffer that is provided.

Parameters:

Buffer in which the time is written. The buffer must have a capacity of at least 26 characters.

Example:

```
char timeBuffer[64];
getLocalTimeString(timeBuffer);
// Now in timeBuffer, for example:
// "Fri Aug 21 15:22:24 1998"
```

3.7.7 Accessing CANoe Environment Variables

getValue

Syntax

```
int getValue(EnvVarName); // Form 1
float getValue(EnvVarName); // Form 2
long getValue(EnvVarName, char buffer[]); // Form 3
long getValue(EnvVarName, byte buffer[]); // Form 4
long getValue(EnvVarName, byte buffer[], long offset); // Form 5
```

Function:

Determines the value of the environment variable with the identifier EnvVar-Name. The data type of the value returned is determined by the type of environment variable (int for discrete (Form 1), float for continuous environment variables (Form 2)). When (character) string environment variables (Form 3) and

environment variables with data bytes (Form 4 and 5) are used, the most current value is written to a buffer, which is given with the function call.

Parameter

Environment variable name, buffer for the return value (Forms 3, 4 and 5) and, when using Form 5, the offset for the first data byte to be copied.

Return:

Forms 1 and 2 : most current value of the environment variable.

Forms 3, 4 and 5: number of bytes copied into the buffer.

Example:

```
int val;
float fval;
char cBuf[25];
byte bBuf[64];
long copiedBytes;

// assigns the value of the environment variable Switch
// to val
val = getValue(Switch);
// assigns the value of the environment variable
// Temperature to fval
fval = getValue(Temperature);
// read the string environment variable NodeName
copiedBytes = getValue(NodeName, cBuf);
// read data out of the environment variable DiagData
copiedBytes = getValue(DiagData, bBuf);
// read data from the environment variable DiagData at
// Position 32
copiedBytes = getValue(DiagData, bBuf, 32);
```

putValue

Syntax

```
putValue(EnvVarName, int val);           // Form 1
putValue(EnvVarName, float val);         // Form 2
putValue(EnvVarName, char val[]);        // Form 3
putValue(EnvVarName, byte val[]);        // Form 4
putValue(EnvVarName, byte val[], long vSize); // Form 5
```

Function:

Allocates the value val to the environment variable with the identifier EnvVarName. Integers are stored in discrete environment variables (Form 1), floating

point numbers in continuous environment variables (Form 2). The contents of a character string are copied into string environment variables (Form 3). When environment variables with data bytes are used (Forms 4 and 5), the data bytes are copied from the buffer into the environment variable.

Parameter:

Environment variable name, new value of the environment variable (Forms 1 and 2), or buffer with the new data (Forms 3, 4 and 5). When Form 5 is used, the number of bytes to be copied.

Example:

```
byte dataBuf[64];  
// Assign the value 0 to the environment variable Switch  
putValue(Switch,0);  
// Assign the value 22.5 to the environment variable  
  Temperature  
putValue(Temperature, 22.5);  
// Copy the string "Master" into the environment  
  variable nodeName  
putValue(NodeName, "Master");  
// Copy 64 bytes of the contents of the buffer into the  
  environment variable DiagData  
putValue(DiagData, dataBuf, 64);
```

callAllOnEnvVar()

Function:

Executes the **on envVar ...** event procedures for all environment variables. This may be necessary at the measurement start to initialize environment variables, set timers that are activated when environment variable values change, or to send messages on the bus with start values of environment variables.

Example:

```
on start {
    write("Traffic light module started");

    // Call all envvar procedures of this model and
    // thus consider the START VALUES of all environment
    // variables for:
    //     initialization of all message variables
    //     starting of any timers
    //     sending messages (output) with start values
    CallAllOnEnvVar();
}
```

3.7.8 Intel/Motorola Formats

Arithmetic in CAPL is performed in Little Endian Format (Intel). The following swap functions are used to interchange bytes in the transition from and to Big Endian Format (Motorola):

word **swapWord(word x)**
int **swapInt(int x)**
dword **swapDWord(dword x)**
long **swapLong(long x)**

Function:

Interchanges the bytes of the parameter

Parameters:

Value whose bytes are to be swapped.

Return:

Value with bytes interchanged.

Example:

```
bigEndian = swapInt(1234); /* Generation of constant
                           1234 for Motorola processor */
```

3.7.9 Trigonometry and Mathematical Functions

double sin(double x)

Function:

Calculates the sine.

Parameters:

Value in radians whose sine is to be calculated.

Example:

```
x = sin(PI); /* Result 0.0 */
```

double cos(double x)

Function:

Calculates the cosine.

Parameters:

Value in radians whose cosine is to be calculated.

Example:

```
x = cos(PI); /* Result -1.0 */
```

double sqrt(double x)

Function:

Calculates the square root

Parameters:

Value whose square root is to be calculated.

Example:

```
x = sqrt(4.0); /* Result: 2.0 */
```

double exp(double x)

Function:

Calculates an exponential function

Parameters:

Exponent to base e

Example:

```
x = exp(1.0); /* Result: 2.7182... */
```

long abs(long x)

double abs(double x)

Function

Returns the absolute value

Parameters:

Value whose absolute value is to be returned.

Return:

Absolute value of x.

dword random(dword x)

Function:

Calculates a random number between 0 and x

Parameters:

Maximum value that the function should return

Example:

```
x = random(2500); // Generates random number between 0
                  and 2500
```

3.7.10 File Functions

It is often necessary to save variables or measurement parameters over multiple measurements. File functions were made available to CAPL for this purpose. The files used have the same structure as the widely-used format of MS-Windows INI files. File accesses were conceived for ease of use and reliability. Therefore, they are not very fast.

The opportunity to use these file functions presents itself in the *on start* event procedure or when a value changes.

To ensure real-time capability the files used are loaded before the start of measurement.

long fileWriteString(char section[], char entry[], char value[], char filename[])

Function:

Opens the file *filename*, finds the section *section* and writes the value *value* to the variable *entry*. If *entry* already exists, the old value is overwritten. The result of the function is the number of characters written, or 0 in case of error.

long fileWriteInt(char section[], char entry[], long def, char filename)

Function:

Analogous to the function **fileWriteString()**, but instead of writing a text, it writes a Long variable to the file. If the function result is 0, an error has occurred; otherwise the function was executed successfully.

Example:

```
fileWriteInt("DeviceData", "DeviceAddr", 2, "TEST.INI");
```

This call writes the following entry to the file TEST.INI:

```
[DeviceData]  
DeviceAddr=2
```

long fileWriteFloat(char section[], char entry[], float def, char filename[])

Function:

Analogous to the function **fileWriteString()**, but instead of writing a text it writes a float variable to the file.

**long fileReadString(char section[], char entry[], char def[], char buffer[],
long buflen, char filename[])**

Function:

Finds the variable *entry* in the section *section* of the file *filename*. Its contents (value) are written to the buffer *buffer*. Its length *must* be passed correctly in *buflen*. If the file or *entry* is not found, the default value *def* is copied to *buffer*.

long fileReadInt(char section[], char entry[], long def, char filename[])

Function:

Finds the variable *entry* in the section *section* of the file *filename*. If its value is numeric, this number is returned as the function result. If the file is not found or *entry* is not found, or if *entry* does not contain a valid number, the default value *def* is returned as the function result.

Example:

```
myAddress=fileReadInt(  
    "DeviceData", "DeviceAddr", 0, "TEST.INI");
```

If this call is made according to the notation given in the example for **fileWriteInt()**, the variable *myAddr* is assigned the value 2. If the entry *DeviceAddr* does not exist in the file TEST.INI, the default value 0 is assigned.

float fileReadFloat(char section[], char entry[], float def, char filename[])

Function:

Analogous to the function **fileReadInt()** for floating point numbers.

long fileReadArray(char section[], char entry[], char buffer[], long bufferlen, char filename[])

Function:

Searches for the variable *entry* in the section *section* of the file *filename*. Its contents are interpreted as a list of byte values. The numbering format is decimal or - if it has the prefix 0x - hexadecimal. The numbers are delimited by a blank character, tab, comma, semi-colon or slash. The *buffer* is filled with up to *bufferlen* bytes.

Return:

Number of characters written.

Example:

File TEST.INI:

```
[DATA]
FIELD=1, 2, 3, 0x20, 100
```

```
int len;
char buf[20];
len = fileReadArray(
    "DATA", "FIELD", buf, elCount(buf), "TEST.INI");
```

The result *len* is 5. The array *buf* is filled with the values 1, 2, 3, 32, 100.

3.7.11 Sequential File Access

The CAPL compiler provides you with various functions for sequential access to files:

long seqFileLoad(char fileName)

Function:

This function opens the file with the name *fileName* for read access.

Return:

The return value is the file handle, which must be passed as a parameter for read operations. If an error occurs, the return value is an error code ≤ 0 .

long seqFileClose(long file)

Function:

This function closes the file specified by the handle *file*. All buffers used by the system are cleared upon closing.

Return:

If successful the function returns the value 0. If an error occurs a nonzero error code is returned.

long seqFileGetBlock(char buffer[], dword bufferSize, long file)

Function:

The function reads at most *bufferSize* characters from the file identified by the handle *file* to the array *buffer*. In doing so, the position pointer is incremented by the number of characters successfully written. If an error occurs the value of the position pointer will be indeterminate.

Return:

The return value contains the number of characters successfully read. It may be smaller than *bufferSize* if an error occurred or the end-of-file was reached.

long seqFileGetLine(char buffer[], dword bufferSize, long file)**long seqFileGetLineSZ(char buffer[], dword bufferSize, long file)**

Function:

The function reads at most *bufferSize*-1 characters from the file specified by the handle *file* to the array *buffer*. After an end-of-line or end-of-file character, no further characters are read. The end-of-line character is not copied to *buffer*.

The function **seqFileGetLine()** does *not* terminate the character string with '\0'; **seqFileGetLineSZ()** terminates the character string with '\0'.

Return:

The return value contains the number of successfully read characters. It is negative if an error occurs.

Note: The CAPL function **seqFileGetLineSZ()** corresponds to the function **fgets()** of the standard C library.

long seqFileRewind(long file)

Function:

The function resets the file specified by the handle *file* back to the beginning.

Return:

If successful the function returns the value zero. If an error occurs a nonzero error code is returned.

Example

In the sample CAPL program the test file TEST.DAT is opened at measurement start, and after pressing the 'a' key it is copied byte-by-byte to message 100 and is transmitted on the bus in blocks of 8 bytes at 10 ms intervals. Pressing the 'r' key causes the file to be reset to its beginning, and it is re-transmitted on the bus.


```

variables {
    message 100 msg = {DLC=8};
    char filename[50] = "test.dat";
    long handle;
    msTimer ReadNextBlock;
}

on start {
    // /Open data file on start of measurement
    handle = seqFileLoad(filename);
    if (handle <= 0) {
        write ("Error opening file");
        write("Error code = %d", handle); stop();
    }
    else {
        write ("File %s opened",filename);
        write ("Press key a to start reading the file");
    }
}

on stopMeasurement {
    // Close data file on stop of measurement
    if (handle > 0) {
        if (seqFileClose(handle)==0) write("closed");
        else write ("error closing file");
    }
}

// Start file transmission
on key 'a' { setTimer(ReadNextBlock,10); }

// Reset file transmission
on key 'r' {
    seqFileRewind(handle);
    setTimer(ReadNextBlock,10);
}

// Transmit data file in blocks of 8 bytes
on timer ReadNextBlock {
    int n;
    char buffer[8]= "          ";
    n = seqFileGetBlock(buffer,8,handle);
    msg.byte(0)= buffer[0]; msg.byte(1)= buffer[1];
    msg.byte(2)= buffer[2]; msg.byte(3)= buffer[3];
    msg.byte(4)= buffer[4]; msg.byte(5)= buffer[5];
    msg.byte(6)= buffer[6]; msg.byte(7)= buffer[7];
    output(msg);
    if (n >= 0) setTimer(ReadNextBlock,10);
    else write("No character read");
}

```

3.7.12 String Functions

long atol(char s[])

Function:

This function converts the string *s* to a LONG number. The numbering system is decimal. If the string begins with 0x, base 16 is used. Leading blank characters are skipped over when reading.

void ltoa(long val, char s[], long base)

Function:

The number *val* is converted to a string *s*, whereby *base* indicates the numbering system base between 2 and 36. The string *s* must be large enough to accept the converted number!

long snprintf(char dest[], long len, char format[], ...)

Function:

This function corresponds to the C function `snprintf()`. Additionally, the parameter *len* indicates the maximum length of the array *dest*. The total length of the resulting string must not exceed 100!

The function result is the number of characters written. The format string has the same meaning as for the function `write()` and is described there.

long strlen(char s[])

Function:

The function result is the length of the string *s*.

strncat(char dest[], char src[], long len)

Function:

This function appends *src* to *dest*. The parameter *len* indicates the maximum length of *src* and *dest*. This function assures a terminating '\0'. Consequently, a maximum of *len*-1 characters are copied.

strncpy(char dest[], char src[], long len)

Function:

This function copies *src* to *dest*. The parameter *len* indicates the maximum length of *src* and *dest*. The function assures a terminating '\0'. Consequently, a maximum of *len*-1 characters are copied.

long strncmp(char s1[], char s2[], long len)

Function:

This function compares *s1* with *s2* for a maximum of *len* characters. If they are identical the function result is 0. If *s1* is less than *s2* the result is -1, else it is +1.

3.7.13 Speech Support and Debugging

long elCount(...)

Function:

Determines the number of elements in an array.

Parameters:

Array of any desired type

Return:

Number of elements

Example:

```
void bsp(int ar[])
{
    int i;
    for(i=0; i < elCount(ar); i ++)
        ...
}

void bsp2(byte ar[][] )
{
    int i, j;
    for(j=0; j < elCount(ar); j ++ )
        for(i=0; i<= elCount(ar[j]); i ++ )
            ...
}
```

runError(long err, long)

Function:

Initiates a run-time error. Outputs an error message in the Write window, giving the error number and the passed number, and it then terminates the measurement.

Parameters:

Numbers which are represented as instructions in CANalyzer. The values less than 1000 are reserved for internal purposes. The second parameter is reserved for future expansions.

Example

```
if (RPM < 0) runError(1001,1);
```

fileName()

Function:

Output of CAPL program name in the Write window. Helpful for debugging purposes.

Example:

```
fileName();
```

3.8 Examples

To practice running the following example it is best if you always begin with a new, empty configuration (Menu item: *File|New configuration*). A CAPL program always runs in a CAPL program block, which you can insert at any of the hotspots in the measurement setup .

Note: CAPL programs which are to transmit messages *must* always be inserted in the transmit branch of CANalyzer's measurement setup. The graphics menu indeed allows you to insert CAPL programs directly before the analysis blocks in the measurement setup. However, since the data flow in the measurement setup is directed from left to right, messages that are transmitted from there only reach the analysis blocks to the right and do not reach the CAN bus. Therefore, you should only insert CAPL programs intended for analysis here.

Then, in the popup menu assign a file name to the CAPL program blocks you have inserted in the measurement setup , e.g. EXAMPLE1.CAN. From the *Edit* menu open the CAPL Browser (cf. section 3.3), with which you can enter and compile the sample programs. After compiling you can immediately test the programs by starting the measurement.

Make sure that you parameterize the CAN controllers properly before the measurement start. In each case you must configure the bus parameters correctly. To follow along and practice running the sample programs you should connect the two CAN controllers on your card to one another using the supplied cable ("short circuiting"). This gives you a real CAN bus with two stations that you can control entirely from CANalyzer. In this case, remember to set the bus parameters so that they are exactly the same for both controllers.

3.8.1 Transmission of Messages

This example relates to the transmission of messages. First we will show how to transmit cyclically. Then the example will be expanded to transmission upon a key press and transmission as a reaction to the receipt of a message.

Task statement:

Messages with the identifier 180 should be cyclically transmitted every 100 msec. The first two data bytes should be interpreted as 16-bit variables, and they should be incremented by 10 after each transmission.

Solution:

First a global variable is declared with the name **sendTimer** and type **msTimer** (Timer based on milliseconds). This is done in Browser's upper right pane:

```
variables {  
    msTimer sendTimer; /* Timer for period. Send */  
}
```

To activate the timer generated in this way, it must be set in the start event procedure. To input the start event procedure select the event type **System** in Browser tree. In the popup menu of the procedures text editor you then select a new **Start** event procedure which is called at the measurement start.

The only action at the measurement start is to set **sendTimer** to 100 msec.

```
on start {  
    setTimer(sendTimer,100); /* Schedule timer */  
}
```

To define what should occur when the timer elapses you now set up a timer event procedure. To do this, select the event type **Timer** in the Browser tree, and create a procedure of this type. As a result, a template for a timer event procedure appears in the Procedures Text Editor, in which you enter the name of the particular timer in front of the opening procedure bracket, in this case **sendTimer**.

Within the procedure brackets, first a local variable is declared: **sendMsg** of the type **message** with identifier 180. The variable **sendMsg** is initialized with data length 2 and data contents 0.

The first instruction in the event procedure is to reset the timer **sendTimer** to 100 msec. This causes the event procedure to be run cyclically every 100 msec. Then the **output()** function outputs the message **sendMsg** at the program block's output, and it is sent on the bus. After transmission the content of the first data word is increased by 10.

After compiling this event procedure the name **sendMsg** appears in the procedure window. It can be selected directly for further handling.

```
on timer sendTimer {

    message 180 sendMsg = { DLC = 2 };

    // Schedule timer period.
    setTimer(sendTimer,100);
    output(sendMsg); // Send
    // Increment data word by 10
    sendMsg.word(0) = sendMsg.word(0) + 10;
}
```

The program is started from CANalyzer's main menu item *Start|Start*. The transmitted messages can be observed in the Trace and Statistics windows. It is transmitted as long as desired until the measurement is ended by pressing the <ESC> key.

Task solution with symbolic database

The **on timer** procedure used solves the task by accessing the message identifier and data bytes directly. This is allowed and is quite appropriate to the task. Nevertheless, the CAPL program cannot be well-maintained flexibly by mixing the application (output of a counter value) and bus-related information (message, data bytes): For example, the first two data bytes of the message would no longer be sufficient if the task were now expanded to have the counter also capture values up to 100,000. The CAPL program would therefore have to be modified.

To better separate the application task from data transport, CANalyzer offers you the option of describing CAN data symbolically. Instead of working with identifiers and data bytes, you then work with symbolic names in CAPL. Then task expansions requiring a change to the data transport layer can - for the most part - be handled by modifying this symbolic description. It is then no longer necessary to make a change to the application i.e. in this case a change to the CAPL program.

A database for the above task statement should include a symbolic description of the message with identifier 180 and a signal *Counter* which is comprised of the first two bytes of this message:

Message Name	StateInfo	Signal name	Counter
Message ID:	180	Signal type	Standard signal
Number of bytes	2	Start bit	0
		Number of bits	16

Therefore, the **on timer** procedure for solving this task could be written as follows:

```
on timer sendTimer {

    message StateInfo sendMsg; // DLC from database

    setTimer(sendTimer,100);
    output(sendMsg);
    // Increment counter by 10
    sendMsg.Counter = sendMsg.Counter + 10;
}
```

Please note that the type definition for the message to be sent includes both the key word **message** and the message name **StateInfo**. Only afterwards does the variable name **sendMsg** occur. To react to messages in event procedures you use the symbolic message name from the database and no variable names. For example, you would write **on message StateInfo {}** to react to the occurrence of the message with identifier 180.

First task expansion:

Supplementally, now the message with identifier 210 should be transmitted whenever the spacebar is pressed. The data contents of byte 3 should indicate how often the message has already been transmitted.

Solution:

You should set up a key event procedure to define what should occur in response to pressing the spacebar. The name of a key event procedure is either the corresponding ASCII character in single quotation marks or a symbolic name (cf. section 3.5.4). In the Browser tree select the event type **Keyboard**. In the template for the new key event procedure, replace the text **<newKey>** by the spacebar character ' '.

Within the procedure brackets, first the local variable **spaceMsg** is declared. This message is transmitted when the procedure is run through. The 3rd data byte is used directly as the counter for transmission frequency.

```
on key ' ' {
    message 210 spaceMsg = {DLC=4, DIR=TX, byte(2)=0};

    output(spaceMsg); /* Send */
    spaceMsg.byte(2) = spaceMsg.byte(2)++; /*Count along*/
}
```

Second task expansion:

Supplementally, now message 121 should be sent out in response to receiving the message with identifier 120. Furthermore, for test purposes a screen message should also be output.

Solution:

You would set up an appropriate message event procedure to define what should occur when a specific message is received. This involves creating an event procedure of the type **on message**. In the procedure template you would replace the text `<newMessage>` by the identifier to which the reaction should take place: **120**.

Within the procedure brackets, first the local variable **responseMsg** is declared. When the procedure is run through, receipt of the message 120 is reported on the screen via the **write()** function. Afterwards, the reply message is transmitted.

```
on message 120 {  
  
    message 121 responseMsg = { DLC = 0, DIR = TX };  
  
    write("Msg 120 received");  
    output(responseMsg); /* Send reply */  
}
```

In the event procedure the following condition could be used to determine whether a message was actually received or perhaps transmitted:

```
if (this.dir == RX) { ... }
```

Since this query is not used, the event procedure reacts identically to received and transmitted messages.

It would only be possible to test this event procedure if a message with identifier 120 were actually received (or transmitted). This can be generated by a separate bus station or by CANalyzer itself. A quick test might be, for example, to select the identifier 210 instead of identifier 120 - as programmed above - which is sent when the spacebar is pressed.

3.8.2 Gateway Application

In the following example, a gateway application is to be programmed, in which the two buses are connected by CANalyzer. Additionally, a PC card must of course be implemented with two CAN controllers, each with separate bus connection leads. In contrast to the previous example, here CAPL programming must include a precise specification of which bus should be used to transmit or receive messages.

Task statement:

A gateway should be programmed between two buses, which essentially lets all messages pass unhindered in both directions. However, when the message with identifier 34 is transferred from Bus 1 to Bus 2 it should be "falsified" by setting byte 4 to 0.

Solution:

A node is inserted in CANalyzer's transmit branch, to which the CAPL program described below is assigned.

Message 34 is modified as follows in the transfer from Bus 1 to Bus 2:

```
on message CAN1.34 {    /* On receipt of message 34 by
                        Bus 1 */

    message CAN2.34 sendMsg;
    if (this.dir!=RX) return; // React only to receiving
    sendMsg=this; // Copy receive message to transmit
                        message
    sendMsg.byte(4) = 0; // Change contents of message
    output(sendMsg); // Transmission
}
```

A bridge function is implemented for the remaining messages: All messages from Bus 1 are transmitted to Bus 2, and vice versa.

```
on message CAN1.* {
    // Entry of '*' identifies all messages
    // that are not processed in any other
    // procedure.
    message CAN2.* sendMsg;
    if (this.dir!=RX) return;
    sendMsg = this;
    output(sendMsg);
}

on message CAN2.* {
    message CAN1.* sendMsg;
    if(this.dir!=RX) return;
    sendMsg = this;
    output (sendMsg);
}
```

3.8.3 CAPL-Driven Triggering

In this section we will show, with a very simple example, how a CAPL program can be used to initiate triggering of data recording to a file (Logging).

Task statement:

The user is observing the system on the screen and would like initiate triggering by a key press ('t' key).

Solution:

In parameterizing the logging block, CAPL is selected as the *Trigger type*. As a result, data logging will be initiated by the CAPL function **trigger()**. Please note that the function only triggers logging if it is located before the logging branch or is in CANalyzer's transmit branch.

The CAPL program consists of a single event procedure for the 't' key, in which triggering is initiated.

```
on key 't' { // On key press of the key 't'
            // (Without shift)

    trigger(); // Initiate triggering
}
```

Naturally, there are also CAPL programs for triggering which look significantly more complicated. For example, by adding a message event procedure triggering could also be initiated when a certain message with certain data contents is received.

Since global variables (Flags) can be introduced, you can also formulate trigger conditions that are a function of a combination of different events.

3.8.4 Use of the Symbolic Database

In this example we will show how symbolic signals can be accessed. This requires that a project database be loaded in CANalyzer.

This database must contain a message with the symbolic name **ioData** on which the engine speed information is coded in the signal **N**. It is also necessary to have another message with the name **EngineData** and the signal for the zero moment **NM**.

Task statement:

Available on CAN2 from an analog I/O module is the scaled engine speed information **N**. This should be used to calculate a zero moment signal **NM** which is to be output cyclically on CAN1.

Solution:

The value of the **N** signal is acquired, and is converted to **NM** which is saved temporarily in a variable **nullMoment**. This value is accessed in the timer function, and the cyclic output is generated:

```
variables {
    int nullMoment = 0;    /* Calculated value */
    int c1 = 10, c2 = 20; /* Conversion constants */
    msTimer t1;           /* Timer */
}
```

```
on start {
    setTimer(t1,100);    /* Setting of timer */
}
```

```
on message CAN2.ioData {
    nullMoment=this.N*c1+c2; /* Linear transformation */
}
```

```
on timer t1 {
    message CAN1.EngineData m; /* Data carrier */
    setTimer(t1,100);          /* Setting of timers */
    m.NM = nullMoment;          /* Entry of active
                                value */
    output(m);                  /* Send out on CAN1 */
}
```

Task expansion:

Calculation of the zero moment should be table-based.

Solution:

A table is created with result values. The table is accessed by means of indices, whereby the value range is checked.

```
variables {
    int nullMoment = 0;          /* Calculated value */
    int values[20] = {           /* Table */
        0,0,3,8,15,24,35,48,63,80,
        99,118,140,164,190,218,220,
        218,210,195
    }
    msTimer t1;                  /* Timer */
}
```

```
on message CAN2.ioData {
    int index;                   /* Auxiliary variable */
    index = this.N;              /* Value as index */
    if (index < 0) index = 0;     /* Range check */
    else if (index>=20) index = 19;
    nullMoment = values[index]; /* Conversion
                                with table */
}
```

A typical task expansion would be to execute the computation as an interpolation over a data array.

Note: Please note that in this solution information on the CAN protocol layer flows into the CAPL program: In this recommended solution the CAN controller numbers are part of the CAPL program. If the user wanted to connect to the card's CAN controllers in another way, the program code would have to be modified. Section 4.3.1 describes how you could avoid these undesired dependencies of the CAPL application on the concrete configuration.

4 Special Topics

4.1 System Description

4.1.1 Overview of the Programs

The following executable programs are part of CANalyzer:

- With the CANdb++ Editor you create or modify databases (*.DBC) which contain the symbolic information for CANalyzer. This includes network nodes and symbolic names for messages and signals.
- In the CAPL Browser you create CAPL programs for the transmit and analysis branches of the measurement setup. Instead of using message identifiers and data bytes, with the help of the database you can also work with message and signal names.
- The CANalyzer main program is used to measure and simulate CAN systems. You can associate one or more databases to any configuration from *File|Database*.

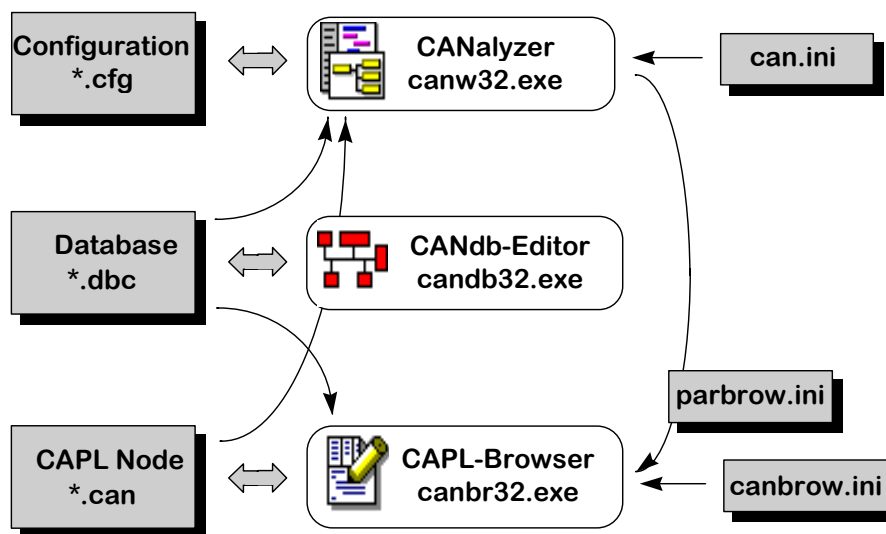


Figure 68: CANalyzer system overview

Start options for CANalyzer and Browser are provided in the linked INI files. If you are starting Browser from CANalyzer's measurement setup, a temporary file PARBROW.INI is automatically generated with the proper start options and is passed to Browser.

4.1.2 CANalyzer Architecture

In the course of a measurement the PC-card registers CAN messages on the bus and passes them to the measurement setup, and from there to the specified paths in the data flow plan and on to the evaluation and analysis blocks at the far right of the

plan. During a measurement two program modules work closely together to this purpose: First the CANalyzer real-time library (CANRT.DLL) retrieves the information arriving at the card, provides them with a time stamp and shifts them to a ring buffer.

In the course of a measurement the PC-card registers CAN messages on the bus and passes them through the simulation setup to the measurement setup, and from there to the specified paths in the data flow plan and on to the evaluation and analysis blocks at the far right of the plan. During a measurement two program modules work closely together to this purpose: First the CANalyzer real-time library (CANRT.DLL) retrieves the information arriving at the card, provides them with a time stamp and shifts them to a ring buffer.

In a second step these data are read out by the actual main program (CANW32.EXE) and are evaluated in the function blocks on the right-hand side of the data flow plan.

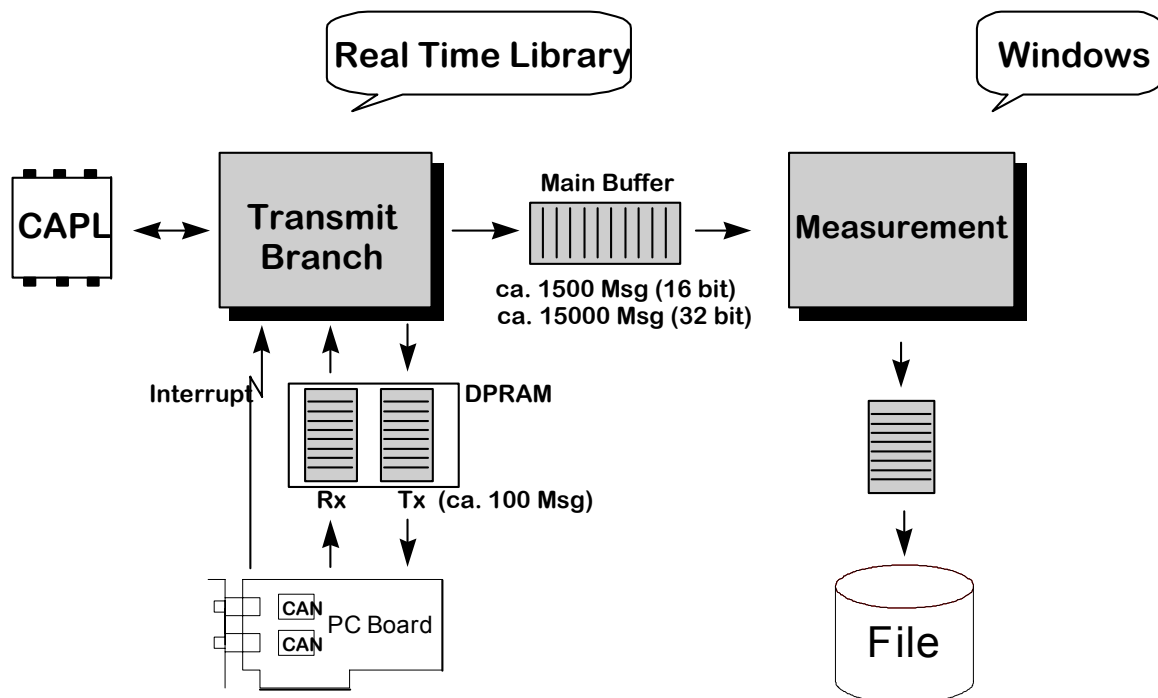


Figure 69: Internal structure of CANalyzer

You can influence the data flow in both program modules by inserting function blocks in the measurement setup. In this context the real-time module is comprised of the PC-card block, all function blocks between the card block and the transmit branch, and all blocks in the transmit branch itself. All other function blocks are used to configure the data flow in the evaluation branch.

If you insert blocks in the real-time library, i.e. CANalyzer's transmit branch/, you should be make sure that they do not demand too much computing time, so that system reaction times are not lengthened. Moreover, in CAPL programs you may only access files from here using special precautionary measures (cf. section 3.7.10).

Note: If you overload Windows severely by other programs during a measurement, there may be a delay in reading data out of the ring buffer. Nevertheless, the time stamp for the events, which for example is displayed in the trace window, remains accurate even in this case.

4.2 CANalyzer in Load and Overload Operation

4.2.1 Behavior in Load Situations

At high bus loading the computing power of your PC may be insufficient - under some circumstances - to simultaneously operate the more complex evaluation and display functions (Statistics window with statistical report, Data and Graphic window with many signals, Trace window in the chronological output mode, etc.). To prevent an impending data loss the program therefore has mechanisms for detecting and handling load situations.

If the rate of messages registered by the card is so high that CANalyzer cannot follow along with processing any longer, the ring buffer between the real-time library and the CANalyzer- application runs full. CANalyzer detects this when a "High water mark" limit is exceeded (cf. Figure 70), and it automatically switches over to a load mode in which display functions are reduced to provide more computing time for internal data processing.

When the high-load limit of the ring buffer is exceeded, the display of messages in the Trace window is interrupted briefly under load operation to provide other analysis blocks with more computing power. However, it is possible that not all messages will be displayed in the window any longer during the measurement. You can recognize this load situation during the measurement by an exclamation point (!) in the first column of the window. Although not all messages are shown, no data are lost. After you stop the measurement, the entire set of information is available to you in the Trace window, Graphic window and in logging.

4.2.2 Behavior with Data Loss

If the ring buffer overruns in spite of these measures, as a user you are immediately informed of this data loss:

An occurring data loss is registered in ASCII logging. The '*' symbol appears in the line after which the data loss occurred. In the configuration dialog for the Log file the *Lost Data Message Box* option also allows you to have a data loss shown in a separate message window at the end of measurement.

The Bus Statistics window also indicates to you a data loss during an overload situation with the '@' symbol. However, please note that the display of bus load and received messages continue to function properly, since this information is provided by the interface card and does not need to be computed by the main program first. Consequently, the Bus Statistics display allows you to estimate the extent of the data loss.

4.2.3 Fill Level Indicator

To allow you to observe the ring buffer between the real-time library and the main Windows program more precisely, the program has a fill level indicator.

You can view the fill level during the measurement in the left corner of the status bar. If the ratio between arriving and processed events is balanced, the indicator has a green color. However, if it appears in red, significantly more events are arriving than can be processed at the given time. This is a clear indication that the system is overloaded. If the queue fill level reaches an alarming limit, the system attempts to relieve the load by selectively deactivating individual evaluation windows. If desired, you can have the queue's state displayed with the help of the entry `ShowMainQueue=0` in the `[System]` section of the `CAN.INI` file.

You can also have the fill level displayed in the Write window. To do this, set a minimum value of `WriteLevel = 3` in the `[Environment]` section of the `CAN.INI` file. Then, when the ring buffer overflows the Write window shows the report "Load transition: NORMAL->DATA LOST" and informs you of the data loss. After the overload situation has ended (e.g. after a brief burst on the bus) you are similarly informed as soon as a normal situation has been restored. You can recognize this in the Write window by the report "Load transition: QUEUE HIGH -> NORMAL".

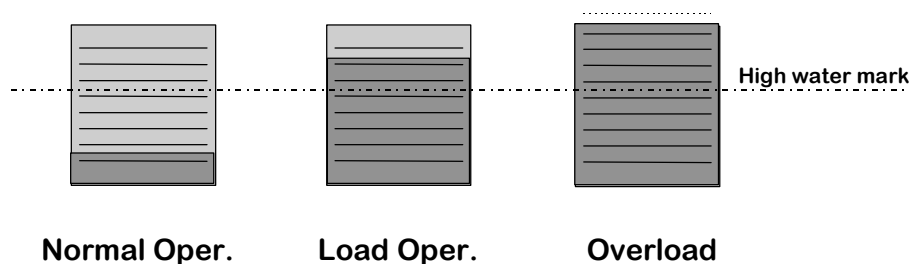


Figure 70: Fill Level of the Ring Buffer

For testing purposes you can provoke overload situations yourself by taking hold of the title bar of the main window with the mouse or by moving the window on the screen. This will interrupt the main program's data display until you release the title bar again. The work of the real-time library, however, remains unaffected by these actions. If messages are being registered on the bus, the ring buffer will be filled without the data being able to be processed by the main program, and as a result the queue will overrun. As soon as you release the title bar you can observe the effects of this overrun in the individual windows.

4.2.4 Optimizing Performance

To make it easier for you to configure CANalyzer for high bus loads, a performance wizard is provided under the menu items *Help|Performance|Optimization* and *Help|Performance|Optimization 2*. This wizard performs a two-stage performance optimization. A report is generated in the Write window listing the configuration options utilized in the optimization as well as their previous and new values. You can undo the changes made at any time.

The wizard optimizes the following options:

- Output mode and update cycle for the Trace window
- Update cycle for bus statistics

- Trigger option *Write full buffer to file* (Logging)
- Buffer size for the logging trigger
- Update mode and update cycle for the Data window
- Buffer size and drawing mode for the Graphic window.

The wizard function *Help|Performance|Undo* cancels the changes of a previous optimization of configuration options. The operations required for this are also summarized in a report.

Please note that only those values can be restored which have not been changed in the configuration dialogs since the optimization.

4.2.5 Configuration Options at High Bus Load

Besides the automatic deactivation function, which CANalyzer initiates in load situations, you can manually switch the following analysis blocks to less computing-intensive modes:

Trace Window

In the Trace window choose the output mode *Fixed position for cyclic update*. As a result, the window contents are no longer updated with each new arriving message, rather they are only updated cyclically. If necessary select a longer update time.

Data Window

If you have configured many signals in the Data window, you should choose the cyclic refresh mode here (*Configure Timer* entry in the Data window's popup menu) and enter a cycle time of maximum 500 milliseconds. The window is then only updated cyclically which saves on computing resources.

Graphic Window

If you have configured many signals in the Graphic window, you should choose a relatively large user-defined refresh (200 ms to 2 s) in the Measurement Setup Dialog (*Options* item in the popup menu) . This defines how often the graphic window display should be updated. Small values result in a fluid representation of the signal response, but they place high demands on computer resources, and with slower computers this might lead to performance problems. High values, on the other hand, reduce computing demands but lead to a more disturbed and delayed display of the signal response.

Statistics Window

Deactivate the statistics report in the Statistics window's popup menu to save on computing power. Furthermore, the averaging time can be selected in the configuration dialog of the Statistics block. Short time intervals place high demands on computer resources and might result in severely oscillating lines in the window. Very long averaging times make the display slower, but also less computing intensive.

If you insert blocks in the real-time library, i.e. in CANalyzer's transmit branch, you should ensure that they do not demand too much computing time, so that they do not increase system reaction times. Furthermore, you may only access files in CAPL programs if you observe special preventive measures (cf. section 3.7.10)

On the other hand, it may be advisable at high bus loading to perform a data reduction as early as in the real-time library to relieve loading in the evaluation branches of CANalyzer.

It is not possible to predict an optimal configuration of the measurement setup for all situations. Cyclic updating indeed saves computing time, but also leads to poorer representation of the information. Under some circumstances it may be advisable to completely disconnect analysis branches that are not needed in the measurement setup (*Insert Break* item in the hot-spot's popup menu) or reduce the volume of data at the input to the measurement setup using filter functions. Moreover, you might also try to insert individual CAPL programs between the real-time library and the evaluation branches and observe the behavior of the program during another measurement run.

To filter out certain messages from the measurement setup, pass filters and blocking filters are provided as insertable function blocks. Furthermore, the supported PC cards with acceptance filtering (*Messages* item in the popup menu of the card icon in the measurement setup) also offer you the option of filtering out certain messages in hardware, thereby relieving both the real-time library and the main Windows program of the need to evaluate unnecessary information.

4.3 Working with Databases

In CANalyzer's symbolic mode you can address CAN messages and data contents by the symbolic names of the associated databases. The sections below offer you instructions which you should observe if you use databases when working with CANalyzer.

4.3.1 Associating the Database

In the database selection dialog opened by the menu command *File|Associate database*, you define the databases with which you want to work.

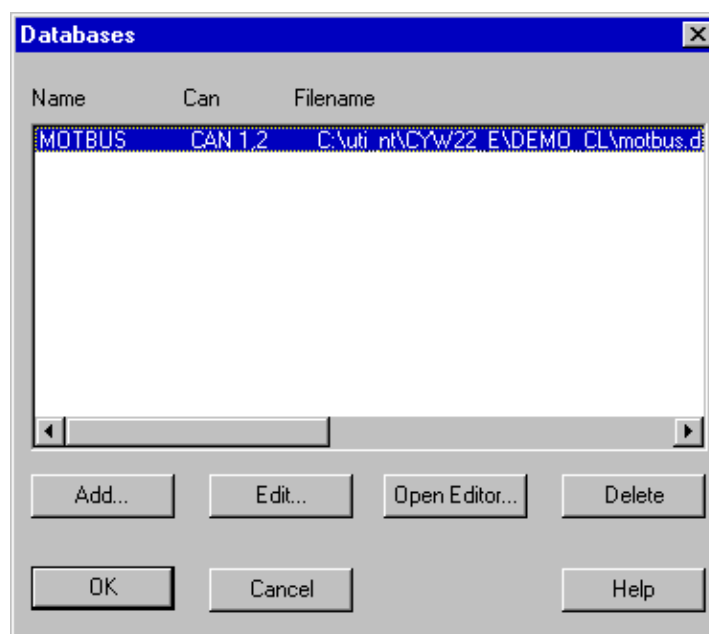


Figure 71: Database selection dialog

Under *Add...* you can associate new databases in a file selection dialog. If no line is highlighted in the list of databases, the new database is added at the end. Otherwise it is added above the highlighted line. You can also highlight the empty last line to have the new database inserted at the end of the list.

In the database list, the first column shows the database name which, for example, might be used to resolve ambiguities as a name qualifier. In the second column are the CAN chips assigned to the database. In the last column you will find the complete file name for the database. The sequence of databases in the list is utilized to resolve ambiguities of symbolic names and message identifiers.

By activating the *Edit...* button you arrive at the Edit Database dialog. Here you can define the database name as well as the assignment of the database to a CAN chip.

The name of the database is determined as follows. If the database includes the attribute *DBName*, then its value is used as the name. If the attribute does not exist, the name is derived from the file name. The user can overwrite the name. Database names must begin with a character and may not contain any blank or special characters.

The database name must be unique. If a name that is automatically determined already exists, it is assigned a unique name according to the sequence of names *CANdb1*, *CANdb2*, etc.

You can assign both CAN controllers (CAN1 and CAN2) to exactly one database. All other databases are assigned to the two controllers jointly. This assignment defines the default selector for message definitions in CAPL.

4.3.2 Use of Multiple Databases

For large systems it may be sensible to distribute the descriptions of messages and signals to several partial databases. When operating CANalyzer on two buses it also makes sense to describe each system by its own database.

CANalyzer supports the simultaneous use of multiple databases. You can configure the databases that you would like to associate with CANalyzer using the menu command *File|Associate database*. Afterwards you can use the symbolic names for messages and signals from all databases in all function blocks and in CAPL. To do this, you would simply enter the symbolic name in the appropriate input box. There is a list of all symbolic names in the signal selection dialogs which you can open by activating the small buttons next to the individual input boxes. You would select the desired symbolic names there.

If you are using more than one database, the messages in the databases that follow the first database are *qualified* with the database name, i.e. the message name is preceded by the database name followed by two colons. However, you will only need to use these qualified names to resolve ambiguities. As long as the symbolic names are unique among all databases, you can forego qualification of symbolic names in all function blocks and when editing CAPL programs.

4.3.3 Resolving Ambiguities

When multiple databases are used, it is essentially possible to have ambiguities in the use of symbolic names. These ambiguities must then be resolved by the program. On the one hand, messages arriving from the bus via one of the two CAN controllers and logged by the program can have different symbolic names in two databases. On the other hand, the user may wish to configure function blocks or measurement windows with different messages that have the same name in different databases.

Ambiguities of the first type are resolved by the sequence in which you have entered the databases in the list of the database selection dialog. Furthermore, you have the option of associating a prioritized database to each of the two CAN controllers. For messages received by this controller, this database then has highest priority in the symbolic association. Only if the symbolic name is not found there, does the program search all other databases indicated in the database selection dialog, and this is done in the sequence defined there.

The search sequence in the database list in the database selection dialog is also used to resolve name conflicts when configuring measurement windows and function blocks. In such a case, the name is associated with the message in the database that is highest in the list. However, you can also resolve ambiguities of this type by qualifying symbolic names.

See CANalyzer's online Help function for examples of resolving ambiguities.

4.3.4 Checking for Consistency of Symbolic Data

In CANalyzer's symbolic mode the CAN messages are addressed using symbolic names from an associated database. Therefore, CANalyzer checks the consistency of the database and the active configuration in the following situations:

- At the program start,
- When a new database is associated,
- When CANalyzer is restarted after a change to the database.

In this consistency check, the symbolic names of all CAN messages used in the measurement and simulation setups are compared to names in the database. If the message name has been changed in the database, an automatic adaptation is made to this name, and an appropriate message appears on the screen. If CANalyzer could find neither the name nor the message ID in the database, the user receives an error message. In this case the measurement cannot be started until the particular message is removed from the configuration.

4.4 Working with Multiple Channels

CANalyzer supports up to 32 (virtual and real) CAN channels. Consequently, you can also use multiple CAN cards to transmit and receive messages. This section describes what must be taken into consideration when working with multiple channels.

4.4.1 Channel Definition

The number of CAN channels you wish to use is configured in the Channel Definition Dialog.

You can access this dialog from the main menu item *Configure | CAN channels...* on the menu bar or from the PC card icon in the measurement setup. In addition to affecting the measurement itself, the channel definition also affects the inputs that are possible in the various configuration dialogs. Only defined channels are offered for selection.

The channels are allocated to the CAN chips registered in the CAN hardware configuration of your computer's Control Panel. Chip allocation is only meaningful in Online mode. In Offline mode, in which messages are replayed from a file, it is irrelevant.

To change the default chip allocation, open the *CAN Hardware* component of your computer's Control Panel. The CAN Hardware Configuration Dialog appears, in which you can modify the chip allocations of the channels.

The chip allocation is also shown in the Hardware Configuration Dialog (cf. section 2.2.1).

Note: The number of channels is configuration-specific. It is saved in the configuration file and is restored when loading the configuration.

4.4.2 Channels in Online Mode

In Online mode messages from the transmit branch are transmitted on one or more real buses, and in the measurement setup they are received by one or more real buses. The defined channels correspond to these real buses with their controllers.

If you specify more than 2 CAN channels the following conditions must be satisfied to be able to start a measurement:

- Your active CAN driver must support more than 2 CAN channels. If this is not the case, you will receive a warning if you specify more than 2 CAN channels.
- A real or virtual CAN controller must be assigned to each channel. If this is not the case, you will be requested to make such an assignment.

In the Channel Definition Dialog you can choose whether or not a consistency check should be performed after configuration. The consistency check covers database assignments and the configuration of all function blocks with the exception of CAPL blocks. The check monitors whether invalid channels are referenced. If this is the case an inconsistency is reported. These reports can be output to the Write window if desired.

With CAPL blocks a determination of whether all referenced channels are valid is not made until compilation. A warning is output if any channels are invalid. Therefore, it is advisable to recompile all nodes after each new definition of channels.

If you use undefined channels, CANalyzer behaves as follows in Online mode:

- Channel configuration does *not* cause any filtering of messages in the data flow plan.
- When *receiving* on controllers not assigned to a defined channel, the received messages are passed through the measurement setup.
- When *transmitting* from a Generator block or Replay block on the right of the transmit branch to an undefined channel, the transmit request is similarly passed through.
- An error is reported in the Write window for a sender in the transmit branch as soon as the transmit request is given to an undefined channel. The message is not transmitted.
- CAPL blocks do not transmit messages to which an undefined channel is assigned.

4.4.3 Channels in Offline Mode

In Offline mode the channels correspond to those channels on which the played-in messages were logged.

Consequently, each message is played-in on the channel on which it was logged. The channels in Offline mode correspond to the channels used during logging to the log file. Therefore, you should define the number of channels such that it corresponds to the number of channels that were configured for logging to the log file.

If you use undefined channels CANalyzer behaves as follows in Offline mode:

- The channel configuration does *not* cause any filtering of messages in the data flow plan.
- If messages are played-in which are assigned to an undefined channel, these messages are passed through the measurement setup.
- When *transmitting* from a Generator block or Replay block on the right of the transmit branch to an undefined channel, the transmit request is passed through.

CAPL blocks do not send out messages to which no defined channel is allocated.

4.5 The DDE Interface

The DDE interface (Dynamic Data Exchange) gives Windows programs the capability of exchanging data with CANalyzer. The Windows program (e.g. Excel, Word or

Explorer) referred to as the DDE Client establishes a connection to CANalyzer - the DDE Server - and requests data from or transmits commands to CANalyzer.

All DDE services provided by CANalyzer are summarized under the service name `CANalyzerDDEMLServer`. You must communicate this name to your Client application. Finally, certain services are incorporated under a topic name. CANalyzer provides various services under the System topic. The data format `CF_TEXT` (ASCII text) is supported exclusively for this purpose.

For instructions on how to use existing Windows programs for data communication via DDE please refer to documentation for these programs. Of course, you can also write your own Windows application (e.g. in Visual Basic, Visual C or Borland C), which communicates with CANalyzer as a DDE Client. A typical DDE application that involves loading configurations with Explorer is described in section 0.

4.5.1 DDE Services

The DDE interface combines data requests and command actions under the concept of a *Transaction*. Four different classes of transactions are differentiated:

Data request (Peek)

Data of the DDE server can be read selectively using this transaction. For example, this can be used to query whether the measurement is currently running, what mode CANalyzer is in, or what value an environment variable currently has.

Data manipulation (Poke)

Data can be manipulated at the DDE server with this transaction. It can be used by other programs, for example to modify the values of environment variables in CANoe.

Reporting loop (Advise loop)

The advise loop provides the capability of informing the DDE client selectively of value changes to specific data.

Execution of DDE commands (Execute)

DDE clients can selectively transmit commands to the DDE server and thereby control the server remotely. For example, a DDE client can have CANalyzer load a specific configuration, start the measurement or switch over to Offline mode.

CANalyzer can execute any number of transactions simultaneously, without restrictions.

DDE utilizes a three level hierarchy to specify the data transmitted in transactions. At the very top of this hierarchy is the *Service name* by which a client addresses the server when establishing a connection. Like most DDE servers, CANalyzer provides exactly one service name - the character string `CANalyzerDDEMLServer`. On the level below this, logical data units are combined for each service under the *Topic name*. CANalyzer provides various services under the topic name *System* which is described below. The data format that CANalyzer exclusively supports for this is `CF_TEXT` (ascii text). Finally, on the lowest level is the *Item*, which describes a data

unit uniquely. The server passes this to the client in the course of the transaction. An item can be an integer, a character string or a bitmap. You can use the items *ConfigurationRequest* and *DynamicStateRequest* of the *System* topic to query information regarding the system's current state.

4.5.2 The DDE *System* Topic

With the *System* topic you can query system information from CANalyzer as well as transmit commands to CANalyzer.

For the purpose of querying system information, CANalyzer provides a DDE channel with which a DDE client can read status messages (Peek) and be informed of status changes (Advise Loop).

In this process, the items *ConfigurationRequest* and *DynamicStateRequest* transfer data on the momentary state of the configuration. The state can be monitored by an Advise Loop or queried by a Peek to the particular item.

The return values for the items *ConfigurationRequest* and *DynamicStateRequest* documented in the following two tables are logically combined with OR. Thus, for example a return value of 6 means that the configuration was modified and Online mode is active.

Return values of *ConfigurationRequest*:

Value	Meaning
1	Configuration has a name
2	Configuration modified/not saved yet
4	Mode is Online

Return values of *DynamicStateRequest*:

Value	Meaning
0	Measurement is not running
4	Measurement is running

A data request to the item *Formats* returns the supported data formats. At this time the only format supported is TEXT. This refers to the clipboard format CF_TEXT according to DDEML convention.

The DDE *Execute* transaction initiates execution of a CANalyzer command via DDE. At this time there is still no genuine return to indicate the success or result of the command. Nevertheless, some effects of the command can be queried by the items *ConfigurationRequest* and *DynamicStateRequest*.

4.5.3 The DDE *Execute* Transaction

Commands used to control CANanalyzers remotely consist of a null-terminated command string.

This command string consists of a command token, and if necessary command parameters that are separated by commas and may be optionally enclosed in round brackets. String parameters may not be placed in quotation marks. The command string may be enclosed in square brackets (DDEML convention). A single *Execute* transaction can only transmit one command, i.e. scripts or command sequences are not permitted. The command tokens are not sensitive to upper/lower case and are delimited by blank characters.

The following commands are supported:

Command	Meaning	Remarks
Start	Starts the measurement	Is ignored during a measurement run
Stop	Stops the measurement	Is ignored if measurement is not running
ToOffline	Switches to Offline mode	Is ignored during measurement or in Offline mode
ToOfflineCopy	Switches to Offline mode (Copy)	Is ignored during measurement or in Offline mode
ToOnline	Switches to Online mode	Is ignored during measurement or in Online mode
ToOnlineCopy	Switches to Online mode (Copy)	Is ignored during measurement or in Online mode
LoadCfg <name>	Loads the configuration <name>	If <name> is empty the File Selection dialog is opened.
SaveCfg <name>	Saves the configuration under the name <name>	If <name> is empty the File Selection dialog is opened.

Examples of *Execute*

The following lines show some examples of command strings that can be transmitted with the *Execute* transaction:

```
LoadCfg c:\canwin\demo_cl\motbus.cfg
LoadCfg (c:\canwin\demo_cl\motbus.cfg)
[LoadCfg (c:\canwin\demo_cl\motbus.cfg)]
LoadCfg
Start
Stop
```

4.5.4 Loading Configurations with the Explorer

To load CANalyzer configurations (CFG files) directly via Explorer it is insufficient to simply link the extension CFG to the application CANW32.EXE in Explorer. Namely, if CANalyzer has already been started and you attempt to load a new configuration by double clicking a CFG file, you get an error message, because in this case after the double click Explorer tries to start CANalyzer a second time.

However, with the help of the DDE interface Explorer can be configured to load the desired configuration after the program has been started. To do this, proceed as follows:

Deleting the Link

In Explorer delete any existing links to CFG files. First, double click a CFG file. If a program starts then there is already an existing link to CFG files. However, if double clicking instead causes the *Open With...* dialog to appear, then there is no link yet and you can create a new link directly.

You can be certain of deleting a link by deleting it in the registry. (There are various types of links; some of them may be deleted - possibly in a simple way - directly in Explorer.) To delete the link in the registry, first open the registry editor, e.g. by *Start|Run regedit <Enter>*.

Then search for the key HKEY_CLASSES_ROOT\.cfg and delete it in the key's local popup menu.

Creating the New Link

You can create a new link in Explorer from the menu item *View|Folder options*. On the *File types* page click the *New Type* button. The *Add New File Type* dialog appears. Here you first press the *Change Icon* button.

Browse for the CANalyzer application EXEC32\CANW32.EXE, select the CANalyzer icon and confirm with *OK*.

In the *Description of type* text box enter the description to be displayed in Explorer, e.g. "CANalyzer configuration". Of course, the associated extension must be CFG here. Leave the text box *Content type (MIME)* empty. The text box *Default extension for content type* is probably deactivated. Also leave this box empty.

Then add a new action by pressing the *New* button. The *New Action* dialog appears. Here you enter an *Open* as the action. Select the *Application used to perform this action* by browsing for the CANalyzer application EXEC32\CANW32.EXE. Afterwards, mark the option *Use DDE* and enter - in the drop-down portion of the dialog - the DDE information as described in Figure 72.

Now you can double click configuration files with the extension CFG. This will work in Explorer and in file groups. If CANalyzer has not yet been started, the program will start up, and after the startup the desired configuration is loaded (flashing effect). If CANalyzer has already been loaded, the configuration is switched over. This also works with shortcuts. That is, you can create shortcuts of configuration files on the

desktop and in program groups which - when double clicked - behave like the file itself.

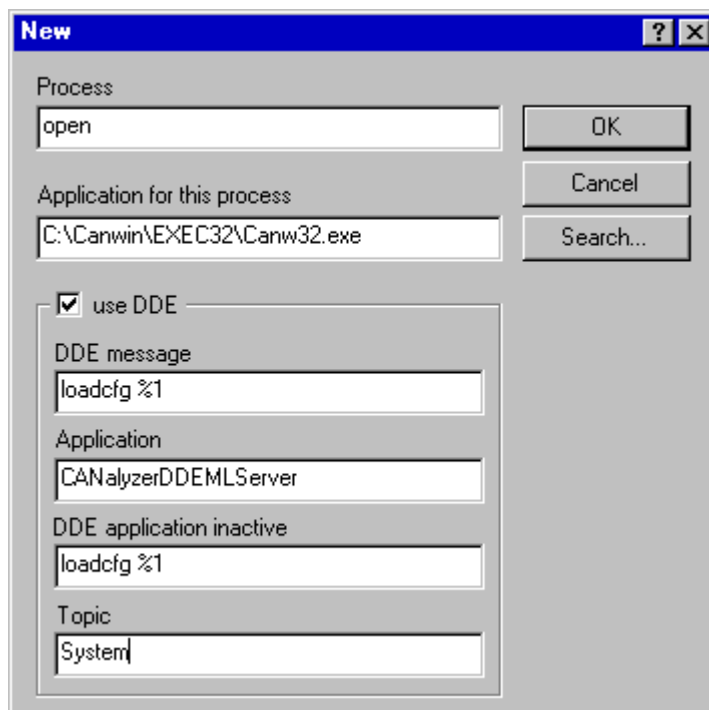


Figure 72: Creating the DDE link for the Explorer

Note: Since CANalyzer and CANoe use the same extension (CFG) for their configurations, at any given time on a single system only one of the two programs can be configured as described here. However, another action (e.g. Open CANoe) could be added for the file type CFG to open the second program.

4.5.5 COM-Server

A COM-Server is implemented in addition to the DDE-services. It helps the program to be gated or controlled by other applications. Besides accessing configuration-specific data it is also possible to control the measurement. You can also call CAPL functions, read signal values, and both read and write access environment variables.

Control can either be realized by COM-ready script environments (ActiveX Scripting: VBScript, JScript, Perl, ...) or by stand-alone applications, which can be written with RAD development environments (Visual Basic, Delphi, ...) or in C/C++.

Please see the Online-Help for a more detailed description of the COM-Server.

5 Appendix

5.1 Particularities of the Demo Version

In the demo version of CANalyzer a demo driver which does not require a PC-card is connected to the PC instead of a regular PC-card driver. However, the functions of this driver are very limited. Primarily, it ensures that all messages which are transmitted are returned as received messages with a accurate time stamps.

Therefore, to be able to work with the demo version, a generator or program block which generates transmit messages must be inserted in the transmit branch. Messages generated in this way can then be captured, evaluated and saved.

The bus parameter options and message setup which are selected by clicking on the PC card icon in the measurement setup are irrelevant for the demo version and can be disregarded.

Aside from the PC card and the associated card driver, the demo version is a complete version. In particular, evaluation and memory storage of messages and CAPL programming can be tested without limitations.

5.2 File Name Extensions Used

In addition to the files which are created during installation, the program uses a number of other files, e.g. files which save measurement setups or logging data. Different extensions are used to differentiate the file types:

Extension	File type
AC	Firmware for download in CAN-AC2 cards
ASC	ASCII log file
BAK	Backup for CAPL source text
BIC	Project database (Internal format)
BIN	Firmware for download in CAN-AC2 cards
CAN	CAPL source text
CBF	CAPL program file (compiled)
CFG	Configuration file for measurement setup
DBC	Project database (ASCII format)
DLL	Runtime libraries
EXE	Program files
GEN	Saved generator block
HLP	Help files
HEX	Intel Hex file as firmware for download
INI	Windows initialization files
LOG	Binary log file, replay file
SBN	Firmware for download with PCMCIA CANcard
TXT	ASCII documents with additional information
WMF	Resources in Windows metafile format
WIR	Write documents with additional information

5.3 Troubleshooting

See section 5.4 for a list of error messages occurring during initialization or during a measurement. In this section tips are offered for handling problems of a general nature.

CANalyzer will not start

CFG file destroyed? Often it is helpful to delete the active configuration file MYCONFIG.CFG. First, the file should be backed up under a different name so that its contents are not lost. After the problem has been cleared up it can be renamed back to MYCONFIG.CFG.

CANalyzer runs too slowly

Power managers, which are particularly common on notebook computers, may not be installed for CANalyzer operation. Among other things, the power manager deprives the application of CPU for long periods of time. Therefore, transmit times are incorrect and messages can be lost when a power manager is installed. To remove the power manager from your system, please compare the instructions in the hardware installation guide.

For less powerful computers it may also be advisable to reduce the resolution of the system clock. The time stamps for messages may then be less accurate, but fewer demands are placed on computer CPU. To do this, enter the following line in section [PCBOARD] of the CAN.INI file:

```
Timerrate = 200
```

or under some circumstances even the value:

```
Timerrate = 400
```

These correspond to time resolutions of 2 or 4 milliseconds, respectively.

Card cannot be initialized

Timeout ...

With error messages of this type, CANalyzer cannot establish a connection to the CAN hardware. Check the installation of the CAN card. You will find hints for troubleshooting in the hardware installation guide.

Above all, notebook PCs frequently use a power manager. This must be deactivated!

Error message: "Error in transmission"

Immediate state change of CAN controller to ERROR PASSIVE

Bus not connected? The bus connection should be checked, and possibly also the pinout of the connector being used.

Terminating resistor present? In particular, the CAN AC2 version with 82527 controllers reacts sensitively to a missing bus termination.

No partner on the bus? If the bus is only connected to one of the two CAN controllers, and there are no other bus nodes, the controller does not receive any acknowledge for transmission attempts.

Baud rate and output control set? The controller register can be programmed by the popup menu of the CAN-card icon. See section 2.2 for a more detailed explanation.

Error message:

**An error has occurred in ...
For further information please report the
LINE and FILE to our Hotline**

At the current level of technology it is impossible to develop completely error-free programs that are non-trivial. Unfortunately, this is also true of CANalyzer. To better localize and correct system errors that occur very seldom, CANalyzer has diagnostic mechanisms that report these errors. Please be sure to write down the exact text of the error message. With this information our telephone Customer Support can then help you to correct the problem more quickly.

5.4 System Messages in the Write Window

In the following section you will find a description of the most important CANalyzer system messages that are output to the Write window.

Assertion <Error number> in <File name> <Condition>

At the current level of technology, it is not possible to develop completely error-free programs that are non-trivial. Unfortunately, this also applies to CANalyzer. To better localize and correct system errors that occur very seldom, CANalyzer has diagnostic mechanisms for reporting these errors. Please be sure to write down the exact text of the error message. With this information our telephone Customer Support can then help you to correct the problem more quickly.

board error: <Error number>

An error has occurred at the interface to the CAN card. Please refer to section 5.5 for more detailed information with the help of the error number.

Bus at <Baud rate> baud.

At the measurement start CANalyzer reports the baud rate to which the PC-card was initialized in the active configuration. (See section 2.2 regarding configuration of the PC-card.)

can board: board timeout

See *Timeout* message in section 5.5

can board: dpram overflow

See *DPRAM overflow* message in section 5.5

can board error: <Error number>

An error has occurred at the interface to the CAN card. Please refer to section 5.5 for more detailed information with the help of the error number.

can board: rx queue is full

See *RX buffer overflow* message in section 5.5

can board: rx queue is full and will be deleted

See *RX buffer overflow* message in section 5.5

can board: rx reg overrun

See *RX register overflow* message in section 5.5

can board: tx queue is full

See *TX-buffer full* message in section 5.5

CANcard box trigger command <Channel> <Command>

For low-speed applications with the CANcard you can switch both trigger channels of the bus driver box at measurement start and at measurement stop, or switch them with the CAPL function `setportbits()`. You would enter the commands in section [CANcard] of the file `CAN.INI`. When switching channels an appropriate message appears in the Write window.

Delayed loading of replay block

Large replay files might not fit in the Windows main memory under some circumstances. In such cases CANalyzer reloads parts of the replay file during the measurement. This warning message appears if the data cannot be reloaded quickly enough. You must then assume that the messages will be output on the bus with a time delay.

DEMO Version: Measurement stopped !!

This message informs you that you have terminated the measurement in a Demo version.

Empty replay block ignored

A replay file that you would like to play back through a replay block, is empty. The replay block is therefore ignored.

Error parsing file <File name>: No database assigned

When you play back replay files that you previously recorded in symbolic mode with the Logging function, you should make sure that the same database is associated for the playback. For ASCII files, message identifiers are not saved, but

rather the associated symbolic names from the database. If you use another database for the playback, the symbolic names might not be interpreted - or might be misinterpreted - when the replay file is read-in.

Error: Unknown environment variable in <File name> in line <Line number>

The ASCII replay file contains an environment variable that was not found in the database. Make sure that the database you used to record the file agrees with the database that you are using to play back the file.

Error: Insufficient memory to create Replay block!

At the measurement start the replay file that you associated with the replay block is first read into the main memory. This message appears if Windows cannot provide enough main memory to load the replay block. You should close some applications before you restart the measurement.

Error: Insufficient memory to load offline source!

At the measurement start in Offline mode, the offline data source is first read into main memory from the offline file. This message appears if Windows cannot provide enough main memory to load the offline data source. You should close some applications before restarting the measurement.

Error: Unknown environment variable in <File name> in line <Line number>

The ASCII replay file contains an environment variable that was not found in the database. Verify that the database you used to record the file is identical to the database you are using to play back the file.

Load transition:

The ring buffer between the real-time library and the CANalyzer main program has a fill level indicator. If the buffer is full or is overflowing, an appropriate message appears (NORMAL->QUEUE HIGH bzw. NORMAL->DATA LOST) in the Write window. If the load is then reduced (possibly after a burst of messages), you are also informed of this by the message QUEUE HIGH -> NORMAL.

No Windows timer available for animation

A Windows operating system timer is used for the animation function in offline mode. If no timer is available any longer, this message will appear. The measurement is then terminated. Close some applications and use the Windows timer before you restart the measurement in animation mode.

**No interrupts received from CAN card!!
System has problems. Check CAN board or CAN.INI for IRQ
configuration. IRQ currently = <IRQ>**

Communication between the PC-card and CANalyzer is normally controlled by an interrupt. This error message indicates that CANalyzer has not received any interrupts from the card. Be sure to also make the interrupt channel for the expansion card known to CANalyzer in the file CAN.INI. The exact procedure is described in the installation instructions. The IRQ number given in the error message corresponds to the active interrupt channel that is set in your project file.

**PB1/2 initialized by CAN.INI: <Initialization value>
PB3 initialized by CAN.INI: <Initialization value>
PB1/2 initialized by USRDLL: <Initialization value>
PB3 initialized by USRDLL: <Initialization value>**

For low-speed applications the low-speed drivers ("piggy backs") on the CAN-AC2 card can be initialized via the project file CAN.INI, with CAPL functions setPortBits() and enablePortBits() or via a run-time library (USRDLL) belonging to the configuration. The initialization values from the project file or run-time library are output in the Write window. Please refer to the installation manual for the low-speed accessory kit for instructions on how to configure the low-speed drivers.

PC-Card could not be found. Error number <Error number>

CANalyzer has not detected the PC-card at the program start. The error number can be obtained from the list of error messages for the CAN interface (section 5.5). Verify that you have installed the card as described in the installation instructions, and that the options in the project file CAN.INI agree with the hardware options (memory area, interrupt channel).

Replay: Loading was delayed

Large replay files might not fit in the Windows main memory under some conditions. In such cases CANalyzer reloads parts of the replay file during the measurement. This warning message appears if the data cannot be reloaded quickly enough. You must assume that the messages will be output on the bus with a time delay.

RTC has to be activated for polling mode (CAN.INI)

You have toggled CANalyzer from the interrupt-driven standard mode to polling mode in the project file CAN.INI. In this mode you must also activate the PC's real-time clock (RTC) as the time base. To do this, set the entry Frequency to a value between 8 and 11 in section [RTC] of the project file.

Please note that by default CANalyzer operates in the interrupt-driven mode. You should only use polling mode in exceptional situations, to increase throughput under conditions of high base load, or if you cannot or do not wish to work with interrupts.

RTC is activated (see CAN.INI section RTC)

You have selected the PC's real-time clock (RTC) as the time base in the project file CAN.INI (section [RTC], entry Frequency is not equal to 0). Normally, time management is driven by the timer on the PC-card (Frequency = 0). The PC's real-time clock should only be activated in exceptional situations (e.g. in Polling mode).

SetTimer warning: timer still active. (Number is the timer corresponding to the n-th [0.,1,...] ON TIMER in text file of CAPL program)

You are using a timer in a CAPL program that was set by the function `setTimer()`, but before the last set timer elapsed. In a CAPL program with multiple timers you can localize a timer by opening the CAPL program with a text editor and counting through the `on timer` event procedures - starting with 0 - from top to bottom. The timer number indicated in the warning will match the procedure number of the event procedure counted in this manner.

Timer Warning: no completion function, SetTimer ignored

In a CAPL program with the function `setTimer()` you are attempting to set a timer for which you have not defined an `on timer` event procedure. The function call is therefore ignored.

Warning: can't set timer; no completion function

In a CAPL program with the function `setTimer()` you are attempting to set a timer for which you have not defined an `on timer` event procedure. The function call is therefore ignored.

Warning: Empty Replay block ignored

A replay file that you would like to play back over a replay block is empty. The replay block is therefore ignored.

5.5 List of Error Messages to the CAN Interface

Error messages related to communication between CANalyzer and the CAN PC-card as well as errors on the CAN bus or in the CAN card firmware appear in this list. In each case, a clear text and an assigned error number are given. Some of these messages are hardware-specific and therefore do not occur with all card types.

Message was not transmitted (14)

The last transmit request was not executed by the CAN controller. This may be due to the error status of the controller, or due to the accumulation of too many higher priority messages.

DLC error (1128, 111, 112)

With the CANIB card a smaller DLC is specified in the DPRAM setup than should be transmitted. This size must be adjusted in the CAPL generator block or replay block.

This error can trigger other consequential errors. When problems are encountered the computer should be rebooted with a cold start!

Due to a firmware error in CANIB, a DLC equal to 0 cannot be processed. Therefore, it is set to at least 1 in the driver. However, an adjustment for calculating the size of the available DPRAM will not be implemented until Version 2.0.

Incorrect controller no. (3,10,113)

An attempt was made to access a nonexistent CAN controller. Most CAN cards supported by CANalyzer have two controllers. But there are also cards with just one controller.

Remedy: Look for a **message CANn...** in the CAPL programs, where *n* is greater than the number of existing controllers. This must be replaced by a correct number.

Incorrect module number (109)

No CANIB was found at the address given. If the port address has been jumpered over on the card, this must be communicated to CANalyzer See the manual.

Incorrect checksum (1368)

The CAN controller detected a faulty CRC checksum.

Incorrect terminating code (1432)

For CANIB cards this is the error code for general firmware problems. These occur primarily in BUSOFF and ERROR_PASSIVE states.

Incorrect card type (8)

CANalyzer card driver and hardware are not compatible.

Remedy: The correct CANalyzer version should be started, or another CAN card should be installed.

Error in statistics request ()

The CAN card or its firmware does not support bus statistics.

FIFO entry>16 (114)

CANIB has reported invalid data to CANalyzer. A potential cause for this is a RX buffer overrun. This cannot be detected directly if the CANIB was not initialized correctly. If its jumpers are transposed the user should follow the instructions in the CANalyzer manual.

Interrupt not found (108,2016)

Proper communication could not be established with the CANIB card. A check should be made to determine which interrupt is jumpered on the CANIB card, and whether there could be collisions with other hardware. It is often the case that there is an overlap conflict with the mouse interrupt. The CANIB driver itself can determine the interrupt allocated to CANIB. If this is unsuccessful the IRQ number can be entered explicitly starting with Version 2.0 (see BOARDCFG.INI).

No reply from CAN controller (106)

The firmware could not establish a connection to the CAN controller. This is an indication of a defective CAN card.

No messages in RX buffer (1)

No message received (7)

No data are currently being received by the card.

Command from driver not supported (6,11,1528)

CANalyzer has sent a command to the card driver which it or the firmware does not recognize. Example: A bus statistics request to a card without the appropriate logic.

RX buffer overrun (101)

The receive buffer could not accept any more received messages.

There are several methods for remedying this situation:

- Insert breaks in branches of the data flow plan which are not needed. In an extreme case the statistics block, data block and trace block can be disconnected by breaks. The measurement is recorded with the logging block and afterwards is evaluated offline.
- For Basic-CAN controllers a reduction in the data stream can be achieved by acceptance filtering. Except in special applications the second controller in particular may be completely disconnected by this method.
- For Full-CAN controllers data reduction can be accomplished by striking messages in the message setup in conjunction with filter blocks.
- Switching-off the statistics report or other unnecessary options.

RX register overrun (105)

The Basic-CAN 82C200 controller has only two internal registers for accepting messages. At higher bus frequencies and higher message rates, newly arriving messages overwrite this buffer before the firmware can read out the register.

Correction: Use acceptance filtering.

Timeout while accessing card (4,232)

Communication problems with the firmware occur during a measurement.

Remedy: Terminate measurement and restart. If this does not help, the reset button can be pressed for some cards. Otherwise, the PC should be rebooted.

Timeout during card initialization (0)

No reply from the card (1400)

No connection could be established with the firmware when the CAN card was initialized.

TX buffer full, TX-REQUEST rejected (2)

The transmit buffer is still full. The new transmit request cannot be processed. There are three possible reasons for this:

CANalyzer is transmitting data faster than the firmware can receive them and pass them to the CAN controller. This may occur, for example, if higher priority messages are being transmitted on the CAN bus.

- The number of messages transmitted one directly after another in a CAPL program is larger than the transmit buffer. This problem occurs primarily when transmission is executed in a loop in CAPL programs:

```
for (i=0;i<50;i=i+1) output(Msg);
```

Remedy: Fast transmission by setting **msTimers** and in reaction to the timer event.

- The CAN controller being addressed is in the BUSOFF state and therefore cannot accept transmit requests any longer. This can be detected in the bus statistics window.

Unknown transmit identifier (100,1496)

A message which is to be transmitted is not entered in the message setup of the Full-CAN controller.

Invalid DPRAM address (107)

When the CAN card was initialized, an illegal identifier was entered for the DPRAM address. See card documentation.

6 Index

!

! Load symbol · 4-3

<

<F10> key · 1-20

7

72005 · 2-11, 2-13

8

82527 · 2-11, 2-13
82C200 · 2-11, 2-13

A

abs() · **3-17**, 3-58
Absolute value · 3-58
Acceptance code · 2-13
Acceptance filtering · 2-13, 2-14, 4-6
Acceptance filters · 2-10
Acceptance mask · 2-13, 2-14
Acknowledge · 2-12
Activity indicator · 2-28
Advise loop · 4-11
Ambiguities · 2-26, 3-16, 3-22
Analysis blocks · 3-66, 4-5
Analysis branch · 4-6
Animate · 2-25, 2-68, 2-69
Animation factor · 2-18
Array · 3-21, 3-38, **3-61**, 3-64, 3-65
Array size determination · 3-17
ASCII · 2-65, 3-69, 5-2
Assertion · **5-4**
Attribute · 3-14, 3-15
Attribute name · 3-14
Attribute value · 3-14, 3-15
Averaging time · 2-40, 2-43, 4-5
Axis options · 2-31, 2-33

B

Backus-Naur form · 3-23
Bar-type display · 2-28
Basic image · 2-40
Basic-CAN controller · 2-9
Baud rate · 2-43, 3-39
Baudrate · 2-10, 2-11

Beep · 3-44
binary · 2-65
Binary · 2-65
Bit time · 2-12
Bit timing · 2-12
Bit timing · 2-12
Blocking filter · 4-6
Branches · 1-3, 2-4
Break · 2-67, 2-68, 2-69
Break condition · 2-68
Breakpoint · 2-68
Browser tree · 3-18, 3-20
BTL cycles · 2-10, 2-12
BTR programming · 3-42
Burst · 2-56
Bus driver box · 5-5
Bus load · 2-5, 2-9, 2-37, 2-42, 4-3
Bus load measurement · 2-43
Bus loading · 4-3, 4-6
Bus parameter · 3-66
Bus parameters · 2-10
Bus registers · 2-10
Bus statistics · 2-14, 2-42, 2-43, 2-66
Bus statistics display · 4-3
Bus statistics information · 2-43, 2-66
Bus Timing Register · 2-11
BUSOFF · 3-27, 3-39
Button · 1-21

C

callAllOnEnvVar() · 3-56
CAN card · 2-9
CAN channel · 4-9
CAN.INI · 2-7, 2-44, 2-65, 2-68, 3-2, 3-36, 4-4
CAN-AC2 · 5-2
CANalyzer without card · 2-18
CANcard · 2-9, 5-2
CANcardX · 2-9, 3-12
cancelTimer() · **3-11**, 3-17, 3-50
CANdb++ · 1-23, 2-19, 4-1
canOffline() · 3-48
canOnline() · 3-48
CAPL · 2-4, 2-62, 3-19, 5-2, 5-9, 5-11
CAPL Browser · 4-1
CAPL program · 2-44, 3-2, 3-9, 3-23, 3-24, 3-71, 3-72
CAPL Program · 2-45
CAPL_DLL_INFO_LIST · 3-37
Card clock · 2-16
Card driver · 2-15, 2-17
Channel · 2-9
Channel definition · 2-9, 4-9, 4-10
Check box · 1-21
Chip allocation · 2-9, 4-9
CIF files · 2-8
Clipboard · 2-43, 3-20
Colors in the Graphics window · 2-37
Comment · 2-45, 3-22, 3-46
Comment box · 1-21

Compilation time · 3-20
 Compile time · 3-21
 Compiler · 2-9, 3-2, 3-5, 3-13, 3-19, 3-20, 3-21, 3-22, 3-36, 3-44, 3-46
 Condition primitives · 2-70
 Configuration · 1-20, 1-21, 2-4, 2-18, 2-49, 2-58, 4-8, 4-9
 Configuration descriptions · 2-8
 Configuration file · 2-2, 2-7
 Configuration file formats · 2-8
 Configuration management · 2-7
 Configuration options · 2-5, 2-37
 ConfigurationRequest · 4-12
 Context menus · 1-20
 Control · 5-4
 Control box · 1-21
 Conversion
 Log files · 2-72
 Conversion formula · 2-57, 3-13, 3-14
 Copying blocks · 2-5
 cos() · **3-17**
 Cosine · 3-58
 CPC/PP · 3-42, 3-43
 Cycle time · 2-28, 2-56, 3-14, 4-5
 Cyclic update · 2-64, 4-5

D

Data flow · 1-4, 2-4, 2-5, 2-15, 2-16, 2-44, 2-48, 2-49, 2-58, 3-3, 3-9, 3-66, 4-1, 4-2, 5-10
 Data flow diagram · [1-3](#), 2-4, 2-5, 2-6, 2-16, 2-67
 Data Length Code · 3-33
 Data loss · 4-3, 4-4
 Data reduction · 2-68, 4-6
 Data source · 2-5, 2-9, 2-43, 2-59, 2-65, 2-67, 2-69
 Data throughput · 2-43
 Database · 2-4, 2-19, 2-24, 3-25, 3-32, 4-8, 4-9
 Database attribute · 3-15
 Database editor · 3-15
 Database Editor · 4-1
 DDE · 4-10, 4-11, 4-12, 4-14
 Deactivation function · 4-5
 Debugging · 3-65, 3-66
 Decimal representation · 3-43, 3-44
 Decrementer · 2-57
 Delay times · 2-14
 Demo version · 3-43, 5-1
 Dialog box · 1-20
 Difference markers · 2-34
 Difference measurement mode · 2-34
 DIR · **3-7**, 3-26, 3-31, **3-33**, 3-34, 3-69, 3-70
 DLC · 3-6, 3-7, 3-26, 3-31, **3-33**, 3-68, 3-69, 3-70
 double · **3-4**, **3-5**, 3-24, 3-32, 3-35, 3-37, 3-57, 3-58
 DPRAM · 5-9, 5-11
 Driver options · 2-14, 2-66
 Drop and drag · 3-20
 Drop down list · 1-21
 Dynamic · 4-10
 DynamicStateRequest · 4-12

E

elCount() · **3-5**, 3-17
 End of measurement · 2-31, 2-33, 2-62, **3-7**, **3-11**
 Environment variable · 2-21, **3-11**, 3-13, 3-15, 3-16
 ERROR · 2-42
 Error counter · 3-12
 Error frame · 2-4, 2-21, 2-42, 2-49, 2-56, 2-63, **3-7**, 3-34
 Error message · 3-32, 4-9, 5-3, 5-8
 Error message · 2-63
 Error Passive · 5-3
 ERRORACTIVE · 3-27
 Error-Frame · 2-42
 ERRORPASSIVE · 3-7
 Evaluation branch · 3-9, 3-10, 4-6
 Evaluation functions · 2-59
 Event procedure · 3-7
 Event procedures · 3-9, 3-23, 3-27, 3-28, 3-34
 Execute transaction · 4-13
 Explorer · 4-13, 4-14
 Exponential function · 3-58
 Export
 Logging files · 2-72
 Extended identifier · 2-21, 3-47
 Extended Identifier · 2-40

F

File access · 3-61
 File format · 3-20
 File icon of the log file · 1-17
 File name extensions · 5-2
 File types · [4-14](#)
 fileName() · 3-66
 fileReadInt() · **3-60**
 Fill level indicator · 4-3
 Filter · 1-4, 2-5, 2-16, 2-19
 Fit · 2-40
 Floating point numbers · 3-4, **3-5**, **3-60**
 Font · 2-44
 Format entry · 3-19, 3-43
 Formats of configurations · 2-8
 Function block · 2-5
 Function generator · 2-52
 Function table · 3-36

G

Gateway · 3-1, 3-2, **3-8**, 3-70
 Generator block · 1-4, 2-6, 2-15, 2-16, 2-49, 2-50, 2-51, 2-52, 2-53, 2-54, 3-3, 5-2
 getCardType() · 3-42
 getLocalTime() · 3-53
 getLocalTimeString() · 3-54
 getValue() · **3-11**, 3-17
 Global options · 2-8

Grammar · 3-23
Grid · 2-36

H

Hardware configuration · 4-9
Help · 1-23, 2-14, 3-27
hexadecimal · 2-13, **3-61**
Hexadecimal · 3-14
Hexadecimal representation · 3-44
High load · 2-57
High-load operation · 4-3
Hotspots · 1-4, 2-16

I

IEEE · 3-5
Import of configuration descriptions · 2-8
Incrementer · 2-57
Infinite loop · 2-17
INI file · 2-7, 2-44, 2-65, 2-68, 3-36, 4-4, 5-3
INI-Datei · 3-2
Initialization of environment variables · 3-56
inport() · **3-17**, 3-41
inportLPT() · 3-41
Input box · 1-21, 2-71
Integer value · 3-15
Integer values · 3-15
Integer variable · 3-18, 3-19
Intel · 3-57
Interactive generator block · 2-54
Interrupt · 5-10

K

Key codes · 3-10
Keyboard · 3-69
Keyboard control · 2-58
Keyboard event · 3-9
Keyboard handling · 3-10

L

Language elements · 3-23
Layout · 2-4, 2-43
Line type · 2-32
Load operation · 4-3, 4-4
Load situation · 4-3
Load transition · 4-4, 5-6
Log file · 2-48
 converting · 2-72
 exporting · 2-72
 file icon · 1-17
Log file name · 3-46
Logging
 time period of · 2-62
Logging export · 2-72
Long-duration measurement · 2-64
Low-speed driver · 5-7

M

Main memory · 2-63
Measurement configuration · 1-20
Measurement control · 3-17
Measurement marker · 2-34
Measurement point · 2-32
Measurement setup · 2-4, 2-15, 2-44, **2-48**, 2-49, 2-58, 4-2
Measurement start · 2-6, 2-9, 2-33, 2-66, **3-4**, 3-7, 3-18, 3-66, 3-67
Measurement value acquisition · 2-32
Memory protection violations · 3-5
Menu operation · 1-20
Message attribute · 2-16, 3-15, 3-34
Message declaration · 3-6
Message display · 4-3
Message Explorer · 2-19, 2-20, 3-22
Message rate · 2-40
Message variable · 3-8
Messages window · 3-20
Mode signal · 2-52, 2-57
Mode-dependent signal · 2-52
Motorola · 3-13, 3-57
Moving blocks · 2-5
msTimer · **3-4**, **3-5**, **3-11**, 3-24, 3-49, 3-50, 3-67, 3-72, 3-73
Multiplex message · 2-52, 2-57
Multisignal mode · 2-29, 2-33, 2-34

N

Network nodes · 2-4
Nodes · 2-4
Nonterminal · 3-23
Notebook · 5-3
Numbering system · 2-8, 3-64

O

OCR · 3-40, 3-42
Offline mode · 2-67, 2-68, 2-69, 2-70
on key · 3-69
on message · 3-70, 3-71
Online mode · 2-5, 2-6, 2-9, 2-15, 2-16, 2-59, 2-69
Operating instructions · 1-19
Operating mode · 1-21, 2-18
Optimization · 2-28, 2-37, 4-4
 Trace window · 2-24
Options button · 1-21
outport() · **3-17**, 3-40
outportLPT() · 3-40
Output mode · 2-32, 4-3, 4-5
output() · **2-45**, 3-1, 3-17, 3-38, 3-67, 3-68, 3-69, 3-70, 3-71
output(this) · **2-45**, 3-9, 3-10, 3-13
Overload situation · 4-3, 4-4
Overload symbol · 2-67

P

Panels · 3-18, 3-20, 3-22
Parameter check · 3-16
PARBROW.INI · 3-20
PC card · 2-7, 2-9, 2-12, 2-13, 4-1, 4-2
PC-card · 2-5, 2-69
Peak display · 2-28
Performance · 2-28, 2-43
Performance optimization · 4-4
Performance wizard · 4-4
PG · 3-33
PGN · 3-31
Phase model · 2-4, 2-18
Piggybacks · 5-7
Point measurement mode · 2-34
Poke · 4-11
Polling mode · 5-7
Post-trigger time · 2-6, 2-63
Power manager · 5-3
Prescaler · 2-10
preStart · **3-7, 3-11**
Pre-trigger time · 2-63
Primitive · 2-70
Priority sequence · 3-9
Procedure template · 3-18
Procedures list · 3-20
Program block · 2-15, 3-1
Program start · 1-3, 2-1, 3-28
Program structure · 3-23
Project database · 3-13, 3-34, 3-72
Project directory · 2-7
Project file · 2-7
putValue() · **3-11**, 3-17

R

Radio button · 1-21
Random number generator · 3-59
Raw value · 2-57
Reaction times · 4-2, 4-6
Real channel · 2-9, 4-9
Real-time capability · 3-59
Real-time clock · 5-7, 5-8
Real-time library · 3-5, 4-2, 4-3, 4-4, 4-5, 4-6
Receive error counter · 3-33
Receive messages · 2-16
Receive time point · 2-9
Receiving chip · 3-8
Refresh · 4-5
Remote frame · 2-56
Replay block · 2-48
Representation formats · 2-8
Representation parameters · 2-25
Reset · 2-68, 3-39
resetCan() · 3-39, 3-40
Right mouse button · 1-20
Ring buffer · 2-63, 4-2, 4-3, 4-4
RTC · 5-7, 5-8
RTR · 3-33, 3-34
runError() · **3-5**, 3-21, 3-38
Runtime error · 3-36

Run-time error · 3-5, 3-21, 3-38
Rx attribute · 2-16, 2-40
Rx buffer overrun · 2-43
Rx error counter · 2-42
RXREMOTE · 3-33

S

Sample programs · 3-66
Samples · 2-11
Sampling point · 2-10
Sampling time point · 2-12
Scaling · 2-40
scalings · 2-36
Search condition · 2-70
Selectors · 3-6, 3-7, 3-24
Semantics · 3-23, 3-29
seqFileClose() · 3-61
seqFileGetBlock() · 3-62
seqFileGetLine() · 3-62
seqFileGetLineSZ() · 3-62
seqFileLoad() · 3-61
seqFileRewind() · 3-62
Sequential file access · 3-61
setLogFileName() · 3-46
setMsgTime() · 3-52
setTimer() · **3-5, 3-11**, 3-17, 3-49, 3-67, 3-68, 3-69
Shortcuts · 2-58
Signal · 2-19, 2-72
Signal curve · 2-32
Signal Explorer · 1-13, 1-14, 2-20, 3-22
Signal identifier · 2-26
Signal response generator · 2-52
Signal selection dialog · 2-32
Signal value · 2-51
Signal-based logging export · 2-72
Simulation · 2-4, 2-18
Simulation configuration · 4-2
Simulation setup · 1-20, 2-4, 2-18, 4-2, 4-9
Simulation setup window · 2-4
sin() · **3-17**
Sine · 3-57
Single-signal mode · 2-29, 2-33, 2-34
Single-step mode · 2-69
SJA 1000 · 2-9, 2-11, 2-13
SJW · 2-11
Slow motion · 2-68
Sound card · 3-45
sprintf() · 3-64
Stack overflow · 3-21, 3-38
Standard deviation · 2-41
Standard Layout · 2-43
Start · 2-68
Start bit · 2-57
Start options · 4-1
Statistical report · 4-3
Statistics report · 2-40, 2-43, 4-5
Statistics window · 2-40
Step · 2-69
Stop · 2-62
stop() · **2-6**, 2-68, 3-17, 3-48
swapLong() · 3-57
swapWord() · 3-57

Symbol selection dialog · 3-22
Symbolic database · 3-72
Symbolic databases · 2-18
Symbolic triggering · 2-71
Synchronization edge · 2-12
Synchronization jump width · 2-11, 2-12
Syntax · 3-23, 3-29, 3-30
sysExit() · 3-45
sysMinimize() · 3-45
System directory · 2-1
System loading · 2-28
System messages · 2-43
System time · 3-52, 3-53, 3-54

T

Terminal · 3-23
Text editor · 3-20
this · **3-10**, **3-11**, 3-12, 3-13, 3-31, 3-34
Time base · 2-69
Time basis · 2-69
Time management · 3-17, 3-49, 3-50
Time resolution · 5-3
Time stamp · 2-16, 2-23
Time window · 2-61, 2-63
timeDiff() · 3-52
timeNow() · **3-17**, 3-52
Timeout · 5-3, 5-10, 5-11
Timer · 3-1, **3-4**, 3-24, 3-28, 3-34, 3-67
ToOffline · 4-13
ToOfflineCopy · 4-13
Toolbar · 2-31, 3-19, 3-20, 3-21
ToOnline · 4-13
ToOnlineCopy · 4-13
Topic name · 4-11
Topic System · 4-12
Trace Watch functionality · 2-24
Trace Watch window · 2-24
Trace window · 2-20
 Configuration of columns · 2-23
 Optimization · 2-24
Transmission of messages · 2-54
Transmission time · 2-18
Transmit block · 2-15
Transmit branch · 2-15, 3-71
Transmit error counter · 3-33
Transmit list · 2-55, 2-56
Trapezoidal function · 2-53
trigger · 3-2
Trigger · 2-59, 2-61, 2-62, 2-63, 2-66, 2-70
Trigger block · 2-66, 2-67

Trigger condition · 2-49, 2-50, 2-51, 2-55, 2-56, 2-59, 2-63, 2-70, 3-72
Trigger mode · 2-60
Trigger time point · 2-56
Trigger type · 2-62, 3-72
trigger() · 3-2, 3-17, 3-46, 3-72
Triggerblock · 2-67
Triggering · 2-63, 3-2, 3-46, 3-71, 3-72
Triggering of logging · 3-46
Trigonometry · 3-57, 3-58
Tx attribute · 2-17, 2-40
Tx error counter · 2-42
TXDATA · 3-31, 3-34
TXREMOTE · 3-31, 3-34
TXREQUEST · 2-14, **3-7**, 3-31, **3-33**, 3-34
TXREQUESTDATA · 3-31, 3-34
TXREQUESTREMOTE · 3-31, 3-34
TxRq attribute · 2-17

U

Undo function · 2-10

V

Value range check · 3-5
Value table · 2-52
VAN · 3-42, 3-43
Virtual channel · 2-9, 4-9

W

Warning Limit · **3-7**, **3-12**
Wildcard · 3-25
WIN.INI · 3-44
Window control · 2-6
Window layout · 2-43, 2-44
Windows timer · 5-6
Working directory · 2-7
write() · **2-43**, **3-11**, 3-17, 3-43, 3-64, 3-70
writeToLog() · **3-17**, 3-46
writeToLogEx() · **3-17**, 3-46

Z

Zoom · 2-40
Zoom mode · 2-5